Київський столичний університет імені Бориса Грінченка

Володимир Соколов

ТЕХНОЛОГІЇ БЕЗПЕКИ ВЕБРЕСУРСІВ

Навчальний посібник

для здобувачів другого (освітнього) рівня вищої освіти
спеціальності 125/F5 «Кібербезпека та захист інформації»
галузі знань 12/F «Інформаційні технології»

Київ — 2027

УДК УДК 004.056:004.738.5(075.8)
ББК 32.973.202
С594

Препринт

Соколов В. Ю. Технології безпеки вебресурсів : навчальний посібник для здобувачів другого (освітнього) рівня вищої освіти. Київ, КСУБГ, 2027, 91 с. — 17 розділів.

Навчальний посібник охоплює теоретичні основи та практичні аспекти технологій безпеки вебресурсів. Розглянуто вебресурси як об'єкт захисту, архітектури вебплатформи, модель порушника та простір загроз; нормативно-правове й стандартизаційне забезпечення в Україні та Європейському Союзі; математичні та кількісні основи оцінювання захищеності; моделювання загроз і управління ризиками; захист транспортного рівня та інфраструктури доставки; безпеку браузера й клієнтської частини; протидію ін'єкційним атакам; автентифікацію й керування сеансами; авторизацію та федеративну ідентифікацію; безпеку API й мікросервісних архітектур; захист від відмови в обслуговуванні та автоматизованих загроз; безпеку серверної, хмарної й контейнерної інфраструктури; безпеку ланцюга постачання програмного забезпечення; безпеку вебзастосунків зі штучним інтелектом; тестування захищеності; моніторинг, виявлення вторгнень і реагування на інциденти; безпечний життєвий цикл розроблення, управління безпекою та напрями наукових досліджень.

Кожен розділ завершується висновками та питаннями для самоперевірки, серед яких — дослідницькі завдання, придатні як відправна точка для наукової статті чи фрагмента дисертаційного дослідження. Посібник призначено для магістрантів галузі знань 12/F «Інформаційні технології» спеціальності 125/F5 «Кібербезпека та захист інформації», а також буде корисним науковим і науково-педагогічним працівникам, фахівцям із захисту інформації та кібербезпеки.

© Володимир Соколов, 2027
© Київський столичний університет імені Бориса Грінченка, 2027

Зміст

Перелік умовних скорочень **1**

Вступ **5**

1 Вебресурси як об'єкт захисту: архітектури, модель порушника й простір загроз **9**
1.1 Поняття та класифікація вебресурсів 9
1.2 Еволюція архітектур вебресурсів 11
1.3 Протокол HTTP і вебплатформа як середовище виконання . 12
1.4 Властивості безпеки вебресурсів 14
1.5 Модель порушника 15
1.6 Простір загроз і багаторівнева поверхня атаки 18
1.7 Український контекст: атаки на державні вебресурси 21
1.8 Термінологічний апарат і структура курсу 23
Висновки до розділу 24
Питання для самоперевірки 24

2 Нормативно-правове та стандартизаційне забезпечення безпеки вебресурсів **27**
2.1 Структура нормативного поля 27
2.2 Законодавство України 29
2.3 Право Європейського Союзу 31
2.4 Міжнародні стандарти 34
2.5 Галузеві рамки OWASP 36
2.6 Оцінювання відповідності та сертифікація 37
2.7 Порівняльний аналіз вимог України та ЄС 39
2.8 Кейс: визначення застосовних вимог 39
Висновки до розділу 40
Питання для самоперевірки 41

3 Математичні та кількісні основи оцінювання захищеності вебресурсів **43**
3.1 Постановка задачі кількісного оцінювання захищеності . . . 43
3.2 Ентропія та стійкість секретів 44
3.3 Моделі підбору облікових даних 46
3.4 Теорія статистичного виявлення 48

3.5 Моделі масового обслуговування для оцінювання стійкості до відмови в обслуговуванні 50
3.6 Атакові графи та марковські моделі 52
3.7 Оцінки вразливостей і ймовірнісна пріоритизація 54
3.8 Розгорнутий приклад комплексного розрахунку 56
Висновки до розділу 57
Питання для самоперевірки 58

4 Моделювання загроз і управління ризиками вебзастосунків 61
4.1 Процес управління ризиками 61
4.2 Моделювання загроз вебзастосунку 63
4.3 Бази знань про атаки та слабкості 66
4.4 Дерева атак 67
4.5 Кількісне оцінювання ризику 68
4.6 Безпека за проєктом 70
4.7 Невизначеність оцінок 71
4.8 Кейс: модель загроз і реєстр ризиків порталу державних послуг 73
Висновки до розділу 74
Питання для самоперевірки 75

5 Захист транспортного рівня та інфраструктури доставки 77
5.1 Роль транспортного рівня в захисті вебресурсу 77
5.2 Протокол TLS 1.3 78
5.3 Інфраструктура відкритих ключів вебу 83
5.4 HSTS і захист від зниження до HTTP 86
5.5 HTTP/2 і HTTP/3: безпекові аспекти 87
5.6 Шифрування ClientHello і постквантовий перехід 88
5.7 DNS як ланка ланцюга довіри 90
5.8 CDN і зворотні проксі 92
5.9 Кейс: модернізація транспортного рівня державного порталу 93
Висновки до розділу 95
Питання для самоперевірки 95

6 Безпека браузера та клієнтської частини вебзастосунків 99
6.1 Модель безпеки браузера 99
6.2 Міжсайтовий скриптинг 101
6.3 Політика безпеки вмісту і Trusted Types 103
6.4 Підробка міжсайтових запитів і атрибут SameSite 104
6.5 Міжпоходжувальний обмін ресурсами 106
6.6 Клікджекінг і заголовки ізоляції 107
6.7 Сторонній код у браузері 109
6.8 Побічні канали і розширення браузера 110

6.9 Кейс: захист клієнтської частини кабінету громадянина . . . 112
Висновки до розділу . . . 113
Питання для самоперевірки . . . 114

7 Ін'єкційні атаки та безпечна обробка даних 117
7.1 Загальна модель вразливості . . . 117
7.2 Безпечна робота з реляційними базами даних . . . 119
7.3 Безпечне звернення до інших інтерпретаторів . . . 120
7.4 Контроль вихідних запитів сервера . . . 122
7.5 Безпечна робота з файлами . . . 123
7.6 Валідація, канонікалізація та обробка винятків . . . 124
7.7 Кейс: програма запобігання ін'єкціям у життєвому циклі . . . 125
Висновки до розділу . . . 126
Питання для самоперевірки . . . 126

8 Автентифікація та керування сеансами 129
8.1 Фактори та рівні автентифікації . . . 129
8.2 Паролі: зберігання та перевірка . . . 130
8.3 Протидія масовим атакам на облікові дані . . . 131
8.4 Багатофакторна та фішингостійка автентифікація . . . 132
8.5 Відновлення облікового запису . . . 134
8.6 Керування сеансами . . . 134
8.7 Українські засоби електронної ідентифікації . . . 135
8.8 Кейс: підсистема автентифікації державного порталу . . . 136
Висновки до розділу . . . 137
Питання для самоперевірки . . . 138

9 Авторизація, федеративна ідентифікація та керування доступом 141
9.1 Моделі керування доступом . . . 141
9.2 Порушення контролю доступу та їх уникнення . . . 143
9.3 OAuth 2.0/2.1: делегована авторизація . . . 144
9.4 Федеративна ідентифікація . . . 145
9.5 Безпечне застосування токенів JWT . . . 146
9.6 Нульова довіра для вебдоступу . . . 146
9.7 Кейс: централізована авторизація державного порталу . . . 147
Висновки до розділу . . . 148
Питання для самоперевірки . . . 148

10 Безпека API та мікросервісних архітектур 151
10.1 Архітектурні стилі API та їхня безпека . . . 151
10.2 OWASP API Security Top 10 . . . 152
10.3 Інвентаризація та тіньові API . . . 153

10.4 Захист периметра API 154
10.5 Міжсервісна авторизація 155
10.6 Кейс: захист API державного порталу 155
Висновки до розділу 156
Питання для самоперевірки 157

11 Захист від відмови в обслуговуванні та автоматизованих загроз 159
11.1 Таксономія атак на доступність 159
11.2 Пом'якшення атак на доступність 161
11.3 Автоматизовані загрози та протидія ботам 162
11.4 Вебекрани прикладного рівня (WAF/WAAP) 163
11.5 Український досвід 164
11.6 Кейс: захист доступності державного порталу 164
Висновки до розділу 165
Питання для самоперевірки 166

12 Безпека серверної, хмарної та контейнерної інфраструктури вебресурсів 169
12.1 Посилення конфігурації 169
12.2 Хмара та модель спільної відповідальності 170
12.3 Контейнери та оркестрація 171
12.4 Інфраструктура як код і виявлення помилок конфігурації . . 172
12.5 Резервне копіювання й відновлення 173
12.6 Кейс: посилення інфраструктури державного порталу 173
Висновки до розділу 174
Питання для самоперевірки 175

13 Безпека ланцюга постачання програмного забезпечення та систем керування вмістом 177
13.1 Транзитивний ризик залежностей 177
13.2 Класи атак на ланцюг постачання 178
13.3 Прозорість складу: SBOM і VEX 179
13.4 Цілісність збирання: SLSA, Sigstore, CI/CD 180
13.5 Системи керування вмістом і сторонні скрипти 180
13.6 Регуляторні вимоги: Акт про кіберстійкість 181
13.7 Кейс: захист ланцюга постачання державного порталу 181
Висновки до розділу 182
Питання для самоперевірки 183

14 Безпека вебзастосунків зі штучним інтелектом і перспективні загрози 185
14.1 Архітектура вебзастосунків з LLM . . . 185
14.2 Ін'єкція підказок . . . 186
14.3 Автономія, інструменти й витік даних . . . 187
14.4 Отруєння та цілісність джерел . . . 188
14.5 Штучний інтелект у захисті вебресурсів . . . 188
14.6 Перспективні технології . . . 188
14.7 Регуляторний контекст . . . 189
14.8 Кейс: безпечний помічник державного порталу . . . 189
Висновки до розділу . . . 190
Питання для самоперевірки . . . 191

15 Тестування захищеності вебресурсів 193
15.1 Методології та місце тестування в життєвому циклі . . . 193
15.2 Автоматичні засоби тестування . . . 194
15.3 Ручне тестування й оцінювання на проникнення . . . 195
15.4 Програми винагород і координоване розкриття . . . 195
15.5 Метрики покриття та ефективності . . . 196
15.6 Кейс: програма тестування державного порталу . . . 197
Висновки до розділу . . . 198
Питання для самоперевірки . . . 198

16 Моніторинг, виявлення вторгнень і реагування на інциденти вебресурсів 201
16.1 Журналювання . . . 201
16.2 Централізоване збирання й кореляція . . . 202
16.3 Телеметрія захисних засобів і виявлення аномалій . . . 202
16.4 Реагування на інциденти . . . 203
16.5 Цифрова криміналістика та відновлення . . . 205
16.6 Кейс: моніторинг і реагування державного порталу . . . 205
Висновки до розділу . . . 206
Питання для самоперевірки . . . 207

17 Безпечний життєвий цикл розроблення, управління безпекою та напрями наукових досліджень 209
17.1 Безпечний життєвий цикл розроблення . . . 209
17.2 Моделі зрілості та DevSecOps . . . 210
17.3 Безпека як властивість платформи . . . 211
17.4 Метрики програми безпеки . . . 211
17.5 Підготовка кадрів . . . 212
17.6 Напрями наукових досліджень . . . 212

17.7 Методологія дисертаційного дослідження 212
17.8 Кейс: побудова програми безпеки вебресурсів відомства . . . 213
Висновки до розділу 214
Питання для самоперевірки 214

Список використаних джерел **217**

Додаток. Завдання для самостійної роботи **239**

Перелік умовних скорочень

БД	база даних
ЄС	Європейський Союз
КЕП	кваліфікований електронний підпис
ПЗ	програмне забезпечення
ПД	персональні дані
СУБД	система управління базами даних
AAL	Authentication Assurance Level (рівень автентифікації)
ABAC	Attribute-Based Access Control
ACME	Automatic Certificate Management Environment
AiTM	Adversary-in-the-Middle (проксі-фішинг)
API	Application Programming Interface (програмний інтерфейс)
ASVS	Application Security Verification Standard (OWASP)
BFLA	Broken Function Level Authorization
BOLA	Broken Object Level Authorization
CAA	Certification Authority Authorization
CDN	Content Delivery Network (мережа доставки вмісту)
CERT-UA	Урядова команда реагування на комп'ютерні надзвичайні події України
CMS	Content Management System (система керування вмістом)
CORS	Cross-Origin Resource Sharing
CRA	Cyber Resilience Act (Акт ЄС про кіберстійкість)
CSP	Content Security Policy
CSPM	Cloud Security Posture Management
CSRF	Cross-Site Request Forgery
CT	Certificate Transparency (прозорість сертифікатів)
CVD	Coordinated Vulnerability Disclosure
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
DAST	Dynamic Application Security Testing
DBSC	Device Bound Session Credentials

DDoS	Distributed Denial of Service
DFD	Data Flow Diagram (діаграма потоків даних)
DPoP	Demonstrating Proof of Possession
ECH	Encrypted Client Hello
EPSS	Exploit Prediction Scoring System
FaaS	Function as a Service
FAIR	Factor Analysis of Information Risk
FIDO2	Fast Identity Online 2 (WebAuthn + CTAP)
GDPR	General Data Protection Regulation
HSTS	HTTP Strict Transport Security
IaaS	Infrastructure as a Service
IaC	Infrastructure as Code (інфраструктура як код)
IAM	Identity and Access Management
IAST	Interactive Application Security Testing
IDOR	Insecure Direct Object Reference
JWT	JSON Web Token
LLM	Large Language Model (велика мовна модель)
MFA	Multi-Factor Authentication (багатофакторна автентифікація)
ML-KEM	Module-Lattice Key Encapsulation Mechanism (FIPS 203)
mTLS	Mutual TLS (взаємний TLS)
NIST	National Institute of Standards and Technology
OAuth	Open Authorization
OIDC	OpenID Connect
ORM	Object-Relational Mapping
OWASP	Open Worldwide Application Security Project
PaaS	Platform as a Service
PASTA	Process for Attack Simulation and Threat Analysis
PKCE	Proof Key for Code Exchange
PKI	Public Key Infrastructure (інфраструктура відкритих ключів)
PPV	Positive Predictive Value (прогностична цінність)
QUIC	Quick UDP Internet Connections
RAG	Retrieval-Augmented Generation
RASP	Runtime Application Self-Protection
RBAC	Role-Based Access Control

ReBAC	Relationship-Based Access Control
ROSI	Return on Security Investment
SAST	Static Application Security Testing
SBOM	Software Bill of Materials
SCA	Software Composition Analysis
SIEM	Security Information and Event Management
SLSA	Supply-chain Levels for Software Artifacts
SOP	Same-Origin Policy (політика того самого походження)
SQL	Structured Query Language
SRI	Subresource Integrity (цілісність підресурсів)
SSDF	Secure Software Development Framework (NIST)
SSDLC	Secure Software Development Life Cycle
SSRF	Server-Side Request Forgery
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
TLS	Transport Layer Security
TOTP	Time-based One-Time Password
VEX	Vulnerability Exploitability eXchange
WAF	Web Application Firewall (вебекран прикладного рівня)
WAAP	Web Application and API Protection
WSTG	Web Security Testing Guide (OWASP)
XSS	Cross-Site Scripting (міжсайтовий скриптинг)
XXE	XML External Entity

Вступ

Вебресурси стали основною формою надання інформаційних і транзакційних послуг. Через вебінтерфейси та програмні інтерфейси (API) громадяни отримують державні послуги, здійснюють платежі, навчаються й лікуються, а організації керують бізнес-процесами, ланцюгами постачання й навіть промисловими об'єктами. Відповідно вебзастосунки є найпоширенішою точкою входу для зловмисників: помилки контролю доступу, ін'єкції, вади автентифікації й неправильні конфігурації десятиліттями очолюють рейтинги вразливостей, а атаки на ланцюг постачання програмного забезпечення, відмова в обслуговуванні терабітного масштабу та зловживання вебзастосунками зі штучним інтелектом формують нові класи загроз. Специфіка захисту вебресурсів полягає в тому, що вони за визначенням відкриті для анонімних користувачів з усього світу, виконують код у середовищі, яке не контролює власник (браузер клієнта), і складаються з великої кількості сторонніх компонентів, кожен з яких може стати джерелом компрометації.

Досвід України останніх років зробив ці питання предметом не лише академічного інтересу. Масові дефейси й руйнівні атаки на державні вебресурси напередодні повномасштабного вторгнення, постійні розподілені атаки на відмову в обслуговуванні на портали органів влади, банків і медіа, компрометації облікових записів і витоки персональних даних перетворили безпеку вебресурсів на складову національної стійкості. Одночасно цифровізація державних послуг (портал і застосунок «Дія», електронні реєстри) підвищила цінність вебінфраструктури як об'єкта атаки. Змінилося й нормативне поле: оновлено законодавство про кібербезпеку та захист державних інформаційних ресурсів, триває гармонізація з вимогами Європейського Союзу, зокрема Директивою NIS 2 і Регламентом про кіберстійкість.

Мета і завдання навчальної дисципліни

Мета курсу — формування в магістрантів знань, умінь і навичок із застосування теоретичних основ та технологій забезпечення безпеки вебресурсів для захисту інформаційних ресурсів і послуг.

Завдання курсу — отримання теоретичних знань та практичних умінь з дослідження й застосування технологій безпеки вебресурсів та набуття таких компетентностей:

- здатність застосовувати сучасні технології безпеки під час проєктування, розроблення та експлуатації захищених вебресурсів;
- здатність оцінювати захищеність і знаходити раціональні рішення щодо захисту вебзастосунків, програмних інтерфейсів та інфраструктури їх доставки;
- здатність до вдосконалення й наукового дослідження засобів і технологій забезпечення безпеки вебресурсів.

Структура посібника

Матеріал згруповано у чотири логічні блоки.

Блок I. Засади (розділи 1–4) формує понятійний апарат: вебресурси як об'єкт захисту, архітектури вебплатформи та простір загроз; нормативно-правове й стандартизаційне поле; математичні та кількісні основи оцінювання захищеності; моделювання загроз і управління ризиками.

Блок II. Механізми та технології захисту вебзастосунків (розділи 5–11) присвячено власне засобам: захисту транспортного рівня та інфраструктури доставки, безпеці браузера й клієнтської частини, протидії ін'єкційним атакам, автентифікації та керуванню сеансами, авторизації й федеративній ідентифікації, безпеці API та мікросервісних архітектур, захисту від відмови в обслуговуванні та автоматизованих загроз.

Блок III. Платформи, інфраструктура та нові класи загроз (розділи 12–14) охоплює безпеку серверної, хмарної й контейнерної інфраструктури, безпеку ланцюга постачання програмного забезпечення та систем керування вмістом, безпеку вебзастосунків зі штучним інтелектом.

Блок IV. Операційна безпека та управління (розділи 15–17) охоплює тестування захищеності, моніторинг і реагування на інциденти, а також безпечний життєвий цикл розроблення, управління програмою безпеки й методологію наукових досліджень.

Як працювати з посібником

Посібник розраховано на здобувачів другого (освітнього) рівня вищої освіти, які вже володіють базовими знаннями з інформаційної безпеки, комп'ютерних мереж, вебтехнологій і криптографії. Виклад побудовано як систематизація на рівні моделей, доказових аргументів і критичного аналізу чинних рішень, а не як первинне ознайомлення з предметом чи інструкція з експлуатації конкретних інструментів. Формули супроводжуються поясненням позначень і числовими прикладами, які варто відтворити самостійно. Фрагменти коду й конфігурацій наведено лише там, де вони пояснюють механізм атаки чи захисту.

Питання для самоперевірки наприкінці кожного розділу неоднорідні: частина перевіряє відтворення матеріалу, частина потребує аналізу й порівняння, а завершальні питання сформульовано як дослідницькі завдання — вони можуть слугувати відправною точкою для наукової статті або фрагмента дисертаційного дослідження. Магістрантові рекомендується опрацьовувати їх у зв'язці з власною темою.

Посилання на джерела наведено в квадратних дужках; повний список подано наприкінці видання. Перевагу віддано чинним нормативним документам, національним та міжнародним стандартам, специфікаціям IETF, W3C і WHATWG, галузевим рамкам OWASP і рецензованим науковим працям. Усі нормативні акти, стандарти й рейтинги загроз слід перевіряти на чинність редакції на момент використання: вебтехнології та ландшафт загроз змінюються особливо швидко. Практичні вправи з атакувальними техніками допустимо виконувати лише на власних стендах або в межах письмово погодженого тестування на проникнення.

Термінологічне зауваження

У виданні вживаються терміни **вебресурс**, **вебзастосунок**, **вебсервер** відповідно до чинного правопису (разом). Під *вебресурсом* розуміємо сукупність вебсайтів, вебзастосунків і програмних інтерфейсів, доступних через протокол HTTP, разом з інфраструктурою, що забезпечує їх функціонування. Англомовні терміни, що не мають усталеного українського відповідника, наводяться в дужках мовою оригіналу при першому вживанні.

РОЗДІЛ 1

Вебресурси як об'єкт захисту: архітектури, модель порушника й простір загроз

Вебресурси стали основною формою надання інформаційних, фінансових і державних послуг. Водночас вебресурс за визначенням відкритий для будь-кого в Інтернеті: його вхідні точки доступні анонімному порушникові, клієнтська частина виконується на пристрої, який власник не контролює, а значна частина коду надходить від сторонніх постачальників. Розділ формує системне бачення вебресурсу як об'єкта захисту — від класифікації й архітектур до моделі порушника і простору загроз, на які спиратимуться наступні розділи. Окремо проаналізовано досвід атак на українські державні вебресурси 2022–2025 рр.

1.1 Поняття та класифікація вебресурсів

1.1.1 Базові означення

Вебресурс — сукупність інформаційних і програмних засобів, доступ до яких надається за протоколом HTTP (HTTPS) за уніфікованим ідентифікатором ресурсу (URI), разом з інфраструктурою, що забезпечує їхнє функціонування. З правового погляду вебресурс є різновидом інформаційно-комунікаційної системи, тож на нього поширюються вимоги законодавства про захист інформації [37] і кібербезпеку [40] (розділ 2).

Усередині цього поняття розрізняють кілька видів, що відрізняються саме з погляду безпеки. **Вебсайт** призначений переважно для подання інформації, а логіка зосереджена в системі керування вмістом (CMS). **Вебзастосунок** (web application) має стан, облікові записи, права доступу й обробляє персональні чи фінансові дані. **Вебпрограмний інтерфейс** (web API) повертає структуровані дані іншим програмам, тому браузерні механізми захисту тут не діють. **Односторінковий застосунок** (Single-Page Application, SPA) обмінюється із сервером лише даними через API; **прогресивний вебзастосунок** (PWA) додає сервіс-воркери і локальні сховища. Окремий клас становлять **державні вебресурси** — портали електронних послуг, вебінтерфейси реєстрів, офіційні сайти органів влади. Їх вирізняє поєднання масового доступу, обробки персональних даних усього населення [38], юридичної значущості операцій [34] і символічної цінності для політично мотивованого порушника.

1.1.2 Рівні об'єкта захисту

Запит користувача проходить через кілька середовищ із різними власниками й моделями довіри. Доцільно виділити п'ять рівнів об'єкта захисту: *клієнт і браузер* (пристрій, розширення, код застосунку в браузері, локальні сховища; розділ 6); *канал* (DNS, TLS, вузли мережі доставки вмісту CDN, зворотні проксі; розділи 5, 11); *вебзастосунок* (серверна логіка, сеанси, автентифікація, авторизація, обробка вхідних даних, де локалізується більшість вразливостей з OWASP Top 10 [231] і CWE Top 25 [54]; розділи 7–10); *платформа та інфраструктура* (вебсервер, ОС, контейнери, хмарні сервіси, секрети; розділ 12); *сторонні компоненти* (бібліотеки, плагіни CMS, сторонні скрипти, зовнішні API, конвеєри збирання), що пронизують усі інші рівні (розділ 13). Засоби захисту одного рівня не компенсують слабкостей іншого: TLS не захищає від ін'єкції SQL, а перевірка вхідних даних на сервері безсила проти скомпрометованого стороннього скрипту в браузері.

1.1.3 Класифікація вебресурсів за моделлю довіри

З погляду безпеки вебресурси доцільно класифікувати за моделлю довіри: хто є користувачем і клієнтом, які дані обробляються і наскільки значущі наслідки порушення (табл. 1.1).

Таблиця 1.1 — Класи вебресурсів та їхні безпекові характеристики

Клас	Модель довіри	Характерні ризики
Інформаційний сайт (CMS)	анонімні читачі, кілька редакторів	дефейс, вразливі плагіни, захоплення адмінпанелі, розміщення шкідливого вмісту
Вебзастосунок з обліковими записами	автентифіковані користувачі з різними ролями	порушення контролю доступу, викрадення сеансів, XSS, ін'єкції, витік ПД
Вебпрограмний інтерфейс (API)	програмні клієнти з токенами	BOLA/IDOR, надмірне розкриття даних, зловживання частотою, тіньові API
SPA/PWA	логіка частково в браузері, токени в клієнті	DOM-XSS, зберігання токенів, сторонні скрипти, кешування в сервіс-воркері
Електронна комерція	клієнти, платіжні провайдери	скимери на сторінках оплати, боти, шахрайство, вимоги PCI DSS [243]
Державний портал і реєстр	усе населення, юридично значущі операції	масові витоки ПД, DDoS, дефейс, атаки на ідентифікацію, ланцюг постачання підрядників

Зі зростанням значущості операцій зростає не лише цінність ресурсу для порушника, а й кількість незалежних доменів довіри: для державного порталу вона охоплює громадян, розпорядників реєстрів, банки-постачальники ідентифікації, підрядників, хмарних і CDN-провайдерів. Тому вимоги до державних вебресурсів встановлюються не лише галузевими

рамками на кшталт OWASP ASVS [226], а й нормативними актами щодо захисту державних інформаційних ресурсів [32].

1.2 Еволюція архітектур вебресурсів

1.2.1 Від статичних сторінок до мікросервісів

Історію вебархітектур зручно розглядати як послідовність етапів, на кожному з яких змінювалися розподіл логіки між клієнтом і сервером і модель безпеки (табл. 1.2). Спільний шлюзовий інтерфейс (CGI) уперше дозволив запускати на сервері програму з параметрами від користувача, і з ним з'явилася ін'єкція команд ОС. Стек LAMP і CMS зробили типовою трирівневу архітектуру, а з нею прийшли SQLi, XSS, CSRF і масові атаки на CMS [280, 305]. Односторінкові застосунки перенесли логіку в браузер, а сервер став постачальником даних через API в стилі REST [127]: клієнтський код повністю видимий і модифікований порушником, тож будь-яка перевірка в браузері має дублюватися на сервері. Мікросервіси перетворили внутрішні виклики функцій на мережеві запити, що потребують міжсервісної автентифікації (розділ 10), а serverless і edge-обчислення переносять код у середовище провайдера, де основними об'єктами захисту стають ідентичності, ролі й конфігурація.

Таблиця 1.2 — Етапи еволюції вебархітектур і зміна моделі безпеки

Етап	Розподіл логіки	Нові класи загроз і зміна моделі безпеки
Статичні сторінки	лише сервер файлів	обхід каталогів, розкриття вмісту; периметр = сервер
CGI	програми на сервері	ін'єкція команд, переповнення буфера, неперевірені параметри
LAMP, CMS	трирівнева, логіка на сервері	SQLi, XSS, CSRF, вразливі плагіни, масові атаки на CMS
AJAX, SPA	логіка в браузері, сервер = API	DOM-XSS, зберігання токенів, CORS, надмірні дані в API
Мікросервіси	десятки сервісів, шлюз API	міжсервісна довіра, SSRF, тіньові API, секрети в конфігурації
Serverless, edge	код у середовищі провайдера	надмірні права ролей, помилки конфігурації, спільна відповідальність
CDN, WAF/WAAP	проміжні вузли з розкриттям TLS	довіра до провайдера, обхід до джерела, концентрація ризику
LLM-агенти	рішення ухвалює модель	ін'єкція підказок, надмірна автономія (розділ 14)

Кожен етап не скасовує загроз попереднього: ін'єкції команд ОС і SQL досі є в CWE Top 25:2025 [54]. Сучасний вебресурс поєднує CMS, SPA, мікросервіси, хмарні функції й CDN, тому його поверхня атаки є *об'єднанням* поверхонь атаки всіх використаних стилів.

1.2.2 Узагальнена архітектура та домени довіри

На рис. 1.1 наведено узагальнену архітектуру сучасного вебресурсу з доменами довіри. Браузер користувача належить до домену, який власник не контролює. Периферійний рівень (CDN, WAF, захист від DDoS) зазвичай належить провайдерові, якому делеговано розкриття TLS і фільтрацію трафіку. Шлюз API, сервіси й дані належать власникові, але часто розміщуються в хмарі під спільною відповідальністю з провайдером. Сторонні постачальники — платіжні системи, постачальники ідентичності (OpenID Connect [260]), аналітика, сервіси штучного інтелекту — обмінюються даними із сервісами через API, а браузер безпосередньо завантажує їхні скрипти.

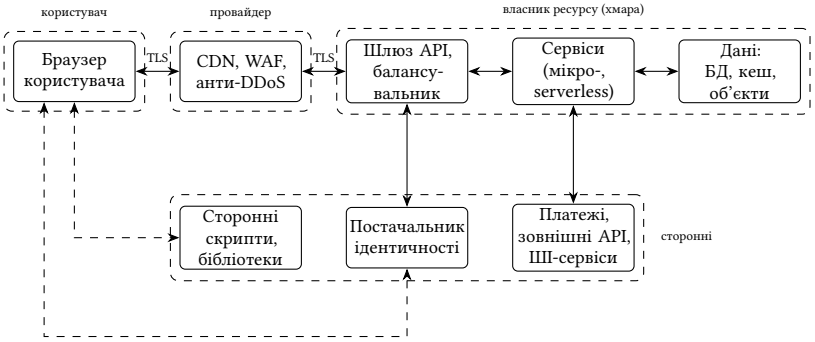


Рис. 1.1 — Узагальнена архітектура сучасного вебресурсу та домени довіри

З рис. 1.1 випливає: межа між доменами довіри — це місце, де дані мають бути перевірені (вхідні дані від браузера, відповіді зовнішніх API, токени); провайдер CDN/WAF бачить увесь трафік відкритим і концентрує ризик; сервер-джерело, доступний в обхід CDN, нейтралізує весь периферійний захист (розділ 11); у браузері код власника і сторонніх постачальників виконується з однаковими правами. Це узгоджується з принципом нульової довіри [258]: жоден сегмент не вважається довіреним лише через своє розташування.

1.3 Протокол HTTP і вебплатформа як середовище виконання

1.3.1 Версії HTTP і безстановість

HTTP визначає семантику, незалежну від версії: методи, коди стану, заголовки, подання [129]. У текстовому HTTP/1.1 розбіжність тлумачення заголовків `Content-Length` і `Transfer-Encoding` між проксі та сервером спричиняє контрабанду запитів (request smuggling); двійкове мультиплексуван-

ня HTTP/2 відкрило нові вектори відмови в обслуговуванні (розділ 11); у HTTP/3 [80] поверх QUIC шифрування TLS 1.3 [255] вбудоване в транспорт, тож мережевий моніторинг втрачає видимість і контроль зміщується до кінцевих точок.

Принциповою властивістю HTTP є **безстановість**: кожен запит обробляється незалежно. Стан сеансу відтворюється поверх протоколу за допомогою cookie [70] або токенів у заголовку `Authorization`. Звідси фундаментальна вразливість вебу: *володіння ідентифікатором сеансу еквівалентне автентифікації.* Хто отримав cookie сеансу — через XSS, перехоплення, фіксацію чи витік журналів, — той діє від імені користувача без пароля і без багатофакторної автентифікації (розділ 8).

1.3.2 Походження та політика того самого походження

Центральним поняттям моделі безпеки вебплатформи є **походження** (origin), означене в RFC 6454 [71] як трійка $o = \langle \text{схема}, \text{хост}, \text{порт} \rangle$. Два URL мають однакове походження лише тоді, коли збігаються всі три компоненти. **Політика того самого походження** (Same-Origin Policy, SOP) забороняє сценаріям одного походження *читати* дані іншого, але браузер дозволяє *надсилати* запити до іншого походження (зображення, форми, скрипти) і автоматично додає до них cookie. Ця асиметрія є коренем атак CSRF і міжсайтових витоків, а механізми контрольованого послаблення SOP (CORS, `postMessage`) стають джерелом помилок конфігурації [152, 121] (розділ 6). Важливо, що SOP захищає походження *одне від одного*, але не *від коду, який сторінка сама завантажила*: скрипт, підключений з іншого домену, виконується з правами походження сторінки і може читати DOM, вміст форм і cookie без атрибута `HttpOnly`.

Отже, браузер фактично є операційною системою для недовіреного коду: він ізолює процеси сайтів, керує доступом до пристроїв і сховищ, а одночасно виконує сценарії кількох суб'єктів з різними намірами [58]. Відповіддю на цю модель є ізоляція сайтів, CSP [106], заголовки ізоляції походжень і Trusted Types.

1.3.3 Кількісна оцінка залежності сторінки від стороннього коду

Нікіфоракіс і співавтори встановили, що 88,45 % із 10 000 найпопулярніших сайтів включають хоча б один віддалений JavaScript-сценарій [216]; Лауінгер і співавтори на вибірці понад 133 тис. сайтів показали, що 37 % з них використовують бібліотеку JavaScript з відомою вразливістю [185]. За даними Web Almanac 2024, 92 % сторінок мають хоча б одного стороннього постачальника, медіана становить 27 постачальників для мільйона найпопулярніших сайтів і 66 — для тисячі найпопулярніших, а сценарії становлять 30,5 % сторонніх запитів [290].

Введемо **частку стороннього коду** сторінки

$$\theta = \frac{\sum_{j \in T} s_j}{\sum_{j \in S} s_j}, \tag{1.1}$$

де S — множина виконуваних ресурсів сторінки (сценаріїв і модулів WebAssembly), $T \subseteq S$ — ресурси з походжень, які не контролює власник, s_j — розмір j-го ресурсу. Якщо сторінка включає код n незалежних постачальників, кожен з яких може бути скомпрометований протягом року з імовірністю q_i, то за відсутності механізмів, що обмежують дію скомпрометованого коду (SRI, CSP, ізоляція в iframe), імовірність хоча б одного шкідливого включення за рік

$$P_{\text{комп}} = 1 - \prod_{i=1}^{n} (1 - q_i) \approx 1 - e^{-n\bar{q}}. \tag{1.2}$$

Приклад: за медіанних $n = 27$ і $\bar{q} = 0{,}002$ маємо $P_{\text{комп}} \approx 1 - e^{-0{,}054} \approx 5{,}3\,\%$ на рік; для сайту з верхньої тисячі ($n = 66$) — $1 - e^{-0{,}132} \approx 12{,}4\,\%$. Компрометація одного популярного постачальника (як у випадку Polyfill.io 2024 р.) одночасно зачіпає всі сайти, що його включають (розділ 13). Формула (1.2) вказує три напрями зниження ризику: зменшення n (самостійне розміщення бібліотек), зменшення q_i (вибір постачальників, фіксація версій) і розрив зв'язку між компрометацією постачальника та виконанням коду в контексті сторінки.

1.4 Властивості безпеки вебресурсів

Класична тріада «конфіденційність–цілісність–доступність» [15, 269] для вебресурсів доповнюється автентичністю, підзвітністю й приватністю. Специфіка вебу полягає не в переліку властивостей, а в тому, *де* і *ким* вони забезпечуються: частину механізмів реалізує браузер, яким власник не керує, частину — сторонні провайдери (табл. 1.3).

Таблиця 1.3 — Властивості безпеки вебресурсів

Властивість	Вебспецифіка й типові механізми	Розділи
Конфіденційність	TLS, HSTS, захист cookie, мінімізація даних у відповідях API, ізоляція походжень	5, 6, 10
Цілісність	цілісність вмісту сторінки (SRI, CSP), захист від ін'єкцій і CSRF, цілісність ланцюга постачання	6, 7, 13
Доступність	стійкість до DDoS прикладного рівня, CDN, обмеження частоти, обробка винятків	11, 12
Автентичність	сертифікати сервера, автентифікація користувачів і сервісів, ключі доступу (passkeys)	5, 8, 9
Підзвітність	журналювання дій, прив'язка до ідентичності, підпис транзакцій, аудит	8, 16
Приватність	мінімізація відстеження, керування згодою, обмеження сторонніх скриптів	2, 6

Конфіденційність у вебі порушується не лише перехопленням, а й надмірним розкриттям даних самим застосунком: API, що повертає повний об'єкт користувача, розкриває всі його поля кожному, хто відкриє інструменти розробника. *Цілісність* охоплює і дані на сервері, і код, що виконується в браузері; другий вимір порушують дефейси, скимери і компрометація сторонніх бібліотек. *Доступність* визначається найслабшою ланкою ланцюга DNS–CDN–джерело–база даних. *Автентичність* двобічна: користувач має бути впевнений у легітимності ресурсу, а ресурс — що запит надійшов від користувача і з його волі (протидія CSRF і clickjacking); NIST SP 800-63-4 [284] вимагає для вищих рівнів гарантії фішингостійкої автентифікації з прив'язкою автентифікатора до походження (WebAuthn [300]). *Приватність* має власні загрози — відстеження між сайтами, відбиток браузера, витоки через сторонні скрипти — і регулюється окремим законодавством [249, 38].

Властивості можуть суперечити одна одній: журналювання створює сховище персональних даних, CDN підвищує доступність, але розширює коло суб'єктів, що бачать трафік, протидія ботам погіршує приватність. Такі компроміси мають бути явно обґрунтовані в моделі загроз (розділ 4).

1.5 Модель порушника

1.5.1 Класичні моделі та їхня адаптація до вебу

Модель Долева–Яо, у якій порушник повністю контролює мережу, для вебу одночасно надто сильна (більшість атак не потребує контролю над мережею) і надто слабка (не охоплює ключової можливості — *змусити браузер жертви виконати дії* з її cookie). Тому Ахаве і співавтори [58] запропонували ієрархію: **вебпорушник** (web attacker) контролює власні сайти з дійсними сертифікатами і може заманити на них жертву, але не контролює

мережу; **мережевий порушник** (пасивний чи активний) додатково перехоплює і змінює трафік; **порушник-гаджет** (gadget attacker) може вбудувати обмежений вміст (зображення, рекламний блок, сценарій) у легітимний сайт. До ієрархії природно додати порушника з доступом до серверної інфраструктури (інсайдер) і порушника, що контролює ланцюг постачання.

Для практичного проєктування модель порушника доцільно задавати кортежем

$$\mathcal{A} = \langle \Pi,\ \Lambda,\ K,\ R,\ M \rangle, \tag{1.3}$$

де Π — позиція щодо системи (зовнішній анонімний, зареєстрований чи привілейований користувач, інсайдер, постачальник, державний актор); Λ — рівні, на яких діє порушник (підрозділ 1.1); K — апріорні знання (вихідний код, архітектура, облікові дані, вразливості нульового дня); R — ресурси (час, ботнет, кошти, персонал); M — мотивація (фінансова, ідеологічна, розвідувальна, руйнівна). Фінансово мотивований порушник обирає найлегші цілі, а державний актор — конкретну ціль, витрачаючи ресурси, непропорційні її вартості. Типові профілі наведено в табл. 1.4; їхні техніки доцільно описувати в термінах CAPEC [104] і MITRE ATT&CK [207]. Відповідно до нормативних документів ТЗІ [25] і вимог до кіберзахисту об'єктів критичної інфраструктури [17] модель порушника є обов'язковим елементом проєктної документації системи захисту (розділ 4).

Таблиця 1.4 — Типові профілі порушника вебресурсів

Профіль	Позиція, ресурси, мотивація	Характерні можливості й атаки
Автоматизований масовий сканер	зовнішній; ботнет; фінансова	сканування адресного простору, експлуатація відомих CVE, підбір облікових даних
Анонімний зовнішній порушник	зовнішній; помірні; різна	ручний пошук вразливостей, ін'єкції, XSS, фішингові двійники
Автентифікований користувач	зареєстрований; знає API; фінансова	IDOR/BOLA, підвищення привілеїв, зловживання бізнес-логікою
Інсайдер	розробник, адміністратор; повні знання	зловживання доступом, закладки в коді, витік секретів і даних
Постачальник	підрядник, бібліотека, CDN, SaaS	компрометація оновлень, сторонніх скриптів, CI/CD
Хактивіст	зовнішній; ботнет-волонтери; ідеологічна	DDoS, дефейси, витоки для розголосу
Державний актор (APT)	будь-яка; значні; розвідувальна, руйнівна	поєднання всіх можливостей, атаки на підрядників, тривала прихована присутність, знищення даних

1.5.2 Кількісна модель автоматизованого масового сканування

Найчисленнішим порушником є автоматизований сканер. За звітом Imperva 2025 р., автоматизований трафік уперше за десятиліття перевищив людський і становить 51 % вебтрафіку, з них 37 % припадає на шкідливих ботів [55]. Дурумерік, Вустров і Халдерман показали, що інструмент ZMap з одного комп'ютера обстежує весь адресний простір IPv4 менш ніж за 45 хв [118], а сканування всього Інтернету стали буденною практикою [116]. Час повного обстеження адресного простору $|\mathcal{N}|$ зі швидкістю r проб за секунду

$$T_{\text{скан}} = \frac{|\mathcal{N}|}{r}. \tag{1.4}$$

Для IPv4 ($|\mathcal{N}| = 2^{32} \approx 4{,}29 \cdot 10^{9}$) і максимальної швидкості гігабітного Ethernet для мінімальних кадрів ($r \approx 1{,}49 \cdot 10^{6}$ пакетів/с) маємо $T_{\text{скан}} \approx 2\,900$ с ≈ 48 хв, що близько до [118]. Для IPv6 (лише підмережа /64 містить 2^{64} адрес) повний перебір неможливий, і цілі знаходять через журнали прозорості сертифікатів (Certificate Transparency) і DNS. Тому *новий вебресурс слід вважати виявленим з моменту випуску для нього TLS-сертифіката* (розділ 5).

Якщо кожна з N незалежних спроб експлуатації (проб вхідних точок, варіантів корисного навантаження, пар «логін–пароль») успішна з імовірністю p, то ймовірність успіху хоча б однієї спроби і кількість спроб, потрібна

для досягнення ймовірності P^*,

$$P_N = 1 - (1-p)^N, \qquad N_{\min}(P^*) = \left\lceil \frac{\ln(1-P^*)}{\ln(1-p)} \right\rceil. \tag{1.5}$$

Приклад: для обходу фільтра з $p = 0{,}01$ на спробу досягнення $P^* = 0{,}95$ потребує $N_{\min} = \lceil \ln 0{,}05 / \ln 0{,}99 \rceil = 299$ спроб — тривіальна кількість для автоматизованого засобу. Обмеження частоти до 10 запитів на хвилину розтягує атаку до 30 хв, але не зупиняє ботнет із тисячами адрес (розділи 3, 11).

1.5.3 Гонка між експлуатацією та виправленням

Нехай після оприлюднення вразливості ($t = 0$) масова експлуатація починається в момент t_0, а далі спроби проти кожного вразливого ресурсу надходять пуассонівським потоком з інтенсивністю λ, кожна успішна з імовірністю p. Якщо ресурс виправлено в момент W, імовірність його компрометації до виправлення

$$P_{\text{злам}}(W) = 1 - \exp\bigl(-\lambda p\,(W - t_0)^+\bigr), \qquad (x)^+ = \max(x, 0). \tag{1.6}$$

Нехай для прикладу $t_0 = 72$ год, $\lambda = 0{,}1$ год$^{-1}$, $p = 0{,}5$. Оновлення за 48 год дає $P_{\text{злам}} = 0$; за 96 год — $1 - e^{-1{,}2} \approx 0{,}70$; за два тижні ($W = 336$ год) — $1 - e^{-13{,}2} \approx 1$. Отже, щомісячний цикл оновлень неприйнятний для критичних вразливостей вебресурсів, а основою пріоритизації стають каталог KEV та оцінки EPSS [171] (розділ 3). Дефейс державних сайтів у січні 2022 р. (підрозділ 1.7.3) є ілюстрацією саме цієї гонки.

1.6 Простір загроз і багаторівнева поверхня атаки

1.6.1 Багаторівнева поверхня атаки

Поверхня атаки — сукупність точок, через які порушник може ввести дані в систему, отримати з неї дані або вплинути на її поведінку. Манадхата і Вінг запропонували вимірювати її як суму внесків методів, каналів і елементів даних системи, кожен із яких зважено відношенням потенціалу шкоди до зусиль порушника [193]:

$$AS = \sum_{m \in M} \frac{d(m)}{e(m)} + \sum_{c \in C} \frac{d(c)}{e(c)} + \sum_{i \in I} \frac{d(i)}{e(i)}, \tag{1.7}$$

де M, C, I — доступні порушникові методи (кінцеві точки HTTP і API), канали (HTTPS, WebSocket, gRPC) і елементи даних (файли, записи БД, cookie); $d(\cdot)$ — потенціал шкоди (наприклад, привілеї, з якими виконується метод); $e(\cdot)$ — зусилля (рівень прав, потрібний для доступу). Метрика (1.7) відносна; вона вказує два шляхи зменшення поверхні атаки — вилучення

непотрібних кінцевих точок і підвищення вимог до автентифікації, а «тіньові» API збільшують AS непомітно для захисника (розділ 10).

Якісно поверхню атаки вебресурсу подають рівнями (рис. 1.2), уздовж яких проходять дві вертикалі: *ланцюг постачання*, що зачіпає всі рівні, і *журналювання та реагування*, без яких атака будь-якого рівня залишається непоміченою.

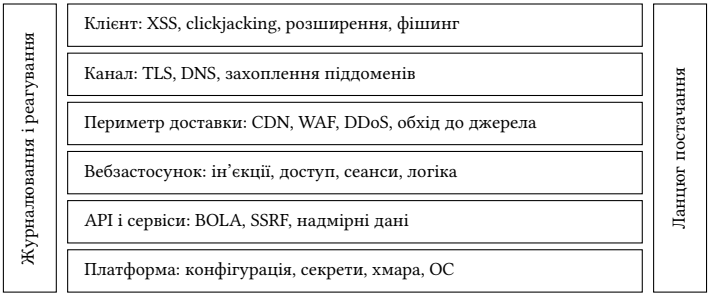


Рис. 1.2 — Багаторівнева поверхня атаки вебресурсу

1.6.2 OWASP Top 10:2025 і CWE Top 25:2025

Редакцію OWASP Top 10:2025 оприлюднено в листопаді 2025 р.; вона ґрунтується на даних тестування понад 2,8 млн застосунків і близько 175 тис. записів CVE, з 589 проаналізованих класів CWE до десяти категорій віднесено 248, вісім категорій визначено за даними, дві — за опитуванням спільноти [231]. Порівняно з редакцією 2021 р. з'явилися дві нові категорії — A03 і A10, SSRF включено до A01, неправильна конфігурація піднялася з п'ятого на друге місце, ін'єкції опустилися з третього на п'яте (табл. 1.5).

Таблиця 1.5 — Категорії OWASP Top 10:2025 і розділи посібника

Код	Категорія	Зміст	Розділи
A01	Broken Access Control	порушення контролю доступу, IDOR, SSRF	9, 10
A02	Security Misconfiguration	помилки конфігурації серверів, хмари, заголовків	12
A03	Software Supply Chain Failures	вразливі й скомпрометовані залежності, CI/CD	13
A04	Cryptographic Failures	слабка або відсутня криптографія	5
A05	Injection	SQLi, XSS, ін'єкція команд	6, 7
A06	Insecure Design	вади проєктування і бізнес-логіки	4, 17
A07	Authentication Failures	вади автентифікації і сеансів	8
A08	Software or Data Integrity Failures	цілісність коду, оновлень, десеріалізація	7, 13
A09	Security Logging & Alerting Failures	недостатнє журналювання й сповіщення	16
A10	Mishandling of Exceptional Conditions	неналежна обробка помилок і винятків	7

Рейтинг CWE Top 25:2025 (MITRE і CISA) підтверджує домінування вебспецифічних слабкостей: перше місце посідає міжсайтовий скриптинг CWE-79 (оцінка 60,38), друге — ін'єкція SQL CWE-89 (28,72), третє — підробка міжсайтових запитів CWE-352 (13,64), четверте — відсутність авторизації CWE-862 (13,28), що піднялася на п'ять позицій; до рейтингу входять також обхід шляху CWE-22, ін'єкція команд ОС CWE-78, необмежене завантаження файлів CWE-434, десеріалізація CWE-502 і SSRF CWE-918 [54]. OWASP Top 10 є переліком *ризиків* для навчання й постановки вимог, CWE Top 25 — рейтингом *слабкостей*; через різний рівень абстракції і методику класифікації їх доцільно використовувати спільно [30]. Для API, LLM-застосунків і верифікації існують окремі переліки OWASP [235, 238, 226].

1.6.3 Композиція загроз і ланцюги атак

Реальні атаки поєднують кілька вразливостей: захоплення облікового запису може складатися зі збереженого XSS, викрадення токена зі сховища браузера і його використання до API. Тому захищеність оцінюють за *ланцюгами атак*. Імовірність успіху ланцюга ℓ з m_ℓ послідовних етапів і хоча б одного з L альтернативних ланцюгів

$$P_\ell = \prod_{j=1}^{m_\ell} p_{\ell j}\big(\mathcal{A}, X\big), \qquad P_\Sigma = 1 - \prod_{\ell=1}^{L}\big(1 - P_\ell\big), \tag{1.8}$$

де $p_{\ell j}$ — імовірність успіху j-го етапу для порушника $\mathcal{A}$ за заходів захисту X. Формула припускає незалежність етапів і ланцюгів; її уточнення на атакових графах розглянуто в розділах 3 і 4. Захід, що обриває обов'язковий

етап, обнуляє P_ℓ незалежно від інших етапів; проте за кількох ланцюгів ризик визначається сукупністю: якщо $P_1 = 0{,}4$, $P_2 = 0{,}3$, $P_3 = 0{,}2$, то $P_\Sigma = 1 - 0{,}6 \cdot 0{,}7 \cdot 0{,}8 = 0{,}664$, а повне закриття найімовірнішого ланцюга знижує ризик лише до $0{,}44$. Звідси принцип глибокого захисту.

1.6.4 Основні класи загроз

Узагальнюючи галузеві рамки [231, 54], огляд загроз ENISA [119] і CyBOK [120], основні класи загроз вебресурсам розподілено між розділами посібника за рівнями рис. 1.2: транспорт, DNS і доставка (розділ 5), клієнтська частина (6), ін'єкції та обробка даних (7), автентифікація й сеанси (8), авторизація і федерація (9), API (10), відмова в обслуговуванні та боти (11), платформа й хмара (12), ланцюг постачання і CMS (13), застосунки зі ШІ (14).

1.7 Український контекст: атаки на державні вебресурси

1.7.1 Хронологія атак 2022–2025 рр.

Атаки на державні вебресурси України передували вторгненню і тривають досі, що робить український досвід унікальним джерелом даних про державного порушника.

Дефейси та WhisperGate (січень 2022 р.). У ніч на 14 січня 2022 р. було атаковано близько 70 державних вебсайтів, зокрема МЗС і МОН; на сторінках з'явилося повідомлення українською, російською і польською мовами; СБУ і Мінцифри заявили, що витоку даних не відбулося, а за оцінками фахівців використана вразливість була відома заздалегідь [28]. Аналітики Unit 42 встановили, що для дефейсу використано вразливість CVE-2021-32648 механізму скидання пароля October CMS [146]. 15 січня Microsoft повідомила про руйнівну програму WhisperGate, що маскувалася під програму-вимагача, але знищувала головний завантажувальний запис і файли без можливості відновлення [90].

DDoS-атаки (лютий–березень 2022 р.). 15 лютого 2022 р. DDoS-атака порушила роботу вебресурсів ПриватБанку, Ощадбанку, сайтів Міноборони і ЗСУ, а також порталу «Дія»; роботу відновили того ж вечора [53]. Атаку назвали наймасштабнішою в історії України: середня потужність сягала десятків гігабіт за секунду, витоку даних не виявлено, натомість зафіксовано ознаки участі іноземних спецслужб [111]. 23 лютого DDoS-атака на урядові сайти збіглася з поширенням вайпера HermeticWiper [90]. За даними Держспецзв'язку, з 15 лютого до середини березня 2022 р. Україна зазнала понад 3 000 DDoS-атак, з рекордом 275 атак за добу і піковою потужністю понад 100 Гбіт/с [10].

Динаміка 2023–2025 рр. За даними CERT-UA, у 2025 р. оброблено 5 927 кіберінцидентів проти 4 315 у 2024 р. (зростання на 37,4 %); найчастіше атаку-

вали органи місцевого самоврядування (35,7 %) і державні органи (19,7 %), а кількість фішингових інцидентів зросла вдвічі [49]. Огляд національної системи кібербезпеки за 2025 р. (НКЦК при РНБО) фіксує зниження кількості інцидентів критичного і високого рівнів із 59 до 12, доповнення псевдохактивістами DDoS-атак і дефейсів атаками на промислові системи, збій IT-систем Укрзалізниці 23 березня 2025 р., що вивів з ладу онлайн-продаж квитків, вебсайт і застосунок, і масові фішингові двійники порталів уряду та «Дії» [29].

1.7.2 Цифрові державні послуги як об'єкт захисту

Портал і застосунок «Дія» (понад 150 послуг і 22,7 млн користувачів [29]) поєднують усі класи табл. 1.1: портал, вебзастосунок з обліковими записами, API для мобільного застосунку й реєстрів, юридично значущі операції з підписом. У термінах моделі (1.3) такий ресурс одночасно є ціллю масового сканера, шахраїв, хактивістів і державного актора. Звідси пріоритети: доступність критична соціально, автентичність ресурсу для громадянина (протидія фішинговим двійникам) важить не менше за автентифікацію громадянина, а цілісність журналів потрібна для юридичної значущості операцій. Нормативну основу захисту становлять закони про кібербезпеку [40] і критичну інфраструктуру [39], координацію реагування здійснюють Держспецзв'язку [8] і CERT-UA [9] (розділи 2, 16).

1.7.3 Кейс: дефейс державних вебсайтів як ланцюг атаки

Вразливість CVE-2021-32648 стосувалася версій October CMS до 1.0.472 і 1.1.5: механізм скидання пароля порівнював код скидання нестрогим оператором `==`, тож передавання логічного значення `true` замість коду обходило перевірку і дозволяло встановити новий пароль адміністратора [146]. Це вада автентифікації (A07:2025), посилена використанням застарілого компонента (A03:2025). Реконструйований ланцюг має етапи: (1) виявлення сайтів на October CMS; (2) наявність невиправленої версії; (3) скидання пароля адміністратора; (4) зміна вмісту сторінки. Паралельно діяв другий ланцюг — розгортання вайпера WhisperGate в мережах українських організацій [90].

Оцінимо ймовірності за (1.8) для типового сайту на цій CMS. Для порушника, що веде масове сканування (1.4), $p_1 \approx 0{,}95$; частку невиправлених інсталяцій через кілька місяців після виправлення приймемо $p_2 = 0{,}5$; логічна вада не потребує підбору, $p_3 \approx 0{,}95$; доступ адміністратора гарантує зміну вмісту, $p_4 \approx 1$. Отже, $P_1 \approx 0{,}95 \cdot 0{,}5 \cdot 0{,}95 \approx 0{,}45$. Своєчасне оновлення (виграш гонки (1.6)) зводить p_2 до $0{,}05$, а P_1 — до $0{,}045$. Обмеження доступу до адміністративного інтерфейсу CMS довіреними адресами або VPN знижує p_3 для зовнішнього порушника, скажімо, до $0{,}05$; разом два заходи дають $P_1 \approx 0{,}95 \cdot 0{,}05 \cdot 0{,}05 \approx 0{,}0024$, тобто ризик знижується майже у

200 разів. Моніторинг цілісності сторінок атаці не запобігає, але скорочує час дефейсу. Для 70 сайтів із $P_1 = 0{,}45$ очікувана кількість успішних дефейсів становить ≈ 32, а за двох заходів імовірність хоча б одного дефейсу $1 - (1 - 0{,}0024)^{70} \approx 0{,}15$.

Кейс ілюструє три висновки: державний порушник використовує масові техніки і відомі вразливості, тому базова гігієна дає найбільший ефект; концентрація ресурсів на одній платформі робить окрему ваду системною, і події компрометації сайтів групи не є незалежними; видимий дефейс може супроводжувати приховану руйнівну операцію, тож реагування має включати розслідування в інфраструктурі розробника й хостингу (розділ 16).

1.8 Термінологічний апарат і структура курсу

Базові поняття вебресурсу, вебсайту, вебзастосунку і вебпрограмного інтерфейсу означено в підрозділі 1.1, походження — у підрозділі 1.3; **сайт** (site) — схема разом із зареєстрованим доменом. **Сервер-джерело** (origin server) — сервер з оригінальним вмістом на противагу вузлам CDN. **Сеанс** — послідовність запитів, пов'язаних спільним станом автентифікації. **Слабкість** (weakness) — тип вади, що за певних умов може призвести до **вразливості**, тобто конкретної вади, яку можна використати для порушення властивостей безпеки (термінологія CWE [105]); **експлойт** — засіб такого використання; **вектор атаки** — шлях, яким порушник досягає вразливості. **Дефейс** (defacement) — несанкціонована зміна вигляду вебсторінки. **Брандмауер вебзастосунків** (WAF) фільтрує запити HTTP на прикладному рівні, а **WAAP** (Web Application and API Protection) додатково охоплює API, ботів і DDoS. Загальні терміни вживаються відповідно до глосарію [269], національних стандартів [15] і ISO/IEC 27034 [163].

Структура курсу відображає рух від загального до конкретного і від механізмів до управління (рис. 1.3). Блок II іде за рівнями поверхні атаки рис. 1.2 від транспорту до доступності, блок III охоплює інфраструктуру, ланцюг постачання і ШІ, блок IV — операційну безпеку й управління. Для кожної категорії OWASP Top 10:2025 відповідні розділи вказано в табл. 1.5. Методологічною основою курсу є CyBOK [120] і фундаментальні праці з безпеки систем [61] і вебзастосунків [305, 151].

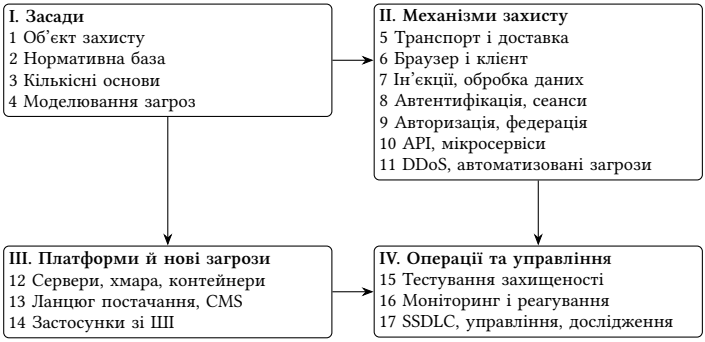


Рис. 1.3 — Структура курсу та зв'язки між блоками розділів

Висновки до розділу

Вебресурс відрізняється від інших інформаційних систем відкритістю вхідних точок для анонімного порушника, виконанням частини коду в неконтрольованому браузері і глибокою залежністю від сторонніх постачальників. Еволюція архітектур не скасовувала загроз попередніх етапів, тож поверхня атаки сучасного ресурсу є об'єднанням поверхонь усіх використаних стилів. Безстановість HTTP робить ідентифікатор сеансу еквівалентом автентифікації, а політика того самого походження не захищає сторінку від коду, який вона сама завантажила; за медіани 27 сторонніх постачальників навіть малі ймовірності їх компрометації дають відчутний річний ризик (1.2). Модель порушника для вебу має враховувати ієрархію «вебпорушник — мережевий порушник — порушник-гаджет» і параметри (1.3); найчисленнішим порушником є автоматизований сканер, для якого обстеження всього IPv4 займає менше години, тому час виправлення критичних вразливостей має вимірюватися годинами й днями (1.6). OWASP Top 10:2025 і CWE Top 25:2025 підтверджують домінування порушень контролю доступу, помилок конфігурації, збоїв ланцюга постачання, XSS та ін'єкцій, а захищеність слід оцінювати за ланцюгами атак (1.8). Український досвід 2022–2025 рр. — дефейси через застарілу CMS, вайпери, масовані DDoS, фішингові двійники порталів — показує, що державний порушник здебільшого використовує відомі вразливості, а базова гігієна, захист адміністративних інтерфейсів і готовність до реагування дають найбільший ефект.

Питання для самоперевірки

1. Дайте означення вебресурсу, вебсайту, вебзастосунку і вебпрограмного інтерфейсу. Чим вони відрізняються з погляду безпеки?

2. Назвіть п’ять рівнів об’єкта захисту вебресурсу й наведіть приклад їх взаємної некомпенсованості.
3. Порівняйте класи вебресурсів (табл. 1.1) за моделлю довіри. Чому для державних порталів кількість доменів довіри максимальна?
4. Користуючись табл. 1.2, поясніть, як перехід до SPA і мікросервісів змінив модель безпеки вебзастосунку.
5. Опишіть домени довіри на рис. 1.1. Які наслідки має доступність сервера-джерела в обхід CDN?
6. Чому безстановість HTTP робить ідентифікатор сеансу еквівалентом автентифікації? Які атрибути cookie зменшують цей ризик?
7. Чи мають однакове походження адреси `https://shop.example`, `https://shop.example:8443` і `http://shop.example`? Від чого не захищає політика того самого походження?
8. За формулою (1.2) обчисліть річну ймовірність компрометації сторінки з кодом 40 сторонніх постачальників при $\bar{q} = 0{,}003$. Як зміниться результат, якщо 30 бібліотек розмістити самостійно?
9. Які властивості доповнюють тріаду «конфіденційність–цілісність–доступність» для вебресурсів (табл. 1.3)? Наведіть приклад конфлікту двох властивостей.
10. Порівняйте вебпорушника, мережевого порушника і порушника-гаджета. Задайте параметри моделі (1.3) для автентифікованого користувача інтернет-банкінгу.
11. За формулою (1.4) оцініть час сканування всього IPv4 зі швидкістю 10^5 проб/с. Чому для IPv6 така оцінка втрачає сенс?
12. Користуючись (1.6), визначте максимальний час виправлення, за якого ймовірність компрометації не перевищує 0,1, якщо $t_0 = 48$ год, $\lambda = 0{,}2$ год$^{-1}$, $p = 0{,}5$.
13. Назвіть нові категорії OWASP Top 10:2025 і чотири перші позиції CWE Top 25:2025. Чим відрізняється призначення цих рамок?
14. Які уроки для захисту державних вебресурсів випливають з атак січня–лютого 2022 р.? Оцініть за (1.8) ефект лише багатофакторної автентифікації адміністраторів у кейсі підрозділу 1.7.3.
15. Дослідницьке: запропонуйте методику вимірювання частки стороннього коду θ (1.1) за кількістю викликів чутливих API браузера, а не за розміром, і обґрунтуйте її використання для оцінки ризику ланцюга постачання клієнтського коду.
16. Дослідницьке: побудуйте модель порушника для порталу цифрових державних послуг в умовах воєнного стану, що враховує хактивістів, шахраїв і державного актора, і визначте, як змінюється пріори-

тетність властивостей табл. 1.3 порівняно з комерційним вебзастосунком.

РОЗДІЛ 2

Нормативно-правове та стандартизаційне забезпечення безпеки вебресурсів

Вебресурс одночасно перебуває в полі дії кількох нормативних режимів: законодавства про кібербезпеку, захист інформації і персональних даних, права ЄС, міжнародних стандартів, галузевих стандартів платіжних карток і відкритих рамок OWASP. Фахівцю потрібно вміти визначати, які з них застосовні до конкретного ресурсу, хто є суб'єктом зобов'язань і як юридичні вимоги перетворюються на перевірювані технічні заходи. Розділ спирається на модель об'єкта захисту з розділу 1, систематизує нормативне поле України та ЄС, стандарти і рамки OWASP, механізми оцінювання відповідності й завершується кейсом для державного вебпорталу та інтернет-магазину.

2.1 Структура нормативного поля

2.1.1 Рівні регулювання

Нормативне забезпечення безпеки вебресурсів доцільно подавати як ієрархію з шести рівнів, що різняться юридичною силою і деталізацією (рис. 2.1). *Закони* України, регламенти і директиви ЄС визначають об'єкти захисту, суб'єктів, обов'язки й відповідальність, але за принципом технологічної нейтральності майже не називають механізмів. *Підзаконні акти* (постанови КМУ, акти Держспецзв'язку, імплементаційні акти Єврокомісії) деталізують процедури. *Технічні нормативи* (НД ТЗІ, профілі безпеки, гармонізовані стандарти) конкретизують вимоги. *Міжнародні стандарти* (ISO/IEC, NIST) формують спільну мову управління безпекою. *Галузеві рамки*, насамперед OWASP, найдетальніше описують вебспецифічні вимоги. *Договірні вимоги* (угода з еквайрером, умови хмарного провайдера, технічне завдання) часто є каналом, яким добровільний стандарт стає обов'язковим.

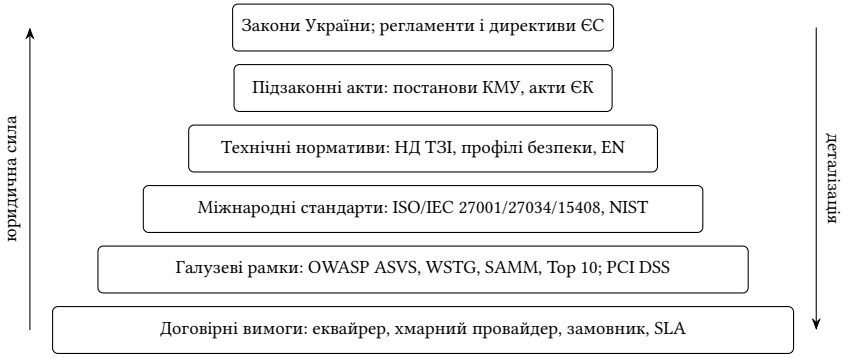


Рис. 2.1 — Рівні нормативного забезпечення безпеки вебресурсів

Норма закону про заходи, що відповідають «сучасному рівню техніки» (state of the art), тлумачиться через стандарти нижніх рівнів. Тому OWASP ASVS [226] і PCI DSS [243] фактично визначають, що вважати належними заходами в розумінні ст. 32 GDPR [249] чи ст. 21 NIS 2 [114], хоча ці акти на них не посилаються.

2.1.2 Суб'єкти зобов'язань

Виділяють п'ять категорій суб'єктів. **Власник вебресурсу** відповідає за захист інформації і персональних даних, повідомлення про інциденти й вибір постачальників. **Розробник або виробник програмного забезпечення** з Актом про кіберстійкість ЄС [253] отримує власні обов'язки щодо безпеки продукту протягом життєвого циклу. **Хостинг-, хмарний і CDN-провайдер** відповідає за свою частину моделі спільної відповідальності (розділ 12). **Еквайрери й платіжні оператори** транслюють вимоги платіжних систем на торговців через договори. **Державні органи** встановлюють вимоги, наглядають і допомагають у реагуванні. Множину вимог, застосовних до об'єкта o, можна записати як

$$\mathcal{R}(o) = \bigcup_{k=1}^{K} \big\{\, r \in \mathcal{R}_k \;:\; \varphi_k(o) = 1 \;\wedge\; \psi_r(o) = 1 \,\big\}, \tag{2.1}$$

де $\mathcal{R}_k$ — вимоги k-го режиму (закону, стандарту, договору); $\varphi_k(o) \in \{0,1\}$ — предикат застосовності режиму («обробляє державні інформаційні ресурси», «пропонує товари особам у ЄС», «приймає картки»); $\psi_r(o)$ — предикат застосовності окремої вимоги (вимога 6.4.3 PCI DSS діє, лише коли платіжна сторінка формується в браузері клієнта). Режими перекриваються (багатофакторна автентифікація адміністраторів задовольняє кілька $\mathcal{R}_k$), тому доцільний єдиний реєстр вимог (підрозділ 2.6).

2.2 Законодавство України

2.2.1 Кібербезпека та реформа 2025 року

Закон «Про основні засади забезпечення кібербезпеки України» [40] відносить до об'єктів кіберзахисту системи органів влади, державні інформаційні ресурси та об'єкти критичної інформаційної інфраструктури, отже, і портали послуг, вебінтерфейси реєстрів та офіційні сайти. Атаки на державні ресурси 2022–2024 рр. (розділ 1) і курс на інтеграцію з ЄС зумовили реформу: Законом від 27.03.2025 № 4336-IX [33] змінено закони про кібербезпеку і про захист інформації в ІКС. Запроваджено національну систему реагування на кіберінциденти й обміну інформацією про загрози, посади фахівців з кібербезпеки в органах влади й на об'єктах критичної інфраструктури, оцінювання стану кіберзахисту з механізмом аудиту; розширено контрольні повноваження Держспецзв'язку. Концептуально головне — відмова від обов'язкової комплексної системи захисту інформації (КСЗІ) на користь ризик-орієнтованого підходу: власник впроваджує заходи базових і галузевих **профілів безпеки** і підтримує їх протягом життєвого циклу системи, що гармонізує підхід із NIS 2 [114] і практиками NIST [220].

2.2.2 Захист інформації: від КСЗІ до авторизації з безпеки

Закон «Про захист інформації в інформаційно-комунікаційних системах» [37] установлює вимоги до захисту державної інформації та інформації з обмеженим доступом. До 2025 р. її обробляли в системах із КСЗІ, підтвердженою атестатом за результатами державної експертизи; порядок створення КСЗІ визначав НД ТЗІ 3.7-003-05 [27], критерії оцінювання — НД ТЗІ 2.5-004-99 [26], загальні правила — постанова КМУ № 373 [32]. Для вебресурсів із безперервним розгортанням ця модель погано пасувала: атестат фіксував стан на дату експертизи.

Постановою КМУ від 18.06.2025 № 712 [11] затверджено Порядок авторизації з безпеки інформаційних, електронних комунікаційних, інформаційно-комунікаційних і технологічних систем та Порядок розроблення і затвердження профілів безпеки. **Авторизація з безпеки** — рішення про допуск системи до експлуатації на підставі документально підтвердженого впровадження заходів профілю. За рік нової процедури авторизовано 682 системи, з них 485 — у першому півріччі 2026 р. [56]. Ризиком переходу [2] є залежність якості від зрілості управління ризиками у власника. Для вебресурсу профіль задає мінімальний набір заходів, а конкретні механізми (WAF, політика безпеки вмісту, сканування залежностей) власник обирає й обґрунтовує; рамки OWASP (підрозділ 2.5) заповнюють проміжок між профілем і реалізацією.

2.2.3 Персональні дані, довірчі послуги, е-комерція, хмарні послуги

Закон «Про захист персональних даних» [38] поширюється на облікові записи, адреси, ідентифікатори пристроїв і cookie. Порівняно з GDPR [249] у ньому немає розгорнутих принципів захисту даних за задумом, загального обов'язку повідомлення про порушення у фіксований строк і співмірних санкцій. Закон «Про електронні довірчі послуги» [34] визначає режим кваліфікованого електронного підпису, важливий для автентифікації на державних порталах і юридичної значущості поданих документів (розділ 8). Закон «Про електронну комерцію» [35] регулює укладання електронних правочинів, що вимагає цілісності й доказовості процесу. Закон «Про хмарні послуги» [42] розмежовує відповідальність постачальника і споживача, установлює вимоги до постачальників, що обробляють державні ресурси, і їхній реєстр, відкривши можливість розміщувати державні вебресурси в хмарі.

2.2.4 Державні вебресурси, критична інфраструктура та інституції

Для вебсайтів органів виконавчої влади технічні вимоги, зокрема захист від несанкціонованого доступу й модифікації, встановлює Порядок функціонування вебсайтів [36]. Закон «Про критичну інфраструктуру» [39] встановлює категоризацію об'єктів, постанова КМУ № 518 [17] — загальні вимоги до кіберзахисту (ризики, доступ, моніторинг, реагування, неперервність). Вебресурс стає частиною об'єкта критичної інформаційної інфраструктури, якщо через нього надаються життєво важливі послуги. Інституційну систему [40, 41] формують Держспецзв'язку [8] (регулятор: профілі, авторизація, контроль), CERT-UA [9] (повідомлення про інциденти, координація реагування), Національний координаційний центр кібербезпеки при РНБО, кіберполіція і СБУ.

2.2.5 Юридична відповідальність

Розділ XVI Кримінального кодексу [20] встановлює відповідальність за несанкціоноване втручання в роботу систем (ст. 361 — дефейс, компрометація сервера), створення і збут шкідливих засобів (ст. 361-1 — вебшели), збут інформації з обмеженим доступом (ст. 361-2 — продаж викрадених баз), несанкціоновані дії особи, що має доступ (ст. 362 — інсайдер), порушення правил експлуатації чи захисту відповідальною особою, що спричинило шкоду (ст. 363), і перешкоджання роботі систем масовим розповсюдженням повідомлень (ст. 363-1, зокрема DDoS). Перевірка чужого ресурсу без дозволу може кваліфікуватися за ст. 361, тож тестування потребує договору або програми координованого розкриття (розділ 15). Зведення актів — у табл. 2.1.

Таблиця 2.1 — Основні акти законодавства України та їхнє значення для вебресурсів

Акт	Значення для вебресурсу
ЗУ № 2163-VIII зі змінами ЗУ № 4336-IX	об'єкти кіберзахисту; система реагування; фахівці; аудит
ЗУ № 80/94-ВР; КМУ № 373, № 712	захист державних ресурсів; профілі; авторизація замість атестата КСЗІ
ЗУ «Про критичну інфраструктуру»; КМУ № 518	ресурси, що надають життєво важливі послуги
ЗУ «Про захист персональних даних»	облікові записи, cookie, журнали
ЗУ «Про електронні довірчі послуги»	КЕП і електронна ідентифікація
ЗУ «Про електронну комерцію»	доказовість електронних договорів
ЗУ «Про хмарні послуги»	державні ресурси в хмарі; реєстр
Наказ № 327/225	вебсайти органів виконавчої влади
ККУ, ст. 361–363-1	кримінальна відповідальність

2.3 Право Європейського Союзу

2.3.1 Директива NIS 2

Директива (ЄС) 2022/2555 [114] (транспонування до 17.10.2024) поширюється на суттєві й важливі суб'єкти. Для вебсфери ключові сектор цифрової інфраструктури (DNS, реєстри TLD, хмарні обчислення, центри обробки даних, CDN, довірчі послуги) і цифрові постачальники (онлайнові майданчики, пошукові системи, соціальні мережі). Правило розміру (середні й великі підприємства) не діє для DNS, реєстрів TLD і кваліфікованих надавачів довірчих послуг. Регламент 2024/2690 [101] деталізує для них технічні вимоги. Ст. 21 вимагає «відповідних і пропорційних» заходів, зокрема безпеки ланцюга постачання, розроблення, оброблення вразливостей, криптографії і багатофакторної автентифікації; ст. 20 встановлює відповідальність органів управління, ст. 23 — багатоетапне повідомлення про інциденти. Інтернет-магазин власних товарів не належить до секторів Директиви, а майданчик для сторонніх продавців — належить: предикат φ_k у (2.1) визначає бізнес-модель, а не технологія.

2.3.2 Акт про кіберстійкість

Регламент (ЄС) 2024/2847 (CRA) [253] запровадив горизонтальні вимоги до **продуктів із цифровими елементами**, включно з *рішеннями віддаленої обробки даних*, без яких продукт не виконує функцій. Він охоплює тиражоване вебпрограмне забезпечення, що постачається в межах комерційної

діяльності (системи керування вмістом, платформи е-комерції, браузери, менеджери паролів, WAF, бібліотеки), і невід'ємний від продукту хмарний бекенд (наприклад, сервер мобільного застосунку). «Чистий» SaaS регулюється NIS 2; некомерційне вільне ПЗ виведено з-під дії, для стюардів відкритого коду діє полегшений режим [89].

Суттєві вимоги додатка I стосуються властивостей продукту (відсутність відомих експлуатованих вразливостей, безпечна конфігурація за замовчуванням, керування доступом, мінімізація поверхні атаки, журналювання, безпечне оновлення) та оброблення вразливостей (SBOM, розділ 13; безкоштовні оновлення безпеки; тестування; координоване розкриття) протягом періоду підтримки, як правило не менше п'яти років. Важливі продукти класів I і II та критичні оцінюються суворіше; їх описує регламент 2025/2392 [103]: до класу I належать браузери, менеджери паролів, системи керування ідентичністю, до класу II — міжмережеві екрани і системи виявлення вторгнень. CRA чинний з 10.12.2024; положення про органи оцінювання діють з 11.06.2026; повідомлення про активно експлуатовані вразливості й серйозні інциденти — з 11.09.2026 (раннє попередження за 24 год, повідомлення за 72 год, остаточний звіт до 14 днів після виправлення); решта вимог і маркування CE — з 11.12.2027 (рис. 2.2).

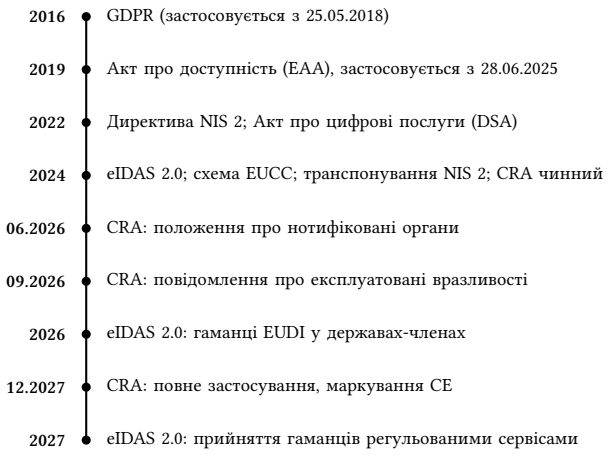


Рис. 2.2 — Хронологія актів ЄС, що визначають вимоги до безпеки вебресурсів

2.3.3 Загальний регламент захисту даних

За ст. 3(2) GDPR [249] застосовується до контролера поза ЄС, що пропонує товари чи послуги особам в ЄС або відстежує їхню поведінку (а відстеження через cookie — типова функція вебресурсу); такий контролер призначає

представника в ЄС (ст. 27). Для безпеки визначальні ст. 25 (**захист даних за задумом і за замовчуванням**: мінімізація, псевдонімізація, приватні налаштування), ст. 32 (заходи з урахуванням сучасного рівня техніки: шифрування, конфіденційність, цілісність, доступність, стійкість, відновлення, регулярне тестування) і ст. 33 (повідомлення наглядового органу протягом 72 год), доповнена ст. 34 щодо повідомлення суб'єктів. Вимірювання банерів згоди виявили масову реєстрацію згоди до її надання [198]: відповідність слід перевіряти технічно, а не за документацією.

2.3.4 eIDAS 2.0, DSA та EAA

Регламент 2024/1183 (eIDAS 2.0) [251] запроваджує гаманець цифрової ідентичності (EUDI Wallet), який регульовані й великі онлайнові сервіси зі строгою автентифікацією мають приймати, — новий механізм автентифікації з вибірковим розкриттям атрибутів (розділ 8); ст. 45 вимагає від браузерів визнавати кваліфіковані сертифікати вебсайтів (QWAC) (розділ 5). Акт про цифрові послуги (DSA) [250] з 17.02.2024 регулює посередницькі послуги і платформи: повідомлення про незаконний вміст, прозорість, простежуваність продавців. Акт про доступність (EAA) [113] з 28.06.2025 охоплює, зокрема, е-комерцію; він не безпековий, але вимагає доступності механізмів захисту (CAPTCHA, багатофакторна автентифікація, тайм-аути сеансу) для людей з інвалідністю (розділ 11).

2.3.5 Санкції та строки повідомлення

Акти ЄС задають граничний штраф як більшу з двох величин — фіксованої суми і частки світового річного обороту T:

$$F_{\max} = \max\bigl(F_{\text{фікс}},\ \alpha\, T\bigr), \tag{2.2}$$

де для GDPR (ст. 83(5)) $F_{\text{фікс}} = 20$ млн євро, $\alpha = 4\,\%$; для суттєвих суб'єктів NIS 2 — 10 млн євро і 2 %, для важливих — 7 млн євро і 1,4 %; для CRA (суттєві вимоги) — 15 млн євро і 2,5 %. Поріг $T^{*} = F_{\text{фікс}}/\alpha$, нижче від якого визначальна фіксована сума, становить 500 млн євро для GDPR і NIS 2 та 600 млн євро для CRA. Для $T = 2$ млрд євро граничний штраф за GDPR — $\max(20;\ 80) = 80$ млн євро, за CRA — $\max(15;\ 50) = 50$ млн євро; для $T = 40$ млн євро граничні суми (20 і 15 млн) перевищують половину обороту, тобто для малого бізнесу санкції непропорційно суворі.

Один інцидент (компрометація через вразливість власного продукту з витоком персональних даних) може запустити кілька паралельних процедур. Якщо t_0 — момент обізнаності, $d_{k,j}$ — строк j-го повідомлення в режимі k, то

$$t_{k,j} - t_0 \le d_{k,j} \quad \forall k, j; \qquad d^{*} = \min_{k,j} d_{k,j}, \tag{2.3}$$

де для NIS 2 і CRA перше повідомлення — 24 год, друге — 72 год, для GDPR — 72 год. Строк $d^* = 24$ год визначає вимогу до виявлення і класифікації (розділ 16): якщо на узгодження рішення відводиться 6 год, середній час виявлення й класифікації має бути меншим за 18 год із запасом на варіабельність. Порівняння актів — у табл. 2.2.

Таблиця 2.2 — Акти ЄС, що впливають на безпеку вебресурсів

Акт	Адресат	Ключові вимоги	Санкції
NIS 2	суттєві й важливі суб'єкти; цифрова інфраструктура, майданчики	ст. 21 заходи; ст. 23 повідомлення 24/72 год	10 млн/2 %; 7 млн/1,4 %
CRA	виробники продуктів із цифровими елементами	додаток I; SBOM; повідомлення з 11.09.2026	15 млн/2,5 %
GDPR	контролери й обробники, зокрема поза ЄС	ст. 25, 32; ст. 33 — 72 год	20 млн/4 %
eIDAS 2.0	держави-члени, довірчі сторони	гаманець EUDI; QWAC	національні
DSA	посередницькі послуги, платформи	повідомлення й дії; простежуваність	до 6 %
EAA	постачальники продуктів і послуг	доступність, у т.ч. механізмів захисту	національні

2.4 Міжнародні стандарти

2.4.1 ISO/IEC 27001 і 27002

ISO/IEC 27001:2022 і 27002:2022 (ДСТУ EN ISO/IEC 27001:2023 [15] і 27002:2023 [16]) задають вимоги до системи управління інформаційною безпекою (СУІБ) і 93 заходи за темами: організаційні (37), людські (8), фізичні (14), технологічні (34) [21]. Для вебресурсів ключові заходи безпечного розроблення: 8.25 «Безпечний життєвий цикл розроблення», 8.26 «Вимоги до безпеки застосунків», 8.27 «Архітектура захищених систем і принципи інженерії», 8.28 «Безпечне кодування» (новий), 8.29 «Тестування безпеки під час розроблення та приймання»; суміжний новий захід — 8.23 «Вебфільтрація» (контроль доступу персоналу до зовнішніх сайтів). ISO/IEC 27001 сертифікує *систему управління*, а не захищеність застосунку: сертифікована організація може експлуатувати вразливий ресурс, якщо ризик формально прийнято.

2.4.2 ISO/IEC 27034 і ISO/IEC 15408

ISO/IEC 27034 [163] переносить логіку СУІБ на рівень застосунку: **організаційна нормативна база** (ONF) містить політики і заходи; **заходи безпе-**

ки застосунку (ASC) мають опис цілі, реалізації й верифікації; **цільовий рівень довіри** задає набір ASC для конкретного застосунку. Рівні довіри близькі до L1–L3 ASVS, тому ASVS часто використовують як готовий каталог ASC. Загальні критерії (ISO/IEC 15408) [161] оцінюють продукти на відповідність профілю захисту з рівнями гарантій EAL1–EAL7; для вебзастосунків це дорого, але релевантно для WAF і, через схему EUCC [102], для продуктів, критичних за CRA.

2.4.3 PCI DSS 4.0.1

PCI DSS [243] — договірна вимога для всіх, хто зберігає, обробляє чи передає дані карток. З 31.03.2025 обов'язкові вимоги, що прямо стосуються вебресурсів. Вимога 6.4.1 зобов'язує захищати загальнодоступні вебзастосунки від відомих атак, 6.4.2 — застосовувати автоматизоване рішення, що виявляє і блокує вебатаки (фактично WAF, розділ 11). Вимога 6.4.3: кожен скрипт платіжної сторінки, що виконується в браузері, має бути авторизований, мати гарантовану цілісність і обґрунтування в переліку. Вимога 11.6.1: механізм виявлення несанкціонованих змін заголовків HTTP і вмісту платіжних сторінок у вигляді, отриманому браузером, щонайменше щотижня або з періодичністю за цільовим аналізом ризиків. Це відповідь на скимери Magecart; засоби реалізації розглянуто в розділі 6. У 2025 р. PCI SSC вилучила 6.4.3 і 11.6.1 з анкети SAQ A (вбудована форма чи перенаправлення), але додала критерій прийнятності: торговець підтверджує, що його сайт не вразливий до атак скриптами [158]. Відповідальність не зникла, а перейшла в декларацію.

2.4.4 NIST CSF 2.0 і SSDF

NIST CSF 2.0 [221] організує результати за шістьма функціями — «Керувати» (нова), «Ідентифікувати», «Захищати», «Виявляти», «Реагувати», «Відновлювати» — у 22 категоріях і 106 підкатегоріях; поточний і цільовий профілі візуалізують розрив. Інструмент багатостандартної оцінки кіберзрілості з CSF 2.0 як базовою шкалою і відображенням на ISO/IEC 27001:2022, COBIT 2019 і CIS Controls v8 запропоновано в [51]. SSDF (NIST SP 800-218) [275] групує практики безпечного розроблення в чотири групи: підготовка організації (PO), захист ПЗ (PS), створення захищеного ПЗ (PW), реагування на вразливості (RV), — і значною мірою перекривається з вимогами CRA щодо оброблення вразливостей.

2.4.5 Відповідність між стандартами

Практичним інструментом є таблиця відповідності (crosswalk). Якщо $M \subseteq \mathcal{R}_A \times \mathcal{R}_B$ — відношення відповідності вимог документів A і B, то покриття

$$\operatorname{cov}(A \to B) = \frac{\left|\{\, b \in \mathcal{R}_B : \exists\, a \in \mathcal{R}_A,\ (a,b) \in M \,\}\right|}{|\mathcal{R}_B|}. \tag{2.4}$$

Покриття (2.4) несиметричне і завищує еквівалентність, бо M здебільшого фіксує тематичну, а не змістову відповідність: захід 8.28 «відповідає» десяткам вимог ASVS, але загальна політика кодування не гарантує виконання жодної з них. Тому слід розрізняти повну, часткову і тематичну відповідність (табл. 2.3).

Таблиця 2.3 — Відповідність вимог стандартів і рамок для вебресурсів

Тема	ISO/IEC 27002	PCI DSS	NIST	ASVS 5.0
Безпечний ЖЦ	8.25, 8.27	6.1, 6.2	SSDF PO, PW	V15
Вимоги до застосунку	8.26	6.2.4	SSDF PW.1	усі розділи
Безпечне кодування	8.28	6.2.4	SSDF PW.5	V1, V2, V5
Тестування	8.29	6.2.3, 11.3, 11.4	SSDF PW.7, PW.8	(метод — WSTG)
Вебатаки ззовні	8.20, 8.26	6.4.1, 6.4.2	CSF PR.PS, DE.CM	V4, V13
Скрипти в браузері	8.28	6.4.3, 11.6.1	CSF PR.DS	V3
Автентифікація	5.17, 8.5	8.3, 8.4	SP 800-63	V6, V7, V10
Журналювання	8.15, 8.16	10.2	CSF DE.CM	V16
Вразливості	8.8	6.3	SSDF RV	V15

2.5 Галузеві рамки OWASP

2.5.1 Стандарт верифікації ASVS 5.0

OWASP ASVS 5.0.0 [226] (травень 2025 р.) — найдетальніший відкритий каталог вимог до вебзастосунків: близько 350 вимог у 17 розділах: V1 — кодування і санітизація; V2 — валідація і бізнес-логіка; V3 — безпека клієнтської частини; V4 — API і вебсервіси; V5 — обробка файлів; V6 — автентифікація; V7 — сеанси; V8 — авторизація; V9 — самодостатні токени; V10 — OAuth і OIDC; V11 — криптографія; V12 — захищені комунікації; V13 — конфігурація; V14 — захист даних; V15 — безпечне кодування й архітектура; V16 — журналювання та обробка помилок; V17 — WebRTC. Вимоги сформульовано як перевірювані твердження про властивості застосунку. Рівні кумулятивні: L1 — мінімальний для всіх застосунків; L2 — для застосунків із чутливими даними чи транзакціями; L3 — для найкритичніших (крити-

чна інфраструктура, медицина, великі фінансові операції). ASVS дає змогу коректно сформулювати технічне завдання («ASVS 5.0 рівня L2, крім V17»).

2.5.2 WSTG, SAMM, Top 10 і API Security Top 10

WSTG [232] описує методику тестування за категоріями від збирання інформації до бізнес-логіки, клієнтської частини та API: ASVS відповідає на питання «*що* має бути істинним», WSTG — «*як* це перевірити» (розділ 15). SAMM [229] оцінює організацію: п'ять бізнес-функцій (управління, проєктування, реалізація, верифікація, операції) по три практики і три рівні зрілості (розділ 17). OWASP Top 10:2025 [231] — документ обізнаності, а не стандарт: категорії навмисно широкі, тож «відповідність Top 10» у технічному завданні не має перевірюваного змісту. API Security Top 10:2023 [235] виконує ту саму роль для API (розділ 10).

Співвідношення документів подано на рис. 2.3: на рівні організації ISO/IEC 27001 і CSF 2.0 задають цілі та управління ризиками, SAMM і SSDF — процеси розроблення; на рівні застосунку ISO/IEC 27034 задає рівень довіри, ASVS — вимоги, WSTG — методику верифікації; Top 10 слугує для пріоритизації й навчання. Нормативні акти і PCI DSS посилаються на результати, а не на документи OWASP, проте настанови до вимоги 6.2.4 PCI DSS серед прикладів галузевих джерел наводять і OWASP.

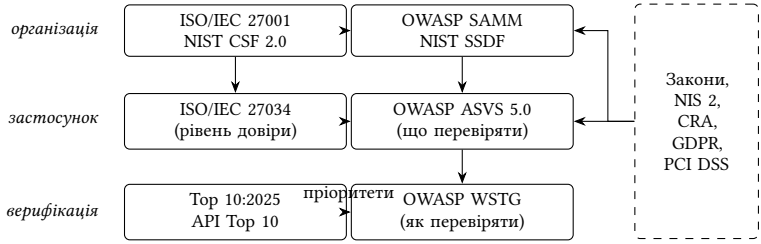


Рис. 2.3 — Співвідношення стандартів управління і рамок OWASP

2.6 Оцінювання відповідності та сертифікація

2.6.1 Національна система та міжнародна сертифікація

Для державних систем основним механізмом стала авторизація з безпеки [11]: власник категоризує систему, обирає базовий і галузевий профілі, впроваджує й оцінює заходи та подає документи до Держспецзв'язку, яка ухвалює рішення протягом 10 днів [56]; доповнює її аудит стану кіберзахисту [33]. Для вебресурсу з частими релізами це вимагає безперервного моніторингу й автоматизованих перевірок у конвеєрі CI/CD.

Сертифікацію СУІБ за ISO/IEC 27001 проводять за трирічним циклом із щорічними наглядовими аудитами; перевіряючи постачальника, слід з'ясувати, чи входить потрібний вебсервіс до області сертифікації. Стандарти аудиту ISO 19011 і ISO/IEC 27007 порівняно з погляду автоматизації в [50]. Оцінювання за PCI DSS залежить від рівня торговця: найбільші подають звіт про відповідність (ROC) кваліфікованого оцінювача (QSA), інші — анкету самооцінювання: SAQ A (вбудована форма чи перенаправлення), SAQ A-EP (сторінка торговця впливає на формування платіжної форми), SAQ D (інші випадки); зовнішні ресурси щокварталу сканує затверджений постачальник (ASV). Для продуктів схема EUCC [102] може підтверджувати відповідність CRA; для класу I можливе самооцінювання за гармонізованими стандартами, для класу II обов'язкове оцінювання третьою стороною [253].

2.6.2 Кількісне оцінювання рівня відповідності

Нехай $\mathcal{R} = \{r_1, \dots, r_N\}$ — застосовні вимоги (2.1), $w_i > 0$ — вага (3 — обов'язкова вимога закону чи профілю, 2 — договору чи стандарту, 1 — рекомендація), $c_i \in \{0;\ 0{,}5;\ 1\}$ — ступінь виконання. **Зважений індекс розриву відповідності**:

$$G = 1 - \frac{\sum_{i=1}^{N} w_i\, c_i}{\sum_{i=1}^{N} w_i}, \qquad G \in [0; 1]. \tag{2.5}$$

Індекс (2.5) компенсаторний: виконання багатьох другорядних вимог маскує невиконання критичної. Тому для обов'язкових вимог $\mathcal{R}_{\text{обов}}$ вводять блокувальну умову, а пріоритет решти визначають за питомим зменшенням індексу на одиницю витрат C_i:

$$\min_{i:\, r_i \in \mathcal{R}_{\text{обов}}} c_i = 1, \tag{2.6}$$

$$\pi_i = \frac{\Delta G_i}{C_i} = \frac{w_i\,(1 - c_i)}{C_i \sum_{j=1}^{N} w_j}. \tag{2.7}$$

Доки (2.6) не виконано, обов'язкові вимоги мають абсолютний пріоритет незалежно від π_i.

Числовий приклад. Реєстр містить 50 вимог: 15 обов'язкових ($w = 3$), 25 стандартних ($w = 2$), 10 рекомендацій ($w = 1$); $\sum w_i = 105$. Обов'язкові: 13 — повністю, 2 — частково; стандартні: 18 — повністю, 5 — частково, 2 — не виконано; рекомендації: 4 — повністю, 2 — частково, 4 — не виконано. Зважена сума $3(13 + 1) + 2(18 + 2{,}5) + 1(4 + 1) = 88$, звідки $G = 1 - 88/105 \approx 0{,}162$; умова (2.6) не виконана. Доведення двох обов'язкових вимог зменшує G на $3/105 \approx 0{,}029$, до $0{,}133$. Далі порівняймо дві невиконані стандартні вимоги: моніторинг платіжних сторінок (11.6.1, $C = 40$ людино-год) і усунення

вади заголовків ($C = 4$ людино-год). За однакового $\Delta G = 2/105$ значення π відрізняються вдесятеро ($4{,}8 \cdot 10^{-4}$ проти $4{,}8 \cdot 10^{-3}$), тож друга виконується першою. Модель (2.7) не враховує залежностей між вимогами і їхньої ризикової цінності (11.6.1 знижує ризик реального скимінгу), тому її доповнюють аналізом ризиків (розділ 4).

2.7 Порівняльний аналіз вимог України та ЄС

Реформа 2025 р. наблизила національну модель до європейської: обидві ґрунтуються на ризик-орієнтованому підході, відповідальності суб'єкта і безперервному підтриманні заходів (табл. 2.4).

Таблиця 2.4 — Порівняння підходів України та ЄС до регулювання безпеки вебресурсів

Аспект	Україна	ЄС
Рамкові акти	ЗУ № 2163-VIII, № 80/94-ВР зі змінами ЗУ № 4336-IX	NIS 2, CRA, GDPR
Сфера дії	державні ресурси, КІ, об'єкти кіберзахисту	суттєві й важливі суб'єкти, у т.ч. приватні цифрові
Підтвердження захищеності	авторизація з безпеки за профілями; раніше атестат КСЗІ	самооцінювання, аудит, сертифікація; нагляд
Регулювання продуктів	немає горизонтального акта	CRA (з 11.12.2027), EUCC
Повідомлення про інциденти	CERT-UA; національна система реагування	24/72 год, 1 міс. (NIS 2); 72 год (GDPR)
Персональні дані	ЗУ № 2297-VI; без строкового повідомлення	GDPR: ст. 25, 32–34; до 4 % обороту
Відповідальність керівництва	посадові особи, ст. 363 ККУ	ст. 20 NIS 2: персональна
Електронна ідентичність	КЕП, Дія.Підпис	eIDAS 2.0, гаманець EUDI

Напрями гармонізації: поширення вимог на приватних цифрових, хмарних, DNS- і CDN-провайдерів за NIS 2; вимоги до безпеки програмних продуктів за зразком CRA, які для українських розробників, що працюють на ринок ЄС, однаково стають фактичним стандартом; строкове повідомлення про порушення безпеки персональних даних; узгодження профілів безпеки з ASVS і ISO/IEC 27002, щоб одна міра закривала кілька режимів.

2.8 Кейс: визначення застосовних вимог

2.8.1 Державний вебпортал електронних послуг

Портал приймає заяви громадян: автентифікація через КЕП, доступ до кількох реєстрів через API, розміщення в хмарі постачальника з реєстру. За моделлю (2.1) застосовні закони про кібербезпеку й захист інформації [40, 37] зі змінами [33] (авторизація з безпеки за базовим і галузевим профіля-

ми [11], посадові особи з кібербезпеки, взаємодія з CERT-UA); у разі категоризації як об'єкта критичної інформаційної інфраструктури — постанова № 518 [17]; закони про персональні дані [38], довірчі [34] і хмарні послуги [42]; вимоги до сайтів органів влади [36]. Технічне наповнення профілю доцільно задати через ASVS: L2 для всіх розділів і L3 для V6–V10 (автентифікація, сеанси, авторизація, токени, федерація), бо вади доступу до реєстрових даних зачіпають мільйони громадян; інтеграції з реєстрами оцінюються за API Security Top 10, процеси розробника — за SSDF і SAMM.

2.8.2 Український інтернет-магазин із продажем у ЄС

Магазин (оборот 12 млн євро, 80 працівників) продає власні товари в Україні та ЄС, приймає картки й має мобільний застосунок. Застосовні закони про персональні дані й е-комерцію [38, 35]; GDPR через ст. 3(2) [249] (представник у ЄС, ст. 25, 32, процедура ст. 33); PCI DSS через договір з еквайрером [243]; EAA [113] як суміжна вимога. NIS 2 і DSA не застосовні, доки магазин не стане майданчиком для сторонніх продавців. Сайт не є продуктом за CRA, але мобільний застосунок, що розповсюджується в ЄС у межах комерційної діяльності, разом із бекендом може кваліфікуватися як продукт із цифровими елементами, тож до 11.12.2027 магазин має підготуватися до обов'язків виробника [253]. Ключове рішення — спосіб прийому карток: власна форма означає SAQ A-EP або D з вимогами 6.4.3 і 11.6.1; вбудована форма провайдера — SAQ A з обґрунтованим підтвердженням стійкості до атак скриптами [158], що на практиці потребує тих самих механізмів (сувора політика безпеки вмісту, моніторинг сторінки). Оскільки $T = 12$ млн євро $< T^*$, граничний штраф за GDPR (2.2) — 20 млн євро, більше за річний оборот.

Для обох ресурсів далі формують єдиний реєстр вимог, оцінюють стан за (2.5) з перевіркою (2.6) і планують усунення розривів за (2.7). Технології обох ресурсів майже однакові, а вимоги різні: їх визначають статус власника, ринок і бізнес-модель.

Висновки до розділу

Нормативне забезпечення безпеки вебресурсів багаторівневе: закони встановлюють обов'язки й санкції, підзаконні акти і профілі — процедури, міжнародні стандарти — мову управління, а рамки OWASP і PCI DSS — найдетальніші перевірювані вимоги, які через договори і тлумачення «сучасного рівня техніки» фактично стають обов'язковими. В Україні Закон № 4336-IX і постанова КМУ № 712 замінили модель атестата КСЗІ ризик-орієнтованою авторизацією з безпеки за профілями, наблизивши національну систему до NIS 2, але без регулювання приватного цифрового сектору і продуктів. У ЄС NIS 2, GDPR і CRA (повідомлення з 11.09.2026, повне застосування з

11.12.2027) створюють для українських розробників і ресурсів, що працюють на ринок ЄС, вимоги зі строками повідомлення 24/72 год і штрафами (2.2), що для малого бізнесу перевищують річний оборот. Для практики корисні модель застосовності (2.1), блокувальні вимоги плюс індекс (2.5) і пріоритизація (2.7).

Питання для самоперевірки

1. Назвіть шість рівнів нормативного забезпечення (рис. 2.1) і поясніть, як галузеві рамки набувають обов'язковості.
2. Які п'ять категорій суб'єктів зобов'язань виділяють для вебресурсів? Наведіть приклад організації, що належить до трьох категорій.
3. Поясніть зміст предикатів φ_k і ψ_r у моделі (2.1) на прикладі PCI DSS.
4. Які зміни запровадив Закон № 4336-IX? Чим авторизація з безпеки відрізняється від атестата КСЗІ?
5. Які статті Кримінального кодексу застосовні до дефейсу, DDoS-атаки і продажу викраденої бази? Яке значення має ст. 363 для власника ресурсу?
6. За яких умов вебресурс підпадає під NIS 2? Чому інтернет-магазин власних товарів не є її суб'єктом?
7. Яке вебпрограмне забезпечення охоплює CRA і чим він відрізняється за сферою дії від NIS 2? Назвіть ключові дати.
8. Обчисліть за (2.2) граничні штрафи за GDPR і CRA для оборотів 100 млн і 1 млрд євро. Що означає поріг T^*?
9. За (2.3) визначте допустимий середній час виявлення інциденту, якщо на узгодження рішення відводиться 8 год, а коефіцієнт запасу дорівнює 1,5.
10. Які заходи ISO/IEC 27002:2022 стосуються безпечного розроблення? Чому сертифікат ISO/IEC 27001 не гарантує захищеності вебзастосунку?
11. Сформулюйте вимоги 6.4.3 і 11.6.1 PCI DSS і поясніть, як зміна SAQ A 2025 р. вплинула на відповідальність торговця.
12. Поясніть за рис. 2.3 і табл. 2.3 співвідношення ASVS, WSTG, SAMM і Top 10. Чому «відповідність Top 10» — некоректна вимога?
13. Обчисліть (2.5) для прикладу з підрозділу 2.6 після виконання всіх обов'язкових і двох невиконаних стандартних вимог. Чи виконується (2.6)?
14. Дослідницьке: запропонуйте некомпенсаторну модель агрегування відповідності, що враховує залежності між вимогами і їхню ризикову цінність, та порівняйте її з (2.5) і (2.7).

15. Дослідницьке: розробіть методику автоматизованого відображення вимог базового профілю безпеки на ASVS 5.0 і ISO/IEC 27002 з оцінкою покриття (2.4) і розрізненням повної, часткової та тематичної відповідності.

РОЗДІЛ 3

Математичні та кількісні основи оцінювання захищеності вебресурсів

Розділ 1 увів модель порушника (1.3), оцінку кількості спроб (1.5), модель гонки між експлуатацією і виправленням (1.6) та композицію ланцюгів атак (1.8), а нормативні вимоги (розділ 2) здебільшого сформульовано якісно: «достатня стійкість», «обмеження частоти», «своєчасне усунення». Розділ систематизує апарат, що перетворює такі вимоги на числа: теорію вгадування, моделі підбору облікових даних, теорію статистичного виявлення, теорію масового обслуговування, марковські й байєсові моделі атак та оцінки CVSS, EPSS, KEV і SSVC, і завершується розгорнутим числовим прикладом. Самі механізми захисту розглянуто в розділах 8, 9, 11 і 16.

3.1 Постановка задачі кількісного оцінювання захищеності

3.1.1 Вебресурс як об'єкт кількісного оцінювання

Розглянемо вебресурс як систему $\mathcal{W} = \langle \mathcal{E}, \mathcal{S}, \mathcal{C}, \mathcal{M}, \mathcal{V} \rangle$, де $\mathcal{E}$ — вхідні точки (сторінки, кінцеві точки API, форми входу); $\mathcal{S}$ — секрети й ідентифікатори (паролі, токени сеансу, ключі API, ідентифікатори об'єктів); $\mathcal{C}$ — обчислювальні ресурси (робочі процеси, з'єднання, пам'ять); $\mathcal{M}$ — механізми захисту (гешування паролів, обмеження частоти, MFA, WAF, журналювання); $\mathcal{V}$ — відомі вразливості компонентів. З моделі порушника (1.3) для кількісних оцінок найважливіші ресурси: кількість адрес ботнету, швидкість запитів, обчислювальна потужність і бюджет. Захищеність характеризується вектором показників

$$\mathbf{q} = \big(H_\infty,\ P_{\text{онл}}(T),\ C_{\text{офл}},\ \text{PPV},\ \lambda_a^*,\ P_{\text{ціль}},\ \text{Cov}\big),$$

кожна компонента якого відповідає окремій властивості й моделі (табл. 3.1). Задача *оцінювання* — обчислити $\mathbf{q}$ для заданих $\mathcal{W}$ і порушника, задача *синтезу* — обрати параметри (довжину токена, вартість гешування, ліміт спроб, поріг тривоги, порядок виправлень), що забезпечують цільові значення. Майже всі показники визначаються *економікою порушника*: вартістю запиту, спроби вгадування чи години роботи графічного процесора.

Таблиця 3.1 — Показники кількісного оцінювання захищеності вебресурсу

Властивість	Показник	Модель і вихідні дані	Підр.
Стійкість секретів	H_∞, $\tilde{\lambda}_\beta$, імовірність колізії	Розподіл секретів, кількість ідентифікаторів	3.2
Автентифікація (онлайн)	$P_{\text{онл}}(T)$, кількість зламаних записів	Крива вгадування, ліміт спроб, повторне використання паролів	3.3
Автентифікація (офлайн)	$C_{\text{офл}}$, GPU-год, долари	Функція гешування, параметри, обладнання	3.3
Виявлення	P_D, P_{FA}, PPV	Розподіли ознак, базова частота	3.4
Доступність	ρ, W, λ_a^*	Інтенсивності, час обслуговування, кількість процесів	3.5
Ланцюги атак	$P_{\text{ціль}}$, кількість кроків	Атаковий граф, імовірності переходів	3.6
Вразливості	Покриття, ефективність	CVSS, EPSS, KEV, бюджет	3.7

3.1.2 Вимоги до моделей

Модель, придатна для обґрунтування рішень, має бути *явною* щодо припущень (незалежність спроб, пуассонівський потік, стаціонарність розподілу паролів), *спостережуваною* (параметри оцінюються з журналів або відкритих даних), *консервативною* (за невизначеності обирають значення, вигідні порушникові), супроводжуватися *аналізом чутливості* і давати результати в *порівнюваних* одиницях (імовірність, години, долари). Такі моделі є джерелом оцінок імовірності для управління ризиками (розділ 4).

3.2 Ентропія та стійкість секретів

3.2.1 Ентропія Шеннона, мін-ентропія і ентропія вгадування

Нехай секрет X набуває значень $x_1, \dots, x_N$ з імовірностями $p_1 \geq p_2 \geq \cdots \geq p_N$. Три класичні міри невизначеності:

$$H(X) = -\sum_{i=1}^{N} p_i \log_2 p_i, \qquad H_\infty(X) = -\log_2 p_1, \qquad G(X) = \sum_{i=1}^{N} i\, p_i, \tag{3.1}$$

де H — ентропія Шеннона; H_∞ — мін-ентропія, що визначає успіх першої найкращої спроби; G — ентропія вгадування, тобто середня кількість спроб порушника, який перебирає значення в порядку спадання ймовірності. Для рівномірного розподілу $H = H_\infty = \log_2 N$, $G = (N+1)/2$; для секретів, обраних людиною, ці міри розходяться на порядки. Мессі [197] показав, що для $H(X) \geq 2$ біт

$$G(X) \;\geq\; 2^{H(X)-2} + 1, \tag{3.2}$$

але оцінки зверху не існує: розподіл з довгим «хвостом» може мати високу H і дуже малу H_∞. Тому ентропія Шеннона *не є мірою стійкості* паролів проти вгадування.

Боно [81] запропонував часткові міри: *β-успішність* λ_β — частку записів, що зламуються за β найкращих спроб на кожен, і *α-роботу* — середню кількість спроб для зламу частки α записів. У бітах

$$\lambda_\beta = \sum_{i=1}^{\beta} p_i, \qquad \tilde{\lambda}_\beta = \log_2 \frac{\beta}{\lambda_\beta}, \tag{3.3}$$

де $\tilde{\lambda}_\beta$ — довжина *випадкового* секрету, еквівалентного розподілу для порушника з β спробами; $\tilde{\lambda}_1 = H_\infty$. Метрика λ_β описує онлайнову атаку з лімітом спроб, α-робота — офлайнову.

3.2.2 Емпіричні дані про паролі та закон Ципфа

Боно проаналізував анонімізовані гістограми паролів $69\,301\,337$ користувачів Yahoo! і отримав $H_\infty = 6{,}5$ біт (найпоширеніший пароль обрали 1,08 % користувачів), $\tilde{\lambda}_{10} = 9{,}1$ біт, тобто $\lambda_{10} \approx 1{,}8\,\%$, а α-робота для $\alpha = 0{,}25$ і $0{,}5$ становила 17,6 і 21,6 біт [81]. Отже, проти онлайнової атаки паролі еквівалентні випадковим секретам менше 10 біт, проти офлайнової — близько 20. Ван і співавтори на кількох великих корпусах витоків показали, що розподіл паролів добре описується законом Ципфа [298]:

$$p_i = \frac{i^{-s}}{\sum_{j=1}^{N} j^{-s}}, \qquad 0 < s < 1, \tag{3.4}$$

де i — ранг пароля за частотою, s — показник, N — кількість різних паролів. Для ілюстрації візьмемо $s = 0{,}8$, $N = 10^7$. Розрахунок дає $H \approx 20{,}5$ біт, $H_\infty \approx 6{,}9$ біт, $G \approx 1{,}7 \cdot 10^6$ (узгоджено з (3.2): $2^{18{,}5} + 1 \approx 3{,}6 \cdot 10^5$), а $\lambda_{10} = 2{,}9\,\%$, $\lambda_{100} = 6{,}7\,\%$, $\lambda_{1000} = 12{,}8\,\%$, $\lambda_{10^6} = 61{,}7\,\%$, що близько до емпіричних даних. Правила складу пароля майже не змінюють верхівку розподілу, тоді як *заборона найпоширеніших паролів* безпосередньо зменшує λ_β; тому NIST SP 800-63B-4 вимагає перевірки за списками скомпрометованих і поширених паролів замість правил складу [284], а українські дослідники пов'язують рівні вимог до ентропії (від 40–64 біт для відкритих даних до 112–128 біт для критичної інфраструктури) з використанням менеджерів паролів [52].

3.2.3 Токени сеансу та парадокс днів народження

Серверні секрети можуть бути рівномірними. OWASP вимагає для ідентифікатора сеансу не менше 64 біт ентропії від криптографічно стійкого генератора [227]. Нехай порушник надсилає r спроб за секунду, активні S се-

ансів, ідентифікатор має b випадкових біт. Імовірність угадати хоча б один чинний ідентифікатор за час T і очікуваний час до успіху

$$P_{\text{сеанс}}(T) = 1 - \Big(1 - \frac{S}{2^b}\Big)^{rT} \approx 1 - \exp\Big(-\frac{rTS}{2^b}\Big), \qquad \bar{T} = \frac{2^b}{rS}. \tag{3.5}$$

Для $b = 64$, $S = 10^5$, $r = 10^4$ с$^{-1}$ маємо $\bar{T} \approx 585$ років і ймовірність успіху за рік $\approx$ 0,17 %; для $b = 32$ — секунди, для $b = 128$ — близько 10^{-22} за рік. Тому 64 біти — мінімум із запасом на зростання S і r; від викрадення токена через XSS чи журнали ентропія не захищає (розділи 6, 8).

Друга вимога — *унікальність*. Для n випадкових b-бітових ідентифікаторів імовірність колізії (парадокс днів народження)

$$P_{\text{кол}}(n) \approx 1 - \exp\Big(-\frac{n^2}{2^{\,b+1}}\Big), \qquad n_{0,5} \approx 1{,}18 \cdot 2^{b/2}, \tag{3.6}$$

де $n_{0,5}$ — кількість, за якої колізія рівноймовірна. Для 32-бітових ідентифікаторів за $n = 10^5$ маємо $P_{\text{кол}} \approx 0{,}69$, для 64-бітових за $n = 10^9$ — 2,7 %, для UUID версії 4 (122 випадкові біти) за $n = 10^{12}$ — близько 10^{-13}. Довжину обирають за жорсткішою з умов (3.5) і (3.6).

Для ідентифікаторів об'єктів (номер замовлення, документа) непередбачуваність має інше значення. За послідовних ідентифікаторів порушник майже кожним запитом влучає в чужий об'єкт, за випадкових 64-бітових і $n = 10^6$ об'єктів — з імовірністю $n/2^{64} \approx 5 \cdot 10^{-14}$. Проте це *не є* контролем доступу: ідентифікатори витікають через журнали, Referer і спільні посилання, а першопричиною IDOR залишається відсутня перевірка авторизації (A01:2025) [231]. Випадкові ідентифікатори зменшують лише масштаб масового перебору (розділ 9).

3.3 Моделі підбору облікових даних

3.3.1 Онлайновий підбір з обмеженням частоти

Нехай вебресурс дозволяє не більше r спроб на обліковий запис за одиницю часу і блокує автентифікатор після L послідовних невдач. Тоді кількість спроб щодо одного запису за час T, імовірність зламу цільового запису й очікувана кількість зламаних записів за розпилення паролів (password spraying) по k найпоширеніших паролів на кожен з A записів

$$n(T) = \min\big(\lfloor rT \rfloor,\ L\big), \qquad P_{\text{онл}}(T) = \lambda_{n(T)}, \qquad \mathbb{E}[X_{\text{розп}}] = A\,\lambda_k, \quad k < L, \tag{3.7}$$

де λ_β визначено в (3.3). Ліміт L обмежує ризик *окремого* запису значенням λ_L незалежно від тривалості атаки, але не ризик *сукупності* записів: розпилення залишається нижче порога блокування. NIST SP 800-63B-4 обмежує кількість послідовних невдалих спроб значенням не більше 100 [284].

За моделлю Ципфа ($s = 0{,}8$) $\lambda_{100} = 6{,}7\,\%$, а розпилення п'яти паролів по $A = 2 \cdot 10^5$ записах дає $A\lambda_5 \approx 4\,300$ зламаних записів. Якщо заборонено 10^4 найпоширеніших паролів і користувачі, що обрали б їх, розподіляються пропорційно за рештою розподілу, ці показники падають до 0,07 % і приблизно семи записів; припущення оптимістичне, але порядок ефекту зберігається. Ліміт за IP-адресою проти ботнету слабкий (розділ 11).

3.3.2 Заповнення облікових даних

Заповнення облікових даних (credential stuffing) використовує пари «логін–пароль» з витоків інших сервісів, тож успіх визначає поведінка користувачів: за оцінкою Даса й співавторів, 43–51 % користувачів повторно використовують пароль на різних сайтах, а багато інших застосовують передбачувані перетворення [110]. Очікувана кількість захоплених записів

$$\mathbb{E}[X_{\text{зап}}] = A\,\rho_L\,\rho_R\,(1-\rho_M)\,(1-d), \tag{3.8}$$

де ρ_L — частка записів, для яких у порушника є пара з витоків; ρ_R — частка повторного використання; ρ_M — частка записів зі стійкою до цієї атаки MFA; d — імовірність блокування спроби засобом протидії ботам. Для $A = 2 \cdot 10^5$, $\rho_L = 0{,}3$, $\rho_R = 0{,}45$ без MFA і протидії ботам маємо 27 000 записів — на порядок більше, ніж за розпилення. Модель мультиплікативна, тож найсильніші заходи обнуляють множник: обов'язкова фішингостійка MFA, ключі доступу (passkeys), що усувають пароль, і перевірка паролів за базами витоків (розділ 8). Протидія ботам лише зменшує масштаб, а ліміт L не діє: на запис витрачається одна-дві спроби.

3.3.3 Вартість офлайнового підбору

Після викрадення бази гешів сервер не контролює частоту спроб, і захищає лише вартість функції гешування. Вартість перевірки N_g кандидатів і обмеження швидкості для Argon2

$$C_{\text{офл}} = \frac{N_g}{3600\,R}\,c_h, \qquad R_{\text{Argon2}} \lesssim \frac{B_{\text{пам}}}{3\,t\,m}, \tag{3.9}$$

де R — швидкість перевірки на одному GPU (спроб/с); c_h — вартість GPU-години; $B_{\text{пам}}$ — пропускна здатність пам'яті (байт/с); t — кількість проходів по пам'яті обсягом m байт (кожен блок — два читання й один запис). Через сіль $N_g = A\beta$ для β спроб на запис.

Візьмемо вимірювання hashcat 6.2.6 на RTX 4090 [108]: MD5 — $1{,}64 \cdot 10^{11}$ спроб/с, SHA-256 — $2{,}2 \cdot 10^{10}$, PBKDF2-HMAC-SHA256 (999 ітерацій) — $8{,}87 \cdot 10^6$, bcrypt з вартістю 5 — $1{,}84 \cdot 10^5$, scrypt з $N = 2^{14}$ — 7126. Для параметрів, рекомендованих OWASP [227], швидкості перераховано пропорційно кількості ітерацій, для Argon2id оцінено за (3.9) з $B_{\text{пам}} \approx 10^{12}$ байт/с; $c_h = 0{,}5$ дол. (табл. 3.2).

Таблиця 3.2 — Оцінка вартості офлайнового підбору на одному GPU класу RTX 4090

Функція і параметри	R, спроб/с	Простір $36^8 \approx 2{,}8{\cdot}10^{12}$: час; вартість	$2{\cdot}10^5$ записів $\times\, 10^3$ спроб: GPU-год; вартість
MD5	$1{,}6{\cdot}10^{11}$	17 с; <0,01 дол.	<0,01; ≈0
SHA-256	$2{,}2{\cdot}10^{10}$	2 хв; 0,02 дол.	<0,01; ≈0
PBKDF2-SHA256, 600 000	$1{,}5{\cdot}10^{4}$	6 GPU-років; $2{,}7{\cdot}10^{4}$ дол.	3,8; 1,9 дол.
bcrypt, вартість 10	$5{,}8{\cdot}10^{3}$	16 GPU-років; $6{,}8{\cdot}10^{4}$ дол.	9,7; 4,8 дол.
bcrypt, вартість 12	$1{,}4{\cdot}10^{3}$	62 GPU-роки; $2{,}7{\cdot}10^{5}$ дол.	39; 19 дол.
scrypt, $N = 2^{17}$, $r = 8$	$\approx 9{\cdot}10^{2}$	100 GPU-років; $4{,}4{\cdot}10^{5}$ дол.	62; 31 дол.
Argon2id, 19 МіБ, $t = 2$	$\lesssim 8{\cdot}10^{3}$	$\gtrsim$11 GPU-років; $\gtrsim 4{,}6{\cdot}10^{4}$ дол.	$\gtrsim$6,6; $\gtrsim$3,3 дол.

Отже, швидкі геш-функції непридатні: восьмисимвольний пароль з малих літер і цифр перебирається за хвилини [52]. Повільні функції захищають *випадкові* паролі, але не обрані людиною: перевірка 10^3 найпоширеніших кандидатів для кожного з $2 \cdot 10^5$ записів з bcrypt (вартість 12) коштує близько 19 дол. і за $\lambda_{1000} = 12{,}8\,\%$ розкриває близько $25\,500$ паролів. Мінімальні параметри Argon2id за вартістю на GPU порівнянні з bcrypt з вартістю 10, тому RFC 9106 рекомендує значно більші параметри ($m = 2$ ГіБ, $t = 1$ або $m = 64$ МіБ, $t = 3$) [79]. Повільне гешування без заборони поширених паролів і MFA недостатнє (розділ 8).

3.4 Теорія статистичного виявлення

3.4.1 Критерій Неймана–Пірсона і ROC-крива

WAF, система виявлення вторгнень чи модуль протидії ботам перевіряють гіпотези: $\mathcal{H}_0$ — запит легітимний, $\mathcal{H}_1$ — атака. Якщо відомі щільності ознак запиту $f_0(x)$ і $f_1(x)$, то за лемою Неймана–Пірсона найпотужніше правило за заданої ймовірності хибної тривоги α

$$\Lambda(x) = \frac{f_1(x)}{f_0(x)} \underset{\mathcal{H}_0}{\overset{\mathcal{H}_1}{\gtrless}} \eta, \qquad P_{FA} = \Pr\{\Lambda > \eta \mid \mathcal{H}_0\} = \alpha, \qquad P_D = Q\big(Q^{-1}(P_{FA}) - d'\big), \tag{3.10}$$

де η — поріг; P_D — імовірність виявлення; остання рівність справджується для гауссівської моделі з однаковою дисперсією, d' — відстань між середніми в одиницях стандартного відхилення, $Q(\cdot)$ — доповнювальна функція стандартного нормального розподілу. Залежність $P_D(P_{FA})$ за зміни порога — ROC-крива. Приклад: за $d' = 3$ і $P_{FA} = 10^{-3}$ маємо $P_D = Q(3{,}09 - 3) \approx$

0,46, а за $P_{FA} = 10^{-5}$ — лише 0,10; для $d' = 5$ — 0,97 і 0,77. Отже, дуже мала P_{FA} досяжна лише за рахунок якості ознак, а не налаштування порога.

3.4.2 Проблема базової частоти

Аксельссон показав, що для систем виявлення вторгнень вирішальною є P_{FA}, бо частка атак у потоці подій мізерна [68]. Нехай π — апріорна частка шкідливих запитів. За формулою Байєса частка істинних тривог серед усіх тривог (positive predictive value)

$$\mathrm{PPV} = \frac{P_D\,\pi}{P_D\,\pi + P_{FA}\,(1-\pi)}, \qquad P_{FA} \;\le\; \frac{P_D\,\pi\,(1-\mathrm{PPV}^*)}{\mathrm{PPV}^*\,(1-\pi)}, \tag{3.11}$$

де друга нерівність — вимога до детектора для цільового значення PPV^*.

Числовий приклад. Вебресурс обробляє 10^7 HTTP-запитів на добу, з них атакувальних $\pi = 10^{-5}$ (100 запитів), $P_D = 0{,}99$. За $P_{FA} = 10^{-3}$ маємо 99 істинних і близько 10 000 хибних тривог, PPV $\approx 1\,\%$; якщо первинний аналіз тривоги триває 5 хв, потрібно 842 людино-години на добу — понад сто аналітиків. За $P_{FA} = 10^{-4}$ — 1099 тривог, PPV $\approx 9\,\%$, 92 людино-години; лише за $P_{FA} \approx 10^{-5}$ PPV сягає 50 % (199 тривог, 17 людино-годин), а для $\mathrm{PPV}^* = 0{,}9$ потрібна $P_{FA} \le 1{,}1 \cdot 10^{-6}$. Криві (3.11) наведено на рис. 3.1.

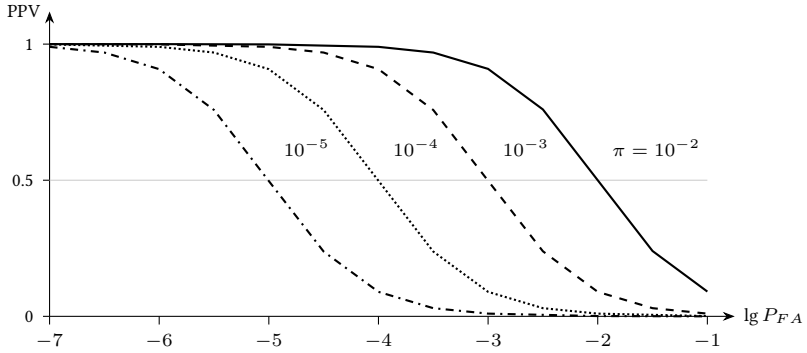


Рис. 3.1 — Частка істинних тривог залежно від імовірності хибної тривоги для різних базових частот ($P_D = 0{,}99$), розраховано за (3.11)

Зменшення базової частоти на порядок вимагає на порядок меншої P_{FA}. Звідси наслідки для WAF і моніторингу (розділи 11, 16): блокувальні правила мають працювати з мінімальною P_{FA}, решта — в режимі журналювання; кореляція кількох умовно незалежних ознак (сигнатура, аномальна відповідь, подальша поведінка клієнта) зменшує ефективну P_{FA} приблизно до добутку окремих значень; оцінки детекторів на збалансованих наборах даних без перерахунку на реальну π вводять в оману. Добове навантаження

$N_{\text{зап}}\big(P_D\pi + P_{FA}(1-\pi)\big)\tau$, де τ — час аналізу тривоги, робить поріг ще й *організаційним* параметром: перевантажені аналітики пропускають тривоги, що фактично зменшує P_D.

3.5 Моделі масового обслуговування для оцінювання стійкості до відмови в обслуговуванні

3.5.1 Модель M/M/1 і закон Літтла

Вебсервер з одним робочим процесом, пуассонівським потоком запитів інтенсивністю λ і експоненційним часом обслуговування із середнім $1/\mu$ описується моделлю M/M/1 [177]:

$$\rho = \frac{\lambda}{\mu}, \qquad L = \frac{\rho}{1-\rho}, \qquad W = \frac{1}{\mu - \lambda}, \qquad L = \lambda W \quad (\rho < 1), \tag{3.12}$$

де ρ — завантаження, L — середня кількість запитів у системі, W — середній час перебування; остання рівність — закон Літтла, справедливий для будь-якої стаціонарної системи, що дозволяє перевіряти вимірювання за журналами. Для $\mu = 100$ с$^{-1}$ за $\lambda = 50$ маємо $W = 20$ мс, за $\lambda = 90$ — 100 мс, за $\lambda = 99$ — 1 с: порушникові достатньо підняти завантаження до 0,95–0,99.

3.5.2 Багатоканальна модель M/M/c і формула Ерланга C

Для сервера з c робочими процесами, навантаженням $a = \lambda/\mu$ і $\rho = a/c < 1$ імовірність очікування в черзі дає формула Ерланга C [177]:

$$C(c,a) = \frac{\dfrac{a^c}{c!}\,\dfrac{c}{c-a}}{\displaystyle\sum_{k=0}^{c-1}\frac{a^k}{k!} + \frac{a^c}{c!}\,\frac{c}{c-a}}, \qquad W_q = \frac{C(c,a)}{c\mu - \lambda}, \qquad W = W_q + \frac{1}{\mu}. \tag{3.13}$$

Приклад: $c = 16$, час обслуговування 20 мс ($\mu = 50$ с$^{-1}$), пропускна здатність 800 запитів/с. За $\lambda = 400$ ($\rho = 0{,}5$) $C = 0{,}009$, $W_q \approx 0{,}02$ мс; за $\lambda = 760$ ($\rho = 0{,}95$) $C = 0{,}78$, $W_q = 19{,}5$ мс, $W = 39{,}5$ мс і за законом Літтла $L \approx 30$; за $\lambda = 790$ $W \approx 114$ мс. Багатоканальна система довше зберігає малу затримку, але різкіше деградує поблизу насичення (рис. 3.2).

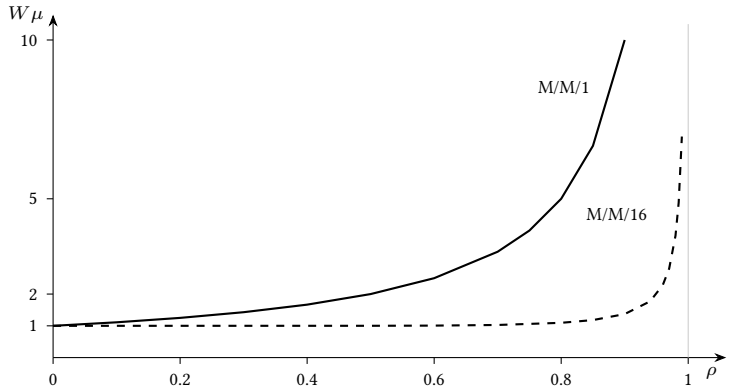


Рис. 3.2 — Нормований час перебування $W\mu$ залежно від завантаження для моделей M/M/1 і M/M/16, розраховано за (3.12) і (3.13)

3.5.3 Неоднорідні запити і вплив варіативності

У вебзастосунку більшість запитів обробляються за мілісекунди, а пошук, звіти чи експорт — за секунди. Для системи M/G/c з коефіцієнтом варіації часу обслуговування c_s застосовують наближення Аллена–Канніна

$$W_q^{\mathrm{M/G/c}} \;\approx\; W_q^{\mathrm{M/M/c}} \cdot \frac{1 + c_s^2}{2}, \qquad c_s^2 = \frac{\operatorname{Var} S}{(\mathbb{E}S)^2}, \tag{3.14}$$

тобто черга зростає пропорційно c_s^2. Додамо до легітимних запитів попереднього прикладу ($\lambda_l = 400$ с$^{-1}$ по 20 мс) $\lambda_a = 3$ с$^{-1}$ «дорогих» запитів по 2 с: $\mathbb{E}S = 34{,}7$ мс, $\rho = 0{,}875$, $c_s^2 \approx 49$, і очікування зростає з 8,8 мс (M/M/c) до близько 220 мс, а за $\lambda_a = 3{,}5$ с$^{-1}$ — до 0,7 с. Крайній випадок — алгоритмічна складність: Кросбі і Воллах показали, що підібрані ключі, які спричиняють колізії в геш-таблицях, перетворюють операції зі складністю $O(1)$ на $O(n)$ і дозволяють малим потоком вивести сервер з ладу [109]; сучасні аналоги — ReDoS і вкладені запити GraphQL (розділ 10).

3.5.4 Коефіцієнт підсилення атаки

Стійкість до прикладної відмови в обслуговуванні характеризує мінімальна інтенсивність атакувальних запитів λ_a^*, що доводить сервер до насичення. З умови $\rho = (\lambda_l s_l + \lambda_a s_a)/c < 1$, де s_l, s_a — середній час обслуговування легітимного й атакувального запиту,

$$\lambda_a^* = \frac{c - \lambda_l s_l}{s_a}, \qquad \kappa = \frac{s_a}{s_l}, \qquad A_{\text{ек}} = \frac{\text{витрати сервера на запит}}{\text{витрати порушника на запит}}, \tag{3.15}$$

де κ — асиметрія вартості запиту; $A_{\text{ек}}$ — економічний коефіцієнт підсилення. У прикладі $\lambda_a^* = (16 - 400 \cdot 0{,}02)/2 = 4$ запити/с; за розміру запиту 2 КБ це близько 64 кбіт/с — потік одного домашнього комп'ютера, непомітний для мережевого захисту від DDoS, тоді як звичайними запитами ($\kappa = 1$) знадобилося б 400 запитів/с. Для атак відбиттям аналогом κ є відношення розміру відповіді до розміру запиту (розділ 11). Напрями захисту: зменшення s_a (кешування, індекси, ліміт вибірки), окремий пул процесів для дорогих кінцевих точок, обмеження частоти саме дорогих операцій і перенесення вартості на клієнта. Ці моделі доповнюють українські дослідження низькошвидкісних HTTP-атак, що виснажують з'єднання сервера малим потоком [31].

3.6 Атакові графи та марковські моделі

3.6.1 Атакові графи

Атаковий граф узагальнює формулу (1.8): вершини — стани порушника або умови (вразливість, доступ), дуги — дії (XSS, заповнення облікових даних, IDOR, SSRF). Шейнер і співавтори запропонували автоматичне генерування атакових графів методом перевірки моделей і пошук мінімальних наборів заходів, що розривають усі шляхи до цілі [268]. Якісну побудову графів розглянуто в розділі 4, тут — їхні ймовірнісні інтерпретації.

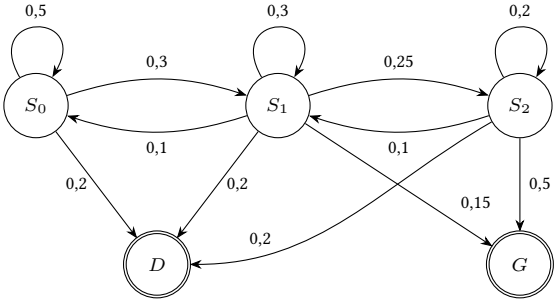
3.6.2 Поглинальний марковський ланцюг

Якщо порушник на кожному кроці просувається, залишається на місці, відступає або його виявляють, процес описує поглинальний марковський ланцюг з перехідною матрицею в канонічній формі

$$\mathbf{P} = \begin{pmatrix} \mathbf{Q} & \mathbf{R} \\ \mathbf{0} & \mathbf{I} \end{pmatrix}, \qquad \mathbf{N} = (\mathbf{I} - \mathbf{Q})^{-1}, \qquad \mathbf{B} = \mathbf{N}\mathbf{R}, \qquad \mathbf{t} = \mathbf{N}\mathbf{1}, \tag{3.16}$$

де $\mathbf{Q}$ — переходи між транзитивними станами; $\mathbf{R}$ — переходи в поглинальні стани («ціль досягнуто» G, «порушника виявлено» D); елемент n_{ij} фундаментальної матриці $\mathbf{N}$ — середня кількість відвідувань стану j зі стану i; b_{ik} — імовірність поглинання станом k; t_i — середня кількість кроків до поглинання.

Приклад наведено на рис. 3.3: з розвідки S_0 порушник за крок з імовірністю 0,3 отримує сеанс користувача S_1 (заповнення облікових даних, XSS), з імовірністю 0,2 його зупиняють; із сеансу з імовірністю 0,25 отримує функції адміністратора S_2, з імовірністю 0,15 одразу отримує дані через IDOR, з імовірністю 0,2 його виявляють; з S_2 з імовірністю 0,5 вивантажує дані.



S_0 — розвідка; S_1 — сеанс користувача; S_2 — функції адміністратора; G — дані вивантажено; D — порушника зупинено

Рис. 3.3 — Поглинальний марковський ланцюг атаки на вебзастосунок

Обчислення за (3.16) дає $b_{0G} \approx 0{,}30$, $b_{1G} \approx 0{,}50$, $b_{2G} \approx 0{,}69$ і $t_0 \approx 3{,}5$ кроку. Обов'язкова MFA, що зменшує ймовірність $S_0 \to S_1$ до 0,1 (решта — у D), знижує b_{0G} до 0,09; виправлення IDOR (перехід $S_1 \to G$ замінюється на виявлення) — до 0,15; обидва заходи — до 0,05. Модель враховує повторні спроби й повернення, але припускає відсутність пам'яті порушника, а ймовірності переходів доводиться оцінювати експертно або за даними тестувань на проникнення (розділ 15).

3.6.3 Байєсові атакові графи

У байєсовому атаковому графі [244] кожна вершина v — бінарна випадкова величина «умову досягнуто», а таблиці умовних імовірностей задаються типом вершини. Для вершин «І» (потрібні всі передумови) і «АБО» (достатньо однієї) за припущення незалежності

$$P_{\text{І}}(v) = p_v \prod_{u \in \mathrm{pa}(v)} P(u), \qquad P_{\text{АБО}}(v) = 1 - \prod_{u \in \mathrm{pa}(v)} \bigl(1 - p_{uv} P(u)\bigr), \tag{3.17}$$

де $\mathrm{pa}(v)$ — батьківські вершини; p_v, p_{uv} — імовірності успіху експлуатації за наявності передумов. Приклад: сеанс досягається через збережений XSS ($p = 0{,}3$) АБО заповнення облікових даних ($p = 0{,}2$): $P(S_1) = 1 - 0{,}7 \cdot 0{,}8 = 0{,}44$; доступ адміністратора вимагає сеансу І IDOR в API адміністратора ($p = 0{,}6$): $P(S_2) = 0{,}264$; ціль досягається через S_2 АБО незалежну SSRF ($p = 0{,}1$): $P(G) \approx 0{,}34$. Виправлення IDOR ($p = 0{,}05$) знижує $P(G)$ до 0,12, а CSP, що зменшує успішність XSS до 0,05, — лише до 0,23. Головна перевага підходу — оновлення апостеріорних імовірностей за спостереженнями моніторингу, що дає змогу динамічно пріоритизувати реагування [244]; обмеження — зростання таблиць для вершин з багатьма батьками і суб'єктивність p_v.

3.7 Оцінки вразливостей і ймовірнісна пріоритизація

3.7.1 CVSS 4.0

CVSS версії 4.0 (FIRST, листопад 2023 р.) оцінює *технічну тяжкість* вразливості за шкалою 0–10 [131]. Базові метрики охоплюють експлуатованість (вектор атаки AV, складність AC, нова метрика вимог до атаки AT, привілеї PR, взаємодія з користувачем UI — «немає», «пасивна», «активна») і вплив окремо на вразливу (VC, VI, VA) і подальші системи (SC, SI, SA), що замінило неоднозначну метрику Scope версії 3.1. Група загрози містить зрілість експлойта (E), група середовища — вимоги до властивостей безпеки, додаткові метрики на оцінку не впливають; позначення CVSS-B, CVSS-BT, CVSS-BE, CVSS-BTE вказують використані групи, рівень «високий» відповідає 7,0–8,9, «критичний» — 9,0–10,0. CVSS є мірою тяжкості, а не ризику: за даними Денисюка (понад 300 тис. записів NVD) лише близько 0,45 % CVE потрапляють до каталогу експлуатованих вразливостей [7], тож пріоритизація лише за CVSS змушує виправляти безліч вразливостей, які ніколи не будуть атаковані.

3.7.2 EPSS, каталог KEV і SSVC

EPSS оцінює ймовірність того, що для вразливості буде зафіксовано експлуатаційну активність упродовж наступних 30 днів [171]. Модель навчається на даних сенсорів про спроби експлуатації та ознаках CVE і щоденно оновлює оцінки; третю версію описано в [172], четверту випущено в березні 2025 р. Каталог KEV агентства CISA містить вразливості з *підтвердженою* експлуатацією і чіткими заходами усунення; для федеральних органів США він пов'язаний з обов'язковими строками виправлення, на кінець 2025 р. налічував близько 1 500 записів [94]. SSVC замінює числову оцінку деревом рішень: за станом експлуатації, технічним впливом, автоматизованістю, значущістю для місії і впливом на суспільний добробут вразливість відносять до дії Track, Track*, Attend або Act [96]. Порівняння — у табл. 3.3.

Таблиця 3.3 — Порівняння систем оцінювання вразливостей

Система	Що оцінює	Результат	Обмеження
CVSS 4.0	Технічну тяжкість за умови експлуатації	Оцінка 0–10, вектор	Не є ймовірністю; група середовища рідко заповнюється
EPSS	Імовірність експлуатації за 30 днів	$p \in [0,1]$, процентиль	Глобальна, без урахування середовища; залежить від сенсорів
KEV	Факт підтвердженої експлуатації	Так / ні	Запізнюється; лише підтверджене
SSVC	Рішення для зацікавленої сторони	Track, Track*, Attend, Act	Якісна; потребує знання місії й активів

3.7.3 Пріоритизація як задача оптимізації

Джейкобс і співавтори формалізували вибір стратегії виправлень через метрики інформаційного пошуку [170]. Нехай V — вразливості в інвентарі, p_i — імовірність експлуатації, c_i — вартість усунення, $x_i \in \{0,1\}$ — рішення про виправлення, B — бюджет циклу. Тоді

$$\mathrm{Cov} = \frac{TP}{TP+FN}, \quad \mathrm{Eff} = \frac{TP}{TP+FP}, \quad \max_{\mathbf{x}} \sum_{i \in V} x_i p_i \ \text{ за } \sum_{i \in V} x_i c_i \le B, \tag{3.18}$$

де TP — виправлені вразливості, що згодом експлуатувалися; FN — невиправлені експлуатовані; FP — виправлені неексплуатовані. Покриття показує частку закритих реальних загроз, ефективність — частку не марно витрачених зусиль. Задача є задачею про рюкзак; жадібний розв'язок (сортування за p_i/c_i) близький до оптимального, коли $c_i \ll B$. Для вебресурсів у p_i доцільно враховувати й досяжність компонента з Інтернету.

В умовному прикладі табл. 3.4 стратегія за CVSS $\ge 7{,}0$ потребує 900 виправлень за ефективності 3,3 %, що ненабагато краще за випадковий вибір, а «KEV або EPSS $\ge 0{,}1$» з 140 виправленнями дає більше покриття. Числа умовні, але співвідношення відповідає емпіричним висновкам [170, 172]. Згідно з моделлю гонки (1.6) вразливості з KEV і високим EPSS слід виправляти за години чи дні, решту — у плановому циклі.

Таблиця 3.4 — Порівняння стратегій виправлення (умовний приклад: 2000 вразливостей, 45 експлуатуються)

Стратегія	Зусилля	TP	Покриття	Ефективність
Виправляти все	2000	45	100 %	2,3 %
CVSS $\geq$ 9,0	250	12	26,7 %	4,8 %
CVSS $\geq$ 7,0	900	30	66,7 %	3,3 %
EPSS $\geq$ 0,1	120	27	60,0 %	22,5 %
KEV або EPSS $\geq$ 0,1	140	33	73,3 %	23,6 %
Те саме або CVSS $\geq$ 9,0 на зовнішніх ресурсах	300	38	84,4 %	12,7 %

3.8 Розгорнутий приклад комплексного розрахунку

Розглянемо вебпортал електронних послуг: $A = 2 \cdot 10^5$ облікових записів, 10^7 запитів на добу, пікове легітимне навантаження 400 запитів/с, 16 робочих процесів (типовий запит 20 мс, пошук за реєстром 2 с). Вихідна конфігурація: паролі без перевірки за списками, bcrypt з вартістю 12, ліміт 100 невдалих спроб, MFA у 60 % користувачів, протидія ботам блокує половину спроб, токени сеансу 128 біт, WAF з $P_{FA} = 10^{-4}$, $P_D = 0{,}99$. Порушник має пари з витоків для $\rho_L = 30\,\%$ користувачів, $\rho_R = 0{,}45$ [110], ботнет і оренду GPU за 0,5 дол./год; паролі розподілені за Ципфом ($s = 0{,}8$).

Результати розрахунку зведено в табл. 3.5. Розпилення паролів оцінено з урахуванням MFA ($4\,300 \cdot 0{,}4$), заповнення облікових даних — як $2 \cdot 10^5 \cdot 0{,}3 \cdot 0{,}45 \cdot 0{,}4 \cdot 0{,}5 = 5\,400$ записів; заходи — список з 10^4 заборонених паролів, обов'язкова фішингостійка MFA для 95 % записів, кореляція двох умовно незалежних ознак WAF з $P_{FA} \approx 10^{-2{,}5}$ кожна, кешування й індексування пошуку ($s_a = 0{,}2$ с) з окремим пулом процесів, виправлення IDOR і перехід до пріоритизації за KEV і EPSS.

Таблиця 3.5 — Зведення результатів розрахунку для вебпорталу електронних послуг

Показник	Формула	Вихідна конфігурація	Після заходів
Угадування сеансу за рік	(3.5)	$<10^{-20}$	без змін
Цільовий онлайновий підбір	(3.7)	6,7 %	0,07 % (список заборонених)
Розпилення паролів, записів	(3.7)	$\approx$1 700	одиниці
Заповнення облікових даних	(3.8)	5 400	675 (MFA 95 %)
Офлайн, 10^3 спроб на запис	(3.9)	19 дол.; $\approx$25 500 паролів	$\approx$1 300 паролів
PPV тривог WAF	(3.11)	9 %; 92 люд.-год/добу	50 %; 17 люд.-год/добу
Поріг прикладного DoS λ_a^*	(3.15)	4 запити/с	40 запитів/с, окремий пул
Досягнення цілі	(3.16)	0,30	0,05 (MFA, IDOR)
Виправлень для покриття $\geq$67 %	(3.18)	900 (CVSS)	140 (KEV, EPSS)

Найбільший ефект дають заходи, що усувають цілі класи атак, а не ті, що лише здорожчують окрему спробу. Результати чутливі до ρ_L, s і ймовірностей переходів, тож на практиці їх подають інтервалами, отриманими методом Монте-Карло (розділ 4).

Висновки до розділу

Майже всі показники захищеності вебресурсу визначаються економікою порушника. Ентропія Шеннона не характеризує стійкість паролів: їхня мінентропія близько 6,5 біт, а проти офлайнової атаки вони еквівалентні випадковим секретам близько 20 біт, тож найефективнішим заходом є заборона найпоширеніших паролів. Серверні секрети мають містити щонайменше 64 випадкові біти, але непередбачуваність ідентифікатора не замінює контролю доступу. Ліміт спроб обмежує ризик окремого запису, але не сукупності записів; заповнення облікових даних описується мультиплікативною моделлю з вирішальною роллю MFA; повільне гешування не захищає популярних паролів. Проблема базової частоти вимагає P_{FA} порядку 10^{-5}–10^{-6} і кореляції ознак. Прикладну відмову в обслуговуванні визначають асиметрія вартості запитів і варіативність часу обслуговування. Атакові графи дозволяють порівнювати заходи за ймовірністю досягнення цілі, а поєднання EPSS і KEV з CVSS дає краще покриття експлуатованих вразливостей за на порядок менших зусиль. Цінність моделі визначають явні припущення й аналіз чутливості.

Питання для самоперевірки

1. Які показники захищеності вебресурсу наведено в табл. 3.1? Чому майже всі вони визначаються економікою порушника?
2. Обчисліть H, H_∞ і G за (3.1) для розподілу $(0{,}5;\ 0{,}25;\ 0{,}125;\ 0{,}125)$. Чому оцінка (3.2) тут не застосовна?
3. Поясніть метрики λ_β і $\tilde{\lambda}_\beta$ (3.3). Яку ймовірність зламу за 10 спроб означає $\tilde{\lambda}_{10} = 9{,}1$ біт?
4. Чому правила складу пароля менш ефективні, ніж список заборонених паролів? Обґрунтуйте за законом Ципфа (3.4).
5. За (3.5) визначте мінімальну довжину токена сеансу, за якої ймовірність угадування за рік не перевищує 10^{-6} для $S = 10^6$ і $r = 10^5$ с$^{-1}$.
6. За (3.6) оцініть кількість 48-бітових випадкових ідентифікаторів, за якої ймовірність колізії досягає 1 %. Чому випадкові ідентифікатори не усувають IDOR?
7. Порівняйте за (3.7) цільовий підбір і розпилення паролів. Чому ліміт невдалих спроб не захищає від розпилення?
8. Які множники моделі (3.8) найефективніше зменшувати і якими заходами?
9. За табл. 3.2 оцініть вартість перевірки 10^4 кандидатів для кожного з 10^6 записів з bcrypt (вартість 12). Яку частку паролів буде розкрито за $\lambda_{10^4} \approx 22\,\%$?
10. Як зміниться P_D за (3.10) для $d' = 4$ при переході від $P_{FA} = 10^{-3}$ до 10^{-5}?
11. За (3.11) визначте P_{FA} для PPV $= 0{,}8$ за $\pi = 10^{-6}$ і $P_D = 0{,}95$. Як це ілюструє рис. 3.1?
12. За (3.13) і законом Літтла обчисліть W і L для $c = 8$, $\mu = 25$ с$^{-1}$, $\lambda = 180$ с$^{-1}$. Чому багатоканальна система різкіше деградує поблизу насичення (рис. 3.2)?
13. Поясніть роль κ і c_s^2 у формулах (3.14) і (3.15). Які архітектурні заходи зменшують кожну з цих величин?
14. Обчисліть за (3.16) імовірність досягнення цілі для ланцюга рис. 3.3, якщо ймовірність виявлення в кожному стані зросте до 0,3 за рахунок імовірності залишитися в стані.
15. Дослідницьке: розробіть методику оцінювання параметрів ρ_L і ρ_R моделі (3.8) для державного вебпорталу за журналами автентифікації без порушення приватності користувачів і обґрунтуйте вимоги до обсягу вибірки.

16. Дослідницьке: запропонуйте модель пріоритизації виправлень для вебресурсів, що поєднує EPSS, KEV, досяжність вразливого коду і модель гонки (1.6), сформулюйте її як задачу (3.18) і запропонуйте метрики для експериментальної перевірки.

РОЗДІЛ 4

Моделювання загроз і управління ризиками вебзастосунків

Розділи 1–3 описали вебресурс як об'єкт захисту і модель порушника, нормативні вимоги та кількісні моделі окремих властивостей — стійкості секретів, виявлення, відмови в обслуговуванні, атакових графів, оцінок CVSS, EPSS і KEV. Управлінського змісту ці результати набувають у системі рішень: які активи захищати, від яких сценаріїв, якими заходами, за яку ціну і який залишковий ризик прийнятний. Розділ розглядає процес управління ризиками, методології моделювання загроз (STRIDE, PASTA, LINDDUN), бази знань CAPEC, CWE і ATT&CK, дерева атак, кількісні моделі ризику (ALE, FAIR, ROSI, модель Гордона–Лоба), безпеку за проєктом і подання невизначеності та завершується моделлю загроз і реєстром ризиків порталу державних послуг.

4.1 Процес управління ризиками

4.1.1 Стандарти та загальна схема

Ризик за ISO 31000 [160] — це вплив невизначеності на цілі; ризик інформаційної безпеки подають поєднанням імовірності (частоти) події та її наслідків. ISO/IEC 27005:2022 [162] конкретизує процес ISO 31000: встановлення контексту, оцінювання ризиків (ідентифікація, аналіз, оцінка), оброблення, прийняття залишкового ризику, а також наскрізні комунікація і моніторинг (рис. 4.1). Редакція 2022 р. розрізняє два підходи до ідентифікації: *подієвий* (event-based), що виходить зі стратегічних сценаріїв — джерело ризику, мотивація, шлях до мети, — і *активо-орієнтований* (asset-based), що перебирає активи, загрози і вразливості. Для вебзастосунку перший зручний на рівні керівництва («масове захоплення облікових записів громадян»), другий — на рівні архітектури, де він поєднується з моделюванням загроз (підрозділ 4.2).

NIST SP 800-30 [217] описує оцінювання ланцюгом «джерело загрози → подія → вразливість (за наявних заходів) → несприятливий вплив» на трьох рівнях: організації, бізнес-процесу та інформаційної системи. CSF 2.0 [221] додав функцію **управління** (GOVERN, GV): категорія GV.RM вимагає визначити стратегію управління ризиками, апетит до ризику і толерантність, GV.SC — ризики ланцюга постачання. Отже, керівництво встановлює критерії прийняття, а команди розробки й експлуатації надають оцінки і варіанти оброблення.

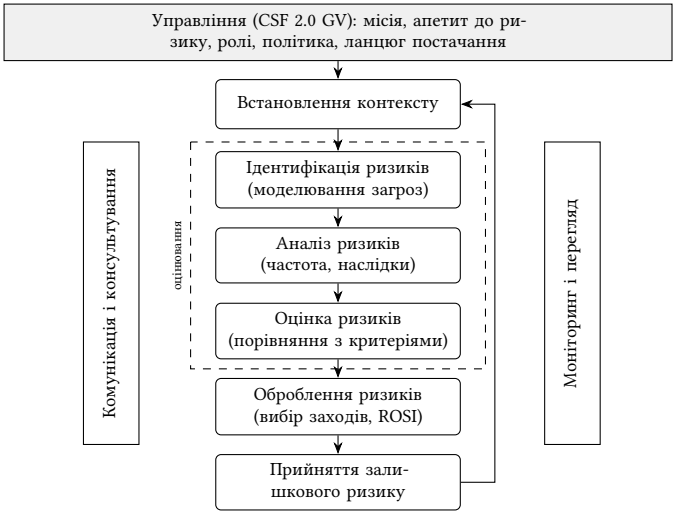


Рис. 4.1 — Процес управління ризиками вебзастосунку (за [162, 160, 221])

Для державних інформаційних ресурсів після змін, внесених Законом № 4336-IX [33], базовим механізмом стали профілі безпеки та авторизація з безпеки [11], які за природою ризик-орієнтовані: вибір заходів профілю обґрунтовують результатами оцінювання ризиків.

4.1.2 Встановлення контексту для вебресурсу

На цьому етапі визначають межі системи, зацікавлені сторони, критерії впливу та прийняття ризику. Межі вебзастосунку майже ніколи не збігаються з межами організації: частину функцій виконують CDN, хмарний провайдер, постачальник ідентичності, платіжний шлюз, сторонні скрипти, а частину — браузер користувача (розділ 1, домени довіри). Тому межі визначають *функціонально* — через потоки даних і межі довіри.

Активами вебресурсу є: *дані* (персональні, платіжні, витяги з реєстрів, ключі API і підпису токенів); *сеанси й токени*, компрометація яких рівнозначна захопленню облікового запису; *облікові записи*, насамперед привілейовані й сервісні; *бізнес-процеси* (оформлення послуги, переказ), цілісність яких не зводиться до цілісності даних; *доступність* у пікові періоди; *репутація і довіра*. Зацікавлені сторони — власник і керівництво (апетит до ризику), користувачі як суб'єкти даних (GDPR [249], Закон «Про захист персональних даних» [38]), команди розробки й експлуатації, постачальники, регулятори та CERT-UA [9]. Критерії впливу задають за кількома вимірами (фінанси, кількість постраждалих, недоступність, регуляторні строки (2.3)) і агрегують за найгіршим виміром, а не сумою балів.

4.2 Моделювання загроз вебзастосунку

4.2.1 Діаграми потоків даних і межі довіри

Моделювання загроз (threat modeling) — структурований аналіз системи з метою виявити, що може піти не так, і вирішити, що з цим робити. Маніфест моделювання загроз [286] (2020) зводить процес до чотирьох запитань: *над чим ми працюємо? що може піти не так? що ми з цим зробимо? чи достатньо добре ми це зробили?* — і наголошує, що «найкращої» методології не існує, а цінність має регулярна практика. Емпіричних порівнянь ефективності методологій бракує [302].

Відповідь на перше запитання дає **діаграма потоків даних** (data flow diagram, DFD) з елементами чотирьох типів: зовнішні сутності, процеси, сховища даних і потоки [270]. До них додають **межі довіри** (trust boundaries), на яких змінюється рівень довіри або власник. Саме на перетинах меж зосереджена більшість загроз: дані з менш довіреного домену мають бути автентифіковані, авторизовані й перевірені.

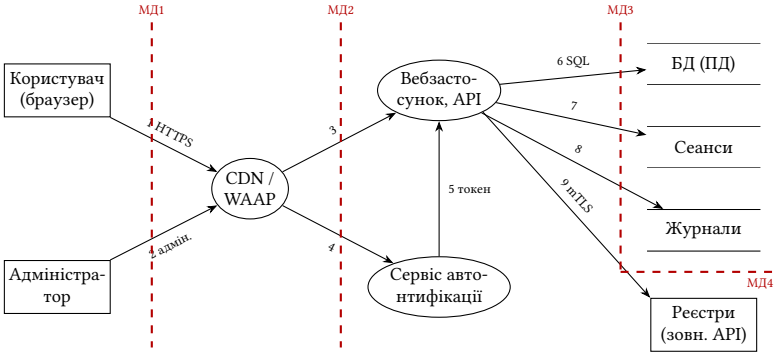


Рис. 4.2 — DFD типового вебзастосунку з межами довіри МД1–МД4 (прямокутник — зовнішня сутність, еліпс — процес, дві лінії — сховище)

На рис. 4.2 МД1 відділяє інтернет від периметра (CDN, вебекран прикладного рівня, протидія ботам), МД2 — периметр від застосунку, МД3 — застосунок від сховищ, МД4 — організацію від зовнішніх реєстрів. Потоки 1 і 2 на МД1 — основне місце підміни й ін'єкцій, підробка токена в потоці 5 означає підміну будь-якого користувача, а спільний шлях адміністративного і користувацького доступу — рішення, яке модель загроз має поставити під сумнів.

4.2.2 STRIDE і STRIDE-per-element

Методологія STRIDE [270] класифікує загрози за порушеною властивістю (табл. 4.1). Це не таксономія атак, а *мнемоніка повноти*: для кожного елемента DFD аналітик ставить шість запитань і фіксує відповіді.

Таблиця 4.1 — Категорії STRIDE у вебзастосунках

Категорія	Властивість	Типові веб-прояви	Заходи (розділ)
Spoofing (підміна)	автентичність	заповнення облікових даних; фішинг з проксі (AiTM); підроблений JWT; викрадений cookie	фішингостійка MFA, ключі доступу; перевірка підпису й аудиторії токена (8, 9)
Tampering (модифікація)	цілісність	SQL- і командні ін'єкції; зміна параметрів ціни; збережений XSS; підміна стороннього скрипту	параметризація; серверна перевірка; CSP, SRI (6, 7, 13)
Repudiation (відмова від авторства)	неспростовність	заперечення транзакції; відсутні або змінювані журнали адміністративних дій	журнали з контролем цілісності; КЕП для юридично значущих дій (16)
Information disclosure (розкриття)	конфіденційність	IDOR; надмірні відповіді API; трасування стека в помилках; SSRF до метаданих хмари	авторизація на рівні об'єктів; мінімізація відповідей; обробка винятків (7, 10)
Denial of service (відмова)	доступність	HTTP-флуд; Rapid Reset; дорогі запити пошуку; ReDoS; блокування облікових записів	CDN, обмеження частоти, квоти, тайм-аути (3, 11)
Elevation of privilege (підвищення)	авторизація	BFLA; масове присвоєння ролі; обхід перевірки в UI; вебшел	централізована авторизація; заборона за замовчуванням (9)

Варіант **STRIDE-per-element** обмежує перебір: для кожного типу елемента DFD застосовні лише певні категорії (табл. 4.2). Сховище журналів додатково є об'єктом загроз неспростовності. Варіант *STRIDE-per-interaction* аналізує пари «джерело–приймач» на межі довіри [270].

Таблиця 4.2 — STRIDE-per-element для DFD рис. 4.2

Елемент	S	T	R	I	D	E	Приклад
Зовнішня сутність	+		+				U: вхід з викраденим паролем
Процес	+	+	+	+	+	+	W: SSTI, IDOR, ReDoS
Потік даних		+		+	+		5: підміна токена
Сховище		+	(+)	+	+		L: видалення записів

Кількість кандидатних загроз для DFD з n_e зовнішніх сутностей, n_p процесів, n_f потоків і n_s сховищ становить $2n_e + 6n_p + 3n_f + 4n_s$; для рис. 4.2 ($n_e = 3$, $n_p = 3$, $n_f = 9$, $n_s = 3$) це $6 + 18 + 27 + 12 = 63$ пари «елемент–категорія». Цінність мають пари з відповіддю «не знаємо» або «залежить від постачальника»: вони формують перелік ризиків для кількісного аналізу.

4.2.3 PASTA: ризик-орієнтоване моделювання

Методологія PASTA (Process for Attack Simulation and Threat Analysis) [289] будує модель «згори вниз» у сім етапів: (1) бізнес-цілі й вимоги відповідності; (2) технічний обсяг; (3) декомпозиція застосунку (DFD, межі довіри, ролі, точки входу); (4) аналіз загроз за даними розвідки та інцидентів; (5) аналіз вразливостей і слабкостей (CWE, результати тестування); (6) моделювання атак (дерева атак, симуляція); (7) аналіз ризиків і впливу із залишковим ризиком. PASTA відповідає не на запитання «що може піти не так з елементом», а «які атаки реальних порушників проти бізнес-функції найімовірніші і скільки вони коштуватимуть». Етапи 4–7 спираються на підрозділи 4.3–4.5; PASTA доцільна для критичних застосунків, а для рядової функціональності достатньо STRIDE-per-element.

4.2.4 LINDDUN: загрози приватності

Система може бути захищеною за STRIDE і водночас надмірно збирати й пов'язувати дані. Методологія LINDDUN [112] використовує ту саму DFD і сім категорій: зв'язуваність (Linkability), ідентифіковуваність (Identifiability), неспростовність (Non-repudiation) як загроза приватності, виявлюваність (Detectability), розкриття інформації (Disclosure), необізнаність суб'єкта (Unawareness) і невідповідність вимогам (Non-compliance). Для вебзастосунків типові зв'язування сеансів через сторонні cookie і відбиток браузера, виявлюваність наявності облікового запису через різні повідомлення форм входу та необізнаність про передачу даних стороннім скриптам. Категорія невідповідності відображає вимоги GDPR [249], а результати LINDDUN є основою оцінки впливу на захист даних (DPIA).

4.3 Бази знань про атаки та слабкості

4.3.1 CAPEC, CWE і ATT&CK

CWE [105] — ієрархічний словник *слабкостей* проєктування й реалізації (CWE-89 — SQL-ін'єкція, CWE-862 — відсутня авторизація), ранжований щорічно в CWE Top 25 [54]. CAPEC [104] — каталог *шаблонів атак* з передумовами, навичками, ресурсами і зв'язками з CWE. MITRE ATT&CK [207] описує *поведінку* реальних порушників: тактики і техніки зі звітів про інциденти. Для вебресурсів ключовою є тактика початкового доступу (Initial Access) з технікою T1190 «Exploit Public-Facing Application»; інші типові техніки наведено в табл. 4.3, а для закріплення — T1505.003 «Web Shell». CWE відповідає на запитання «що неправильно в коді чи проєкті», CAPEC — «як цим скористатися», ATT&CK — «що робить порушник до і після». Ланцюг «CWE → CAPEC → ATT&CK» пов'язує результати статичного аналізу (розділ 15) з правилами виявлення в SIEM (розділ 16).

4.3.2 OWASP Top 10:2025 як каталог ризиків

OWASP Top 10:2025 [231] — каталог *категорій ризику*, кожна з яких агрегує десятки CWE (розділ 1). Він придатний як контрольний список повноти моделі і мова комунікації з керівництвом, але не як одиниця аналізу: A01 охоплює і IDOR у кабінеті громадянина, і SSRF з різними частотами й наслідками. Зв'язки між базами наведено в табл. 4.3; оцінки CVSS 4.0 і EPSS (підрозділ 3.7) слугують вхідними даними для частоти подій у моделі ризику.

Таблиця 4.3 — Зв'язок сценаріїв атак із базами знань

Сценарій	CAPEC	CWE	ATT&CK	Top 10
SQL-ін'єкція в пошуку	66	89	T1190	A05
Збережений XSS у кабінеті	592, 63	79	T1189, T1539	A05
IDOR в API довідок	1	639, 862	T1190	A01
Заповнення облікових даних	600	307, 287	T1110.004, T1078	A07
SSRF до метаданих хмари	664	918	T1190, T1552.005	A01
Шкідлива версія залежності	538	1357	T1195.001	A03
Прикладна відмова в обслуговуванні	125	770, 400	T1499	API4

Відображення між базами не взаємно однозначні: один шаблон CAPEC експлуатує кілька CWE, а T1190 охоплює майже всі серверні вразливості, тобто для веб-рівня ATT&CK надто грубий і потребує доповнення CAPEC. Прикладна відмова в обслуговуванні не має окремої категорії в Top 10:2025 і описується в API Security Top 10 [235]

(API4:2023 «Unrestricted Resource Consumption»). Автоматичне зіставлення CVE → CWE → CAPEC → ATT&CK через різну гранулярність потребує експертної перевірки.

4.4 Дерева атак

4.4.1 Структура та обчислення атрибутів

Дерево атак [265] — ієрархічна модель, корінь якої відповідає меті порушника, внутрішні вузли — підцілям, листя — елементарним діям; вузол «АБО» досягається, якщо досягнуто хоча б одного нащадка, «І» — якщо всіх. Формальна семантика [199] трактує дерево як множину сценаріїв і показує, що атрибут коректно обчислюється знизу вгору, якщо операції для «АБО» і «І» утворюють напівкільце, наприклад $(\min, +)$ для вартості. На відміну від атакових графів (підрозділ 3.6), дерева описують цілі, а не стани системи.

Імовірності за незалежності листків поширюють, як у байєсовому атаковому графі (3.17): $p_{\text{І}} = \prod_k p_k$, $p_{\text{АБО}} = 1 - \prod_k (1 - p_k)$. Для вартості і вигоди раціонального порушника

$$c_{\text{І}} = \sum_k c_k, \qquad c_{\text{АБО}} = \min_k c_k, \qquad U^* = \max_{\sigma \in \Sigma} \bigl(p_\sigma B - c_\sigma \bigr), \tag{4.1}$$

де c_k — вартість листка; Σ — множина сценаріїв (мінімальних наборів листків, що досягають кореня); p_σ, c_σ — імовірність успіху і вартість сценарію; B — вигода порушника. Захист *економічно стримує* раціонального порушника, якщо $U^* < 0$. Незалежне поширення p і c некоректне для рішень: мінімальна вартість і максимальна ймовірність можуть досягатися на різних сценаріях, тому використовують U^* або фронт Парето за (p, c). Дерева атак і захисту [180] додають вузли контрзаходів, що дає змогу оцінювати ефект захисту в тій самій моделі.

4.4.2 Приклад: заволодіння обліковим записом адміністратора

Мета G — заволодіння обліковим записом адміністратора вебпорталу (рис. 4.3). Шлях A1 (І): фішинг пароля і обхід MFA через «втому від push-сповіщень»; A2 (АБО): викрадення сеансу через збережений XSS у панелі модерації або шкідливе ПЗ на робочій станції адміністратора; A3 (АБО): вада логіки — обхід відновлення пароля або виклик функцій адміністрування через API без перевірки ролі (BFLA).

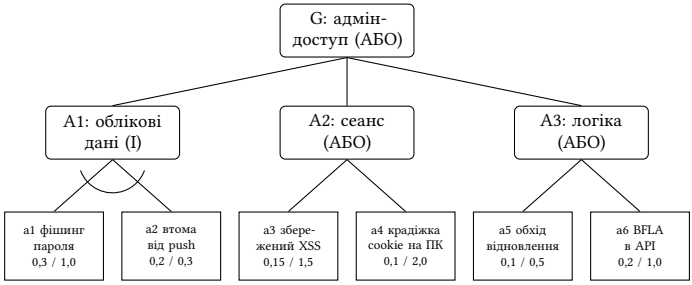


Рис. 4.3 — Дерево атак «заволодіння обліковим записом адміністратора» (у листі: імовірність успіху за рік / вартість в умовних одиницях; дуга — вузол «I»)

Маємо $p_{A1} = 0{,}3 \cdot 0{,}2 = 0{,}06$, $p_{A2} = 1 - 0{,}85 \cdot 0{,}9 = 0{,}235$, $p_{A3} = 1 - 0{,}9 \cdot 0{,}8 = 0{,}28$, $p_G = 1 - 0{,}94 \cdot 0{,}765 \cdot 0{,}72 \approx 0{,}482$; мінімальні вартості $c_{A1} = 1{,}3$, $c_{A2} = 1{,}5$, $c_{A3} = c_G = 0{,}5$. За $B = 10$ корисності сценаріїв: $U(a6) = 0{,}2 \cdot 10 - 1{,}0 = 1{,}0$, $U(a5) = 0{,}5$, $U(a3) = 0$, $U(A1) = -0{,}7$, $U(a4) = -1{,}0$ — раціональний порушник обере BFLA в API, а не фішинг, що зазвичай першим спадає на думку.

Заходи: ключі FIDO2 для адміністраторів (розділ 8) знижують $p(a2)$ до 0,01; CSP з Trusted Types (розділ 6) — $p(a3)$ до 0,03; виправлення відновлення і централізована перевірка ролей (розділ 9) — $p(a5)$ до 0,01, $p(a6)$ до 0,02. Тоді $p_{A1} = 0{,}003$, $p_{A2} = 0{,}127$, $p_{A3} = 0{,}030$, $p_G \approx 0{,}156$ — утричі менше, і всі $U < 0$. Проте залишкову ймовірність тепер визначає a4: викрадений з робочої станції cookie обходить і MFA, і серверні перевірки. Прив'язка сеансу до пристрою (DBSC, розділ 8) і привілейовані робочі станції ($p(a4) = 0{,}02$) дають $p_G \approx 0{,}080$. Типовий висновок: після закриття очевидних шляхів пріоритет зміщується на контроль сеансу і кінцевого пристрою.

4.5 Кількісне оцінювання ризику

4.5.1 ALE і FAIR

Класична модель визначає втрати від однієї події $\mathrm{SLE} = \mathrm{AV} \cdot \mathrm{EF}$ (вартість активу на частку втрати) і річні очікувані втрати як добуток на річну частоту ARO. Методологія FAIR (Factor Analysis of Information Risk) [134] розкладає частоту подій втрат (LEF) на частоту подій загрози (TEF) і вразливість (Vuln), а величину втрат (LM) — на первинні (реагування, відновлення, простій) і вторинні, спричинені реакцією сторонніх (штрафи, позови, відтік клієнтів):

$$\begin{aligned} \mathrm{ALE} &= \mathrm{SLE} \cdot \mathrm{ARO}, \qquad \mathrm{LEF} = \mathrm{TEF} \cdot \mathrm{Vuln}, \\ \mathrm{Vuln} &= \Pr\{\mathrm{TCap} > \mathrm{RS}\}, \qquad \mathrm{LM} = L_{\text{перв}} + p_{\text{втор}}\, L_{\text{втор}}, \end{aligned} \tag{4.2}$$

де TCap — спроможність загрози, RS — стійкість захисту на спільній шкалі, $p_{\text{втор}}$ — імовірність вторинних наслідків. Очікувані річні втрати дорівнюють LEF · E[LM], але кожен множник FAIR вимагає подавати розподілом (підрозділ 4.7).

Приклад. Вебпортал фіксує TEF $= 12$ кампаній заповнення облікових даних на рік; за наявних обмеження частоти й часткової MFA Vuln $= 0{,}25$, тож LEF $= 3$. Первинні втрати події — 200 тис. грн; з імовірністю 0,3 подія набуває розголосу з вторинними втратами 1 500 тис. грн. Тоді LM $= 650$ тис. грн і ALE $= 1\,950$ тис. грн. Орієнтиром порядку величин є галузеві дані (середня світова вартість витоку даних у 2025 р. — 4,44 млн дол. США [157]), що погано переносяться на конкретну організацію. Методику оцінювання комплексних (прямих, непрямих, репутаційних) збитків запропоновано в [47], а кількісну методологію для випадку відсутності фінансових даних про втрати — у [1].

4.5.2 ROSI та економічне обґрунтування заходів

Доцільність заходу j оцінюють рентабельністю інвестицій у безпеку [273]. Щоб урахувати взаємодію заходів, ефект визначають відносно вже впровадженої множини X за мультиплікативною моделлю ефективності:

$$\begin{gathered}\mathrm{ROSI}_j = \frac{\Delta\mathrm{ALE}_j - C_j}{C_j}, \qquad \Delta\mathrm{ALE}_j = \mathrm{ALE}(X) - \mathrm{ALE}(X \cup \{j\}), \\ \mathrm{ALE}(X) = \sum_i \mathrm{ALE}_i^0 \prod_{m \in X} \bigl(1 - e_{im}\bigr),\end{gathered} \tag{4.3}$$

де C_j — річна вартість заходу (амортизація, ліцензії, праця, вплив на швидкість розробки); $e_{im} \in [0;1]$ — частка зниження ризику сценарію i заходом m. У прикладі вище захід вартістю 400 тис. грн (перевірка паролів за списками зламаних, обов'язкова MFA, протидія ботам), що знижує LEF на 70 %, дає $\Delta\mathrm{ALE} = 1\,365$ і ROSI $\approx 2{,}41$. Заходи ранжують жадібно з перерахунком після кожного кроку; для однотипних заходів модель (4.3) завищує ефект, і їх об'єднують.

4.5.3 Модель Гордона–Лоба

Гордон і Лоб [139] розглянули інформаційний набір з потенційними втратами L і ймовірністю порушення без інвестицій v, яку інвестиції z знижують до $S(z, v)$:

$$\begin{gathered}z^{*} = \arg\max_{z \ge 0} \Bigl\{ \bigl[v - S(z,v)\bigr] L - z \Bigr\}, \qquad S(z,v) = \frac{v}{(\alpha z + 1)^{\beta}}, \\ z^{*} < vL/e \approx 0{,}37\, vL,\end{gathered} \tag{4.4}$$

де $\alpha, \beta > 0$ — параметри продуктивності інвестицій; межу vL/e доведено для двох широких класів функцій S. Для S з (4.4) $z^* = \left[(\alpha\beta vL)^{1/(\beta+1)} - 1\right]/\alpha$. За $v = 0{,}6$, $L = 5\,000$ тис. грн, $\alpha = 0{,}002$ (тис. грн)$^{-1}$, $\beta = 1$ маємо $z^* = (\sqrt{6} - 1)/0{,}002 \approx 725$ тис. грн, $S(z^*) \approx 0{,}245$, чисту вигоду $(0{,}6 - 0{,}245) \cdot 5\,000 - 725 \approx 1\,050$ тис. грн; межа $vL/e \approx 1\,104$ не перевищена. Оскільки α і β на практиці невідомі, цінність моделі — у висновках: інвестувати «до нуля ризику» нераціонально, бюджет понад третину очікуваних втрат потребує обґрунтування, а оптимальні інвестиції не обов'язково зростають з v — захист найвразливіших наборів може бути настільки дорогим, що раціональніше захищати набори із середньою вразливістю.

ROSI — інструмент ранжування, а не остаточного рішення: обов'язкові заходи (розділ 2) впроваджують незалежно від нього, цінність детективних заходів погано виражається через ΔALE, а для державних послуг L має включати втрати громадян.

4.6 Безпека за проєктом

4.6.1 Небезпечний дизайн і сценарії зловживання

Категорія A06:2025 «Insecure Design» [231] відокремлює вади *проєкту* від вад *реалізації*: досконала реалізація небезпечного проєкту залишається небезпечною, бо потрібний механізм не передбачено. Приклади: відновлення доступу через «секретні запитання»; відсутність обмеження кількості цінних операцій (масове бронювання талонів, перебір промокодів); довіра до ціни чи ролі від клієнта; багатокрокові процеси без серверної перевірки порядку кроків (CWE-841); відсутність розділення орендарів. Такі вади не виявляють сканери, бо кожен запит синтаксично коректний.

Загрози на рівні вимог описують **сценаріями зловживання** (abuse cases) [200] або **сценаріями неправомірного використання** (misuse cases) [271], які зображують на одній діаграмі з use-сценаріями з відношеннями «загрожує» і «пом'якшує». Наприклад, сценарій «громадянин замовляє витяг» доповнюють сценарієм «бот масово замовляє витяги щодо інших осіб, перебираючи ідентифікатори», що пом'якшується авторизацією на рівні об'єкта і квотами. Сценарії перетворюють на перевірювані вимоги ASVS 5.0 [226]: розділу V2 (валідація та бізнес-логіка), V8 (авторизація), V15 (безпечне кодування й архітектура). Рівень ASVS обирають за ризиком: для державного порталу чи інтернет-банкінгу обґрунтований L2 для всього застосунку і L3 для функцій з найвищим впливом.

4.6.2 Шаблони безпечного проєктування

Шаблони безпечного проєктування усувають цілі класи вад [61, 227]: *повна опосередкованість* — кожен доступ проходить через єдиний серверний механізм авторизації; *заборона за замовчуванням* для нових кінцевих точок

API; *безпечні за замовчуванням API* — шаблонізатори з контекстним кодуванням, ORM з параметризацією; *розділення привілеїв* — адміністративний інтерфейс як окремий застосунок за окремою межею довіри; *мінімізація даних*; *ідемпотентність і транзакційність* бізнес-операцій проти станів гонитви (подвійне нарахування паралельними запитами). Ці шаблони переносять безпеку з дисципліни розробника на властивість платформи (розділ 17).

4.6.3 Моделювання загроз в Agile і DevSecOps

Разовий семінар перед випуском погано узгоджується з ітеративною розробкою. NIST SSDF [275] (PW.1) і OWASP SAMM [229] (Threat Assessment) розглядають моделювання загроз як безперервну діяльність на трьох рівнях: запитання STRIDE для історії користувача, що перетинає межу довіри; модель застосунку, що оновлюється при архітектурних змінах; глибокий аналіз (PASTA, дерева атак) для критичних функцій. **Моделювання загроз як код** (threat modeling as code) описує DFD текстом у репозиторії поруч із кодом, рецензує його як код і генерує діаграми та переліки кандидатних загроз; прикладами є OWASP pytm [228] і OWASP Threat Dragon [230]. Згенерований перелік — лише контрольний список: специфічних вад бізнес-логіки він не виявляє.

4.7 Невизначеність оцінок

4.7.1 Інтервальні оцінки та PERT

Точкові оцінки створюють ілюзію точності: «3 інциденти на рік» насправді означає інтервал. Практика кількісного оцінювання [155, 134] вимагає 90-відсоткових довірчих інтервалів або трійок «мінімум–найімовірніше–максимум». Для трійки (a, m, b) використовують бета-PERT-розподіл, а для величин, що охоплюють порядки (втрати), — логнормальний з параметрами за 90-відсотковим інтервалом $[a; b]$:

$$\mu_{\text{PERT}} = \frac{a + 4m + b}{6}, \qquad \sigma_{\text{PERT}} \approx \frac{b - a}{6};$$
$$\ln X \sim \mathcal{N}(\mu, \sigma^2), \quad \mu = \frac{\ln a + \ln b}{2}, \quad \sigma = \frac{\ln b - \ln a}{2 \cdot 1{,}645}, \quad \mathrm{E}[X] = e^{\mu + \sigma^2/2}. \tag{4.5}$$

Для частоти $(a, m, b) = (1;\ 3;\ 12)$ на рік $\mu_{\text{PERT}} \approx 4{,}17$, $\sigma_{\text{PERT}} \approx 1{,}83$: асиметрія зсуває середнє вправо від моди. Для втрат від серйозного витоку з інтервалом $[200;\ 5\,000]$ тис. грн медіана $\sqrt{200 \cdot 5\,000} = 1\,000$, $\sigma \approx 0{,}978$, $\mathrm{E}[X] \approx 1\,614$ тис. грн — на 60 % більше за медіану через важкий хвіст. «Типове» значення замість середнього систематично занижує ризик найруйнівніших сценаріїв.

4.7.2 Метод Монте-Карло і криві перевищення втрат

Річні втрати моделюють складеним пуассонівським процесом: кількість подій $N \sim \mathrm{Poisson}(\lambda)$, втрати X_k незалежні й однаково розподілені. Результат — **крива перевищення втрат** (loss exceedance curve, LEC):

$$L_\Sigma = \sum_{k=1}^{N} X_k, \qquad \mathrm{E}[L_\Sigma] = \lambda\,\mathrm{E}[X],$$
$$\bar F(\ell) = \Pr\{L_\Sigma > \ell\} \approx \frac{1}{M}\sum_{r=1}^{M} \mathbf{1}\{L_\Sigma^{(r)} > \ell\}, \tag{4.6}$$

де M — кількість ітерацій. Для $\lambda = 0{,}5$ і логнормальних втрат з інтервалом $[200;\,5\,000]$ моделювання з $M = 2\cdot 10^5$ дає $\mathrm{E}[L_\Sigma] \approx 807$ тис. грн (аналітично $0{,}5 \cdot 1\,614$), імовірність хоча б однієї події $1 - e^{-0{,}5} \approx 0{,}39$, $\bar F(1\,000) \approx 0{,}23$, $\bar F(5\,000) \approx 0{,}033$, 95-й і 99-й процентилі — близько 4 000 і 8 400 тис. грн. Якщо керівництво задало *криву толерантності* «втрати понад 5 млн грн на рік — з імовірністю не більше 2 %», ризик неприйнятний, хоча середні втрати (0,8 млн) помірні. На відміну від порядкових матриць «імовірність × вплив», де добуток рангів не має кількісного змісту, LEC адитивна за сценаріями й інтерпретована [155].

4.7.3 Калібрування експертів і байєсове оновлення

Експерт *калібрований*, якщо близько 90 % його 90-відсоткових інтервалів містять істинне значення. Непідготовлені експерти надмірно впевнені, а коротке тренування із зворотним зв'язком суттєво покращує калібрування [155]. Якість бінарних прогнозів вимірюють оцінкою Браєра $\mathrm{BS} = \frac{1}{n}\sum_i (f_i - o_i)^2$, де f_i — прогнозована ймовірність, $o_i \in \{0,1\}$ — результат.

Оцінки оновлюють за даними моніторингу (розділ 16). Якщо апріорну частоту λ описано гамма-розподілом (α_0, β_0), а за час t зареєстровано k подій, то завдяки спряженості з пуассонівською правдоподібністю

$$\lambda \mid k \sim \mathrm{Gamma}(\alpha_0 + k,\ \beta_0 + t), \qquad \mathrm{E}[\lambda \mid k] = \frac{\alpha_0 + k}{\beta_0 + t}. \tag{4.7}$$

Апріорна оцінка 2 серйозні інциденти на рік з $(\alpha_0, \beta_0) = (2;\,1)$ і 1 інцидент за 3 роки дають апостеріорне середнє $(2+1)/(1+3) = 0{,}75$ на рік. Відсутність інцидентів теж є даними, якщо система виявлення здатна їх помітити (підрозділ 3.4).

4.8 Кейс: модель загроз і реєстр ризиків порталу державних послуг

4.8.1 Модель загроз і реєстр ризиків

Портал державних електронних послуг має архітектуру рис. 4.2: $2 \cdot 10^5$ облікових записів громадян (пароль з необов'язковою MFA або КЕП), кабінет посадовців, API довідок і витягів, інтеграції з реєстрами, CDN без розвиненої протидії ботам, сторонні бібліотеки з публічних репозиторіїв. Портал підлягає авторизації з безпеки за профілем безпеки [11], тож реєстр ризиків обґрунтовує вибір заходів. Аналіз STRIDE-per-element дав 63 кандидатні пари, з яких після відкидання закритих типовими заходами лишилося шість сценаріїв, оцінених експертно за (4.2) (табл. 4.4; тис. грн, ARO — подій на рік; для R4 і R5 включено суспільні втрати від недоступності послуг).

Таблиця 4.4 — Реєстр ризиків порталу державних послуг (фрагмент)

ID	Сценарій (STRIDE; база знань)	ARO	SLE	ALE_0	Залиш.
R1	Заповнення облікових даних (S; CAPEC-600)	6	120	720	108
R2	IDOR в API витягів (I; CWE-639)	1,5	600	900	180
R3	Збережений XSS у кабінеті посадовця (S, E; CWE-79)	0,5	800	400	100
R4	Прикладний DDoS у піковий період (D; T1499)	4	150	600	600
R5	Шкідлива залежність у збиранні (T; T1195.001)	0,2	4 000	800	320
R6	Зловживання логікою черги (T, E; CWE-841)	2	100	200	120
	Разом			3 620	1 428

Стовпець «Залиш.» — залишковий ризик після заходів M1, M2, M3, M5 (див. нижче). Власники ризиків: R1, R3 — керівник напряму ідентифікації; R2, R6 — власник продукту; R4 — служба експлуатації; R5 — керівник розробки.

4.8.2 Вибір заходів за ROSI

Розглянуто п'ять заходів: M1 — фішингостійка MFA і ключі доступу, перевірка паролів за списками зламаних, обмеження частоти ($C = 500$; R1: $e = 0{,}85$); M2 — централізована авторизація на рівні об'єктів і функцій з тестами авторизації в CI ($C = 350$; R2: $0{,}8$; R6: $0{,}4$); M3 — CSP з Trusted Types і контекстне кодування ($C = 150$; R3: $0{,}75$); M4 — WAAP з протидією ботам і захистом від DDoS прикладного рівня ($C = 700$; R4: $0{,}7$; R1: $0{,}3$; R6: $0{,}3$); M5 — SBOM, аналіз складу ПЗ, фіксація версій і перевірка цілісності залежностей ($C = 300$; R5: $0{,}6$). Результати жадібного вибору за (4.3) наведено в табл. 4.5.

Таблиця 4.5 — Послідовне обґрунтування заходів (тис. грн на рік)

Крок	C_j	ΔALE_j	ROSI	ALE після	Рішення
M2	350	800	1,29	2 820	впровадити
M3	150	300	1,00	2 520	впровадити
M5	300	480	0,60	2 040	впровадити
M1	500	612	0,22	1 428	впровадити
M4	700	488	−0,30	940	див. текст

Перші чотири заходи мають додатний ROSI; їхня сумарна вартість 1 300 тис. грн узгоджується з орієнтиром Гордона–Лоба $3\,620/e \approx 1\,332$, а сумарна вартість ризику (втрати плюс витрати) зменшується з 3 620 до $1\,428 + 1\,300 = 2\,728$. Захід M4 має від'ємний ROSI навіть першим кроком ($\Delta\text{ALE} = 696$, ROSI $\approx -0{,}01$), а після M1 і M2 його внесок у зниження R1 і R6 частково дублюється. Проте після чотирьох заходів найбільший залишковий ризик — R4 (600 з 1 428), а досвід DDoS-атак на державні ресурси (розділ 1) свідчить, що ARO $= 4$ може бути заниженим. Раціонально переглянути M4: базовий захист від DDoS у межах CDN і окремий пул для дорогих запитів (розділ 11) з перевіркою чутливості методом Монте-Карло.

4.8.3 Залишковий ризик і прийняття

Після M1, M2, M3, M5 залишкові очікувані втрати становлять 1 428 тис. грн на рік (39 % початкових). R2 і R3 (вади застосунку) знижено в 4–5 разів, а домінують R4 і R5, що залежать від зовнішніх сторін. Їх приймає керівництво відповідно до апетиту до ризику з щоквартальним переглядом: частоти R1 і R4 оновлюють за даними моніторингу за (4.7), а R5 — за повідомленнями про компрометацію пакетів (розділ 13). Залишковий ризик — явне управлінське рішення, а не побічний продукт заходів.

Висновки до розділу

Управління ризиками вебзастосунку ґрунтується на ISO/IEC 27005, ISO 31000, NIST SP 800-30 і функції управління CSF 2.0, але потребує функціонального визначення меж через потоки даних і межі довіри та врахування сеансів, бізнес-процесів, доступності й довіри як активів. STRIDE-per-element забезпечує повноту за властивостями безпеки, PASTA пов'язує сценарії з бізнес-цілями, LINDDUN охоплює приватність, а CWE, CAPEC і ATT&CK дають спільну мову проєктування, тестування і моніторингу. Дерево атак показало, що раціональний порушник обирає не найочевидніший шлях, а найдешевшу ваду логіки, і що після закриття очевидних шляхів пріоритет зміщується на захист сеансу й кінцевого пристрою. ROSI ранжує заходи з урахуванням уже впроваджених, модель Гордона–Лоба обмежує раціональний бюджет третиною очікуваних втрат,

а невизначеність подають розподілами і кривими перевищення втрат з байєсовим оновленням; порядкові матриці придатні лише для комунікації. У кейсі порталу державних послуг очікувані втрати знижено з 3,62 до 1,43 млн грн на рік за витрат 1,3 млн грн, а залишковий ризик змістився до доступності та ланцюга постачання (розділи 11 і 13).

Питання для самоперевірки

1. Опишіть етапи процесу управління ризиками за ISO/IEC 27005:2022 (рис. 4.1). Чим подієвий підхід до ідентифікації ризиків відрізняється від активо-орієнтованого?
2. Яку роль відіграє функція GOVERN у CSF 2.0? Хто встановлює критерії прийняття ризику?
3. Чому межі вебзастосунку визначають функціонально? Які його активи відсутні в переліку «дані–системи»?
4. Побудуйте DFD для інтернет-магазину з платіжним шлюзом і позначте межі довіри. Які потоки перетинають найбільше меж?
5. Для кожної категорії STRIDE (табл. 4.1) наведіть по одному прояву в API мобільного банкінгу і відповідний захід.
6. Обчисліть кількість кандидатних пар STRIDE-per-element (табл. 4.2) для DFD з 2 зовнішніми сутностями, 5 процесами, 12 потоками і 4 сховищами. Як скоротити перебір без втрати повноти?
7. Порівняйте STRIDE і PASTA за вхідними даними, результатом і трудомісткістю. Для яких застосунків виправдана PASTA?
8. Проаналізуйте за LINDDUN форму відновлення пароля, що повідомляє «користувача з такою адресою не знайдено».
9. За табл. 4.3 поясніть, чому техніка T1190 надто груба для моделювання загроз вебзастосунку і як це компенсує CAPEC.
10. Для дерева рис. 4.3 обчисліть p_G і c_G, якщо впроваджено лише FIDO2 для адміністраторів. Який сценарій максимізує U^* за (4.1) при $B = 5$?
11. За (4.2) обчисліть ALE для TEF $= 20$, Vuln $= 0{,}1$, первинних втрат 300 і вторинних 2 000 тис. грн з імовірністю 0,2. Яким має бути ефект заходу вартістю 250, щоб ROSI ≥ 1?
12. Поясніть результат Гордона–Лоба (4.4). Чому оптимальні інвестиції не обов'язково зростають з вразливістю набору?
13. Наведіть три вади категорії A06:2025, які не виявляє сканер вразливостей, і для кожної сформулюйте сценарій зловживання та вимогу ASVS.

14. Чому крива перевищення втрат (4.6) інформативніша за матрицю «імовірність × вплив»? Обчисліть $\mathrm{E}[L_\Sigma]$ для $\lambda = 1$ і інтервалу втрат $[100;\ 10\,000]$.
15. Дослідницьке: розробіть метод автоматичного оновлення моделі загроз у конвеєрі CI/CD (threat modeling as code), що виявляє зміни DFD, які перетинають межі довіри, і оцініть його повноту на відкритому вебзастосунку.
16. Дослідницьке: для кейсу підрозділу 4.8 замініть точкові ARO і SLE інтервалами, побудуйте криві перевищення втрат до і після заходів методом Монте-Карло та дослідіть чутливість рішення щодо M4 до невизначеності частоти DDoS-атак.

РОЗДІЛ 5

Захист транспортного рівня та інфраструктури доставки

Розділи 1–4 описали вебресурс як об'єкт захисту, нормативне поле, кількісні моделі та процес управління ризиками. Блок II переходить до механізмів захисту, і першим із них є транспортний рівень: до того як застосунок перевірить пароль чи права доступу, запит має дійти до правильного сервера, не бути прочитаним і зміненим у дорозі, а відповідь — повернутися від автентичного джерела. Цю задачу розв'язує зв'язка протоколу TLS, інфраструктури відкритих ключів (PKI) вебу, системи доменних імен (DNS) та інфраструктури доставки — мереж доставки вмісту (CDN) і зворотних проксі. Розділ розглядає TLS 1.3 і його ключовий розклад, уроки атак на попередні версії, PKI вебу з прозорістю сертифікатів і скороченням строків їх дії, проблему відкликання, механізм HSTS, протоколи HTTP/2 і HTTP/3 (QUIC), шифрування ClientHello, постквантовий перехід, загрози на рівні DNS та архітектурні ризики CDN і завершується кейсом модернізації транспортного рівня державного порталу.

5.1 Роль транспортного рівня в захисті вебресурсу

5.1.1 Що захищає TLS і чого він не захищає

Протокол TLS (Transport Layer Security) [255] забезпечує для каналу між клієнтом і сервером три властивості: *автентичність сервера* (і, за потреби, клієнта) на основі сертифіката X.509 [107]; *конфіденційність* переданих даних; *цілісність* — виявлення будь-якої зміни, вставлення, видалення чи переупорядкування записів. У термінах моделі порушника (розділ 1, підрозділ 1.5) TLS нейтралізує *мережевого* порушника Долева–Яо, який контролює канал, але не кінцеві точки. Від порушника, що діє як легітимний клієнт (ін'єкції, IDOR, заповнення облікових даних), TLS не захищає: зловмисний запит, загорнутий у бездоганний TLS, так само шкідливий. Не захищає він і дані на сервері, в журналах та резервних копіях, а також від компрометації самого сервера чи CDN, де TLS завершується.

Звідси дві практичні тези. По-перше, TLS — *необхідна*, але не достатня умова: без нього безглузді cookie сеансу, токени і паролі, бо їх перехоплює будь-хто на шляху; з ним залишаються всі вади рівня застосунку (розділи 6–10). По-друге, межі TLS-з'єднання не збігаються з межами довіри застосунку: між браузером і вебзастосунком зазвичай є кілька з'єднань

(браузер–CDN, CDN–балансувальник, балансувальник–сервіс), і кожне з них потребує окремого рішення про шифрування та автентифікацію.

5.1.2 Складові транспортного захисту вебресурсу

Захищеність каналу визначає не лише протокол, а ланцюг, у якому кожна ланка може бути найслабшою: (1) *DNS* — ім'я має бути перетворено на адресу правильного сервера; (2) *PKI* — сертифікат має видати лише той, хто контролює домен; (3) *протокол TLS* — узгоджені параметри мають бути стійкими, а реалізація — коректною; (4) *політика браузера* — HSTS, вимоги прозорості сертифікатів, перелік довірених центрів сертифікації; (5) *інфраструктура доставки* — CDN і проксі, які завершують TLS і повторно встановлюють з'єднання до джерела. Історично успішні атаки найчастіше вражали не криптографію, а стики: зниження версії протоколу, видачу сертифіката не тому власнику, захоплення забутого запису DNS, прямий доступ до джерела в обхід CDN. Структура розділу відтворює цей ланцюг.

NIST SP 800-52 Rev. 2 [202] вимагає для державних систем США підтримки TLS 1.3 і забороняє версії нижче 1.2; RFC 8996 [208] офіційно визнав TLS 1.0 і 1.1 застарілими. PCI DSS 4.0.1 [243] вимагає «стійкої криптографії» для передавання даних платіжних карток відкритими мережами, а вимоги до захисту інформації в державних системах України [32] — захисту інформації під час передавання каналами зв'язку із застосуванням засобів криптографічного захисту, що мають підтвердження відповідності.

5.2 Протокол TLS 1.3

5.2.1 Рукостискання

TLS 1.3 (RFC 8446, 2018) [255] став першою версією, спроєктованою з урахуванням формального аналізу: його ядро верифіковано в моделях Tamarin і ProVerif ще на етапі чернеток. Порівняно з TLS 1.2 протокол радикально скорочено (табл. 5.1): вилучено статичний обмін ключами RSA і статичний DH (а отже, кожне з'єднання має **пряму секретність**, forward secrecy), CBC-режими і RC4, стиснення, повторне узгодження, довільні групи DH; залишено лише AEAD-шифри.

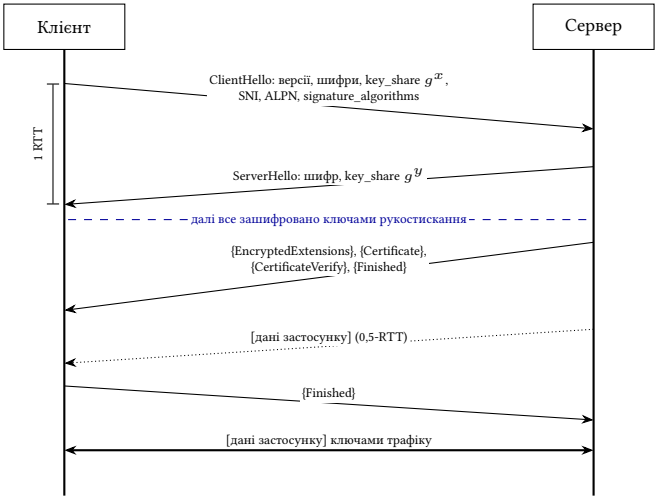


Рис. 5.1 — Повне рукостискання TLS 1.3 з ефемерним (EC)DHE (у фігурних дужках — повідомлення, зашифровані ключами рукостискання; у квадратних — ключами трафіку)

Клієнт у першому ж повідомленні ClientHello надсилає свою частку ефемерного обміну (key_share) для однієї або кількох груп, *вгадуючи* групу, яку обере сервер (рис. 5.1). Якщо вгадав, рукостискання займає один обхід (1-RTT); якщо ні, сервер відповідає HelloRetryRequest, і додається ще один обхід. Сертифікат сервера, на відміну від TLS 1.2, передається вже зашифрованим, що приховує від пасивного спостерігача ідентичність сервера — але не ім'я в розширенні SNI, яке лишається відкритим до впровадження ECH (підрозділ 5.6). Повідомлення CertificateVerify містить підпис сервера над хешем усього попереднього транскрипту, а Finished — MAC над транскриптом; підміна будь-якого поля ClientHello (наприклад, вилучення стійких шифрів) змінює транскрипт і зриває рукостискання. Так TLS 1.3 захищається від атак *зниження параметрів* (downgrade), які в TLS 1.2 і раніше були основним класом практичних атак. Додатково, якщо сервер підтримує TLS 1.3, а з'єднання узгоджено на нижчій версії, він вписує в останні байти поля ServerHello.random спеціальне значення-маркер, і клієнт з підтримкою 1.3 виявляє спробу зниження версії.

5.2.2 Ключовий розклад і пряма секретність

Усі ключі TLS 1.3 виводяться з трьох секретів функцією HKDF [255], що складається з кроків HKDF-Extract (вилучення ентропії) і HKDF-Expand (розгортання з міткою):

$$\begin{aligned} \mathrm{ES} = \mathrm{Extract}(0,\ \mathrm{PSK}), \qquad \mathrm{HS} = \mathrm{Extract}\big(\mathrm{Derive}(\mathrm{ES}),\ g^{xy}\big), \\ \mathrm{MS} = \mathrm{Extract}\big(\mathrm{Derive}(\mathrm{HS}),\ 0\big), \\ k_{\text{тр}} = \mathrm{Expand}\big(\mathrm{MS},\ \text{мітка} \,\|\, H(\text{транскрипт}),\ L\big), \end{aligned} \tag{5.1}$$

де ES, HS, MS — ранній секрет, секрет рукостискання і головний секрет; PSK — попередньо узгоджений ключ (для відновлення сеансу; у повному рукостисканні — нулі); g^{xy} — спільний секрет ефемерного (EC)DHE; $\mathrm{Derive}(\cdot)$ — розгортання з фіксованою міткою; $H(\text{транскрипт})$ — хеш усіх повідомлень рукостискання; L — довжина ключа. Оскільки x і y знищуються після рукостискання, компрометація довгострокового ключа підпису сервера в майбутньому не дає змоги розшифрувати записаний трафік — це і є пряма секретність. У TLS 1.2 зі статичним обміном RSA викрадений закритий ключ сервера розкривав *увесь* колись записаний трафік, що робило стратегію «записати зараз, розшифрувати пізніше» (harvest now, decrypt later) прямою загрозою; у TLS 1.3 вона актуальна лише щодо майбутнього квантового порушника (підрозділ 5.6). Прив'язка ключів до хешу транскрипту забезпечує, що обидві сторони виводять однакові ключі лише за однакового бачення рукостискання.

Набори шифрів TLS 1.3 визначають лише AEAD-алгоритм і хеш для HKDF; механізм обміну (група) і підпис узгоджуються окремими розширеннями. Обов'язковий набір — TLS_AES_128_GCM_SHA256; поширені також TLS_AES_256_GCM_SHA384 і TLS_CHACHA20_POLY1305_SHA256, що ефективніший на пристроях без апаратного прискорення AES. Групи: x25519, secp256r1, secp384r1 і гібридні постквантові (підрозділ 5.6); підписи: ECDSA, RSA-PSS, EdDSA.

Таблиця 5.1 — Порівняння TLS 1.2 і TLS 1.3

Аспект	TLS 1.2 (RFC 5246)	TLS 1.3 (RFC 8446)
Обмін ключами	RSA, DH, ECDHE (статичні — без прямої секретності)	лише ефемерні (EC)DHE або PSK з (EC)DHE
Шифрування	CBC+HMAC, RC4, AEAD	лише AEAD (AES-GCM, ChaCha20-Poly1305, AES-CCM)
Обходи (повне)	2-RTT	1-RTT; 0-RTT при відновленні
Сертифікат	відкритим текстом	зашифрований
Захист від зниження	частковий (Finished)	підпис транскрипту, маркер у random
Повторне узгодження, стиснення	є (атаки Renegotiation, CRIME)	вилучено
Узгодження параметрів	набір шифрів визначає все	набір + окремі розширення груп і підписів

5.2.3 Затримка встановлення з'єднання і 0-RTT

Кількість обходів безпосередньо впливає на швидкодію, а отже, на готовність власників вмикати шифрування повсюдно. Час до надсилання першого байта даних застосунку

$$T_{\text{перш}} = \left(n_{\text{тр}} + n_{\text{TLS}}\right) \cdot \text{RTT} + T_{\text{обч}}, \tag{5.2}$$

де $n_{\text{тр}}$ — обходи транспортного протоколу (1 для TCP, 0 для QUIC, де транспортне й криптографічне рукостискання суміщені); n_{TLS} — обходи TLS (2 для TLS 1.2, 1 для TLS 1.3, 0 для 0-RTT); $T_{\text{обч}}$ — час криптографічних обчислень. За $\text{RTT} = 80$ мс (мобільний клієнт, сервер в іншій країні) маємо: TCP + TLS 1.2 — 240 мс, TCP + TLS 1.3 — 160 мс, QUIC з TLS 1.3 — 80 мс, QUIC з 0-RTT — практично 0 (дані йдуть у першому пакеті). Різниця в 160 мс на кожне нове з'єднання відчутна для користувача, тож TLS 1.3 і QUIC зняли історичний аргумент «шифрування уповільнює сайт».

Режим **0-RTT** (early data) дає змогу клієнту, що вже мав сеанс із сервером, надіслати дані разом із ClientHello, шифруючи їх ключем, виведеним з PSK. Ціна — дві слабкості [255]: ранні дані не мають прямої секретності щодо ключа квитка сеансу і, головне, можуть бути *повторені* (replay) порушником, що перехопив пакет. Тому 0-RTT допустимий лише для *ідемпотентних* запитів (GET статичного вмісту), а сервер або CDN має відхиляти в ранніх даних запити, що змінюють стан (оформлення платежу, зміна пароля), чи повертати код 425 Too Early. Для вебзастосунку з бізнес-операціями безпечне налаштування — вимкнути 0-RTT на джерелі й дозволити його лише для статичного вмісту на CDN.

5.2.4 Уроки атак на попередні версії

Історія атак на SSL/TLS (табл. 5.2) — найкраще обґрунтування дизайнерських рішень TLS 1.3 і водночас нагадування, що протокол захищає лише в коректній реалізації й конфігурації.

Таблиця 5.2 — Резонансні атаки на SSL/TLS та їхні уроки

Атака	Рік	Суть	Урок / відповідь у TLS 1.3
SSL stripping	2009	перехоплення першого HTTP-запиту і проксування без TLS [196]	HSTS і preload (підрозділ 5.4)
BEAST, Lucky13	2011–2013	атаки на CBC і порядок MAC-then-encrypt	лише AEAD
CRIME, BREACH	2012–2013	витік секретів через довжину стисненого шифротексту	стиснення вилучено; BREACH — на рівні HTTP
Heartbleed	2014	вада реалізації OpenSSL: читання пам'яті сервера [117]	безпечні мови, фазинг, ротація ключів
POODLE	2014	зниження до SSL 3.0 і атака на доповнення	заборона застарілих версій
FREAK, Logjam	2015	зниження до «експортних» ключів 512 біт; передобчислення для поширених груп DH [57]	лише стандартні групи, без експортних шифрів
DROWN	2016	ключ, спільний із сервером SSLv2, дозволяє розшифрувати TLS	ізоляція ключів між протоколами
ROBOT	2017	оракул доповнення PKCS#1 v1.5 у RSA-обміні (атака Блейхенбахера)	RSA-обмін ключами вилучено

Урок Heartbleed окремий: атака вражала не протокол, а реалізацію — переповнення буфера в розширенні heartbeat давало змогу читати по 64 КБ пам'яті сервера, зокрема закриті ключі і cookie сеансів. Дослідження [117] показало, що через тиждень після оприлюднення вразливими лишалися близько 11 % найпопулярніших HTTPS-сайтів, а ключі після виправлення перевипустила лише незначна частка постраждалих. Логічна вимога після інциденту з можливим витоком ключа — не лише оновити бібліотеку, а й перевипустити сертифікат і відкликати старий (підрозділ 5.3). Атака Logjam [57] продемонструвала ще один урок: передобчислення дискретного логарифма для однієї поширеної 1024-бітної групи DH дає змогу пасивно розшифровувати значну частку трафіку всіх серверів, що її використовують. Отже, *монокультура параметрів* — системний ризик, а стандартизовані групи мають бути достатньо великими.

5.3 Інфраструктура відкритих ключів вебу

5.3.1 Модель довіри і її слабкості

Автентичність сервера в TLS ґрунтується на сертифікаті X.509 [107], виданому центром сертифікації (ЦС, Certification Authority, CA), якому довіряє браузер. Модель має фундаментальну слабкість: будь-який із понад сотні довірених кореневих ЦС і їхніх підпорядкованих може видати сертифікат на *будь-який* домен. Стійкість системи визначає найслабший ЦС. Інциденти це підтверджують: компрометація нідерландського ЦС DigiNotar (2011) призвела до видачі підробленого сертифіката для доменів Google, який використовували для перехоплення трафіку користувачів Gmail в Ірані, і ЦС був вилучений з усіх браузерів; поступова відмова від довіри до ієрархії Symantec (2017–2018) і недовіра Chrome до нових сертифікатів Entrust (з листопада 2024 р.) стали наслідком систематичних порушень правил видачі.

Правила для публічних ЦС встановлює **CA/Browser Forum** у Базових вимогах (Baseline Requirements) [82], а виконання забезпечують програми кореневих сертифікатів браузерів і операційних систем. Ключовий етап видачі — **перевірка контролю домену** (domain control validation, DCV): ЦС переконується, що заявник контролює домен, за допомогою запису DNS, файлу на вебсервері чи листа на адресу адміністратора. Сертифікати рівня DV (Domain Validated) засвідчують лише контроль домену; OV і EV додають перевірку організації, але браузери прибрали особливу індикацію EV, оскільки користувачі її не помічали.

5.3.2 Прозорість сертифікатів

Прозорість сертифікатів (Certificate Transparency, CT; RFC 6962 [186]) не запобігає помилковій видачі, але робить її *виявлюваною*. Кожен сертифікат перед видачею (як передсертифікат) заноситься до кількох публічних журналів лише з дописуванням, побудованих на деревах Меркла; журнал повертає підписану позначку часу (Signed Certificate Timestamp, SCT), яку вбудовують у сертифікат. Браузери (Chrome з 2018 р., Safari) приймають сертифікат лише з достатньою кількістю SCT від незалежних журналів. Моніторинг журналів (рис. 5.2) дозволяє власнику домену виявити будь-який сертифікат, виданий на його ім'я.

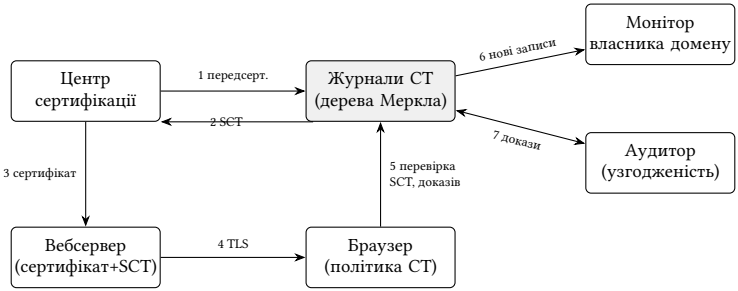


Рис. 5.2 — Екосистема прозорості сертифікатів: журнали, монітори, аудитори

Ефективність CT оцінюють двома величинами: розміром доказу включення, що визначає накладні витрати перевірки, та ймовірністю виявлення помилкової видачі:

$$|\pi_{\text{вкл}}| = \lceil \log_2 n \rceil \cdot h, \qquad P_{\text{вияв}} = 1 - \prod_{i=1}^{k} \bigl(1 - q_i\bigr), \tag{5.3}$$

де n — кількість записів журналу; h — довжина хешу (32 байти для SHA-256); k — кількість журналів, до яких занесено сертифікат; q_i — імовірність того, що моніторинг журналу i своєчасно виявить запис для домену. Для журналу з $n = 2 \cdot 10^9$ записів доказ містить 31 хеш, тобто 992 байти, — перевірка залишається дешевою незалежно від розміру журналу. Якщо власник домену моніторить лише через одного агрегатора з $q = 0{,}9$, а сертифікат має SCT від $k = 2$ журналів, кожен з яких цим агрегатором охоплюється, $P_{\text{вияв}} = 0{,}99$. Ключова умова — щоб моніторинг узагалі був: CT робить помилкову видачу виявлюваною лише для тих, хто дивиться. Для державних доменів доцільний централізований моніторинг усіх сертифікатів у зоні `gov.ua` з передаванням аномалій до CERT-UA [9]. Побічний ефект CT — розкриття імен внутрішніх піддоменів (`test.`, `admin.`), які порушники використовують для розвідки, тому для внутрішніх сервісів застосовують wildcard-сертифікати або приватний ЦС.

Додатковий превентивний механізм — запис **CAA** у DNS (RFC 8659 [145]), що перелічує ЦС, яким дозволено видавати сертифікати для домену; з 2017 р. перевірка CAA обов'язкова для публічних ЦС. Розширення `accounturi` обмежує видачу конкретним обліковим записом ACME, що захищає від видачі на ім'я організації сторонньою особою навіть через той самий ЦС.

5.3.3 Строки дії сертифікатів і автоматизація ACME

Максимальний строк дії публічного TLS-сертифіката послідовно скорочувався: 5 років до 2015 р., 39 місяців, 825 днів (2018), 398 днів (2020). Бюлетень CA/Browser Forum SC-081 (2025) [82] встановив подальший графік (табл. 5.3), за яким до 2029 р. строк дії скоротиться до 47 днів, а період повторного використання результатів перевірки контролю домену — до 10 днів.

Таблиця 5.3 — Графік скорочення строків дії TLS-сертифікатів (SC-081)

Дата набрання чинності	Макс. строк дії	Повторне використання DCV
до 15.03.2026	398 днів	398 днів
з 15.03.2026	200 днів	200 днів
з 15.03.2027	100 днів	100 днів
з 15.03.2029	47 днів	10 днів

Обґрунтування скорочення — обмеження *вікна експлуатації* скомпрометованого ключа чи помилково виданого сертифіката в умовах, коли відкликання працює погано (наступний підрозділ). Якщо ключ компрометується в рівномірно розподілений момент строку дії V, а відкликання ефективно доходить до клієнтів лише з імовірністю ρ за час T_r, то очікуване вікно

$$\mathrm{E}[W] = \rho\, T_r + (1-\rho)\,\frac{V}{2}. \tag{5.4}$$

За $V = 398$ днів, $\rho = 0{,}3$, $T_r = 2$ дні маємо $\mathrm{E}[W] \approx 0{,}6 + 139{,}3 \approx 140$ днів; за $V = 47$ — близько 17 днів, тобто у 8 разів менше без жодної зміни механізму відкликання. Наслідок для операцій: ручне оновлення сертифікатів раз на півтора місяця на сотнях доменів неможливе, тож скорочення строків де-факто примушує до автоматизації за протоколом **ACME** (RFC 8555 [69]), який реалізують Let's Encrypt та інші ЦС. ACME автоматизує перевірку контролю домену (виклики http-01, dns-01, tls-alpn-01), видачу й оновлення. Ризики переносяться на автоматизацію: ключ облікового запису ACME і доступ до API DNS (для dns-01) стають цінними активами, а збій оновлення — причиною масової недоступності. Звідси вимоги: моніторинг строків дії всіх сертифікатів, оновлення за третину строку до закінчення, сповіщення про невдалі спроби.

5.3.4 Проблема відкликання

Механізми відкликання — списки відкликаних сертифікатів (CRL) [107] та онлайн-протокол статусу OCSP — у вебі історично працювали погано. Браузери застосовують політику *soft-fail*: якщо сервер OCSP недоступний, сертифікат приймається, а порушник, що здатний перехоплювати трафік, здатний і заблокувати запит OCSP. До того ж OCSP розкриває ЦС, які сайти

відвідує користувач. Механізм *OCSP stapling* (сервер сам додає свіжу відповідь OCSP до рукостискання) усуває проблему приватності, але не soft-fail.

Відповіддю стали дві тенденції. Браузери перейшли до *централізовано зібраних* списків відкликання, що доставляються з оновленнями (CRLSets у Chrome, CRLite у Firefox — стиснене подання всіх відкликань на основі каскаду фільтрів Блума). ЦС відмовляються від OCSP: Let's Encrypt припинив підтримку OCSP у 2025 р., перейшовши на CRL [189], і запропонував сертифікати строком дії близько шести днів, для яких відкликання практично не потрібне. Отже, архітектура відкликання вебу зміщується від «перевіряй статус кожного сертифіката» до «короткий строк дії плюс централізовані списки для екстрених випадків», що узгоджується з моделлю (5.4): за $V \to 0$ вікно визначає лише швидкість перевипуску.

5.4 HSTS і захист від зниження до HTTP

5.4.1 Атака SSL stripping

Найпростіша і досі актуальна атака на HTTPS не ламає TLS, а обходить його. Користувач вводить в адресному рядку `portal.gov.ua` без схеми, браузер надсилає запит HTTP, сервер відповідає перенаправленням на HTTPS. Порушник на шляху (публічна мережа Wi-Fi, скомпрометований маршрутизатор) перехоплює перший HTTP-запит, сам встановлює HTTPS-з'єднання із сервером і віддає користувачу сторінку по HTTP, переписуючи посилання [196]. Користувач бачить знайомий сайт; відсутність замка помічають одиниці. Перенаправлення з HTTP на HTTPS тут не допомагає, бо порушник саме його і перехоплює.

5.4.2 Механізм HSTS і список попереднього завантаження

Заголовок **HSTS** (HTTP Strict Transport Security, RFC 6797 [150]) `Strict-Transport-Security: max-age=31536000; includeSubDomains; preload` наказує браузеру протягом `max-age` секунд звертатися до домену (і, з `includeSubDomains`, до всіх піддоменів) лише по HTTPS, автоматично переписуючи `http://` на `https://` ще до надсилання запиту, і забороняє користувачу ігнорувати помилки сертифіката. Слабке місце — *довіра при першому використанні* (trust on first use): до першого успішного HTTPS-відвідування браузер про політику не знає. Його закриває **список попереднього завантаження** HSTS (preload list), вбудований у браузери; для включення домен має віддавати HSTS з `max-age` не менше року, `includeSubDomains` і `preload`.

Ефект HSTS кількісно оцінюють числом відвідувань, вразливих до SSL stripping. Нехай користувач відвідує сайт як пуассонівський потік з інтенсивністю r (відвідувань на добу), `max-age` дорівнює M діб; політика втрача-

ється, лише якщо перерва між відвідуваннями перевищить M (імовірність e^{-rM}). Тоді очікувана кількість вразливих відвідувань за період T діб

$$N_{\text{вразл}} = \begin{cases} rT \cdot f_{\text{HTTP}}, & \text{без HSTS,} \\ 1 + rT\, e^{-rM}, & \text{з HSTS,} \\ 0, & \text{preload,} \end{cases} \tag{5.5}$$

де f_{HTTP} — частка відвідувань, що починаються з HTTP (введення без схеми, старі закладки, посилання). Для $r = 0{,}1$ (раз на 10 днів), $T = 365$, $f_{\text{HTTP}} = 0{,}4$: без HSTS — 14,6 вразливих відвідувань на рік; з HSTS і $M = 365$ — практично одне (перше); з HSTS і $M = 1$ доба — $1 + 36{,}5\, e^{-0{,}1} \approx 34$ потенційно вразливих відвідувань (фактично вразливі лише ті з них, що починаються з HTTP, тобто близько 14 — стільки ж, скільки без HSTS). Висновок: короткий `max-age` майже знецінює HSTS, а для державних доменів правильна ціль — preload усієї зони відомства.

Впровадження HSTS з `includeSubDomains` вимагає інвентаризації: будь-який піддомен, що працює лише по HTTP (стара система, принтер, внутрішній сервіс з публічним ім'ям), після цього стане недоступним. Тому практичний порядок такий: інвентаризація піддоменів (зокрема за журналами CT); переведення всіх на HTTPS; HSTS з малим `max-age` і поступове збільшення; `includeSubDomains`; preload. Вилучення домену зі списку preload триває місяці, тож це рішення практично незворотне.

5.4.3 Змішаний вміст і cookie

Сторінка, завантажена по HTTPS, що підключає скрипти чи стилі по HTTP (*активний змішаний вміст*), повністю знецінює TLS: порушник підмінює скрипт і отримує контроль над сторінкою. Сучасні браузери блокують активний змішаний вміст і автоматично переводять пасивний (зображення) на HTTPS. Директива CSP `upgrade-insecure-requests` (розділ 6) допомагає під час міграції. Cookie сеансу мають атрибут `Secure`, щоб ніколи не передаватися по HTTP, і префікс `__Host-`, що забороняє їх встановлення з піддоменів і по HTTP [70]; без цього навіть за HSTS залишаються сценарії перезапису cookie через HTTP-піддомен (розділ 8).

5.5 HTTP/2 і HTTP/3: безпекові аспекти

5.5.1 HTTP/2

HTTP/2 мультиплексує багато потоків в одному TCP-з'єднанні, стискає заголовки (HPACK) і на практиці використовується лише поверх TLS (браузери не підтримують HTTP/2 без шифрування). Мультиплексування створило нові класи атак на доступність: асиметрію між дешевим для клієнта і дорогим для сервера створенням потоку. Атака *HTTP/2 Rapid Reset* (CVE-

2023-44487, 2023) полягала в циклічному відкритті і негайному скасуванні потоків (кадр RST_STREAM): сервер витрачав ресурси на обробку кожного запиту, а обмеження на кількість одночасних потоків не спрацьовувало, оскільки скасовані потоки не враховуються. Атака дала рекордні на той час сотні мільйонів запитів за секунду і детально розглядається в розділі 11. Інший клас — десинхронізація запитів (request smuggling) при трансляції HTTP/2 у HTTP/1.1 на проксі: різне тлумачення меж запиту фронтендом і бекендом дає змогу «підкласти» запит у чуже з'єднання. Захист — наскрізний HTTP/2 або суворе нормалізування на проксі.

5.5.2 QUIC і HTTP/3

QUIC (RFC 9000 [169]) переносить транспорт у простір користувача поверх UDP і вбудовує TLS 1.3 (RFC 9001 [285]): шифруються не лише дані, а й більшість заголовків транспортного рівня, зокрема номери пакетів. HTTP/3 (RFC 9114 [80]) працює поверх QUIC. Безпекові наслідки: (1) middlebox-пристрої (системи запобігання вторгненням, DLP) втрачають видимість, що змушує організації або завершувати QUIC на проксі, або блокувати UDP/443 і повертати клієнтів на TCP; (2) ідентифікатори з'єднань дозволяють міграцію між мережами (Wi-Fi $\rightarrow$ мобільна) без нового рукостискання, але створюють ризик відстеження, тому протокол передбачає їх зміну; (3) оскільки UDP не має рукостискання, сервер QUIC до перевірки адреси клієнта може бути використаний для *підсилення* відбитих атак. RFC 9000 обмежує цей ризик правилом проти підсилення:

$$B_{\text{сер}} \le 3 \cdot B_{\text{кл}}, \qquad B_{\text{кл}} \ge 1200 \text{ байт (дод. Initial)} \;\Rightarrow\; A_{\text{QUIC}} \le 3, \tag{5.6}$$

де $B_{\text{сер}}$ — обсяг, який сервер може надіслати на неперевірену адресу; $B_{\text{кл}}$ — обсяг, отриманий від неї; A — коефіцієнт підсилення (розділ 3, підрозділ 3.5). Клієнт зобов'язаний доповнювати перший пакет Initial щонайменше до 1200 байт, тож навіть після підробки адреси жертви порушник отримує підсилення не більше 3 — порівняно з десятками для відкритих DNS-резолверів чи сотнями для memcached. Для перевірки адреси сервер може надіслати маркер Retry, що коштує додаткового обходу; його вмикають під навантаженням.

5.6 Шифрування ClientHello і постквантовий перехід

5.6.1 Encrypted Client Hello

Навіть з TLS 1.3 спостерігач бачить ім'я сайту в розширенні SNI. Це використовують для цензури і блокування, а також для профілювання. **Encrypted Client Hello** (ECH, RFC 9849 [257]) шифрує «внутрішній» ClientHello з реальним ім'ям відкритим ключем, опублікованим у записі DNS типу HTTPS;

«зовнішній» ClientHello містить лише ім'я фронтального сервера (CDN), спільне для багатьох сайтів. Захист ефективний лише в «натовпі»: якщо за фронтальним ім'ям стоїть один сайт, ECH нічого не приховує. Для організацій ECH має подвійний наслідок: зростає приватність користувачів, але корпоративні засоби фільтрації трафіку за SNI втрачають ефективність. Поширений компроміс — ECH на публічних ресурсах і явне проксування в корпоративній мережі. ECH змістовний лише в поєднанні з шифрованим DNS (DoH, DoT), бо інакше ім'я розкриває DNS-запит.

5.6.2 Постквантова загроза і гібридний обмін ключами

Достатньо потужний квантовий комп'ютер алгоритмом Шора зламає RSA, DH і ECDH, на яких ґрунтується обмін ключами і підписи TLS [73]. Загрози відрізняються за терміновістю. Для *конфіденційності* діє стратегія «записати зараз, розшифрувати пізніше»: трафік, записаний сьогодні, буде розшифровано після появи квантового комп'ютера, тож дані з довгим строком цінності (персональні, медичні, державні) потребують захисту вже зараз. Для *автентичності* загроза актуальна лише з моменту появи квантового комп'ютера, бо підробити підпис у минулому неможливо.

NIST стандартизував механізм інкапсуляції ключа ML-KEM (FIPS 203 [219]) та підпис ML-DSA (FIPS 204 [218]). Для TLS обрано *гібридний* підхід: спільний секрет обчислюють з класичного (X25519) і постквантового (ML-KEM-768) компонентів, тож зв'язка стійка, доки стійкий хоча б один з них [182]. Гібридна група X25519MLKEM768 стандартизована в RFC 10024 і з 2024–2025 рр. увімкнена за замовчуванням у Chrome, Firefox, Safari та на провідних CDN, тож значна частка вебтрафіку вже захищена від «записати зараз». Ціна — розмір (табл. 5.4).

Таблиця 5.4 — Розміри ключів і шифротекстів (байти)

Механізм	Відкр. ключ	Шифротекст / підпис	Застосування
X25519	32	32	обмін, класичний
ML-KEM-768 (FIPS 203)	1 184	1 088	обмін, ПК
X25519MLKEM768	1 216	1 120	обмін, гібрид
ECDSA P-256	64	64–72	підпис, класичний
RSA-2048	256	256	підпис, класичний
ML-DSA-44 (FIPS 204)	1 312	2 420	підпис, ПК
ML-DSA-65 (FIPS 204)	1 952	3 309	підпис, ПК

Збільшення розміру ClientHello оцінюють кількістю пакетів, на які він розбивається:

$$N_{\text{пак}} = \left\lceil \frac{S_0 + \sum_g s_g}{\text{MTU} - h} \right\rceil, \tag{5.7}$$

де S_0 — розмір ClientHello без часток обміну (типово 300–500 байт з SNI, ALPN, ECH); s_g — розміри часток для груп g; h — заголовки IP і TCP (40–60 байт). З лише X25519 ($S_0 = 450$, $s = 32$) $N_{\text{пак}} = 1$; з X25519MLKEM768 і резервною X25519 ($450 + 1\,216 + 32 = 1\,698$ байт) за MTU 1500 — 2 пакети. Сам по собі другий пакет майже не впливає на затримку, але виявив приховані вади реалізацій: деякі мережеві пристрої й сервери некоректно обробляли ClientHello, розбитий на кілька сегментів, що спричинило збої з'єднань під час розгортання. Це конкретний приклад *окостеніння протоколу* (ossification) і аргумент за механізм GREASE — випадкові значення в полях, що змушують реалізації коректно ігнорувати невідоме.

Постквантові *підписи* в сертифікатах — складніша задача: ланцюг із сертифіката сервера, проміжного сертифіката, двох-трьох SCT і підпису CertificateVerify з ML-DSA-44 додасть понад 10 КБ до рукостискання, що помітно для мобільних клієнтів і порушує межі початкового вікна перевантаження TCP. Тому для підписів досліджують альтернативи (компактніші схеми, Merkle Tree Certificates, скорочення ланцюга), а перехід у вебі очікується пізніше, ніж для обміну ключами. Для власника вебресурсу практичні кроки вже сьогодні: увімкнути гібридний обмін на CDN і серверах (сучасні бібліотеки підтримують його з коробки), провести інвентаризацію криптографії (cryptographic inventory) і забезпечити *криптографічну гнучкість* — можливість замінити алгоритм конфігурацією, а не переписуванням коду.

5.6.3 Національна криптографія

В Україні криптографічний захист державних інформаційних ресурсів та кваліфіковані електронні довірчі послуги ґрунтуються на національних стандартах: цифровому підписі на еліптичних кривих ДСТУ 4145-2002 [12], блоковому шифрі «Калина» ДСТУ 7624:2014 [13] і функції гешування «Купина» ДСТУ 7564:2014 [14] [6]. Браузери ці алгоритми в TLS не підтримують, тож публічні державні вебресурси використовують міжнародні алгоритми й публічні ЦС для TLS, а національна криптографія застосовується на прикладному рівні — для кваліфікованого електронного підпису (розділ 8) і захищених каналів між державними системами. Гармонізація з європейськими вимогами в межах eIDAS 2.0 [251], зокрема щодо кваліфікованих сертифікатів автентифікації вебсайтів (QWAC), — окремий напрям інтеграції, у якому позиції регуляторів ЄС і розробників браузерів тривалий час розходилися щодо обов'язковості довіри до QWAC.

5.7 DNS як ланка ланцюга довіри

5.7.1 Атаки на розв'язання імен

DNS визначає, на яку адресу браузер відправить запит; компрометація цієї ланки обходить усі механізми рівня застосунку. Основні сценарії: *отруєн-*

ня кешу резолвера підробленими відповідями (атака Камінського 2008 р. і її варіанти); *захоплення облікового запису в реєстратора* чи провайдера DNS і зміна делегування — так у 2018–2019 рр. в кампанії DNSpionage перенаправлялася пошта й вебтрафік урядових установ низки країн, з отриманням легітимних сертифікатів на перенаправлені домени; *BGP-перехоплення* префікса DNS-сервера чи вебсервера. Атаки на делегування особливо небезпечні, бо після перенаправлення домену порушник проходить перевірку контролю домену і отримує *легітимний* сертифікат — тут рятує лише моніторинг CT (рівняння (5.3)).

DNSSEC (RFC 4033 [66]) підписує записи зони і дозволяє резолверу перевірити їх автентичність від кореня; він захищає від отруєння кешу, але не від захоплення облікового запису в реєстратора, бо порушник змінює і ключі. Захист облікових записів реєстратора (фішингостійка MFA, блокування змін на рівні реєстру — registry lock) є найдешевшим і найефективнішим заходом для критичних доменів. Шифрований DNS (DoH, DoT) захищає приватність і цілісність каналу клієнт–резолвер, але не автентичність даних зони.

5.7.2 Захоплення піддоменів

Захоплення піддомену (subdomain takeover) використовує «висячі» записи DNS: запис `CNAME` вказує на ресурс хмарного провайдера (сховище об'єктів, платформу хостингу), який власник уже видалив, але запис лишився. Порушник реєструє ресурс з тим самим ім'ям у провайдера і отримує контроль над вмістом піддомену організації — із можливістю отримати сертифікат, встановлювати cookie для батьківського домену (якщо атрибут Domain задано широко) і обходити CSP та CORS, що довіряють піддоменам. Масштабне дослідження [191] виявило сотні висячих записів у доменах провідних організацій і університетів. Ризик зростає з кількістю записів, що вказують на зовнішні сервіси, і часом їхнього існування:

$$\mathrm{E}[N_{\text{вис}}] = \sum_{j=1}^{m} \left(1 - e^{-\mu_j t}\right), \qquad P_{\text{захопл}} = 1 - \prod_{j} \left(1 - \left(1 - e^{-\mu_j t}\right) a_j\right), \quad (5.8)$$

де m — кількість записів на зовнішні ресурси; μ_j — інтенсивність виведення ресурсу j з експлуатації без видалення запису (за рік); t — час; a_j — імовірність, що порушник виявить і зареєструє звільнене ім'я (для провайдерів, які дозволяють реєструвати довільні імена, близька до 1). Для $m = 120$ записів з $\mu = 0{,}02$ на рік через 3 роки $\mathrm{E}[N_{\text{вис}}] \approx 120 \cdot 0{,}058 \approx 7$; за $a_j = 0{,}3$ імовірність хоча б одного захоплення $\approx 1 - (1 - 0{,}0175)^{120} \approx 0{,}88$. Отже, без процесу, що пов'язує видалення хмарного ресурсу з видаленням запису DNS, захоплення піддомену у великій організації — майже неминучість.

Заходи: інвентаризація зони і зовнішніх цілей, автоматична перевірка висячих записів, порядок «спершу запис DNS, потім ресурс», інфраструктура як код для DNS (розділ 12).

5.8 CDN і зворотні проксі

5.8.1 Архітектура і перенесення довіри

CDN і зворотні проксі завершують TLS на краю мережі, кешують вміст, фільтрують запити (WAF, протидія ботам) і поглинають DDoS-атаки (розділ 11). З погляду безпеки це *перенесення довіри*: провайдер CDN бачить увесь трафік у відкритому вигляді, зокрема паролі й токени, і має закриті ключі (або делегований доступ до підпису). Отже, CDN — частина довіреної обчислювальної бази вебресурсу і постачальник у сенсі ризиків ланцюга постачання (розділи 4 і 13). Варіанти зменшення довіри: власні ключі в апаратному модулі з віддаленим підписом (keyless SSL), розділення доменів статичного вмісту і чутливих API, наскрізне шифрування чутливих полів на рівні застосунку.

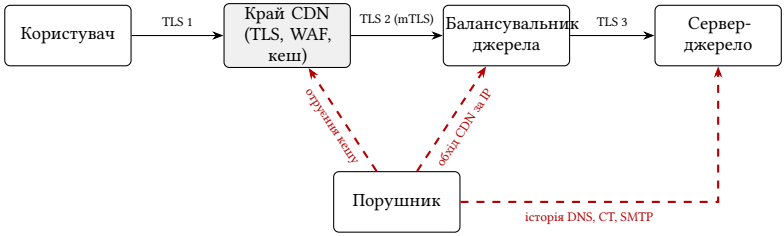


Рис. 5.3 — Інфраструктура доставки: три TLS-з'єднання і типові шляхи атак (штрихові лінії)

5.8.2 Обхід CDN і розкриття джерела

Захист CDN від DDoS і WAF діє лише тоді, коли весь трафік іде через CDN. Якщо порушник знає IP-адресу сервера-джерела, він атакує її напряму (рис. 5.3). Дослідження [292] показало, що для значної частки сайтів за хмарними провайдерами безпеки адресу джерела вдається встановити через історію записів DNS, незахищені піддомени (`direct.`, `ftp.`, `mail.`), заголовки вихідної пошти, сертифікати, видані на IP джерела, і сканування адресного простору з перевіркою відповіді на запит з потрібним заголовком Host. Заходи: дозвіл вхідних з'єднань на джерелі лише з адрес CDN (списки опубліковані провайдерами) або, надійніше, лише через автентифіковане з'єднання — взаємний TLS (mTLS) із сертифікатом CDN чи тунель, ініційований з боку джерела; зміна адреси джерела після переходу під CDN; відсутність прямих записів DNS на джерело.

5.8.3 Атаки на кеш

Кеш створює два класи атак. *Отруєння кешу* (web cache poisoning): порушник надсилає запит з некешованим, але відображуваним у відповіді заголовком (наприклад, `X-Forwarded-Host`), і шкідлива відповідь кешується та роздається всім користувачам. *Обман кешу* (web cache deception) [137]: порушник змушує жертву відкрити URL на кшталт `/account/profile/x.css`; застосунок ігнорує суфікс і віддає особисту сторінку, а CDN за розширенням вважає відповідь статичною і кешує її — після чого порушник читає чужі дані з кешу. Корінь обох — розбіжність між тим, як CDN і застосунок тлумачать запит. Заходи: явні правила кешування за типом вмісту і заголовками `Cache-Control` від застосунку (`private`, `no-store` для персоналізованих відповідей), ключ кешу, що включає всі впливові заголовки, суворе тлумачення шляхів у застосунку, відмова від кешування за розширенням.

5.8.4 Ланцюг з'єднань і рівні шифрування

Між краєм CDN і джерелом трафік має бути зашифровано і автентифіковано так само, як між користувачем і краєм: інакше порушник у мережі провайдера чи в хмарі бачить усе. Поширена помилка — режим «гнучкого» TLS, коли CDN звертається до джерела по HTTP, а користувач бачить замок. Правильна конфігурація — TLS до джерела з перевіркою сертифіката (повний суворий режим) і mTLS, щоб джерело приймало з'єднання лише від CDN. Усередині кластера (балансувальник–сервіси) шифрування забезпечує сервісна сітка (розділи 10 і 12) відповідно до принципу нульової довіри [258]: розташування в «внутрішній» мережі не є підставою для довіри.

5.9 Кейс: модернізація транспортного рівня державного порталу

5.9.1 Вихідний стан

Портал державних послуг з архітектурою, наведеною на рис. 4.2 (розділ 4), обслуговує 30 доменних імен у зоні відомства. Аудит транспортного рівня за рекомендаціями NIST SP 800-52 [202] і OWASP ASVS 5.0 [226] (вимоги до безпечної комунікації) виявив: TLS 1.0 і 1.1 увімкнено «для сумісності»; пріоритет наборів CBC; 0-RTT увімкнено на джерелі; сертифікати оновлюються вручну, двічі за останні два роки строк дії спливав, що спричиняло недоступність; HSTS лише на основному домені з `max-age=86400`; CAA відсутні; моніторингу CT немає; 3 записи CNAME вказують на видалені хмарні ресурси; CDN звертається до джерела по HTTP, адреса джерела відома з історії DNS; cookie сеансу без префікса `__Host-`. Портал обробляє персональні дані з довгим строком цінності (витяги з реєстрів).

5.9.2 План заходів і оцінювання

Заходи впорядковано за схемою розділу 4: спершу ті, що закривають сценарії з найбільшим ризиком (табл. 5.5). Оцінки ARO і ефективності — експертні, грошові величини — у тис. грн на рік.

Таблиця 5.5 — План модернізації транспортного рівня порталу

#	Захід	Сценарій, що закривається	C_j
T1	Видалити висячі CNAME; автоматична перевірка зони; порядок виведення ресурсів	захоплення піддомену, (5.8)	40
T2	mTLS між CDN і джерелом, зміна IP джерела, фільтрація вхідних	обхід WAF і DDoS-захисту	90
T3	Лише TLS 1.2/1.3, AEAD, X25519MLKEM768; 0-RTT лише для статики	зниження, «записати зараз», повтор	30
T4	ACME для всіх доменів, моніторинг строків дії	недоступність, (5.4)	60
T5	HSTS max-age рік, includeSubDomains, preload; __Host- cookie	SSL stripping, (5.5)	25
T6	CAA з accounturi; моніторинг CT з передаванням до CERT-UA	помилкова видача, (5.3)	35

Найвищий пріоритет мають T1 і T2. Захоплення піддомену за (5.8) — вже реалізований стан (3 висячі записи), а не ймовірність; дефейс піддомену державного порталу з фішинговою формою має значні наслідки для довіри громадян, а порушник може встановлювати cookie для батьківського домену. Відкрита адреса джерела знецінює весь захист CDN: за досвідом DDoS-атак на державні ресурси (розділ 11) саме прямі атаки на джерело виводили сайти з ладу. Захід T3 закриває ризик «записати зараз, розшифрувати пізніше» для персональних даних з довгим строком цінності практично безкоштовно, оскільки сучасний CDN і бібліотеки підтримують гібридний обмін за замовчуванням; потрібно лише перевірити сумісність клієнтів (2-пакетний ClientHello, рівняння (5.7)) на тестовому піддомені. T4 одночасно знижує ризик недоступності через прострочений сертифікат (два інциденти за два роки) і готує до 47-денних сертифікатів (табл. 5.3) — без автоматизації графік SC-081 для 30 доменів означає оновлення приблизно кожні півтора дня.

Для T5 розрахунок за (5.5) показує, що поточне значення `max-age=86400` для типового користувача державних послуг ($r \approx 0{,}05$ відвідування на добу) майже нічого не дає: $1 + 18{,}25\, e^{-0{,}05} \approx 18$ вразливих відвідувань на рік проти 1 за річного `max-age` і 0 за preload. Перехід вимагає інвентаризації піддоменів, яку забезпечує моніторинг CT із заходу T6.

5.9.3 Результат

Після виконання плану портал отримує: відсутність висячих записів і процес, що запобігає їхній появі; недоступність джерела в обхід CDN; лише TLS 1.2/1.3 з AEAD і гібридним постквантовим обміном; автоматичне оновлення сертифікатів; HSTS з preload для всієї зони; виявлення помилкової видачі сертифікатів. Сумарна вартість — 280 тис. грн на рік, що значно менше за очікувані втрати від одного дефейсу чи тривалої недоступності. Залишкові ризики: компрометація облікових записів у реєстратора і провайдера DNS (закривається registry lock і фішингостійкою MFA), довіра до провайдера CDN (договірні вимоги, keyless-режим) і перехід на постквантові підписи, який залежить від екосистеми браузерів і ЦС.

Висновки до розділу

Транспортний рівень вебресурсу — ланцюг із DNS, PKI, протоколу TLS, політики браузера та інфраструктури доставки, і практичні атаки зазвичай вражають стики між ланками, а не криптографію. TLS 1.3 усунув основні класи атак попередніх версій: статичний обмін ключами, CBC-режими, стиснення, зниження параметрів — і зменшив затримку до одного обходу, а з QUIC і 0-RTT — практично до нуля; при цьому ранні дані 0-RTT допустимі лише для ідемпотентних запитів. Модель довіри PKI вебу обмежена найслабшим ЦС, тому її доповнюють прозорість сертифікатів (дієва лише за наявності моніторингу), записи CAA і скорочення строків дії до 47 днів, яке компенсує неефективне відкликання і де-факто вимагає автоматизації ACME. HSTS із preload закриває атаку SSL stripping, а короткий `max-age` майже знецінює механізм. QUIC обмежує підсилення втричі, але зменшує видимість для засобів мережевого контролю. ECH приховує ім'я сайту лише в «натовпі» CDN, а гібридний обмін X25519MLKEM768 вже захищає трафік від стратегії «записати зараз, розшифрувати пізніше»; постквантові підписи — наступний, складніший етап. Висячі записи DNS і відкрита адреса джерела за CDN — найпоширеніші і найдешевші в усуненні вади, для яких потрібні не технології, а процеси інвентаризації.

Питання для самоперевірки

1. Які властивості забезпечує TLS і від яких моделей порушника він не захищає? Наведіть три атаки на вебзастосунок, яким TLS не перешкоджає.
2. Опишіть повне рукостискання TLS 1.3 (рис. 5.1). Чому сертифікат сервера в TLS 1.3 передається зашифрованим і що при цьому лишається відкритим?

3. Як TLS 1.3 захищається від атак зниження версії й параметрів? Яку роль відіграють підпис транскрипту і маркер у ServerHello.random?
4. Поясніть ключовий розклад (5.1). Чому статичний обмін ключами RSA не забезпечує прямої секретності і як це пов'язано зі стратегією «записати зараз, розшифрувати пізніше»?
5. За (5.2) обчисліть час до першого байта даних для RTT $= 120$ мс для TCP+TLS 1.2, TCP+TLS 1.3 і QUIC. Які запити допустимо надсилати в 0-RTT і чому?
6. Оберіть дві атаки з табл. 5.2 і поясніть, яке рішення TLS 1.3 їх унеможливлює. Чому Heartbleed не усувається зміною протоколу?
7. Чому модель довіри PKI вебу обмежена найслабшим ЦС? Як CT, CAA і програми кореневих сертифікатів браузерів компенсують цю слабкість?
8. За (5.3) обчисліть розмір доказу включення для журналу з 10^{10} записів. За якої умови CT насправді виявляє помилкову видачу?
9. За (5.4) порівняйте очікуване вікно експлуатації скомпрометованого ключа для строків 200 і 47 днів за $\rho = 0{,}2$, $T_r = 3$ дні. Які операційні ризики створює скорочення строків дії?
10. Чому механізм OCSP у вебі вважають неефективним? Які альтернативи застосовують браузери і ЦС?
11. Опишіть атаку SSL stripping. За (5.5) обчисліть кількість вразливих відвідувань на рік для $r = 0{,}2$ і $M = 30$ діб. Чому для державних доменів метою є preload?
12. Чому QUIC до перевірки адреси клієнта обмежує відповідь утричі (5.6)? Як QUIC змінює можливості мережевого моніторингу організації?
13. Які загрози постквантовий перехід створює для конфіденційності і для автентичності? Чому обмін ключами переводять на гібридний режим раніше, ніж підписи (табл. 5.4, рівняння (5.7))?
14. Опишіть механізм захоплення піддомену. За (5.8) оцініть $\mathrm{E}[N_{\text{вис}}]$ для 300 записів з $\mu = 0{,}05$ на рік через 2 роки і запропонуйте процес, що запобігає проблемі.
15. Дослідницьке: розробіть методику пасивного виявлення адреси сервера-джерела за CDN (історія DNS, CT, заголовки пошти, сканування) і проведіть її оцінювання на вибірці державних вебресурсів з дозволу власників; запропонуйте метрику «прихованості джерела».
16. Дослідницьке: оцініть вплив гібридного постквантового обміну ключами і постквантових підписів на час установлення з'єднання для мобільних клієнтів в українських мережах (3G/4G) на основі вимірювань

і моделі (5.2), (5.7) з урахуванням початкового вікна перевантаження TCP.

РОЗДІЛ 6

Безпека браузера та клієнтської частини вебзастосунків

Розділ 5 забезпечив захищений канал до автентичного сервера. Проте вміст, що надійшов цим каналом, виконується в браузері — спільному середовищі, де одночасно працюють сторінки різних власників, сторонні скрипти, розширення і вбудовані фрейми. Браузер є найбільш поширеною платформою виконання недовіреного коду, і саме тут реалізується значна частина атак на вебзастосунки: міжсайтовий скриптинг, підробка міжсайтових запитів, клікджекінг, витоки між сайтами, компрометація сторонніх скриптів. Розділ розглядає модель безпеки браузера (походження, сайт, ізоляція процесів), міжсайтовий скриптинг і контекстне кодування, політику безпеки вмісту (CSP) і Trusted Types, захист від CSRF і атрибут SameSite, механізм CORS, клікджекінг і заголовки ізоляції, ризики стороннього коду з цілісністю підресурсів (SRI), атаки на побічних каналах (XS-Leaks, Spectre) і розширення браузера та завершується кейсом захисту кабінету громадянина.

6.1 Модель безпеки браузера

6.1.1 Походження і сайт

Фундамент безпеки вебплатформи — **політика того самого походження** (same-origin policy, SOP) [71, 305]. **Походження** (origin) — це трійка «схема, хост, порт»; документ з одного походження може вільно читати DOM, cookie (з уточненнями), сховища і відповіді на запити лише свого походження. Водночас SOP від початку була асиметричною: сторінка *може надсилати* запити на будь-яке походження (форми, зображення, скрипти) і *вбудовувати* чужі ресурси, але *не може читати* відповідь. Саме ця асиметрія породжує CSRF (підрозділ 6.4) і більшість витоків між сайтами (підрозділ 6.8).

Другим поняттям є **сайт** (site) — схема разом з *реєстрованим доменом* (eTLD+1), тобто доменом на один рівень нижче публічного суфікса зі списку Public Suffix List [211]. Для `https://cabinet.portal.gov.ua:443` походження — уся трійка, а сайт — `https://portal.gov.ua`, оскільки `gov.ua` є публічним суфіксом. Формально

$$
\begin{aligned}
o = (\sigma,\ h,\ \pi), \qquad o_1 \equiv_{\text{orig}} o_2 &\iff \sigma_1 = \sigma_2 \wedge h_1 = h_2 \wedge \pi_1 = \pi_2, \\
o_1 \equiv_{\text{site}} o_2 &\iff \sigma_1 = \sigma_2 \wedge \text{eTLD+1}(h_1) = \text{eTLD+1}(h_2),
\end{aligned}
\tag{6.1}
$$

де σ — схема, h — хост, π — порт. Межа *походження* визначає доступ до DOM і відповідей, межа *сайту* — поведінку cookie з атрибутом SameSite,

ізоляцію процесів і розділення кешу. Наслідок для архітектури: усі піддомени одного реєстрованого домену — *один сайт*, тож скомпрометований чи захоплений піддомен (розділ 5, підрозділ 5.7) обходить захист SameSite для основного застосунку. Тому сервіси з різним рівнем довіри (користувацький вміст, документація, тестові середовища) розміщують на *окремих реєстрованих доменах*, а не піддоменах.

6.1.2 Ізоляція сайтів і архітектура браузера

До 2018 р. сторінки різних сайтів могли виконуватися в одному процесі рендерингу, а межу між ними забезпечувала лише логіка рушія. Атаки Spectre [179] показали, що JavaScript здатен через спекулятивне виконання читати довільну пам'ять свого процесу, отже, логічна межа недостатня. Відповіддю стала **ізоляція сайтів** (site isolation) [254]: кожен сайт виконується в окремому процесі з обмеженими привілеями (пісочниці), а процес рендерингу отримує від мережевого процесу лише ті відповіді, які сайт має право читати (механізм CORB/ORB блокує, наприклад, JSON-відповідь іншого сайту, завантажену як «скрипт»). Ізоляція сайтів перетворює SOP з логічної межі на межу *операційної системи*: навіть повна компрометація процесу рендерингу через ваду рушія не дає доступу до даних інших сайтів без додаткової вади пісочниці.

Модель багатопроцесного браузера (рис. 6.1) визначає і ціну атаки: для доступу до файлової системи користувача порушнику потрібен ланцюг з вади рушія JavaScript (виконання коду в процесі рендерингу) і вади пісочниці чи ядра ОС (вихід з неї). Такі ланцюги дорогі і використовуються у цільових атаках, тоді як масові атаки на вебзастосунки діють *у межах* моделі — як легітимний код сторінки, впроваджений через XSS чи сторонній скрипт.

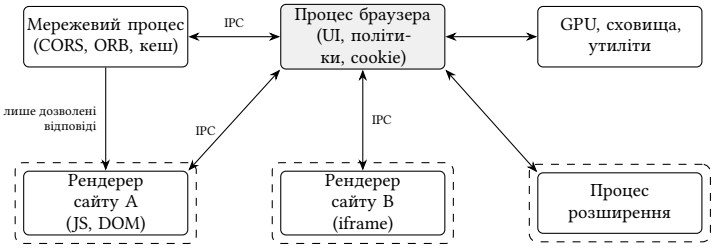


Рис. 6.1 — Багатопроцесна архітектура браузера з ізоляцією сайтів

6.1.3 Межі моделі: що браузер не захищає

Браузер захищає сайти *один від одного*, але не захищає сайт від власного коду. Будь-який скрипт, що виконується в походженні застосунку — власний, сторонній чи впроваджений, — має всі права цього походження: читає DOM, токени у `localStorage`, надсилає запити з cookie користувача, змінює форми. Тому центральна задача безпеки клієнтської частини — контроль того, *який код виконується в походженні*, що й визначає структуру решти розділу: недопущення впровадження (XSS), обмеження наслідків (CSP, Trusted Types), контроль сторонніх скриптів (SRI, ізоляція) і контроль запитів з інших сайтів (CSRF, CORS, Fetch Metadata).

6.2 Міжсайтовий скриптинг

6.2.1 Класи XSS

Міжсайтовий скриптинг (cross-site scripting, XSS, CWE-79) — впровадження порушником коду, що виконується в походженні вразливого застосунку в браузері жертви. У CWE Top 25:2025 [54] XSS посідає перше місце, а в OWASP Top 10:2025 [231] входить до категорії A05 «Injection». Розрізняють: *відбитий* XSS — шкідливі дані з параметра запиту негайно повертаються у відповіді (жертву заманюють посиланням); *збережений* — дані зберігаються на сервері (коментар, ім'я профілю, звернення громадянина) і виконуються в браузері кожного, хто переглядає сторінку; *DOM-орієнтований* — вада повністю в клієнтському коді, що бере дані з джерела (`location.hash`, `postMessage`, `document.referrer`) і передає їх у небезпечний приймач (`innerHTML`, `eval`, `document.write`, атрибут `href` з `javascript:`). Масштабне дослідження [188] з автоматичним відстеженням потоків даних у браузері виявило DOM-XSS на близько 10 % з 5 000 найпопулярніших сайтів — серверні засоби захисту таких вад не бачать, бо шкідливі дані можуть узагалі не досягати сервера.

Наслідки XSS — повний контроль над сеансом жертви в межах застосунку: виконання будь-яких дій від її імені, читання даних, підміна форм (зокрема фішингова форма введення пароля чи даних картки на справжньому домені), викрадення токенів зі сховищ. Атрибут `HttpOnly` захищає cookie сеансу від читання скриптом, але не від *використання*: скрипт порушника надсилає запити, і браузер сам додає cookie. Тому `HttpOnly` — корисне обмеження, а не захист від XSS.

6.2.2 Поширення збереженого XSS

Збережений XSS в інтерфейсі, який переглядає багато користувачів, масштабується без участі порушника. Якщо шкідливий запис розміщено на сторінці, яку U активних користувачів відвідують з інтенсивністю v на добу, то кількість уражених до моменту виявлення й видалення

$$N(t) = U\bigl(1 - e^{-vt}\bigr), \qquad \mathrm{E}[N] = U\Bigl(1 - \frac{\delta}{\delta + v}\Bigr) = \frac{Uv}{\delta + v}, \tag{6.2}$$

де t — час існування запису; δ — інтенсивність виявлення і видалення (за експоненційного розподілу часу до виявлення з середнім $1/\delta$). Для панелі модерації звернень з $U = 400$ посадовців, $v = 0{,}8$ і середнім часом виявлення 5 діб ($\delta = 0{,}2$) маємо $\mathrm{E}[N] = 400 \cdot 0{,}8/1{,}0 = 320$ — уражено 80 % користувачів привілейованого інтерфейсу. Скорочення часу виявлення до 4 годин ($\delta = 6$) дає $\mathrm{E}[N] \approx 47$. Якщо ж корисне навантаження *саморозповсюджується* (черв'як, що від імені жертви публікує себе далі, як черв'як Samy у MySpace 2005 р., який за добу уразив понад мільйон профілів), зростання стає експоненційним. Висновок: збережений XSS в інтерфейсах привілейованих користувачів — критичний ризик, а детективні заходи (звіти про порушення CSP, підрозділ 6.3) мають цінність лише за швидкого реагування.

6.2.3 Контекстне кодування

Корінь XSS — змішування даних і коду (розділ 7): дані користувача потрапляють у контекст, де браузер тлумачить їх як розмітку чи код. Захист — **контекстне кодування виведення** (output encoding): перетворення даних так, щоб у *конкретному* синтаксичному контексті вони залишалися даними [227]. Ключове слово — «контекстне»: правильне для HTML-тексту кодування не захищає в атрибуті без лапок, у JavaScript-рядку чи URL (табл. 6.1).

Таблиця 6.1 — Контексти виведення і відповідні заходи

Контекст	Приклад	Захід
Текст HTML	`<p>$x</p>`	HTML-кодування `< > & " '`
Атрибут у лапках	`<a title="$x">`	кодування атрибута; завжди лапки
URL в атрибуті	`<a href="$x">`	перевірка схеми (лише `https:`), URL-кодування параметрів
JavaScript	`<script>v='$x'</script>`	уникати; передавати через `data`-атрибут або JSON із `\u`-кодуванням
CSS	`style="color:$x"`	уникати; білий список значень
DOM-приймач	`el.innerHTML=x`	`textContent`; санітизація (DOMPurify, Sanitizer API); Trusted Types
Багатий текст	HTML від користувача	санітизація за білим списком тегів і атрибутів

Ручне кодування в кожній точці виведення ненадійне: досить однієї пропущеної. Тому основний захід — *безпечні за замовчуванням фреймворки*: шаблонізатори з автоматичним контекстним екрануванням (React, Angular, Vue, серверні шаблони з автоекрануванням) роблять XSS можливим лише через явні «аварійні виходи» (`dangerouslySetInnerHTML`, `v-html`, `bypassSecurityTrust...`), які легко знайти пошуком і контролювати при

рецензуванні коду. Для HTML від користувача (редактори повідомлень) застосовують санітизацію за білим списком перевіреною бібліотекою; власні регулярні вирази для «очищення від тегів» — типова вада через різночитання парсерів (mutation XSS: розмітка, що після санітизації і повторного розбору браузером змінює структуру).

6.3 Політика безпеки вмісту і Trusted Types

6.3.1 Від білих списків до строгої CSP

Політика безпеки вмісту (Content Security Policy, CSP Level 3) [106] — заголовок, яким сервер повідомляє браузеру, звідки дозволено завантажувати й виконувати скрипти, стилі, фрейми та інші ресурси. CSP — *другий рубіж*: вона не усуває вразливість, але робить впроваджену розмітку нешкідливою, якщо порушник не може виконати скрипт.

Ранні CSP будувалися на *білих списках* доменів (`script-src 'self' cdn.example.com`). Дослідження Google [301] на 100 мільярдах сторінок показало, що 94,7 % політик, які намагалися обмежити скрипти, тривіально обходилися: дозволені домени (CDN, аналітика, API Google) містили JSONP-кінцеві точки чи бібліотеки на кшталт старих версій AngularJS, через які порушник виконує довільний код, не виходячи за білий список. Відповіддю стала **строга CSP** (strict CSP) на основі разових значень (nonce) або хешів:

```
Content-Security-Policy: script-src 'nonce-{random}' 'strict-dynamic';
 object-src 'none'; base-uri 'none'; require-trusted-types-for 'script'
```

Сервер генерує для кожної відповіді випадковий nonce і вставляє його в атрибут кожного легітимного тегу `<script nonce="...">`; браузер виконує лише скрипти з правильним nonce. Директива `'strict-dynamic'` поширює довіру на скрипти, які завантажує вже довірений скрипт (необхідно для завантажувачів модулів), і змушує ігнорувати білі списки доменів. Впроваджена порушником розмітка `<script>` не має nonce і не виконується, а обробники подій у розмітці (`onerror=`) і `javascript:`-URL заборонені за відсутності `'unsafe-inline'`.

Стійкість строгої CSP визначає непередбачуваність nonce. Якщо nonce містить b випадкових бітів, а порушник може зробити q спроб (наприклад, через відбитий XSS з різними вгаданими значеннями), то

$$P_{\text{вгад}} \le \frac{q}{2^{b}}, \qquad b \ge 128 \;\Rightarrow\; P_{\text{вгад}} < 10^{-30} \text{ для } q = 10^{8}, \tag{6.3}$$

тобто перебір неможливий, і реальні обходи строгої CSP пов'язані з *повторним використанням* nonce: кешування сторінки з nonce на CDN (усі користувачі отримують те саме значення), вставлення nonce у статичні шаблони, витік через атрибути, доступні CSS-селекторам, або впровадження

розмітки *всередину* довіреного скрипту. Отже, nonce має генеруватися криптографічно стійким генератором для кожної відповіді, а сторінки з nonce — не кешуватися спільно. Для статичних сайтів замість nonce використовують хеші вмісту скриптів (`'sha256-...'`).

6.3.2 Розгортання і звітування

Впровадження CSP на наявному застосунку починають у режимі `Content-Security-Policy-Report-Only`, коли браузер не блокує, а надсилає звіти про порушення на вказану адресу (`report-to`). Аналіз звітів виявляє вбудовані скрипти і обробники, які треба переписати, перш ніж перемкнути політику в режим блокування. У режимі блокування ті самі звіти стають *детективним* засобом: поява порушень з нових джерел на сторінці, де все переписано, — сигнал можливої атаки, що пов'язує CSP з моніторингом (розділ 16) і з моделлю (6.2).

6.3.3 Trusted Types

Строга CSP не захищає від DOM-XSS у власному коді застосунку: довірений скрипт, що виконує `el.innerHTML = location.hash`, виконується легітимно. **Trusted Types** [297] закривають цей клас на рівні платформи: директива `require-trusted-types-for 'script'` змушує браузер відхиляти рядки в небезпечних приймачах DOM (`innerHTML`, `outerHTML`, `eval`, `src` скрипту та ін.) і приймати лише об'єкти спеціальних типів (`TrustedHTML`, `TrustedScript`, `TrustedScriptURL`), які створюються *політиками* — невеликими функціями, що санітизують чи перевіряють дані. Так безпека DOM-коду зводиться до аудиту кількох політик замість аудиту тисяч точок виклику. Досвід Google, де Trusted Types розгорнуто на більшості продуктів, показав практичне зникнення DOM-XSS у захищених застосунках. Підтримка, яка тривалий час була лише в браузерах на рушії Chromium, поширюється й на інші рушії; у старих браузерах використовують поліфіл і звітування.

6.4 Підробка міжсайтових запитів і атрибут SameSite

6.4.1 Механізм CSRF

Підробка міжсайтового запиту (cross-site request forgery, CSRF, CWE-352) використовує асиметрію SOP: сторінка порушника не може прочитати відповідь застосунку, але може змусити браузер жертви *надіслати* запит — з автоматично доданими cookie сеансу. Автоматичне надсилання форми з `evil.example` на `bank.example/transfer` виконує переказ від імені жертви, якщо застосунок автентифікує запити лише cookie. CSRF посідає третє місце в CWE Top 25:2025 [54]. Відомі варіанти: *вхід під обліковим записом порушника* (login CSRF) [72], коли жертву непомітно автентифікують у засто-

сунку під обліковим записом порушника, і вона вводить свої дані в чужий профіль; CSRF на пристрої в локальній мережі (маршрутизатори) з публічної сторінки.

6.4.2 Заходи захисту

Класичний захід — *токен синхронізатора*: випадкове значення, прив'язане до сеансу, яке сервер вставляє у форми і перевіряє в кожному запиті, що змінює стан. Сторінка порушника його не знає і не може прочитати через SOP. Ймовірність успіху атаки з q спробами за b-бітного токена дорівнює $q/2^b$, як у (6.3). Варіанти без стану на сервері — *подвійне надсилання cookie* (значення в cookie і в параметрі мають збігатися; вразливе до перезапису cookie з піддомену, якщо не використано префікс `__Host-`) і підписаний токен, прив'язаний до ідентифікатора сеансу. Перевірка заголовків `Origin`/`Referer` [72] — корисний додатковий захід.

Атрибут cookie `SameSite` [76] переносить захист на рівень браузера: cookie з `SameSite=Strict` не надсилаються в жодних міжсайтових запитах, з `Lax` — лише при навігації верхнього рівня методом GET (перехід за посиланням), з `None` (обов'язково з `Secure`) — завжди (табл. 6.2). З 2020 р. Chrome трактує cookie без атрибута як `Lax`, що різко скоротило поверхню CSRF.

Таблиця 6.2 — Надсилання cookie залежно від SameSite

Тип запиту з іншого сайту	Strict	Lax	None; Secure
Перехід за посиланням (GET, верхній рівень)	–	+	+
Надсилання форми POST	–	–	+
`fetch`/XHR з `credentials`	–	–	+
`<img>`, `<iframe>` (підресурси)	–	–	+
Запит з того самого сайту (будь-який)	+	+	+

SameSite не замінює токени повністю. По-перше, межа — *сайт*, а не походження (6.1): запит з захопленого піддомену чи з іншого застосунку того самого реєстрованого домену вважається «своїм». По-друге, `Lax` пропускає GET-навігацію, тож операції, що змінюють стан методом GET, лишаються вразливими — це додатковий аргумент за дотримання семантики HTTP [129]. По-третє, застосунки, що свідомо працюють у міжсайтових фреймах чи з федеративним входом, вимушено використовують `None`. Сучасна рекомендація — `SameSite=Lax` або `Strict` для cookie сеансу, токени для операцій, що змінюють стан, і перевірка заголовків Fetch Metadata (підрозділ 6.6). Для API з автентифікацією заголовком `Authorization` (а не cookie) класичний CSRF неможливий, бо браузер не додає заголовок автоматично.

6.5 Міжпоходжувальний обмін ресурсами

6.5.1 Механізм CORS

CORS (Cross-Origin Resource Sharing) [121] — механізм *послаблення* SOP: сервер явно дозволяє сторінкам з інших походжень читати свої відповіді. Для «простих» запитів (GET, POST з типами форм) браузер надсилає запит із заголовком `Origin` і показує відповідь сторінці лише якщо відповідь містить відповідний `Access-Control-Allow-Origin`. Для інших (методи PUT, DELETE, власні заголовки, тип `application/json`) браузер спершу надсилає *попередній запит* (preflight) методом OPTIONS і лише за дозволу — основний (рис. 6.2). Заголовок `Access-Control-Allow-Credentials: true` дозволяє запити з cookie, але тоді `Allow-Origin` не може бути `*`.

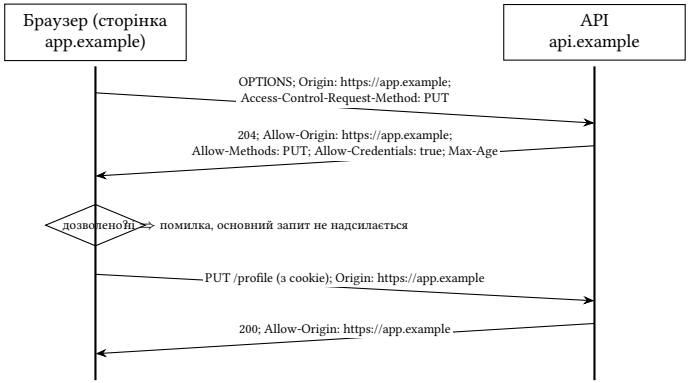


Рис. 6.2 — CORS з попереднім запитом для запиту з обліковими даними

6.5.2 Типові помилки конфігурації

CORS сам по собі не створює захисту — він створює *дозволи*, і помилки конфігурації відкривають дані. Емпіричне дослідження [88] виявило поширені вади: (1) *віддзеркалення* будь-якого значення `Origin` у `Allow-Origin` разом з `Allow-Credentials: true` — будь-який сайт читає особисті дані користувача; (2) перевірка походження підрядком чи регулярним виразом без якорів (`example.com` пропускає `example.com.evil.net` і `evilexample.com`); (3) довіра до `Origin: null`, яке має сторінка з пісочного iframe чи файлу; (4) довіра до всіх піддоменів, що переносить ризик XSS чи захоплення будь-якого піддомену на API; (5) довіра до HTTP-походжень, що дає змогу мережевому порушнику підробити запит. Правильний підхід — явний перелік точних походжень, відсутність `Allow-Credentials` для публічних API, заголовок `Vary: Origin` для коректного кешування. Важливо: CORS — механізм браузера; запити від серверних клієнтів (скриптів, ботів) він не обмежує, тож авторизацію API він не замінює (розділ 10).

6.6 Клікджекінг і заголовки ізоляції

6.6.1 Клікджекінг

Клікджекінг (clickjacking, UI redressing) [154] — вбудовування сторінки застосунку в невидимий фрейм поверх приманки, щоб користувач, клікаючи на видимий елемент, натискав кнопку в застосунку («Підтвердити», «Надати доступ», «Видалити»). Застарілі скрипти «виходу з фрейму» (frame busting) обходилися пісочними фреймами і подіями вивантаження. Надійний захід — директива CSP `frame-ancestors 'none'` (або перелік дозволених вбудовувачів) і застарілий, але широко підтримуваний заголовок

`X-Frame-Options: DENY`. Для критичних дій додатково використовують повторне підтвердження з елементом, який неможливо натиснути випадково.

6.6.2 Сучасні заголовки безпеки

Набір заголовків відповіді дає браузеру інструкції щодо ізоляції документа (табл. 6.3). Fetch Metadata [295] — заголовки *запиту* `Sec-Fetch-Site`, `Sec-Fetch-Mode`, `Sec-Fetch-Dest`, які браузер додає автоматично і які сторінка не може підробити; вони дають серверу змогу реалізувати *політику ізоляції ресурсів*: відхиляти міжсайтові запити до всіх кінцевих точок, крім явно дозволених (навігація верхнього рівня, публічні ресурси). Така політика одним правилом закриває CSRF, значну частину XS-Leaks і атаки на кшталт Spectre щодо даних застосунку.

Таблиця 6.3 — Заголовки безпеки клієнтської частини

Заголовок	Призначення	Рекомендоване значення
Content-Security-Policy	обмеження скриптів, фреймів, base, форм	строга CSP з nonce; `frame-ancestors 'none'`
Strict-Transport-Security	лише HTTPS (розділ 5)	`max-age=31536000; includeSubDomains; preload`
Cross-Origin-Opener-Policy	розрив зв'язку з вікнами інших походжень	`same-origin`
Cross-Origin-Embedder-Policy	вимога явного дозволу для вбудованих ресурсів	`require-corp` (для крос-походжувальної ізоляції)
Cross-Origin-Resource-Policy	хто може вбудовувати ресурс	`same-site` або `same-origin`
Referrer-Policy	обмеження витоку URL	`strict-origin-when-cross-origin`
Permissions-Policy	вимкнення API (камера, геолокація)	лише потрібні, для фреймів — заборона
X-Content-Type-Options	заборона вгадування типу	`nosniff`

Поєднання COOP `same-origin` і COEP `require-corp` переводить документ у стан *крос-походжувальної ізоляції* (cross-origin isolated), у якому браузер гарантує відсутність у процесі чужих даних і тому дозволяє потужні API (`SharedArrayBuffer`, високоточні таймери). Для звичайного вебзастосунку це вимога лише при використанні таких API, а COOP `same-origin` доцільний завжди: він не дає сторінці порушника, яка відкрила застосунок у новому вікні, отримати посилання на його об'єкт `window`, яке використовують для XS-Leaks і атак через `postMessage`.

6.7 Сторонній код у браузері

6.7.1 Ризик сторонніх скриптів

Сторінка типового вебзастосунку підключає десятки сторонніх скриптів: аналітику, рекламу, віджети чатів, платіжні форми, бібліотеки з публічних CDN. Кожен з них виконується з *усіма* правами походження (підрозділ 6.1), тож компрометація будь-якого постачальника рівнозначна збереженому XSS на всіх сайтах-клієнтах. Аналіз мільйонів сторінок [216] показав, що сайти масово включають скрипти з доменів, які надалі можуть бути викуплені (зокрема з помилками в назвах і з доменів, термін реєстрації яких минув). Інциденти підтверджують модель: у 2018 р. угруповання Magecart через змінений скрипт на сайті British Airways викрало дані близько 380 тисяч платіжних карток (у 2020 р. регулятор Великої Британії оштрафував компанію на 20 млн фунтів [159]); у 2024 р. новий власник домену polyfill.io почав віддавати шкідливий код понад 100 тисячам сайтів, що підключали бібліотеку-поліфіл [263]. Сторонні скрипти — клієнтський вимір ризиків ланцюга постачання (A03:2025, розділ 13).

Ризик зростає з кількістю постачальників і транзитивним включенням (скрипт аналітики завантажує ще кілька). Якщо сторінка залежить від n незалежних джерел скриптів, кожне з яких компрометується за рік з імовірністю p_i, і кожне транзитивно завантажує m_i додаткових джерел з імовірністю компрометації p', то

$$P_{\text{компр}} = 1 - \prod_{i=1}^{n} \bigl(1 - p_i\bigr)\bigl(1 - p'\bigr)^{m_i}. \tag{6.4}$$

Для $n = 15$, $p_i = 0{,}005$, $m_i = 2$, $p' = 0{,}003$ маємо $P_{\text{компр}} = 1 - 0{,}995^{15} \cdot 0{,}997^{30} \approx 1 - 0{,}928 \cdot 0{,}914 \approx 0{,}15$ на рік. Для сторінки оплати чи введення пароля це неприйнятно: скорочення до $n = 2$ без транзитивних включень дає близько 0,01.

6.7.2 Цілісність підресурсів і ізоляція

Цілісність підресурсів (Subresource Integrity, SRI) [296] — атрибут `integrity` тегу `<script>` з криптографічним хешем очікуваного вмісту; браузер виконує скрипт, лише якщо хеш збігається. SRI повністю закриває сценарій «підміна файлу на CDN» для *версійованих* бібліотек, але не застосовний до скриптів, які постачальник свідомо змінює (аналітика, віджети), — а саме вони найризикованіші. Для них використовують: *самостійне розміщення* (self-hosting) копії з контролем оновлень; *ізоляцію* — виконання віджета в iframe з окремого походження з атрибутом `sandbox`, обмін через `postMessage` з перевіркою походження; мінімізацію — відсутність сторонніх скриптів на сторінках автентифікації й оплати. Стандарт PCI DSS 4.0.1 [243] (вимоги 6.4.3 і 11.6.1) вимагає інвентаризації та обґрунтування кожного скрипту на сторінках оплати і механізму виявлення несанкціонованих змін — пряма реакція на атаки Magecart. Строга CSP з `'strict-dynamic'` обмежує, *які* скрипти виконуються, але не те, що робить дозволений скомпрометований скрипт; для контролю витоку даних застосовують директиви `connect-src` і `form-action`.

6.8 Побічні канали і розширення браузера

6.8.1 Міжсайтові витоки

Міжсайтові витоки (cross-site leaks, XS-Leaks) [303] використовують інформацію, яку SOP не захищає: кількість фреймів у вікні, факт успішного чи помилкового завантаження ресурсу, час завантаження, розмір відповіді в кеші, перенаправлення, розмір вікна. Сторінка порушника, відкрита жертвою, надсилає запити до застосунку від імені жертви (з її cookie) і спостерігає ці побічні ефекти, отримуючи по біту інформації за запит: «чи автентифікований користувач», «чи містить результат пошуку в поштовій

скриньці слово X», «чи є користувач членом групи». Систематичне дослідження [178] формалізувало десятки класів XS-Leaks і показало, що кожен з основних браузерів був уразливий до більшості з них.

Небезпека XS-Leaks — у можливості *пошуку*: якщо застосунок має кінцеву точку пошуку, відповідь якої відрізняється за наявності результатів, порушник виконує бінарний (або k-арний) пошук у просторі значень секрету:

$$Q = \left\lceil \log_k |\mathcal{S}| \right\rceil, \tag{6.5}$$

де $|\mathcal{S}|$ — потужність множини можливих значень секрету; k — кількість розрізнюваних станів оракула за запит (для бінарного оракула $k = 2$); якщо пошук підтримує префікси, секрет відновлюють посимвольно за $L \cdot \lceil \log_2 |\Sigma| \rceil$ запитів. Для 10-значного номера документа з префіксним пошуком: $10 \cdot \lceil \log_2 10 \rceil = 40$ запитів, що виконуються за секунди, поки відкрита сторінка порушника. Заходи — ті самі, що й для CSRF, але послідовніші: SameSite `Lax`/`Strict`, політика ізоляції ресурсів на основі Fetch Metadata, COOP, CORP, однакові відповіді (розмір, код, час) для чутливих кінцевих точок незалежно від результату і заборона вбудовування.

6.8.2 Spectre і апаратні канали

Атаки класу Spectre [179] перетворили будь-який JavaScript на потенційний засіб читання пам'яті свого процесу через побічні канали кешу процесора. Захист браузерів — ізоляція сайтів, огрубіння таймерів і заборона `SharedArrayBuffer` поза крос-походжувальною ізоляцією; захист власника застосунку — не допускати, щоб його чутливі відповіді потрапляли в процес чужого сайту: CORP і правильний `Content-Type` з `nosniff` (тоді механізм ORB не передасть відповідь у чужий процес), політика ізоляції ресурсів. Отже, ті самі заголовки табл. 6.3 захищають і від логічних, і від апаратних витоків.

6.8.3 Розширення браузера

Розширення з дозволом на доступ до всіх сайтів працюють *поза* моделлю SOP: читають і змінюють вміст будь-якої сторінки, зокрема введені паролі й токени. Ризики — зловмисне розширення, викуп популярного розширення новим власником і компрометація облікового запису розробника з публікацією шкідливого оновлення (наприкінці 2024 р. в такий спосіб було скомпрометовано низку розширень Chrome, зокрема розширення компанії з безпеки даних Cyberhaven). Власник застосунку розширення не контролює; організація контролює їх для своїх працівників через політики керування браузерами: білий список дозволених розширень для робочих станцій посадовців і адміністраторів — обов'язковий захід у моделі загроз

розділу 4 (сценарій a4 рис. 4.3). Маніфест V3 у Chrome обмежив віддалене завантаження коду розширеннями, але не доступ до вмісту сторінок.

6.9 Кейс: захист клієнтської частини кабінету громадянина

6.9.1 Вихідний стан

Кабінет громадянина порталу державних послуг (розділ 4, сценарій R3) — односторінковий застосунок на сучасному фреймворку з автоекрануванням. Аудит за OWASP ASVS 5.0 [226] (розділи V3 «Безпека вебінтерфейсу», V1 «Кодування і санітизація») і WSTG [232] виявив: 4 використання `dangerouslySetInnerHTML` для відображення відповідей посадовців з форматуванням, з них одне без санітизації (збережений XSS у панелі посадовця); CSP з білим списком 11 доменів, серед яких публічний CDN з JSONP-кінцевою точкою, і `'unsafe-inline'`; 14 сторонніх скриптів, 3 з них на сторінці входу; cookie сеансу без префікса `__Host-`, з `SameSite=None`; CORS віддзеркалює будь-яке `Origin` з `Allow-Credentials: true` на API довідок; відсутні `frame-ancestors` і COOP; ендпоінт пошуку по зверненнях повертає 200 або 404 залежно від наявності результатів.

6.9.2 План заходів

Заходи згруповано за класами (табл. 6.4).

Таблиця 6.4 — План захисту клієнтської частини кабінету

#	Захід	Ефект
C1	Точний перелік походжень у CORS, вилучення віддзеркалення	закриває читання даних будь-яким сайтом
C2	Санітизація всіх 4 точок `dangerouslySetInnerHTML`; правило лінтера	усуває відомий XSS
C3	Строга CSP з nonce і `'strict-dynamic'`, спершу Report-Only; `frame-ancestors 'none'`	другий рубіж проти XSS, клікджекінг
C4	Trusted Types з однією політикою санітизації	DOM-XSS на рівні платформи
C5	`__Host-`, `Secure`, `HttpOnly`, `SameSite=Lax` для сеансу; токени для змін стану	CSRF, перезапис cookie
C6	Вилучення сторонніх скриптів зі сторінок входу і заяв; SRI для бібліотек; віджет чату в iframe з пісочницею	(6.4): P з 0,15 до ≈0,01
C7	Fetch Metadata: відхиляти міжсайтові запити до API; COOP, CORP; однакові відповіді пошуку	XS-Leaks, Spectre

Порядок визначено ризиком і вартістю. C1 і C2 — виправлення наявних вад, що експлуатуються прямо: віддзеркалення `Origin` дає змогу будь-якій сторінці, відкритій громадянином, прочитати його довідки, а збережений XSS у панелі посадовця за (6.2) уразив би більшість посадовців за кілька

днів. С3 і С4 перетворюють захист від XSS із дисципліни розробників на властивість платформи; перехід на строгу CSP вимагає переписати вбудовані обробники, тож політика спершу діє в режимі звітів два тижні, а звіти потім передаються до SIEM (розділ 16). С6 найбільше впливає на сценарій масової компрометації: за моделлю (6.4) ймовірність компрометації сторінки входу через сторонній код знижується приблизно в 15 разів. С7 закриває XS-Leak через пошук: за (6.5) номер звернення відновлюється за кілька десятків запитів.

6.9.3 Результат і залишковий ризик

Після заходів XSS у кабінеті можливий лише через ваду політики Trusted Types або в обхід фреймворку, що виявляється рецензуванням; CSRF закривається на двох рівнях; дані API доступні лише власним походженням; сторонній код відсутній на чутливих сторінках. Залишкові ризики: шкідливі розширення в браузерах громадян (організація не контролює; часткова компенсація — прив'язка сеансу до пристрою і повторна автентифікація для критичних дій, розділ 8), компрометація збирання клієнтської частини (розділ 13) і вади рушіїв браузерів, які закриваються їхніми оновленнями.

Висновки до розділу

Модель безпеки браузера ґрунтується на політиці того самого походження і поняттях походження та сайту (6.1), а після Spectre — на ізоляції сайтів у процесах ОС. Браузер захищає сайти один від одного, але не сайт від власного коду, тому центральна задача клієнтської безпеки — контроль того, який код виконується в походженні застосунку. Міжсайтовий скриптинг, перший у CWE Top 25:2025, усувається безпечними за замовчуванням фреймворками з контекстним кодуванням, а збережений XSS у привілейованих інтерфейсах уражає більшість користувачів до виявлення (6.2). Строга CSP з nonce і `'strict-dynamic'` замінила неефективні білі списки, а Trusted Types закривають DOM-XSS на рівні платформи. SameSite і Fetch Metadata перенесли захист від CSRF і XS-Leaks у браузер, проте межею залишається сайт, а не походження. CORS — механізм послаблення, а не захисту, і віддзеркалення `Origin` з обліковими даними є однією з найнебезпечніших помилок конфігурації. Сторонні скрипти мають усі права походження, а ризик їх компрометації швидко зростає з кількістю постачальників (6.4); SRI, ізоляція в iframe і мінімізація на чутливих сторінках є обов'язковими. Заголовки ізоляції (COOP, CORP, CSP, Fetch Metadata) одночасно протидіють логічним витокам і апаратним побічним каналам.

Питання для самоперевірки

1. Дайте означення походження і сайту (6.1). Для пар `https://a.portal.gov.ua` і `https://b.portal.gov.ua`, `https://x.github.io` і `https://y.github.io` визначте, чи є вони одним походженням і одним сайтом.
2. У чому полягає асиметрія SOP і які класи атак вона породжує?
3. Як ізоляція сайтів змінила модель загроз браузера після Spectre (рис. 6.1)? Що вона не захищає?
4. Порівняйте відбитий, збережений і DOM-орієнтований XSS. Чому серверні засоби захисту не виявляють DOM-XSS?
5. За (6.2) обчисліть очікувану кількість уражених для $U = 1\,000$, $v = 0{,}3$ і середнього часу виявлення 2 доби та 1 година. Які висновки щодо моніторингу?
6. Для кожного контексту табл. 6.1 наведіть приклад даних, які обходять HTML-кодування. Чому санітизація регулярними виразами ненадійна?
7. Чому CSP з білими списками доменів зазвичай обходиться? Опишіть строгу CSP і поясніть роль `'strict-dynamic'`.
8. За (6.3) оцініть стійкість nonce з 64 бітів. Які практичні помилки роблять nonce передбачуваним незалежно від довжини?
9. Як Trusted Types змінюють аудит DOM-коду? Чому вони доповнюють, а не замінюють строгу CSP?
10. Поясніть механізм CSRF і login CSRF. Чому `SameSite=Lax` не захищає від CSRF з піддомену і від змін стану методом GET (табл. 6.2)?
11. Опишіть обмін CORS з попереднім запитом (рис. 6.2). Назвіть п’ять типових помилок конфігурації і наслідки кожної.
12. Які заголовки табл. 6.3 захищають від клікджекінгу, XS-Leaks і Spectre? Що таке політика ізоляції ресурсів на основі Fetch Metadata?
13. За (6.4) обчисліть імовірність компрометації сторінки з 25 сторонніми джерелами ($p_i = 0{,}004$, $m_i = 3$, $p' = 0{,}002$). Які заходи і чому застосовні до скриптів, що змінюються постачальником?
14. Як працює XS-Leak через кінцеву точку пошуку? За (6.5) оцініть кількість запитів для відновлення 16-символьного шістнадцяткового токена з префіксним пошуком.
15. Дослідницьке: розробіть метод автоматичної генерації строгої CSP і політик Trusted Types для наявного односторінкового застосунку на основі динамічного аналізу і звітів Report-Only; оцініть повноту і кількість хибних блокувань на відкритих застосунках.

16. Дослідницьке: проведіть вимірювання поширеності сторонніх скриптів, SRI, строгої CSP і заголовків ізоляції на вебресурсах органів влади України (зона `gov.ua`) і побудуйте інтегральний показник ризику клієнтської частини на основі моделі (6.4).

РОЗДІЛ 7

Ін'єкційні атаки та безпечна обробка даних

Розділ 6 розглянув захист клієнтської частини. Цей розділ переходить на серверний бік і подає *захисний* погляд на клас вад, відомий як ін'єкції. Ін'єкції — один з найпоширеніших класів вад вебзастосунків: SQL-ін'єкція (CWE-89) посідає друге місце в CWE Top 25:2025 [54], а категорія A05 «Injection» в OWASP Top 10:2025 [231] об'єднує понад три десятки CWE. Виклад зосереджено не на способах зловживання, а на причині вразливості та інженерних рішеннях, що її усувають: загальній моделі змішування даних і коду, структурному розділенні для різних інтерпретаторів, безпечній роботі із зовнішніми запитами й файлами, валідації та канонікалізації введення і коректній обробці винятків (нова категорія A10:2025). Розділ завершується кейсом побудови програми запобігання ін'єкціям у життєвому циклі розроблення.

7.1 Загальна модель вразливості

7.1.1 Змішування даних і коду як причина

Вади цього класу мають спільну природу: застосунок формує рядок мовою деякого *інтерпретатора* (SQL, оболонка операційної системи, шаблонізатор, XML-парсер, служба каталогів) і поєднує фіксований шаблон з даними користувача. Якщо дані здатні містити синтаксичні елементи мови, межа між *даними* і *кодом* зникає, і структуру команди визначає вже не лише розробник. Інакше кажучи, вразливість виникає тоді, коли канал даних і канал керування збігаються. Розуміння саме причини, а не окремих проявів, дозволяє застосовувати один і той самий принцип захисту до всіх інтерпретаторів.

Су і Вассерманн [281] дали формальне означення безпечного введення: нехай застосунок утворює рядок $q = T[x]$ підстановкою введення x у шаблон T, а $\mathcal{P}_L(q)$ — дерево розбору q граматикою мови L. Введення безпечне, якщо воно цілком розбирається як один літерал у позиції, яку для нього передбачив розробник:

$$\mathrm{Safe}(T, x) \iff \exists\, \tau \subseteq \mathcal{P}_L\big(T[x]\big) : \ \mathrm{yield}(\tau) = x \ \wedge \ \tau \in \mathrm{Lit}_L, \tag{7.1}$$

де τ — піддерево розбору; $\mathrm{yield}(\tau)$ — рядок, який воно породжує; Lit_L — множина піддерев-літералів мови (рядок, число). Рей і Лігатті [248] уточнили, що визначальною є поява у введенні саме *коду* — операторів і ключових слів, а не значень. Модель має прямий практичний наслідок: *фільтра-*

ція «небезпечних символів» ненадійна, бо її повнота залежить від знання всієї граматики та всіх діалектів інтерпретатора, тоді як *структурне розділення* надійне за побудовою — шаблон розбирається окремо від даних, і умова (7.1) виконується для будь-якого x (рис. 7.1).

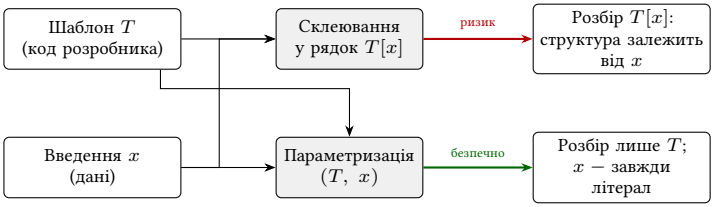


Рис. 7.1 — Склеювання у рядок (дані здатні змінити структуру команди) і параметризація (структура фіксується до підстановки даних)

7.1.2 Спільний принцип захисту

З моделі випливає єдиний принцип: *дані передають інтерпретаторові окремим каналом або в типізованій формі, а не як частину тексту команди.* Таблиця 7.1 зводить інтерпретатори, типові для вебзастосунків, до відповідних реалізацій цього принципу. Решта розділу послідовно розкриває рядки таблиці з погляду інженерної реалізації захисту.

Таблиця 7.1 — Структурні заходи за інтерпретаторами

Інтерпретатор	CWE	Реалізація принципу розділення даних і коду
Реляційна СУБД	89	підготовлені запити з параметрами; об'єктно-реляційне відображення (ORM); білий список для імен стовпців
Документна СУБД	943	типізація введення за схемою; заборона приймати структуровані об'єкти там, де очікується значення
Оболонка ОС	78	виклик програм без оболонки, з масивом аргументів; відмова від передавання введення в командний рядок
Шаблонізатор	1336	шаблони лише з коду застосунку; дані — лише як значення; виконання в пісочниці
XML-парсер	611	вимкнення зовнішніх сутностей і DTD; безпечні типові налаштування парсера
Серіалізація об'єктів	502	формати даних зі схемою (JSON) замість серіалізації довільних об'єктів
Служба каталогів, XPath	90, 643	параметризовані API; кодування спеціальних символів
Журнал, SIEM	117	структуроване журналювання; дані як поля, а не частина рядка

7.2 Безпечна робота з реляційними базами даних

7.2.1 Підготовлені запити

Надійним заходом для реляційних СУБД є **підготовлені запити** (prepared statements, параметризовані запити): застосунок надсилає СУБД текст запиту з позиційними чи іменованими заповнювачами (`?`, `:id`) окремо від значень параметрів. СУБД розбирає і планує запит до того, як отримає значення, тож структура команди фіксується, а параметр за побудовою залишається одним літералом у позиції заповнювача — умова (7.1) виконується для будь-якого значення. На відміну від екранування, підготовлені запити не залежать від знання діалекту й кодувань і не псують даних (ім'я з апострофом зберігається коректно).

Принципове обмеження: параметризувати можна лише *значення*, але не *ідентифікатори* (імена таблиць і стовпців) і не ключові слова (напрямок сортування). Коли такі елементи залежать від введення (сортування за обраним стовпцем, динамічні фільтри), їх не підставляють у текст, а зіставляють із *білим списком* дозволених значень у коді: отримане від користувача ім'я стовпця приймають лише тоді, коли воно належить заздалегідь визначеній множині, і в запит потрапляє константа з цієї множини, а не рядок користувача.

7.2.2 Об'єктно-реляційне відображення і його межі

Об'єктно-реляційне відображення (ORM) будує запити з типізованих виразів мови програмування і параметризує значення за замовчуванням, тому в типовому використанні усуває цілий клас вад на рівні фреймворку. Проте ORM не є автоматичною гарантією: майже всі ORM надають «аварійні виходи» для сирих фрагментів запиту (методи на кшталт `raw`, `whereRaw`, виклики збережених процедур рядком), і саме в них конкатенація повертається. Тому в рецензуванні коду і статичному аналізі (розділ 15) такі виклики позначають як точки підвищеної уваги: їхня частка серед усіх звернень до БД є практичною метрикою ризику. Другий недогляд — динамічні імена стовпців, передані в ORM рядком; їх контролюють тим самим білим списком.

7.2.3 Обмеження наслідків

Структурне розділення усуває причину, але принцип глибокого захисту (розділ 4) вимагає обмежити й можливі наслідки будь-якої залишкової вади. Застосовують: *найменші привілеї* облікового запису БД (окремі облікові записи для читання і запису; відсутність прав на зміну схеми в застосунку); *розділення даних* за орендарями і чутливістю; *мінімізацію відповідей* застосунку (повертати лише потрібні поля, не транслювати повідомлення СУБД користувачеві); *журналювання й виявлення аномалій* у зверненнях до БД (розділ 16). Очікувані збитки від залишкової вади за наявності цих заходів оцінюють мультиплікативно:

$$\text{Impact} = I_0 \cdot \prod_{k} \bigl(1 - m_k\bigr), \tag{7.2}$$

де I_0 — потенційний збиток без обмежувальних заходів; $m_k \in [0;1]$ — частка зниження збитку заходом k (привілеї, розділення, мінімізація). Формула (7.2) — окремий випадок моделі ефективності заходів (4.3) і пояснює, чому навіть за надійного структурного захисту обмеження привілеїв БД лишається обов'язковим: воно знижує I_0 на випадок помилки в будь-якій ланці.

7.3 Безпечне звернення до інших інтерпретаторів

7.3.1 Команди операційної системи

Найбезпечніший підхід — *не залучати оболонку взагалі*: сучасні середовища дозволяють запускати зовнішню програму, передаючи ім'я виконуваного файлу й *масив аргументів* окремими елементами, тож оболонка не розбирає рядок і дані не можуть стати керівними конструкціями. Там, де зовнішня програма не потрібна, перевагу віддають бібліотечним функціям

мови (робота з файлами, мережею) замість виклику утиліт. Якщо аргументом є ім'я файлу чи параметр з обмеженої множини, застосовують білий список значень, а не спробу «очистити» рядок.

7.3.2 Шаблонізатори та інтерпретатори виразів

Серверні шаблонізатори безпечні, доки шаблон є *кодом застосунку*, а дані підставляються лише як значення з автоматичним контекстним екрануванням (розділ 6). Небезпечний антипатерн — побудова самого тексту шаблону з введення користувача [175]: тоді дані потрапляють у позицію коду. Захід — архітектурна заборона динамічно складати шаблони з зовнішніх даних; якщо застосунок свідомо надає користувачам можливість шаблонізації (наприклад, налаштовувані листи), використовують спеціалізований обмежений рушій, що виконується в пісочниці з мінімальним набором доступних операцій і без доступу до рантайму.

7.3.3 Обробники XML і серіалізація

Історичні типові налаштування багатьох XML-парсерів дозволяли *зовнішні сутності* і завантаження DTD, що є джерелом класу вад XXE [278]. Захід простий і повний: вимкнути обробку зовнішніх сутностей і DTD у конфігурації парсера; сучасні бібліотеки дедалі частіше роблять це типовим. Для передавання структурованих даних між системами надають перевагу форматам без виконуваної семантики (JSON) з *перевіркою за схемою*.

Окремої уваги потребує **десеріалізація**: відновлення довільних об'єктів середовища виконання із зовнішнього потоку байтів небезпечне принципово, бо сам формат може приписувати створення об'єктів і виклик їхніх методів [135]. Безпечний підхід — не відновлювати довільні об'єкти з недовірених джерел узагалі, а передавати дані у форматі зі схемою і будувати об'єкти застосунку явним кодом після валідації. Якщо серіалізація об'єктів неминуча, застосовують сувору фіксацію переліку дозволених класів і криптографічний контроль цілісності потоку.

7.3.4 Журнали як інтерпретатор

Запис недовірених даних у журнали теж є ін'єкцією: підроблені переводи рядків спотворюють записи, а підстановки, що тлумачаться підсистемою журналювання, у відомому інциденті Log4Shell (CVE-2021-44228) призводили до звернення до зовнішніх ресурсів під час запису рядка [64]. Захід — *структуроване журналювання*, де дані зберігаються окремими полями, а не вбудовуються в рядок повідомлення, та своєчасне оновлення бібліотек журналювання (розділи 13 і 16).

7.4 Контроль вихідних запитів сервера

7.4.1 Природа ризику SSRF

Окремий клас (CWE-918, у OWASP Top 10 раніше виділений окремо, у редакції 2025 р. — у складі A01) пов'язаний не з інтерпретатором, а з тим, що сервер застосунку сам виконує мережеві запити за адресою, похідною від введення (отримання зображення за посиланням, вебхук, запит до стороннього API). Ризик у тому, що сервер має мережевий доступ, якого немає зовнішній порушник: до внутрішніх сервісів і, у хмарах, до служби метаданих, яка може видавати тимчасові облікові дані [287]. Саме цей механізм був ключовою ланкою відомого інциденту витоку даних Capital One 2019 р. [181].

7.4.2 Заходи

Захист будують у кілька рівнів (рис. 7.2). На рівні *застосунку*: приймати не довільний URL, а ідентифікатор ресурсу з обмеженої множини, або звужувати дозволене до конкретних схем і доменів білим списком; розв'язувати ім'я в адресу і перевіряти, що вона не належить приватним і службовим діапазонам, причому *до* і *під час* з'єднання, щоб уникнути розбіжності між перевіркою і фактичним розв'язанням імені; забороняти перенаправлення на недозволені адреси. На рівні *інфраструктури*: ізолювати вихідний трафік сервера через проксі з політикою призначень; у хмарі — вимагати захищеної версії служби метаданих із сеансовими маркерами (IMDSv2) [60] і обмежувати права ролі екземпляра. Поєднання застосункового білого списку і мережевого обмеження реалізує глибокий захист: помилка на одному рівні компенсується іншим.

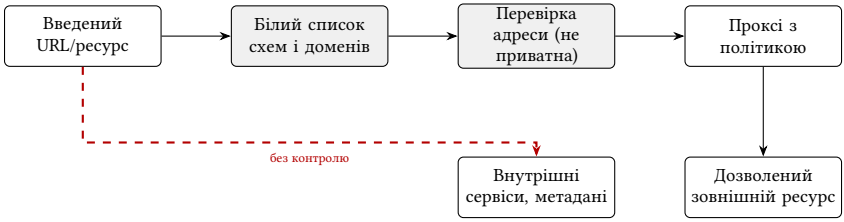


Рис. 7.2 — Багаторівневий контроль вихідних запитів сервера: застосунок (білий список, перевірка адреси) і інфраструктура (проксі, обмеження прав)

7.5 Безпечна робота з файлами

7.5.1 Доступ до шляхів

Коли ім'я чи шлях файлу залежить від введення, виникає ризик виходу за межі призначеного каталогу (CWE-22). Надійний захід не покладається на пошук «небезпечних» послідовностей у рядку, а будує шлях структурно: приймає лише ідентифікатор чи ім'я з обмеженої множини, з'єднує його з фіксованим базовим каталогом, *канонікалізує* повний шлях (розв'язуючи символьні посилання й відносні переходи) і перевіряє, що результат лишається *всередині* базового каталогу. Формально безпечність виражають умовою належності канонічного шляху піддереву:

$$\operatorname{canon}\big(\operatorname{join}(B,\ x)\big)\ \sqsubseteq\ \operatorname{canon}(B), \tag{7.3}$$

де B — базовий каталог; x — похідне від введення ім'я; canon — канонікалізація; $\sqsubseteq$ — відношення «є нащадком або збігається». Перевірку виконують над *канонічним* шляхом, бо порівняння сирих рядків обходиться посиланнями й кодуваннями. Де можливо, взагалі відмовляються від похідних від введення шляхів, зберігаючи відповідність «ідентифікатор → файл» у базі даних.

7.5.2 Приймання вивантажених файлів

Для файлів, що надходять від користувачів, застосовують сукупність заходів: зберігання поза кореневим каталогом вебсервера або в окремому сховищі об'єктів; *згенероване* застосунком ім'я замість наданого користувачем; визначення типу за вмістом, а не за розширенням чи заголовком; обмеження розміру й кількості (розділ 11); віддавання файлів із заголовками, що унеможливлюють їх трактування як активного вмісту (`Content-Disposition: attachment`, `X-Content-Type-Options: nosniff`), і бажано з окремого походження (розділ 6); перевірку антивірусними засобами. Ключова ідея — вивантажений файл ніколи не має потрапляти в позицію, де середовище може його *виконати*.

7.5.3 Забруднення властивостей об'єктів

У середовищах з прототипним успадкуванням існує клас вад (prototype pollution), коли рекурсивне злиття зовнішнього об'єкта з внутрішнім дозволяє записати службові властивості спільного прототипу і в поєднанні з іншими ділянками коду призвести до тяжких наслідків [266]. Заходи стосуються *обробки структурованого введення*: не виконувати глибоке злиття недовірених об'єктів у наявні; відхиляти службові ключі; створювати об'єкти без прототипу для словників даних; перевіряти вхідні об'єкти за схемою, що задає дозволений перелік полів. Це окремий випадок загального принципу — приймати дані в *очікуваній формі*, а не довільні структури.

7.6 Валідація, канонікалізація та обробка винятків

7.6.1 Валідація як доповнення, а не заміна

Валідація введення за білим списком (перевірка типу, діапазону, формату, довжини, належності множині) — важливий, але *другорядний* щодо структурного розділення захід. Вона зменшує поверхню і ловить очевидно некоректні дані, проте не може бути основним захистом від ін'єкцій: багато коректних за змістом значень (прізвища з апострофом, адреси, тексти звернень) містять символи, значущі для того чи іншого інтерпретатора, і валідація, достатньо строга, щоб їх відкинути, зіпсує легітимні дані. Тому валідацію виконують за призначенням поля (а не за «небезпечністю» для конкретної БД), і завжди *на сервері*: клієнтська перевірка (розділ 6) є зручністю для користувача, а не межею довіри.

7.6.2 Канонікалізація і узгодженість кодувань

Багато обходів перевірок спираються на множинність подань одного значення: відсоткове кодування, подвійне кодування, різні форми Unicode, альтернативні роздільники шляху. Тому валідацію й авторизаційні рішення ухвалюють над *канонічною* формою: спершу декодування й **канонікалізація** (зокрема нормалізація Unicode до єдиної форми), потім перевірка. Порядок критичний — перевірка до канонікалізації залишає розрив, яким користуються для обходу. Застосунок також має підтримувати єдине кодування (UTF-8) наскрізно, бо розбіжності кодувань між рівнями самі стають джерелом вад.

7.6.3 Коректна обробка винятків (A10:2025)

Редакція OWASP Top 10:2025 [231] уводить нову категорію A10 «Mishandling of Exceptional Conditions» — неправильне оброблення виняткових ситуацій. Її актуальність для цього розділу подвійна. По-перше, повідомлення про помилки інтерпретаторів, повернені користувачеві (текст помилки СУБД, трасування стека), полегшують пошук і експлуатацію вад і самі є витоком інформації; тому користувачеві повертають узагальнене повідомлення й ідентифікатор події, а подробиці пишуть лише у внутрішній журнал. По-друге, *спосіб* оброблення помилки визначає безпечність поведінки: застосунок має *відмовляти безпечно* (fail securely) — за винятком не продовжувати виконання в невизначеному стані, не трактувати помилку перевірки як дозвіл, звільняти ресурси й зберігати цілісність транзакції. Емпіричні дослідження відмов у промислових системах показують, що значна їх частина спричинена саме неправильним обробленням уже виявлених помилкових умов [304], тому шляхи оброблення помилок потребують такого самого тестування (розділ 15), як і основна логіка. Непродумане оброблення винятків у бізнес-логіці також створює стани, якими зловживають (подвій-

не нарахування за збою платежу) — це пов'язує A10 з безпекою за проєктом (розділ 4).

7.7 Кейс: програма запобігання ін'єкціям у життєвому циклі

7.7.1 Постановка

Відомство супроводжує портретний застосунок надання послуг (розділ 4) зі зверненнями до реляційної БД, зовнішніх реєстрів і файловим сховищем документів. Замість реагування на окремі знахідки поставлено системну ціль — *унеможливити* клас ін'єкцій засобами процесу розроблення, а не лише виправленнями. Вимоги взято з OWASP ASVS 5.0 [226] (розділ V1 «Кодування і санітизація», вимоги до звернень до БД і зовнішніх систем) і SSDF [275].

7.7.2 Заходи програми

Програму побудовано як поєднання стандартизації, автоматизації і обмеження наслідків (табл. 7.2). Пріоритет визначено тим, що найбільший і найдешевший ефект дає усунення *причини* на рівні платформи (єдиний рівень доступу до даних з параметризацією), а перевірки в конвеєрі лише стежать за дотриманням.

Таблиця 7.2 — Програма запобігання ін'єкціям

#	Захід	Рівень
P1	Єдиний рівень доступу до даних з параметризацією; заборона сирих фрагментів поза переліком винятків	платформа
P2	Білі списки для динамічних ідентифікаторів і схем; типізація введення за схемами	код
P3	Виклик зовнішніх програм лише без оболонки; безпечні типові налаштування XML; відмова від десеріалізації довільних об'єктів	платформа
P4	Контроль вихідних запитів: білий список, перевірка адрес, IMDSv2	застосунок + інфраструктура
P5	Структурне формування шляхів (7.3); окреме сховище вивантажень	код
P6	Статичний аналіз і аналіз складу залежностей у конвеєрі; позначення аварійних виходів	CI/CD
P7	Найменші привілеї БД, розділення даних, мінімізація відповідей, узагальнені помилки	експлуатація

7.7.3 Оцінювання ефекту

Ефект оцінюють двома показниками. Перший — *залишкова щільність* точок ризику: частка звернень до інтерпретаторів, що не проходять через безпечний за замовчуванням механізм (сирі фрагменти, динамічні ідентифікатори без білого списку, виклики оболонки). Ціль програми — звести

цю частку до контрольованого переліку винятків, кожен з яких має обґрунтування й додаткову перевірку. Другий — *очікуваний залишковий збиток* за (7.2): навіть якщо попри P1–P6 лишиться одна вада, заходи P7 (найменші привілеї, розділення, мінімізація) знижують I_0. Для оцінки: якщо обмеження привілеїв дає $m_1 = 0{,}6$, розділення даних $m_2 = 0{,}5$, мінімізація відповідей $m_3 = 0{,}3$, то залишковий збиток $I_0 \cdot 0{,}4 \cdot 0{,}5 \cdot 0{,}7 = 0{,}14\, I_0$ — менше ніж сьома частина. Так програма поєднує усунення причини (мета — нульова щільність некерованих точок) з обмеженням наслідків на випадок помилки, що відповідає принципу глибокого захисту (розділ 4).

Висновки до розділу

Ін'єкції різних інтерпретаторів мають спільну причину — змішування даних і коду, формалізоване умовою (7.1), — і тому єдиний принцип захисту: передавати дані інтерпретаторові окремим каналом або в типізованій формі, а не як частину тексту команди. Для реляційних БД його реалізують підготовлені запити й ORM, причому параметризувати можна лише значення, а динамічні ідентифікатори контролюють білим списком; аварійні виходи ORM лишаються точками ризику. Для оболонки ОС безпечним є виклик без оболонки з масивом аргументів, для XML — вимкнення зовнішніх сутностей, для обміну даними — формати зі схемою замість десеріалізації довільних об'єктів, для журналів — структуроване журналювання. Контроль вихідних запитів сервера (SSRF) будують багаторівнево — застосунковий білий список і перевірка адрес разом з мережевою ізоляцією та захищеною службою метаданих. Робота з файлами спирається на структурне формування й канонікалізацію шляху (7.3) і на недопущення виконання вивантаженого вмісту. Валідація за білим списком і канонікалізація доповнюють, але не замінюють структурне розділення, а нова категорія A10:2025 наголошує, що безпечне оброблення винятків — узагальнені повідомлення користувачеві, безпечна відмова, збереження цілісності — є невід'ємною частиною захисту. Системне розв'язання полягає не в реагуванні на окремі знахідки, а в усуненні причини засобами платформи й процесу розроблення з одночасним обмеженням можливих наслідків (7.2).

Питання для самоперевірки

1. Сформулюйте загальну причину ін'єкцій. Чому розуміння причини дає єдиний принцип захисту для різних інтерпретаторів?
2. Поясніть умову безпечності введення (7.1). Чому фільтрація «небезпечних символів» ненадійна, а структурне розділення надійне?

3. Що таке підготовлені запити і чому вони усувають причину, а не лише прояв? Які елементи запиту не можна параметризувати і як їх захищають?
4. За яких умов ORM не гарантує захисту? Запропонуйте метрику ризику на основі аварійних виходів ORM.
5. Чому виклик зовнішньої програми з масивом аргументів без оболонки безпечніший за передавання рядка? Коли зовнішню програму можна взагалі не залучати?
6. Який антипатерн робить шаблонізатор небезпечним? Як безпечно надати користувачам можливість шаблонізації листів?
7. Які типові налаштування XML-парсерів створюють ризик і як їх усувають? Чому десеріалізація довільних об'єктів небезпечна принципово?
8. Поясніть природу ризику SSRF (рис. 7.2). Чому перевірку адреси слід робити і до, і під час з'єднання, і яку роль відіграє IMDSv2?
9. Запишіть умову безпечного доступу до файлу (7.3). Чому перевірку виконують над канонічним, а не сирим шляхом?
10. Перелічіть заходи для безпечного приймання вивантажених файлів. Який спільний принцип їх об'єднує?
11. Чому валідація за білим списком є другорядним, а не основним захистом від ін'єкцій? Наведіть приклад легітимних даних, які строга валідація зіпсує.
12. Чому авторизаційні рішення і перевірки ухвалюють над канонічною формою введення? Що станеться за зворотного порядку?
13. У чому полягає категорія A10:2025 і як принцип «безпечної відмови» пов'язаний із захистом від ін'єкцій та з безпекою за проєктом (розділ 4)?
14. За (7.2) обчисліть залишковий збиток за $m_1 = 0{,}7$, $m_2 = 0{,}4$, $m_3 = 0{,}5$. Поясніть, чому обмеження привілеїв БД лишається обов'язковим навіть за надійної параметризації.
15. Дослідницьке: розробіть метод вимірювання «щільності точок ризику» (частки звернень до інтерпретаторів поза безпечними за замовчуванням механізмами) за вихідним кодом і оцініть його кореляцію з результатами динамічного тестування на вибірці відкритих вебзастосунків.
16. Дослідницьке: запропонуйте архітектурний патерн єдиного рівня доступу до даних, що робить параметризацію єдиним можливим способом формування запиту (сирі фрагменти неможливі без явного виня-

тку), і оцініть його вплив на залишкову щільність вад та на продуктивність розробки.

РОЗДІЛ 8

Автентифікація та керування сеансами

Розділи 5–7 забезпечили захищений канал, безпечну клієнтську частину і безпечну обробку даних на сервері. Наступне питання — хто саме звертається до застосунку. **Автентифікація** встановлює ідентичність суб'єкта, а **керування сеансами** підтримує її між запитами безстанового протоколу HTTP. Вади цих механізмів належать до категорії A07:2025 «Authentication Failures» OWASP Top 10 [231], а слабка автентифікація (CWE-287) і погане керування сеансами (CWE-384) — до CWE Top 25:2025 [54]. Компрометація автентифікації зводить нанівець усі інші заходи: порушник, що діє як легітимний користувач, проходить і TLS, і авторизацію. Розділ розглядає фактори й рівні автентифікації за NIST SP 800-63-4, безпечне зберігання та перевірку паролів, протидію масовим атакам на облікові дані, багатофакторну автентифікацію і фішингостійкі механізми FIDO2/WebAuthn та ключі доступу, відновлення облікового запису, керування сеансами, а також українські засоби електронної ідентифікації (кваліфікований електронний підпис, Дія.Підпис, BankID) і завершується кейсом підсистеми автентифікації державного порталу.

8.1 Фактори та рівні автентифікації

8.1.1 Фактори

Класично розрізняють фактори автентифікації: *знання* (пароль, PIN), *володіння* (апаратний ключ, смартфон, смарткартка) і *притаманність* (біометрія). **Багатофакторною** (MFA) автентифікація є лише тоді, коли поєднує фактори *різної природи*: пароль і код з SMS — два фактори (знання і володіння), а пароль і контрольне запитання — один фактор (обидва знання). Ключовою характеристикою є не кількість кроків, а *незалежність* факторів: компрометація одного не повинна спричиняти компрометацію іншого. Біометрія в вебзастосунках використовується не як фактор, переданий на сервер (відбиток не можна змінити після витоку), а як *локальне розблокування* ключа на пристрої користувача (підрозділ 8.4).

8.1.2 Рівні за NIST SP 800-63-4

Оновлена редакція настанов NIST SP 800-63-4 [141] (2025) розділяє три незалежні виміри: рівень підтвердження особи (IAL), рівень автентифікації (AAL) і рівень федерації (FAL). Для вебзастосунку визначальним є **AAL** — рівень упевненості, що обліковим записом керує його власник (табл. 8.1). Істотна зміна редакції 2025 р. — наголос на *фішингостійкості*: AAL3 ви-

магає автентифікатора, стійкого до перехоплення й ретрансляції (verifier impersonation resistance), яким є криптографічний ключ з прив'язкою до походження (FIDO2). Редакція також відмовляється від вимоги періодичної зміни паролів без причини і від підказок, вводить вимоги до доступності й конфіденційності.

Таблиця 8.1 — Рівні автентифікації AAL (NIST SP 800-63-4)

Рівень	Вимога	Приклад для вебзастосунку
AAL1	один або кілька факторів	пароль; типові послуги з низьким ризиком
AAL2	два різні фактори; криптографічний захист	пароль + TOTP/ключ; кабінет громадянина, оплати
AAL3	апаратний криптографічний автентифікатор, фішингостійкий	FIDO2-ключ; адміністрування, доступ до реєстрів

Вибір AAL — ризик-орієнтоване рішення (розділ 4): для перегляду публічної інформації достатньо AAL1, для дій з персональними даними й платежами обґрунтований AAL2, для привілейованих операцій і доступу до державних реєстрів — AAL3. Рівень може бути *динамічним*: застосунок вимагає сильнішого фактора при зростанні ризику операції (крокова автентифікація, step-up).

8.2 Паролі: зберігання та перевірка

8.2.1 Чому паролі зберігають як результат повільної функції

Пароль ніколи не зберігають у відкритому вигляді й не шифрують оборотно — його зберігають як результат **функції визначення ключа з пароля** (password hashing). До неї ставлять дві вимоги: *унікальна сіль* (salt) для кожного запису, щоб однакові паролі давали різні результати й унеможливлювали попереднє обчислення таблиць; *обчислювальна дорожнеча* й *вимогливість до пам'яті*, щоб масова перевірка кандидатів у разі витоку бази була якомога дорожчою. Сучасний рекомендований вибір — Argon2id [78] (переможець конкурсу Password Hashing Competition), як запасні — scrypt і bcrypt, а за потреби сумісності — PBKDF2 [209] з великою кількістю ітерацій. Вимогливість до пам'яті важлива тому, що вона зводить нанівець перевагу спеціалізованого обладнання, оптимізованого під обчислення, але не під обсяг швидкої пам'яті.

Вартість перевірки одного кандидата для нападника за витоку бази зростає пропорційно параметрам функції. Очікуваний час перебору простору

можливих паролів потужності N з часткою ϕ найімовірніших, що перевіряються першими, оцінюють як

$$T_{\text{переб}} = \frac{\phi\, N \cdot c_H}{R}, \qquad c_H \propto t \cdot m, \tag{8.1}$$

де c_H — вартість одного обчислення функції (зростає з параметром часу t і пам'яті m); R — доступна нападникові швидкість (обчислень за секунду на всьому обладнанні). Параметри t, m добирають так, щоб час перевірки легітимного входу був прийнятним (десятки мілісекунд), а c_H — максимальним у цих межах; збільшення m з 64 до 256 МБ підвищує c_H і водночас різко знижує R для масового обладнання. Це пояснює, чому *вимогливість до пам'яті* цінніша за просте збільшення ітерацій.

8.2.2 Політики паролів за сучасними настановами

Настанови NIST SP 800-63-4 [141] і дослідження практик [132] змінили підхід до політик. Рекомендовано: встановлювати *велику мінімальну довжину* (не менш як 8, краще 15 символів) і допускати довгі парольні фрази та всі символи Unicode, бо довжина важить більше за «складність»; *не нав'язувати* обов'язкових класів символів і регулярної зміни без ознак компрометації (це призводить до передбачуваних модифікацій); *перевіряти* новий пароль за списками раніше скомпрометованих і найпоширеніших і відхиляти збіги. Перевірку за списками витоку реалізують приватно за моделлю k-анонімності [156]: клієнт надсилає сервісу лише перші символи хешу пароля й отримує набір можливих закінчень, не розкриваючи самого пароля. Мета політики — не ускладнити життя користувачеві, а виключити паролі, уже відомі нападникам, бо саме вони використовуються в масових атаках (підрозділ 8.3).

8.3 Протидія масовим атакам на облікові дані

8.3.1 Природа загрози

Через повторне використання паролів витоки з одних сервісів живлять автоматизовані спроби входу на інші — клас загроз, у якому нападник не вгадує пароль конкретної людини, а масово перевіряє відомі пари «логін–пароль» або поширені паролі проти багатьох облікових записів. Це поєднується з переліком автоматизованих загроз OWASP (розділ 11). Ключова характеристика — масштаб: атака розподілена за багатьма адресами й обліковими записами, тож проста блокування за кількістю невдалих спроб на один обліковий запис недостатня.

8.3.2 Заходи захисту

Ефективний захист багаторівневий (табл. 8.2) і поєднує обмеження частоти, виявлення аномалій і, головне, фактори, не вразливі до повторного використання паролів. *Обмеження частоти* й прогресивні затримки застосовують за обліковим записом, за адресою та за пристроєм, з урахуванням розподіленості; корисною є модель черг і виявлення (розділ 3). *Повідомлення форм* уніфікують, щоб не розкривати, чи існує обліковий запис. *Виявлення* спирається на репутацію адрес, ознаки автоматизації й невідповідність звичній поведінці. Проте вирішальним є перехід на *фішингостійку MFA* і ключі доступу: вони роблять знання пароля недостатнім, тому масова перевірка облікових даних перестає давати доступ навіть за правильного пароля.

Таблиця 8.2 — Заходи проти масових атак на облікові дані

Захід	Призначення
Перевірка за списками витоку	виключити заздалегідь відомі паролі
Обмеження частоти й прогресивні затримки	уповільнити перебір за обліковим записом/адресою
Уніфіковані повідомлення форм	не розкривати наявність облікового запису
Протидія ботам (розділ 11)	відсіяти автоматизований трафік
Виявлення аномалій входу	реагувати на нетипові сесії
Фішингостійка MFA, ключі доступу	зробити знання пароля недостатнім
Сповіщення користувача про новий вхід	раннє виявлення компрометації

8.4 Багатофакторна та фішингостійка автентифікація

8.4.1 Межі традиційної MFA

Традиційні другі фактори різняться стійкістю. Коди з SMS вразливі до перехоплення й підміни SIM і вважаються найслабшим варіантом. Одноразові коди за часом (TOTP) і push-підтвердження надійніші, але всі вони *фішингостійкими не є*: сучасні набори для фішингу працюють як проксі в реальному часі (adversary-in-the-middle, AiTM) [204] — підставний сайт передає введені користувачем пароль і одноразовий код на справжній сервіс, а потім викрадає встановлений cookie сеансу. Тому одноразові коди підвищують вартість атаки, але не усувають фішинг, а push-підтвердження додатково вразливі до «втоми» користувача від потоку запитів (сценарій a2 дерева атак, розділ 4).

8.4.2 FIDO2/WebAuthn і ключі доступу

Фішингостійкість забезпечує криптографічна автентифікація з *прив'язкою до походження*. У стандартах FIDO2/WebAuthn [125, 300] на пристрої користувача генерується пара ключів для конкретного походження; закритий ключ не залишає пристрій (апаратний автентифікатор, захищений елемент смартфона) і розблоковується локально (біометрія, PIN). Під час входу браузер підписує виклик сервера разом з *даними про походження*, тож підпис, виданий на підставному домені, недійсний для справжнього — проксі-фішинг AiTM стає неможливим, бо ключ не спрацює для чужого походження. Стійкість підпису визначає стандартна криптографія (розділ 5); практичну межу задає передусім захист закритого ключа на пристрої й процедура його розблокування.

Ключі доступу (passkeys) [126] — облікові дані FIDO, синхронізовані між пристроями користувача через провайдера платформи, що усуває головну перешкоду поширення — втрату єдиного апаратного ключа й складність реєстрації. Вони забезпечують безпарольний вхід рівня AAL2 для масового користувача; для AAL3 застосовують несинхронізовані апаратні ключі, прив'язані до пристрою. Дослідження впровадження [184] показують, що основні перешкоди — не стійкість, а питання відновлення, облік кількох пристроїв і узгодженість підтримки, тож перехід проектують поетапно: спершу як додатковий спосіб, потім як основний, зі збереженням резервного шляху.

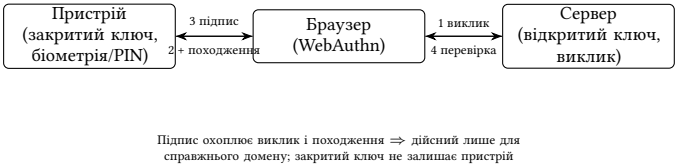


Рис. 8.1 — Фішингостійкий вхід за WebAuthn: прив'язка підпису до походження

8.4.3 Кількісний ефект фішингостійкості

Вплив вибору фактора оцінюють через залишкову річну ймовірність компрометації облікового запису. Якщо $p_{\text{ф}}$ — імовірність того, що користувач піддасться фішингу за рік, β — частка фішингу, що долає другий фактор (для AiTM-проксі близька до 1 щодо одноразових кодів і 0 щодо прив'язаних до походження ключів), $p_{\text{інш}}$ — імовірність компрометації іншими шляхами, то

$$P_{\text{компр}} = 1 - \bigl(1 - \beta\, p_{\text{ф}}\bigr)\bigl(1 - p_{\text{інш}}\bigr). \tag{8.2}$$

Для $p_{\text{ф}} = 0{,}1$, $p_{\text{інш}} = 0{,}01$: лише пароль ($\beta = 1$) дає $P \approx 0{,}109$; пароль з TOTP проти AiTM ($\beta \approx 0{,}9$) — $\approx 0{,}099$ (зниження незначне); ключ доступу ($\beta \approx 0$) — $\approx 0{,}01$, тобто на порядок менше. Висновок кількісно підтверджує якісну тезу: проти сучасного фішингу цінність має саме прив'язка до походження, а не наявність другого фактора як такого.

8.5 Відновлення облікового запису

Відновлення доступу — частий *найслабший* шлях: сувора автентифікація знецінюється, якщо скинути її можна через слабку процедуру. «Секретні запитання» ненадійні (відповіді часто публічні або вгадні) і не рекомендовані. Відновлення через електронну пошту чи телефон переносить довіру на ці канали, тож їх захищають не слабше за сам обліковий запис, а посилання на скидання роблять одноразовими, короткочинними й прив'язаними до запиту. Для безпарольних систем передбачають *кілька* зареєстрованих ключів або резервні коди, що зберігаються як результат повільної функції (8.1). Процедури відновлення для привілейованих облікових записів посилюють (перевірка кількома каналами, участь іншого адміністратора), бо саме вони — ціль цілеспрямованих атак. Рівень упевненості відновлення не повинен бути нижчим за рівень автентифікації, яку воно скидає, — інакше AAL облікового запису фактично визначає найслабша процедура.

8.6 Керування сеансами

8.6.1 Життєвий цикл сеансу

Оскільки HTTP безстановий, після входу застосунок видає **ідентифікатор сеансу**, який браузер повертає з кожним запитом. Вимоги до нього [239]: достатня *ентропія* (щонайменше 128 бітів справжньої випадковості, щоб унеможливити вгадування, — та сама умова, що й для токенів у розділі 3); генерація криптографічно стійким джерелом; відсутність у значенні будь-якої інформації про користувача. Правильне місце зберігання — cookie з атрибутами `HttpOnly` (недоступність скрипту — обмеження наслідків XSS, розділ 6), `Secure` (лише HTTPS, розділ 5), `SameSite` (протидія CSRF) і префіксом `__Host-` (прив'язка до походження, заборона встановлення з піддоменів). Зберігання токена сеансу у `localStorage` небажане, бо робить його доступним будь-якому скрипту сторінки.

8.6.2 Ключові переходи

Безпека сеансу визначається коректністю переходів його життєвого циклу (рис. 8.2): ідентифікатор *оновлюють* одразу після входу і після зміни рівня привілеїв, щоб значення, можливо нав'язане до входу, не стало дійсним (протидія фіксації сеансу); задають *обмеження* часу бездіяльності й абсолютного часу життя відповідно до чутливості; забезпечують *справжнє завер-*

шення сеансу на сервері при виході (а не лише видалення cookie); надають користувачеві перегляд активних сеансів і можливість їх відкликати. Чутливі операції захищають повторною (кроковою) автентифікацією навіть у межах дійсного сеансу.

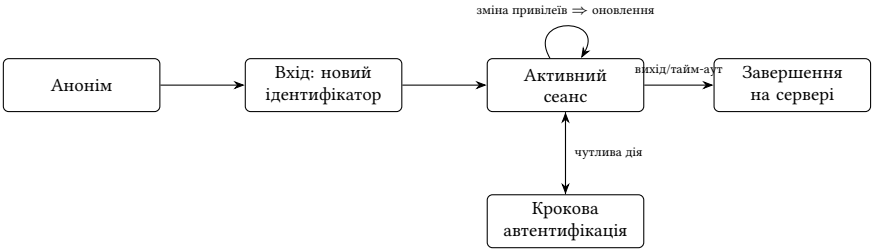


Рис. 8.2 — Життєвий цикл сеансу й ключові переходи

8.6.3 Прив'язка сеансу до пристрою

Навіть за бездоганної автентифікації залишковим ризиком лишається *викрадення встановленого cookie* сеансу (через шкідливе ПЗ на пристрої чи AiTM-проксі): маркер, перенесений на інший пристрій, надає доступ. Механізм **прив'язки сеансу до пристрою** (Device Bound Session Credentials, DBSC) [294] усуває цей клас: браузер зберігає в захищеному елементі пристрою закритий ключ сеансу і періодично підтверджує ним володіння, тож викрадений cookie на іншому пристрої стає марним. У моделі дерева атак (розділ 4) саме прив'язка до пристрою закриває сценарій, що лишався домінантним після впровадження фішингостійкої MFA. Для привілейованих робочих станцій прив'язку доповнюють вимогою апаратного ключа й перевіркою стану пристрою.

8.7 Українські засоби електронної ідентифікації

Правовою основою електронної ідентифікації в Україні є Закон «Про електронну ідентифікацію та електронні довірчі послуги» [34], гармонізований з підходами eIDAS [251]. Для вебресурсів, що надають послуги громадянам і бізнесу, застосовують кілька засобів високого рівня довіри. **Кваліфікований електронний підпис** (КЕП) на захищеному носії або хмарний (Дія.Підпис [22]) забезпечує і автентифікацію, і юридично значуще підписання дій — це криптографічний фактор володіння на основі національних стандартів (розділ 5). Система **BankID** Національного банку [24] дає змогу ідентифікувати особу через банк, у якому вона вже пройшла процедуру знайомства з клієнтом, що відповідає високому рівню підтвердження особи (IAL). Застосування цих засобів для доступу до державних реєстрів і зна-

чущих операцій реалізує рівні AAL2–AAL3 і водночас вимогу неспростовності (розділ 4); дослідження впровадження MFA в державних органах [43] наголошують на потребі резервних шляхів і врахування доступності для різних груп користувачів. Інтеграцію цих засобів проектують за тими самими принципами безпеки сеансів і федерації (розділ 9), а не як окремий «довірений» канал поза загальною моделлю.

8.8 Кейс: підсистема автентифікації державного порталу

8.8.1 Вихідний стан і цілі

Портал державних послуг (розділ 4, сценарій R1) автентифікує громадян паролем з необов'язковим TOTP, посадовців — паролем, адміністраторів — паролем з push-підтвердженням. Аудит за NIST SP 800-63-4 [141] і OWASP ASVS 5.0 [226] (розділи автентифікації і керування сеансами) виявив: пароль зберігається з однією функцією без вимогливості до пам'яті; немає перевірки за списками витоку; MFA необов'язкова; відновлення — через контрольні запитання; ідентифікатор сеансу не оновлюється після входу; cookie без префікса `__Host-`; відсутня прив'язка до пристрою; адміністративний і користувацький входи спільні. Цілі — привести рівні до ризику операцій: AAL2 для громадян, AAL3 для адміністраторів і доступу до реєстрів — і закрити залишковий ризик викрадення сеансу.

8.8.2 План заходів

Таблиця 8.3 — План модернізації автентифікації порталу

#	Захід	Рівень/ефект
A1	Argon2id з вимогливістю до пам'яті; перевірка нових паролів за списками витоку	зберігання, (8.1)
A2	Ключі доступу як основний спосіб для громадян; збереження пароля+TOTP як резерв	AAL2, (8.2)
A3	Апаратні ключі FIDO2 для адміністраторів і доступу до реєстрів; крокова автентифікація	AAL3
A4	Інтеграція КЕП/Дія.Підпис і BankID для значущих дій	неспростовність, IAL
A5	Відновлення без секретних запитань; кілька ключів, резервні коди; посилене відновлення для привілейованих	8.5
A6	Оновлення ідентифікатора після входу; `__Host-`, `HttpOnly`, `Secure`, `SameSite`; завершення на сервері; перегляд сеансів	керування сеансами
A7	Прив'язка сеансу до пристрою (DBSC); окремий домен і посилені станції для адміністрування	залишковий ризик викрадення

Пріоритет надано A1 (дешеве зниження наслідків можливого витоку бази), A2 і A3 (перехід на фішингостійкі фактори, що за (8.2) знижує ймовірність компрометації на порядок і закриває сценарій AiTM), A5 (відновлення

не має бути слабшим за автентифікацію). А6 усуває фіксацію сеансу й узгоджує cookie з розділами 5–6. А7 закриває залишковий ризик викрадення сеансу, що в дереві атак розділу 4 лишався домінантним після сильної автентифікації. Розведення адміністративного й користувацького входів на окремі домени узгоджується з поняттям сайту (розділ 6).

8.8.3 Результат

Після заходів знання пароля перестає бути достатнім для входу; компрометація бази паролів дає нападникові лише дорогу для перебору вибірку (8.1); викрадений cookie марний на іншому пристрої; відновлення й привілейований доступ підняті до відповідного рівня; значущі дії підписуються КЕП. Залишкові ризики: втрата всіх зареєстрованих ключів користувачем (компенсується резервними кодами й процедурою відновлення), компрометація пристрою з локальним розблокуванням ключа (компенсується прив'язкою й моніторингом, розділ 16) і питання доступності безпарольного входу для частини користувачів (збережений резервний шлях).

Висновки до розділу

Автентифікація встановлює ідентичність, а керування сеансами підтримує її в безстановому HTTP; їхня компрометація знецінює решту заходів. Багатофакторність визначає не кількість кроків, а незалежність факторів, а рівні AAL за NIST SP 800-63-4 обирають за ризиком операції, з наголосом редакції 2025 р. на фішингостійкості. Паролі зберігають як результат повільної, вимогливої до пам'яті функції з унікальною сіллю (8.1), а політики спрямовують на довжину і виключення вже скомпрометованих паролів, а не на штучну «складність» і регулярну зміну. Проти масових атак на облікові дані вирішальним є не обмеження частоти саме по собі, а перехід на фактори, не вразливі до повторного використання паролів. Традиційна MFA з одноразовими кодами підвищує вартість атаки, але не усуває сучасний проксі-фішинг AiTM; фішингостійкість забезпечує лише криптографічна автентифікація з прив'язкою до походження (FIDO2/WebAuthn, ключі доступу), що кількісно знижує ймовірність компрометації на порядок (8.2). Відновлення облікового запису не повинно бути слабшим за автентифікацію, яку воно скидає. Безпека сеансу визначається ентропією ідентифікатора, атрибутами cookie і коректністю переходів життєвого циклу, а залишковий ризик викрадення сеансу закриває прив'язка до пристрою (DBSC). Українські засоби — КЕП, Дія.Підпис, BankID — забезпечують високі рівні довіри й неспростовність і інтегруються в загальну модель автентифікації та федерації.

Питання для самоперевірки

1. Чому багатофакторність визначається незалежністю факторів, а не кількістю кроків? Чому пароль і контрольне запитання — це один фактор?
2. Опишіть рівні AAL за NIST SP 800-63-4 (табл. 8.1). Для яких операцій державного порталу обґрунтовані AAL1, AAL2 і AAL3?
3. Чому пароль зберігають як результат повільної, вимогливої до пам'яті функції з унікальною сіллю? Яку роль відіграє сіль, а яку — вимогливість до пам'яті?
4. За (8.1) поясніть, чому збільшення параметра пам'яті ефективніше за просте збільшення ітерацій проти спеціалізованого обладнання.
5. Які сучасні рекомендації щодо політик паролів і чим вони відрізняються від традиційних вимог до «складності» та регулярної зміни?
6. Як реалізують перевірку пароля за списками витоку, не розкриваючи самого пароля (k-анонімність)?
7. Чому обмеження частоти за одним обліковим записом недостатнє проти масових атак на облікові дані? Перелічіть заходи табл. 8.2 і їх призначення.
8. Чому одноразові коди (TOTP, SMS) не є фішингостійкими? Поясніть механізм проксі-фішингу AiTM.
9. Як FIDO2/WebAuthn забезпечує фішингостійкість (рис. 8.1)? Чому підпис, виданий на підставному домені, недійсний для справжнього?
10. За (8.2) обчисліть річну ймовірність компрометації для $p_{\text{ф}} = 0{,}15$, $p_{\text{інш}} = 0{,}02$ для пароля, пароля з TOTP ($\beta = 0{,}9$) і ключа доступу ($\beta = 0$). Який висновок?
11. Чому відновлення облікового запису часто є найслабшим шляхом? Які вимоги ставлять до відновлення і чому його рівень не має бути нижчим за AAL?
12. Перелічіть вимоги до ідентифікатора сеансу й атрибути cookie сеансу. Чому токен не варто зберігати у `localStorage`?
13. Які переходи життєвого циклу сеансу (рис. 8.2) є критичними для безпеки? Навіщо оновлювати ідентифікатор після входу?
14. Який залишковий ризик закриває прив'язка сеансу до пристрою (DBSC) і чому він лишається домінантним навіть після фішингостійкої MFA (зв'язок з деревом атак розділу 4)?
15. Дослідницьке: спроєктуйте й оцініть поетапний перехід державного порталу на безпарольний вхід ключами доступу з урахуванням від-

новлення, кількох пристроїв і доступності для різних груп користувачів; визначте метрики успішності та показник залишкового ризику.

16. Дослідницьке: порівняйте рівні довіри й безпекові властивості КЕП, Дія.Підпис і BankID як засобів автентифікації для вебресурсів органів влади й запропонуйте модель вибору засобу залежно від IAL/AAL операції та вимоги неспростовності.

РОЗДІЛ 9

Авторизація, федеративна ідентифікація та керування доступом

Розділ 8 встановив, *хто* звертається до застосунку. Цей розділ відповідає на наступне питання — *що* цьому суб'єктові дозволено. **Авторизація** (керування доступом) — найчастіше джерело критичних вад вебзастосунків: категорія A01:2025 «Broken Access Control» очолює OWASP Top 10 [231], а відсутня авторизація (CWE-862) входить до верхівки CWE Top 25:2025 [54]. На відміну від ін'єкцій, порушення доступу не має універсального структурного «запобіжника»: кожен об'єкт і кожна дія потребують перевірки, яку легко пропустити. Розділ розглядає моделі доступу (DAC, RBAC, ABAC, ReBAC) і політику як код, типові порушення контролю доступу та архітектурні засоби їх уникнення, протокол авторизації OAuth 2.0/2.1 з сучасними практиками безпеки (RFC 9700), федеративну ідентифікацію OpenID Connect і SAML, безпечне застосування токенів JWT, принцип нульової довіри для вебдоступу та завершується кейсом централізованої авторизації державного порталу з інтеграцією національної системи електронної ідентифікації.

9.1 Моделі керування доступом

9.1.1 Від дискреційної до атрибутної моделі

Керування доступом визначає, чи дозволено суб'єктові s виконати дію a над об'єктом o у контексті c. Історично склалися кілька моделей (табл. 9.1). *Дискреційна* (DAC) закріплює права за власником ресурсу; у чистому вигляді для вебзастосунків вона погано масштабується. *Рольова* (RBAC) [262] вводить проміжний рівень ролей: права призначають ролям, а ролі — користувачам, що радикально спрощує адміністрування у великих системах. *Атрибутна* (ABAC) [153] ухвалює рішення за правилами над атрибутами суб'єкта, об'єкта, дії й середовища, що дає тонку, контекстнозалежну політику (час, місце, рівень довіри пристрою). *Реляційна* (ReBAC) виражає доступ через граф відношень між суб'єктами й об'єктами («є власником», «є членом команди, якій надано документ») і добре відповідає моделі спільного доступу сучасних вебзастосунків; промисловим прикладом масштабованої ReBAC є система Zanzibar [242].

Таблиця 9.1 — Моделі керування доступом

Модель	Основа рішення	Доречність для вебу
DAC	власник надає права	прості випадки, спільний доступ до файлів
RBAC	роль користувача	організаційні системи зі сталими ролями
ABAC	атрибути s, o, a, c	тонка контекстна політика, нульова довіра
ReBAC	граф відношень	спільний доступ, багатокористувацькі документи

На практиці моделі поєднують: ролі задають грубий рівень, атрибути й відношення уточнюють. Формально рішення авторизації — це предикат над суб'єктом, дією, об'єктом і контекстом, обчислений за політикою Π за правилом *заборони за замовчуванням*:

$$\mathrm{Permit}(s,a,o,c) = \bigvee_{r\in\Pi} \mathrm{match}_r(s,a,o,c) \;\wedge\; \neg \bigvee_{d\in\Pi} \mathrm{deny}_d(s,a,o,c), \qquad (9.1)$$

де match_r — спрацювання дозвільного правила r; deny_d — спрацювання заборонного правила (заборони мають пріоритет); за відсутності дозвільного правила доступ *заборонено*. Саме порядок кванторів утілює ключовий принцип: дозвіл надається лише явним правилом, а не відсутністю заборони.

9.1.2 Політика як код

Поширений антипатерн — розсіяні по коду перевірки доступу (`if role == ...`), які важко переглянути на повноту й узгодженість. Сучасний підхід — **політика як код** (policy as code): правила авторизації виносять у декларативну політику, що обчислюється окремим механізмом рішень (policy decision point), а код застосунку лише ставить запит і виконує відповідь. Приклади — Open Policy Agent з мовою Rego [223] і системи на основі ReBAC [242]. Переваги: єдине місце визначення політики, можливість її тестування й аудиту, відокремлення логіки доступу від бізнес-логіки. Архітектурно рішення ухвалює *точка рішення*, а застосовує — *точка застосування* (policy enforcement point) у кожній кінцевій точці; критично, щоб жодна чутлива дія не оминала точку застосування.

9.2 Порушення контролю доступу та їх уникнення

9.2.1 Класи порушень

Категорія A01:2025 охоплює кілька класів, спільна риса яких — *відсутня або неправильна перевірка* на боці сервера (табл. 9.2). Найпоширеніший — доступ до чужого об'єкта за прямим ідентифікатором (CWE-639), коли застосунок перевіряє автентифікацію, але не перевіряє, що саме цей користувач має право на саме цей об'єкт. Інші — відсутня перевірка функції (адміністративна дія доступна через відоме звернення), вертикальне підвищення (отримання вищих прав) і горизонтальне (доступ до даних рівного користувача), довіра до рішення, ухваленого лише на клієнті, масове присвоєння полів, що включає службові (роль, статус), та обхід багатокрокових процесів.

Таблиця 9.2 — Класи порушень доступу та архітектурні заходи

Клас	CWE	Архітектурний захід
Доступ до чужого об'єкта	639	перевірка належності об'єкта в точці застосування; непрямі ідентифікатори
Відсутня перевірка функції	862	заборона за замовчуванням; централізована авторизація
Підвищення привілеїв	269	єдина модель ролей на сервері; тести авторизації
Довіра до клієнта	602	усі рішення на сервері; клієнт — лише зручність
Масове присвоєння	915	білий список полів, доступних для запису
Обхід кроків процесу	841	серверний контроль стану й порядку кроків

9.2.2 Архітектурні засоби

Оскільки універсального «запобіжника» немає, захист будують архітектурно. По-перше, *повна опосередкованість*: кожне звернення до об'єкта чи функції проходить через єдину точку застосування, а не через розсіяні перевірки. По-друге, *заборона за замовчуванням* (9.1): нова кінцева точка недоступна, доки доступ не надано явно; це переносить ризик з «забули заборонити» на «забули дозволити», що виявляється під час тестування одразу. По-третє, *перевірка належності* на рівні запиту до даних: вибірка завжди обмежується об'єктами, доступними поточному суб'єктові (фільтр за власником/відношенням у самому запиті), а не перевіряється після отримання. По-четверте, *білі списки полів* для запису, щоб введення не могло змінити службові атрибути. По-п'яте, *тести авторизації* в конвеєрі (розділ 15): для кожної захищеної дії — автоматична перевірка, що неавторизований суб'єкт отримує відмову. Непрямі ідентифікатори (видача користувачеві непрогнозованого посилання замість послідовного номера) не замінюють перевірку належності, а лише ускладнюють перебір.

Залишкову частку вразливих до порушення доступу кінцевих точок оцінюють як

$$\rho_{\text{ac}} = \frac{n_{\text{прям}}}{N}\big(1 - c\big), \tag{9.2}$$

де N — загальна кількість захищених кінцевих точок; $n_{\text{прям}}$ — кількість тих, що звертаються до об'єктів за ідентифікатором; $c \in [0; 1]$ — покриття автоматичними тестами авторизації. Централізація точки застосування зменшує $n_{\text{прям}}$ (усі проходять єдиний механізм), а тести авторизації збільшують c; разом вони зводять ρ_{ac} до контрольованого мінімуму. Це кількісно пояснює, чому розсіяні перевірки без тестів дають велику залишкову частку навіть за старанного кодування.

9.3 OAuth 2.0/2.1: делегована авторизація

9.3.1 Призначення і ролі

OAuth 2.0 [147] розв'язує задачу *делегованого доступу*: дати застосунку обмежений доступ до ресурсів користувача в іншому сервісі *без передавання пароля*. Ролі: *власник ресурсу* (користувач), *клієнт* (застосунок), *сервер авторизації* (видає токени) і *сервер ресурсів* (приймає токени). Результат — **маркер доступу** (access token) з обмеженим обсягом (scope) і терміном дії. Важливо розрізняти: OAuth — протокол *авторизації* делегованого доступу, а не автентифікації користувача; використання маркера доступу як «доказу входу» є поширеною помилкою, яку виправляє OpenID Connect (підрозділ 9.4).

9.3.2 Потік коду авторизації з PKCE

Рекомендований і для вебзастосунків, і для мобільних та односторінкових клієнтів — **потік коду авторизації** (authorization code flow) з розширенням **PKCE** [259] (рис. 9.1). Клієнт генерує випадковий *верифікатор* і надсилає його хеш (*виклик*) на сервер авторизації; після згоди користувача сервер повертає одноразовий *код*, який клієнт обмінює на токени, пред'являючи верифікатор. Навіть якщо код перехоплено, без верифікатора він марний, що закриває клас атак з перехопленням коду. OAuth 2.1 [148] робить PKCE обов'язковим для всіх клієнтів, вилучає небезпечні потоки (неявний і за паролем) і кодифікує сучасні практики.

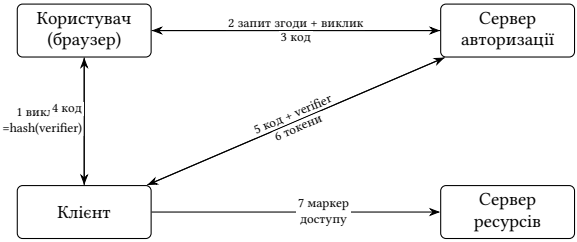


Рис. 9.1 — Потік коду авторизації OAuth з PKCE

9.3.3 Практики безпеки OAuth (RFC 9700)

Формальний аналіз [123] і накопичений досвід узагальнено в RFC 9700 «OAuth 2.0 Security Best Current Practice» [192] (2025). Ключові вимоги: *точна звірка* зареєстрованих URI перенаправлення (без шаблонів і підрядків — запобігає викраденню коду через відкрите перенаправлення); обов'язковий PKCE; параметр `state` і прив'язка до сеансу проти CSRF на потоці; захист від підміни сервера авторизації й *змішування* відповідей кількох серверів (видавець у відповіді має звірятися); короткочинні маркери доступу й *ротація* маркерів оновлення з виявленням повторного використання; прив'язка маркера до клієнта. Для усунення вразливості маркера «на пред'явника» (будь-хто, хто ним володіє, отримує доступ) застосовують **підтвердження володіння** (DPoP) [122] або прив'язку до взаємного TLS: маркер стає дійсним лише разом із ключем клієнта, тож викрадений маркер марний.

9.4 Федеративна ідентифікація

9.4.1 OpenID Connect

OpenID Connect (OIDC) [260] — рівень автентифікації над OAuth 2.0, що розв'язує саме задачу «хто ввійшов»: окрім маркера доступу, сервер видає **токен ідентичності** (ID token) у форматі JWT з перевіреними твердженнями про користувача (ідентифікатор, час автентифікації, рівень AAL). Це дає **єдиний вхід** (SSO): користувач автентифікується в постачальника ідентичності, а застосунки-довірники приймають токен ідентичності. Безпека OIDC успадковує практики OAuth [192] і додає перевірку токена ідентичності: підпис, видавець, аудиторія, час, значення `nonce` проти повторення. Клієнт повинен перевіряти, що токен призначений *саме йому* (аудиторія) і виданий *очікуваним* постачальником, інакше токен, виданий для іншого застосунку, може бути пред'явлений повторно.

9.4.2 SAML і вибір протоколу

SAML — старіший стандарт федерації на основі XML, досі поширений в організаційних системах. Його безпека критично залежить від коректної перевірки XML-підпису: історичні вади реалізацій (обгортання підпису, XSW) дозволяли підміну тверджень; крім того, обробка XML повертає ризики XXE (розділ 7). Для нових вебзастосунків зазвичай обирають OIDC як простіший і менш схильний до таких вад, а SAML лишають для інтеграції з наявними корпоративними постачальниками. У будь-якому разі федерація переносить довіру на постачальника ідентичності, тож його компрометація означає компрометацію всіх довірників — це враховують у моделі загроз (розділ 4) і вимагають від постачальника рівня захисту, не нижчого за найчутливіший довірник.

9.5 Безпечне застосування токенів JWT

JSON Web Token (JWT) [174] — компактний самоописовий токен із твердженнями, підписаний (JWS) видавцем. Його зручність — у можливості перевірити токен без звернення до сховища, але саме це породжує типові вади, зведені в RFC 8725 «JWT Best Current Practices» [267]. Основні вимоги: *завжди звірятися алгоритм* зі списком очікуваних і ніколи не приймати `alg: none` та не дозволяти зловмиснику нав'язати заміну асиметричного алгоритму симетричним; перевіряти *підпис*, *видавця*, *аудиторію* і *час* дії; не зберігати в JWT чутливих даних (payload лише закодований, не зашифрований); враховувати, що *відкликати* самоописовий токен до завершення терміну складно, тому термін роблять коротким, а для сеансів застосунку часто надійніше використовувати серверні сеанси (розділ 8), а не довгочинні JWT. Для сценаріїв, де потрібне відкликання, ведуть список відкликаних ідентифікаторів токенів або використовують короткі маркери доступу з ротованими маркерами оновлення (підрозділ 9.3).

9.6 Нульова довіра для вебдоступу

Принцип **нульової довіри** (zero trust) [258, 299] відкидає припущення, що розташування в «внутрішній» мережі є підставою для довіри: кожен запит автентифікується й авторизується незалежно, з урахуванням ідентичності, стану пристрою й контексту. Для вебдоступу це означає: немає «периметра», за яким застосунки відкриті; доступ до внутрішніх вебресурсів опосередковується шлюзом, що на кожен запит перевіряє автентифікацію й політику (ABAC за атрибутами користувача, пристрою, часу, чутливості ресурсу); привілейовані дії потребують сильнішого підтвердження (крокова автентифікація, розділ 8). Модель узгоджується з політикою як код (точка рішення обчислює доступ динамічно) і з мікросегментацією (розділ 12).

Практично нульову довіру впроваджують поступово, починаючи з найчутливіших ресурсів і привілейованого доступу, бо суцільне перенесення всіх застосунків коштовне.

9.7 Кейс: централізована авторизація державного порталу

9.7.1 Вихідний стан

Портал державних послуг (розділ 4, сценарії R2, R6) має розсіяні по коду перевірки доступу; API довідок приймає ідентифікатор документа й повертає його без перевірки належності (ризик доступу до чужого витягу, CWE-639); адміністративні функції доступні через відомі звернення без окремої перевірки ролі; частина рішень ухвалюється на клієнті; форма оновлення профілю приймає всі поля, зокрема роль (масове присвоєння); вхід громадян інтегрований з національною системою електронної ідентифікації id.gov.ua [23] за OIDC, але токен ідентичності перевіряється лише за підписом, без звірки аудиторії. Аудит за OWASP ASVS 5.0 [226] (розділ авторизації) і дослідженнями моделей доступу для критичної інфраструктури [19] виявив високу залишкову частку (9.2) через відсутність централізації й тестів.

9.7.2 План заходів

Таблиця 9.3 — План централізації авторизації порталу

#	Захід	Клас (табл. 9.2)
Z1	Єдина точка застосування; політика як код (RBAC+ABAC); заборона за замовчуванням	862, повнота
Z2	Перевірка належності в запиті до даних для всіх API об’єктів	639
Z3	Білі списки полів для запису; заборона зміни службових атрибутів	915
Z4	Перенесення всіх рішень на сервер; клієнт — лише відображення	602
Z5	Тести авторизації в CI для кожної захищеної дії	покриття *c*
Z6	Повна перевірка токена OIDC: підпис, видавець, аудиторія, nonce, час	федерація
Z7	Короткі маркери доступу, ротація оновлення, DPoP для доступу до реєстрів; нульова довіра для адміністрування	9.3, 9.6

Пріоритет надано Z2 (прямо експлуатований доступ до чужих витягів — найтяжчий наслідок для персональних даних), Z1 і Z5 (централізація плюс тести за (9.2) системно знижують залишкову частку, а не латають окремі місця), Z6 (без звірки аудиторії токен, виданий для іншого довірника національної системи, міг би бути пред’явлений порталу). Z3 і Z4 усувають масове присвоєння й довіру до клієнта. Z7 посилює захист доступу до дер-

жавних реєстрів і застосовує нульову довіру до адміністративного контуру, узгоджуючись із розділенням доменів (розділи 6, 8).

9.7.3 Результат

Після заходів кожна чутлива дія проходить єдину точку застосування із забороною за замовчуванням; вибірка даних обмежена об'єктами користувача на рівні запиту; службові поля недоступні для запису; токени федерації перевіряються повністю; доступ до реєстрів — за короткими підтвердженими володінням маркерами. Залишкова частка (9.2) зводиться до контрольованого переліку, що покривається тестами. Залишкові ризики: компрометація постачальника ідентичності (вимоги до рівня його захисту, моніторинг), складність політики як джерело помилок (тестування й рецензування політики як коду) і внутрішній зловмисник з легітимними правами (мінімізація привілеїв, розділення обов'язків, журналювання привілейованих дій, розділ 16).

Висновки до розділу

Авторизація визначає, що дозволено автентифікованому суб'єктові, і є найчастішим джерелом критичних вад (A01:2025), бо не має універсального структурного запобіжника: кожен об'єкт і дія потребують перевірки. Моделі доступу — DAC, RBAC, ABAC, ReBAC — на практиці поєднують, а правила виносять у політику як код з окремими точками рішення й застосування. Рішення ухвалюють за принципом заборони за замовчуванням (9.1), а захист будують архітектурно: повна опосередкованість, перевірка належності в запиті до даних, білі списки полів, тести авторизації, що разом зводять залишкову частку вразливих кінцевих точок до мінімуму (9.2). OAuth 2.0/2.1 забезпечує делеговану авторизацію з обов'язковим PKCE, точною звіркою URI перенаправлення й підтвердженням володіння маркером (DPoP) згідно з RFC 9700; OpenID Connect додає автентифікацію з перевіреним токеном ідентичності, а JWT застосовують за практиками RFC 8725 зі звіркою алгоритму, видавця й аудиторії. Нульова довіра відкидає довіру за розташуванням і авторизує кожен запит за ідентичністю, станом пристрою й контекстом. Федерація переносить довіру на постачальника ідентичності, рівень захисту якого має відповідати найчутливішому довірнику.

Питання для самоперевірки

1. Порівняйте моделі DAC, RBAC, ABAC і ReBAC (табл. 9.1) за основою рішення й доречністю для вебзастосунків. Наведіть приклад поєднання моделей.
2. Поясніть предикат рішення (9.1) і принцип заборони за замовчуванням. Чому порядок кванторів утілює цей принцип?

3. Що таке політика як код? Яку роль відіграють точка рішення і точка застосування, і чому жодна дія не повинна оминати останню?
4. Чому порушення контролю доступу не мають універсального структурного запобіжника, як ін'єкції? Назвіть класи порушень табл. 9.2.
5. Опишіть п'ять архітектурних засобів проти порушень доступу. Чому непрямі ідентифікатори не замінюють перевірку належності?
6. За (9.2) поясніть, як централізація точки застосування й тести авторизації знижують залишкову частку. Обчисліть ρ_{ac} для $N = 200$, $n_{\text{прям}} = 80$, $c = 0{,}9$.
7. Яку задачу розв'язує OAuth 2.0 і чому маркер доступу не є доказом автентифікації користувача?
8. Опишіть потік коду авторизації з PKCE (рис. 9.1). Чому перехоплений код без верифікатора марний?
9. Перелічіть ключові вимоги RFC 9700. Що таке підтвердження володіння маркером (DPoP) і яку вразливість маркера-на-пред'явника воно усуває?
10. Чим OpenID Connect доповнює OAuth? Які поля токена ідентичності й чому клієнт повинен перевіряти аудиторію і видавця?
11. Які ризики несе SAML і чому для нових застосунків часто обирають OIDC? Що означає перенесення довіри на постачальника ідентичності?
12. Назвіть типові вади застосування JWT за RFC 8725. Чому не можна приймати `alg: none` і зберігати чутливі дані в payload?
13. Сформулюйте принцип нульової довіри для вебдоступу. Як він пов'язаний з ABAC, кроковою автентифікацією і мікросегментацією?
14. Чому відкликання самоописового JWT складне і які є підходи до його розв'язання?
15. Дослідницьке: розробіть метод автоматичної генерації тестів авторизації за специфікацією API і моделлю доступу та оцініть досягнуте покриття c і виявлення штучно внесених порушень належності на відкритому вебзастосунку.
16. Дослідницьке: запропонуйте модель політики як код (RBAC+ABAC) для доступу до державних електронних реєстрів через систему id.gov.ua з урахуванням рівнів AAL/IAL, вимоги неспростовності та журналювання, і оцініть її супроводжуваність і ризик помилок політики.

РОЗДІЛ 10

Безпека API та мікросервісних архітектур

Сучасні вебресурси дедалі частіше — це не монолітні сайти, а набори **програмних інтерфейсів** (API), якими користуються односторінкові застосунки, мобільні клієнти, партнерські системи й інші сервіси. Логіка розподілена між мікросервісами, що спілкуються між собою, а публічні API стають основною поверхнею атаки. Специфіку відображає окремий перелік — OWASP API Security Top 10:2023 [235], у якому на перших місцях саме порушення авторизації на рівні об'єктів і функцій. Розділ розглядає архітектурні стилі API (REST, GraphQL, gRPC, WebSocket, вебхуки) і їхні безпекові особливості, перелік OWASP API Top 10, проблему інвентаризації та тіньових API, засоби захисту периметра (шлюзи, обмеження частоти, схеми як контракт), міжсервісну авторизацію (mTLS, робочі ідентичності, сервісна сітка) і завершується кейсом захисту API державного порталу. Розділ спирається на авторизацію (розділ 9) і транспортний захист (розділ 5) і доповнює їх специфікою API.

10.1 Архітектурні стилі API та їхня безпека

10.1.1 REST, GraphQL, gRPC, WebSocket

REST [128] — найпоширеніший стиль: ресурси адресують URL, дії виражають методами HTTP, стан не зберігають між запитами. Безпекові наслідки: кожна кінцева точка потребує власної перевірки авторизації (розділ 9), а безстановість означає, що автентифікація передається з кожним запитом (токен). **GraphQL** [140] надає клієнтові єдину кінцеву точку й мову запитів, якою він сам визначає потрібні поля; це зручно, але створює специфічні ризики: складні вкладені запити здатні спричинити надмірне навантаження (підрозділ 10.4), а авторизацію треба перевіряти на рівні кожного *поля* й вузла графа, а не кінцевої точки. **gRPC** [143] — бінарний протокол поверх HTTP/2 для міжсервісної взаємодії зі строгими контрактами; типово застосовується всередині з mTLS. **WebSocket** [124] встановлює двонапрямлений канал; його особливість у тому, що на нього *не поширюється* політика того самого походження і CORS, тож походження перевіряють на сервері вручну (інакше виникає міжсайтове захоплення з'єднання), а авторизацію — не лише під час рукостискання, а й для повідомлень.

10.1.2 Вебхуки

Вебхуки — зворотні HTTP-виклики, якими сервіс сповіщає інший про подію. Як *приймач* вебхука застосунок має перевіряти *автентичність* викли-

ку (підпис тіла спільним секретом), захищатися від *повторення* (мітка часу й одноразовий ідентифікатор), а також враховувати, що оброблення вебхука, яке саме звертається за наданим у ньому URL, є вектором SSRF (розділ 7). Таблиця 10.1 зводить стилі та їхні ключові безпекові акценти.

Таблиця 10.1 — Стилі API та безпекові акценти

Стиль	Застосування	Безпековий акцент
REST	публічні й внутрішні API	авторизація кожної кінцевої точки й об'єкта
GraphQL	гнучкі клієнтські запити	авторизація на рівні полів; обмеження складності
gRPC	міжсервісна взаємодія	mTLS, робочі ідентичності
WebSocket	двонапрямлений обмін	перевірка походження; авторизація повідомлень
Вебхуки	сповіщення між сервісами	підпис, захист від повторення, ризик SSRF

10.2 OWASP API Security Top 10

10.2.1 Домінування вад авторизації

Перелік OWASP API Security Top 10:2023 [235] відображає, що специфічні для API ризики — передусім авторизаційні (табл. 10.2). *Порушення авторизації на рівні об'єкта* (BOLA, API1) — те саме, що доступ до чужого об'єкта (розділ 9), але для API воно особливо поширене, бо клієнт явно передає ідентифікатори об'єктів. *Порушення авторизації на рівні властивостей* (BOPLA, API3) поєднує читання зайвих полів у відповіді й масове присвоєння на запис. *Порушення авторизації на рівні функцій* (BFLA, API5) — доступ до дій, не призначених ролі. Ці три класи підтверджують висновок розділу 9: авторизація — головне джерело ризику, і для API вона потребує перевірки на рівні кожного об'єкта, кожного поля й кожної функції.

Таблиця 10.2 — Вибрані ризики OWASP API Security Top 10:2023

Код	Ризик	Захід
API1	авторизація об’єктів (BOLA)	перевірка належності в запиті до даних
API2	вади автентифікації	практики розділу 8; захист токенів
API3	авторизація властивостей (BOPLA)	білі списки полів на читання й запис
API4	необмежене споживання ресурсів	квоти, обмеження частоти, пагінація
API5	авторизація функцій (BFLA)	заборона за замовчуванням, перевірка ролі
API6	зловживання бізнес-потоками	контроль логіки, протидія ботам
API7	SSRF	контроль вихідних запитів (розділ 7)
API8	хибна конфігурація	посилення, керування конфігурацією (розділ 12)
API9	неналежне керування інвентарем	інвентаризація, виявлення тіньових API
API10	небезпечне споживання сторонніх API	валідація відповідей, обмеження довіри

10.3 Інвентаризація та тіньові API

Специфічний для API ризик — **тіньові** (shadow) і **застарілі** (zombie) API: кінцеві точки, не враховані в документації й не захищені поточною політикою, — старі версії, тестові інтерфейси, забуті інтеграції. Галузеві звіти [261] стабільно вказують на них як на поширену причину інцидентів: захист застосовують до відомих API, а атакують невідомі. Корінь проблеми в тому, що ризик визначає *повнота* інвентаризації: якщо з N наявних кінцевих точок інвентаризовано й захищено частку κ, а кожна неврахована має ймовірність p_v містити ваду, то очікувана кількість відкритих вразливих точок

$$\mathrm{E}[V] = N\,(1-\kappa)\,p_v. \tag{10.1}$$

Для $N = 300, \kappa = 0{,}8, p_v = 0{,}1$ маємо $\mathrm{E}[V] = 6$ — шість відкритих вразливих точок попри захист 80 % відомих. Звідси заходи: ведення *реєстру API* як частини процесу (нова версія не розгортається без реєстрації), автоматичне виявлення кінцевих точок за трафіком і специфікаціями, виведення застарілих версій за графіком, а не їх безстрокове існування. Інвентаризація — передумова будь-якого іншого захисту: не можна захистити те, про існування чого невідомо (зв’язок з розділом 13 щодо складу ПЗ).

10.4 Захист периметра API

10.4.1 Шлюз API

Шлюз API (API gateway) — єдина точка входу, що централізує наскрізні функції: завершення TLS, автентифікацію й передавання перевіреної ідентичності далі, обмеження частоти й квоти, валідацію за схемою, журналювання. Централізація узгоджується з повною опосередкованістю (розділ 9): шлюз — природна точка застосування політик периметра. Проте шлюз *не замінює* авторизацію в сервісах: рішення на рівні об'єкта й поля (BOLA, BOPLA) потребують знання бізнес-контексту, якого на шлюзі немає, тому їх ухвалює сам сервіс. Шлюз закриває периметрові ризики, а доменну авторизацію лишають сервісу — це розподіл відповідальності, а не дублювання.

10.4.2 Схема як контракт безпеки

Специфікація **OpenAPI** [225] (для REST) чи схема GraphQL — не лише документація, а *контракт*, який можна застосувати для захисту: шлюз і сервіс відхиляють запити й відповіді, що не відповідають схемі (невідомі поля, неправильні типи, завеликі значення). Це реалізує позитивну модель безпеки (дозволено лише описане) і одночасно протидіє масовому присвоєнню й надмірним відповідям. Схема також є джерелом для автоматичної генерації тестів (зокрема тестів авторизації, розділ 9) і фазингу (розділ 15). Розбіжність між задекларованою схемою й фактичною поведінкою — сама по собі ризик, тож відповідність контролюють автоматично.

10.4.3 Обмеження частоти та споживання ресурсів

Ризик API4 «необмежене споживання ресурсів» усувають *квотами* й *обмеженням частоти* на рівні користувача, клієнта й операції, *пагінацією* (заборона необмежених вибірок), *тайм-аутами* й *запобіжниками* (circuit breakers) [222]. Для GraphQL додатково обмежують *складність* і *глибину* запиту, бо один синтаксично короткий запит здатен породити комбінаторно великий обсяг роботи [149]. Оцінку вартості запиту обчислюють до виконання:

$$\mathrm{Cost}(q) = \sum_{f \in q} w_f \prod_{a \in \mathrm{anc}(f)} m_a \;\le\; C_{\max}, \tag{10.2}$$

де w_f — базова вартість поля f; $\mathrm{anc}(f)$ — предки поля в дереві запиту; m_a — множник кількості елементів на рівні a (списки); $C_{\max}$ — дозволений бюджет. Запити з $\mathrm{Cost}(q) > C_{\max}$ відхиляють до виконання, що перетворює потенційно необмежене навантаження на кероване. Ці заходи змикаються з протидією відмові в обслуговуванні (розділ 11).

10.5 Міжсервісна авторизація

10.5.1 Ідентичність робочих навантажень

У мікросервісній архітектурі довіра «за розташуванням у внутрішній мережі» неприйнятна (нульова довіра, розділ 9): кожен сервіс має власну **робочу ідентичність** (workload identity) і автентифікується перед іншими. Стандартом де-факто є SPIFFE [276]: сервіс отримує криптографічну ідентичність (SVID) у формі сертифіката чи токена, яку перевіряють за взаємним TLS. **Взаємний TLS** (mTLS) [256] забезпечує, що обидві сторони з'єднання автентифіковані, а канал зашифрований; це закриває і прослуховування внутрішнього трафіку, і підміну сервісу. Облікові дані робочих навантажень роблять короткочинними з автоматичною ротацією, щоб обмежити наслідки витоку.

10.5.2 Сервісна сітка і межі

Сервісна сітка (service mesh) [168] виносить mTLS, політику доступу й телеметрію на рівень інфраструктури (бічні проксі), знімаючи це навантаження з коду сервісів і забезпечуючи однакове застосування політик. Проте сітка забезпечує авторизацію на рівні *сервіс-сервіс* (який сервіс може викликати який метод), але не доменну авторизацію на рівні об'єкта (чи має *цей користувач* доступ до *цього запису*) — остання лишається в сервісі. Поширена помилка — вважати, що mTLS між сервісами усуває потребу авторизації користувача всередині; насправді потрібне наскрізне передавання й перевірка ідентичності користувача (через підписаний токен) на кожному рівні. Таким чином, безпека мікросервісів — це два незалежні шари: інфраструктурний (хто з ким може говорити) і доменний (що дозволено користувачеві), і жоден не замінює інший.

10.6 Кейс: захист API державного порталу

10.6.1 Вихідний стан

API державного порталу (розділи 4, 9) обслуговує мобільний застосунок, вебклієнт і партнерські інтеграції. Аудит за OWASP API Security Top 10:2023 [235], ASVS 5.0 [226] і дослідженням безпеки API в електронному урядуванні [45] виявив: кінцеві точки довідок вразливі до BOLA (ідентифікатор документа без перевірки належності); відповіді містять зайві поля (BOPLA на читання); відсутній реєстр API, виявлено дві незадокументовані старі версії (тіньові API); немає обмеження частоти на дорогих вибірках (API4); GraphQL-кінцева точка аналітики без обмеження складності; внутрішні виклики між сервісами без mTLS, у припущенні довіри до внутрішньої мережі; вебхуки партнерів без перевірки підпису. Портал обробляє персональні дані, тож BOLA й BOPLA мають найтяжчі наслідки.

10.6.2 План заходів

Таблиця 10.3 — План захисту API порталу

#	Захід	Ризик
G1	Перевірка належності в запиті до даних для всіх об'єктних кінцевих точок	API1 (BOLA)
G2	Білі списки полів на читання й запис; валідація за OpenAPI на шлюзі й у сервісі	API3 (BOPLA)
G3	Реєстр API; виявлення й виведення тіньових/застарілих версій	API9, (10.1)
G4	Обмеження частоти, квоти, пагінація; обмеження складності GraphQL	API4, (10.2)
G5	mTLS і робочі ідентичності між сервісами; наскрізне передавання ідентичності користувача	міжсервісна
G6	Підпис і захист від повторення для вебхуків; контроль вихідних запитів	вебхуки, API7
G7	Тести авторизації за схемою в CI для кожної кінцевої точки, об'єкта й поля	покриття

Пріоритет — G1 і G2 (прямо експлуатований доступ до чужих документів і витік зайвих полів персональних даних), G3 (за (10.1) дві тіньові версії — це вже відкриті вразливі точки поза політикою), G4 (захист доступності, зокрема GraphQL за (10.2)). G5 усуває хибне припущення довіри до внутрішньої мережі, але з наголосом, що mTLS не замінює доменну авторизацію G1. G7 робить авторизацію перевірюваною й пов'язує розділ з тестуванням (розділ 15).

10.6.3 Результат

Після заходів кожен об'єкт і поле API захищені перевіркою належності й білими списками; невідомих кінцевих точок немає, старі версії виведено; дорогі й складні запити обмежені бюджетом; внутрішні виклики автентифіковані mTLS, але авторизація користувача перевіряється наскрізно; вебхуки автентичні. Залишкові ризики: складність політик на рівні полів (покривається тестами G7 і схемою), компрометація робочої ідентичності сервісу (короткочинні облікові дані, ротація, моніторинг, розділ 16) і довіра до партнерських API (валідація відповідей, обмеження обсягу довіри, API10).

Висновки до розділу

Вебресурси дедалі частіше складаються з API й мікросервісів, тож API стають основною поверхнею атаки, а специфіку відображає OWASP API Security Top 10:2023, у якому домінують вади авторизації на рівні об'єктів (BOLA), властивостей (BOPLA) і функцій (BFLA). Різні стилі — REST, GraphQL, gRPC, WebSocket, вебхуки — потребують різних акцентів: по-об'єктної авторизації, авторизації на рівні полів і обмеження складності,

перевірки походження, автентичності й захисту від повторення. Інвентаризація — передумова захисту, бо тіньові й застарілі API є відкритими вразливими точками поза політикою (10.1). Шлюз API централізує периметрові функції й застосовує схему OpenAPI як контракт безпеки, але не замінює доменну авторизацію в сервісах. Необмежене споживання ресурсів усувають квотами, обмеженням частоти, пагінацією й обмеженням складності запитів (10.2). Міжсервісна безпека — два незалежні шари: інфраструктурний (mTLS, робочі ідентичності, сервісна сітка — хто з ким може говорити) і доменний (що дозволено користувачеві), причому перший не замінює другий, а ідентичність користувача передається й перевіряється наскрізно.

Питання для самоперевірки

1. Порівняйте REST, GraphQL, gRPC і WebSocket (табл. 10.1) за застосуванням і безпековими акцентами. Чому на WebSocket не поширюється SOP/CORS і що з цього випливає?
2. Які вимоги висувають до приймача вебхука і чому оброблення вебхука може бути вектором SSRF?
3. Чому в OWASP API Top 10:2023 домінують вади авторизації? Поясніть різницю між BOLA, BOPLA і BFLA.
4. Що таке тіньові й застарілі API? За (10.1) обчисліть $\mathrm{E}[V]$ для $N = 500$, $\kappa = 0{,}7$, $p_v = 0{,}08$ і поясніть роль повноти інвентаризації.
5. Які функції централізує шлюз API і чому він не замінює авторизацію на рівні об'єкта в сервісі?
6. Як схема OpenAPI чи GraphQL стає контрактом безпеки? Яку модель безпеки (позитивну чи негативну) вона реалізує?
7. За (10.2) поясніть, чому короткий запит GraphQL може спричинити велике навантаження і як обмеження складності це усуває. Обчисліть вартість для простого прикладу вкладених списків.
8. Що таке робоча ідентичність навантаження і як mTLS забезпечує міжсервісну автентифікацію? Навіщо робити облікові дані сервісів короткочинними?
9. Чому mTLS між сервісами не замінює доменну авторизацію користувача? Опишіть два незалежні шари безпеки мікросервісів.
10. Які периметрові функції доцільно винести в сервісну сітку і які рішення мають лишатися в сервісі?
11. Чому інвентаризацію API вважають передумовою будь-якого іншого захисту? Як організувати реєстр API в процесі розроблення?
12. Які заходи усувають ризик API4 «необмежене споживання ресурсів» для REST і окремо для GraphQL?

13. Як перевіряти автентичність і захищатися від повторення вебхуків? Чому цього недостатньо без контролю вихідних запитів?
14. Поясніть, чому для API авторизацію треба перевіряти на рівні кожного об'єкта, поля й функції, а не лише кінцевої точки.
15. Дослідницьке: розробіть метод автоматичного виявлення тіньових і застарілих API за мережевим трафіком і специфікаціями та оцініть його повноту (частку виявлених неінвентаризованих кінцевих точок) на тестовому середовищі.
16. Дослідницьке: запропонуйте методику наскрізної перевірки ідентичності користувача в ланцюзі мікросервісів (поєднання mTLS робочих ідентичностей і підписаного токена користувача) і оцініть її вплив на затримку та стійкість до підміни внутрішнього виклику.

РОЗДІЛ 11

Захист від відмови в обслуговуванні та автоматизованих загроз

Попередні розділи захищали конфіденційність і цілісність; цей розглядає *доступність* — властивість, атаки на яку (відмова в обслуговуванні, DoS) не потребують обходу автентифікації чи авторизації, а просто вичерпують ресурс. Для державних вебресурсів доступність має особливу вагу: недоступність порталу послуг чи реєстру — це безпосередня шкода громадянам, а досвід 2022–2026 рр. показав, що розподілені атаки є постійним складником гібридних кампаній. Поряд із DoS розділ розглядає ширший клас **автоматизованих загроз** — діяльність ботів, що зловживають легітимною функціональністю (перебір, скупка, вишкрібання даних). Розглянуто таксономію DDoS, підсилення, рекордні атаки 2025–2026 рр. (HTTP/2 Rapid Reset, ботнет Aisuru), засоби пом'якшення (anycast, CDN, скрубінг), перелік OWASP Automated Threats, протидію ботам (CAPTCHA та приватні токени доступу), моделі й оцінювання вебекранів прикладного рівня (WAF/WAAP) та український досвід, із завершальним кейсом. Розділ спирається на кількісні моделі масового обслуговування (розділ 3) і транспортну інфраструктуру (розділ 5).

11.1 Таксономія атак на доступність

11.1.1 Рівні та класи

Атаки на відмову в обслуговуванні класифікують за рівнем, на якому вичерпується ресурс (табл. 11.1). *Об'ємні* (volumetric) заповнюють смугу пропускання великим потоком трафіку; їхня ефективність вимірюється в бітах за секунду. *Протокольні* вичерпують стани з'єднань чи таблиці (напіввідкриті з'єднання, стани на балансувальниках); вимірюються в пакетах за секунду. *Прикладні* (рівня 7) надсилають відносно небагато запитів, кожен з яких дорогий для сервера (складний пошук, генерація звіту); вимірюються в запитах за секунду і найнебезпечніші тим, що трафік виглядає легітимним. Окремий випадок — *асиметричні* атаки, де мала дія нападника спричиняє велику роботу сервера (регулярний вираз з катастрофічним відкотом, розпакування «бомби», дорогий GraphQL-запит, розділ 10).

Таблиця 11.1 — Класи атак на доступність

Клас	Ресурс / міра	Пом'якшення
Об'ємні	смуга / біт·с$^{-1}$	anycast, скрубінг, запас смуги
Протокольні	стани / пакет·с$^{-1}$	SYN cookies, обмеження станів, QUIC-правила
Прикладні	процесор, БД / запит·с$^{-1}$	обмеження частоти, кеш, захист дорогих операцій
Асиметричні	процесор / одиниця	безпечні алгоритми, тайм-аути, ліміти

11.1.2 Підсилення

Об'ємні атаки часто спираються на **підсилення** (розділ 3): нападник надсилає малі запити з підробленою адресою жертви до сторонніх служб, що відповідають значно більшими пакетами на адресу жертви. Коефіцієнт підсилення й необхідна смуга нападника

$$A = \frac{S_{\text{відп}}}{S_{\text{запит}}}, \qquad B_{\text{жертва}} = A \cdot B_{\text{нап}}, \tag{11.1}$$

де $S_{\text{відп}}$, $S_{\text{запит}}$ — розміри відповіді й запиту; B — смуги. Для вразливо налаштованих служб A сягає десятків (DNS), сотень (NTP) і десятків тисяч (memcached) [183], тож навіть скромна смуга нападника перетворюється на потужний потік до жертви. Системний захід проти підсилення — усунення самої можливості: заборона підробки адрес джерела на мережах операторів (BCP 38) і коректне налаштування служб, щоб вони не відповідали невідомим. На боці жертви підсилення гасять поглинанням трафіку в розподіленій інфраструктурі (підрозділ 11.2). Протокол QUIC (розділ 5) обмежує підсилення самого сервера втричі (5.6).

11.1.3 Рекордні атаки 2025–2026 років

Масштаб загрози швидко зростає. Атака **HTTP/2 Rapid Reset** (CVE-2023-44487) [138] використала асиметрію мультиплексування HTTP/2 (розділ 5): швидке відкриття й негайне скасування потоків давало рекордні на той час сотні мільйонів запитів за секунду при малих ресурсах нападника; захист — обмеження частоти створення й скасування потоків і оновлення серверів. Об'ємні рекорди встановлюють ботнети з уражених пристроїв інтернету речей: за традицією Mirai [63] (2016), що уражав пристрої типовими обліковими даними, у 2025–2026 рр. ботнет Aisuru спричинив атаки, які провідні провайдери оцінювали на рівні десятків терабітів за секунду, зокрема рекорд близько 31,4 Тбіт/с [99, 100]. Урок подвійний: по-перше, безпека пристроїв інтернету речей є суспільною проблемою, бо їх використовують проти третіх сторін; по-друге, самостійно поглинути такі обсяги майже не-

можливо, тож захист спирається на спеціалізовану розподілену інфраструктуру.

11.2 Пом'якшення атак на доступність

11.2.1 Поглинання і розподіл

Основою захисту від об'ємних і протокольних атак є *розподіл* і *запас потужності*. **Anycast** оголошує одну адресу з багатьох географічно рознесених вузлів, тож трафік атаки розподіляється між ними й поглинається ближче до джерел, не досягаючи єдиної точки [210]. **Мережі доставки вмісту** (CDN, розділ 5) додають сукупну смугу, що на порядки перевищує можливості окремого сервера, і **центри очищення** (scrubbing centers), які відділяють атакувальний трафік від легітимного. Ефективність поглинання визначає співвідношення сукупної потужності інфраструктури й піку атаки; при достатньому запасі атака не доходить до джерела. Критична умова, повторена з розділу 5: захист діє, лише якщо *весь* трафік проходить через інфраструктуру, а адреса джерела прихована, інакше атака обходить захист прямим зверненням.

11.2.2 Модель вичерпання ресурсу

Для прикладних атак корисна модель масового обслуговування (розділ 3): сервер з c обслуговувачами й інтенсивністю обслуговування μ запитів за секунду насичується, коли сукупна інтенсивність легітимних (λ_ℓ) і атакувальних (λ_a) запитів наближається до пропускної здатності. Частка відмовлених легітимних запитів і умова стабільності

$$\rho = \frac{\lambda_\ell + \lambda_a}{c\,\mu}, \qquad P_{\text{відмова}} \approx \max\!\Big(0,\; 1 - \frac{c\,\mu}{\lambda_\ell + \lambda_a}\Big) \text{ при } \rho > 1, \tag{11.2}$$

показує, що для прикладної атаки нападникові вигідно збільшувати не кількість запитів, а їхню *вартість* (зменшувати μ): кілька дуже дорогих запитів насичують сервер так само, як потік дешевих. Звідси пріоритет захисту дорогих операцій: кешування, обмеження частоти саме дорогих кінцевих точок, винесення важких обчислень у черги з контролем, тайм-аути й ліміти на операцію. Модель також обґрунтовує *ізоляцію*: окремий пул ресурсів для дорогих операцій не дає їм вичерпати обслуговування дешевих.

11.2.3 Стійкість і відновлення

Жоден захист не гарантує абсолютної доступності, тож планують *деградацію* (вимкнення необов'язкових важких функцій під навантаженням зі збереженням критичних), *пріоритезацію* автентифікованих користувачів і черги очікування замість відмов, а також цілі відновлення (RTO/RPO, роз-

діл 16). Готовність до атак перевіряють навчаннями й тестами навантаження в межах, погоджених із провайдером.

11.3 Автоматизовані загрози та протидія ботам

11.3.1 Перелік OWASP Automated Threats

Не всякий шкідливий автоматизований трафік є DoS: значна частина ботів *зловживає легітимною функціональністю*. Перелік OWASP Automated Threats to Web Applications [236] систематизує такі сценарії (OAT): масова перевірка облікових даних (розділ 8), перебір карток, скупка дефіцитних товарів чи талонів, вишкрібання даних (scraping), спам, розвідка. Спільне в них — запити формально коректні, тож класичні засоби їх не відрізняють; потрібне розрізнення *людина проти автомата* й *нормальна поведінка проти аномальної*. Для державних порталів типові сценарії — масове автоматизоване замовлення довідок чи талонів (зв'язок зі сценарієм R6 розділу 4) і вишкрібання персональних даних з публічних реєстрів.

11.3.2 CAPTCHA та приватні токени доступу

Класичний засіб розрізнення — **CAPTCHA** [293], задача, легка для людини й важка для автомата. Її межі відомі: сучасні засоби розв'язують багато типів CAPTCHA автоматично або через сервіси людей-розв'язувачів, а сама вона погіршує доступність і зручність, зокрема для людей з інвалідністю. Тому акцент зміщується на *поведінкові* й *репутаційні* сигнали й на криптографічні підтвердження. **Приватні токени доступу** (Private Access Tokens на основі Privacy Pass) [245] дають змогу пристрою чи платформі *засвідчити*, що запит походить від реального користувача на справжньому пристрої, не розкриваючи його особи: сервер отримує непов'язуваний токен-підтвердження. Це зберігає приватність (на відміну від суцільного відстеження) і зменшує потребу в CAPTCHA. Проте жоден сигнал не є абсолютним, тож протидію ботам будують як багатосигнальне ризик-орієнтоване рішення, а не єдиний бар'єр.

11.3.3 Межі протидії та баланс помилок

Будь-яке розрізнення «бот/людина» має дві помилки: пропуск бота й хибне блокування людини. Якість оцінюють як у теорії виявлення (розділ 3): за частки автоматів π у трафіку, ймовірності виявлення p_d і хибного блокування p_f частка помилково заблокованих серед усіх заблокованих

$$\mathrm{FDR} = \frac{(1-\pi)\,p_f}{(1-\pi)\,p_f + \pi\,p_d}. \tag{11.3}$$

Для $\pi = 0{,}2$, $p_d = 0{,}9$, $p_f = 0{,}05$ маємо $\mathrm{FDR} = 0{,}04/(0{,}04+0{,}18) \approx 0{,}18$ — майже кожне п'яте блокування зачіпає людину. Для державного порталу,

де хибне блокування позбавляє громадянина доступу до послуги, допустимий p_f має бути малим, а реакція — поступовою (додаткова перевірка, а не відмова). Це той самий компроміс, що й для WAF (підрозділ 11.4): чутливіше розрізнення ловить більше ботів ціною більшої шкоди легітимним користувачам.

11.4 Вебекрани прикладного рівня (WAF/WAAP)

11.4.1 Моделі та архітектура

Вебекран прикладного рівня (Web Application Firewall, WAF) аналізує HTTP-запити й відповіді, блокуючи ті, що відповідають ознакам атак. Сучасні рішення об'єднують WAF, протидію ботам, захист від DDoS і безпеку API в єдину платформу — **WAAP** (Web Application and API Protection). WAF працює за двома моделями [65]: *негативною* (блокувати відоме погане за сигнатурами, як типовий набір правил) і *позитивною* (дозволяти лише відоме добре за моделлю нормального трафіку чи схемою API, розділ 10). Позитивна модель потенційно надійніша, але потребує точного опису норми; негативна простіша, але відстає від нових технік. Дедалі частіше додають моделі машинного навчання для виявлення аномалій.

11.4.2 Роль і межі

WAF — важливий *компенсувальний* і детективний засіб, але не заміна безпечної розробки. Він корисний як «віртуальне латання» (швидке блокування експлуатації відомої вади до виправлення коду), як рівень глибокого захисту й як джерело телеметрії для моніторингу (розділ 16). Проте WAF, що діє за формою запитів, у принципі не бачить вад бізнес-логіки й авторизації (BOLA, розділ 10), бо такі запити синтаксично коректні, і може бути обійдений кодуваннями. Тому в моделі ризику (розділ 4) WAF знижує ймовірність експлуатації, але не усуває ваду, а покладатися лише на нього небезпечно.

11.4.3 Оцінювання ефективності

Ефективність WAF оцінюють на двох показниках: *повнота блокування* атак і *хибні спрацювання* на легітимному трафіку. Їх подають ROC-кривою (розділ 3), а робочу точку обирають за вартістю помилок. Корисна інтегральна оцінка — очікувана вартість помилок за одиницю часу

$$\mathrm{E}[C] = \lambda_a\,(1 - p_d)\,C_{\text{пропуск}} + \lambda_\ell\,p_f\,C_{\text{блок}}, \tag{11.4}$$

де λ_a, λ_ℓ — інтенсивності атакувальних і легітимних запитів; $C_{\text{пропуск}}$ — вартість пропущеної атаки; $C_{\text{блок}}$ — вартість хибного блокування. Мінімізація $\mathrm{E}[C]$ визначає робочу точку: для державного порталу $C_{\text{блок}}$ (недоступність послуги громадянину) часто значна, тож WAF налаштовують на

режим спостереження чи м'якого втручання для нових правил, а жорстке блокування вмикають після калібрування. Правила тестують на реальному трафіку в режимі лише-журнал перед увімкненням — так само, як CSP (розділ 6).

11.5 Український досвід

Державні вебресурси України від 2022 р. перебувають під постійними DDoS-атаками як складником гібридних кампаній (розділ 1). Досвід засвідчив ефективність кількох підходів: перенесення критичних ресурсів під захист розподілених провайдерів із достатньою сукупною смугою; географічний розподіл і резервування; координація реагування через CERT-UA [9] і обмін індикаторами; готовність до деградації функціональності зі збереженням критичних послуг. Дослідження протидії DDoS державних ресурсів [4] наголошують на потребі завчасних домовленостей із провайдерами захисту й регулярних навчань, бо розгортання захисту під час атаки менш ефективне. Окремим явищем стала скоординована добровольча кіберактивність («IT Army») [272]; з погляду цього посібника важливий саме *захисний* бік — стійкість власних ресурсів, а будь-які наступальні дії лежать поза межами навчального курсу з безпеки вебресурсів і регулюються законодавством (розділ 2).

11.6 Кейс: захист доступності державного порталу

11.6.1 Вихідний стан

Портал державних послуг (розділ 4, сценарій R4) у пікові періоди (подання звітності, запис на послуги) зазнає і сплесків легітимного навантаження, і цілеспрямованих DDoS. Аудит виявив: CDN без розвиненої протидії ботам і прикладному DDoS; дорогі кінцеві точки (пошук у реєстрі, генерація витягу) без обмеження частоти й окремого пулу; адреса джерела відома (розділ 5); масове автоматизоване замовлення талонів (сценарій R6); відсутність плану деградації; домовленості з провайдером захисту укладаються реактивно. За моделлю (11.2) кілька десятків паралельних дорогих запитів насичують сервер.

11.6.2 План заходів

Таблиця 11.2 — План захисту доступності порталу

#	Захід	Клас загрози
D1	Весь трафік через розподілений захист; приховування адреси джерела (mTLS, фільтрація)	об'ємні, обхід
D2	Обмеження частоти й окремий пул для дорогих кінцевих точок; кеш; тайм-аути	прикладні, (11.2)
D3	Протидія ботам: поведінкові сигнали, приватні токени доступу, поступова перевірка	OAT, R6
D4	Безпечні алгоритми й ліміти проти асиметричних атак (ReDoS, розпакування, GraphQL)	асиметричні
D5	WAAP у режимі калібрування за (11.4); телеметрія в SIEM	компенсувальний
D6	План деградації, пріоритезація автентифікованих, цілі RTO/RPO	стійкість
D7	Завчасні домовленості з провайдером, навчання, обмін із CERT-UA	готовність

Пріоритет — D1 і D2: без проходження всього трафіку через захист і приховування джерела решта заходів обходиться, а ізоляція й обмеження дорогих операцій за (11.2) закривають найдешевший для нападника прикладний вектор. D3 усуває масове автоматизоване замовлення, але з малим p_f (11.3), щоб не блокувати громадян. D5 вмикають поступово, мінімізуючи (11.4), бо хибне блокування позбавляє доступу до послуги. D6 і D7 переносять частину захисту на організаційну готовність, що відповідає уроку українського досвіду.

11.6.3 Результат

Після заходів об'ємні атаки поглинаються розподіленою інфраструктурою; дорогі операції ізольовані й обмежені, тож прикладна атака не насичує сервер; масові боти відсіюються з малою шкодою легітимним користувачам; під навантаженням портал деградує керовано зі збереженням критичних послуг. Залишкові ризики: наднові рекордні об'ємні атаки (залежність від потужності провайдера, запас і координація), витончені «людиноподібні» боти (постійне донавчання моделей, баланс (11.3)) і ризик хибного блокування (моніторинг рівня відмов легітимних користувачів як окрема метрика).

Висновки до розділу

Атаки на доступність не потребують обходу автентифікації й авторизації, а вичерпують ресурс, і для державних вебресурсів недоступність — безпосередня шкода громадянам. Атаки поділяють на об'ємні, протокольні, прикладні й асиметричні; об'ємні часто спираються на підсилення (11.1), системний захист від якого — заборона підробки адрес і коректне налаштування служб. Рекордні атаки 2025–2026 рр. (HTTP/2 Rapid Reset, ботнет Aisuru обсягом десятки терабітів за секунду) показали, що самостійне поглинання

майже неможливе, тож захист спирається на anycast, CDN і центри очищення за умови, що весь трафік проходить через інфраструктуру, а джерело приховане. Для прикладних атак модель масового обслуговування (11.2) обґрунтовує пріоритет захисту дорогих операцій та їх ізоляції. Автоматизовані загрози (OWASP OAT) зловживають легітимною функціональністю, а протидія ботам зміщується від CAPTCHA до поведінкових сигналів і приватних токенів доступу з урахуванням балансу помилок (11.3). WAF/WAAP — важливий компенсувальний і детективний засіб, але не заміна безпечної розробки: він не бачить вад логіки й авторизації, а його робочу точку обирають мінімізацією очікуваної вартості помилок (11.4). Український досвід наголошує на завчасній готовності, розподіленому захисті, деградації й координації через CERT-UA.

Питання для самоперевірки

1. Класифікуйте атаки на доступність за рівнем і ресурсом (табл. 11.1). Чому прикладні атаки найважче відрізнити від легітимного трафіку?
2. Поясніть механізм підсилення (11.1). Чому системний захист — це заборона підробки адрес і налаштування служб, а не лише дії на боці жертви?
3. У чому полягала асиметрія HTTP/2 Rapid Reset? Який подвійний урок дають об'ємні атаки з ботнетів інтернету речей?
4. Як anycast і CDN поглинають об'ємні атаки? Чому захист діє лише за умови проходження всього трафіку через інфраструктуру й приховування джерела?
5. За моделлю (11.2) поясніть, чому для прикладної атаки нападникові вигідніше збільшувати вартість запиту, ніж їх кількість. Як це обґрунтовує ізоляцію дорогих операцій?
6. Обчисліть $P_{\text{відмова}}$ за (11.2) для $c\mu = 1000$ запитів/с, $\lambda_\ell = 400$, $\lambda_a = 1200$. Які заходи знижують ρ?
7. Що таке автоматизовані загрози (OWASP OAT)? Наведіть сценарії, актуальні для державного порталу, і поясніть, чому класичні засоби їх не відрізняють.
8. Які межі CAPTCHA? Як приватні токени доступу розрізняють людину й автомата, зберігаючи приватність?
9. За (11.3) обчисліть частку помилково заблокованих людей для $\pi = 0{,}3$, $p_d = 0{,}95$, $p_f = 0{,}03$. Чому для державного порталу важливий малий p_f?
10. Порівняйте негативну й позитивну моделі WAF. Чому позитивна потенційно надійніша і що для неї потрібне?

11. Чому WAF не замінює безпечну розробку? Які класи вад він у принципі не бачить і чому?
12. За (11.4) поясніть вибір робочої точки WAF. Чому для державного порталу нові правила спершу вмикають у режимі спостереження?
13. Які підходи засвідчив ефективними український досвід протидії DDoS державних ресурсів? Чому завчасна готовність важливіша за реакцію під час атаки?
14. Що таке керована деградація й пріоритезація під навантаженням? Як вони пов'язані з цілями відновлення RTO/RPO?
15. Дослідницьке: розробіть і оцініть багатосигнальну модель протидії ботам для державного порталу, що мінімізує хибне блокування громадян (11.3) за заданого рівня виявлення автоматизованого зловживання, з поступовою (а не бінарною) реакцією.
16. Дослідницьке: побудуйте модель вибору робочої точки WAAP за (11.4) з емпіричними оцінками вартостей пропуску й хибного блокування для послуги електронного урядування та дослідіть її чутливість до зміни інтенсивності атак.

РОЗДІЛ 12

Безпека серверної, хмарної та контейнерної інфраструктури вебресурсів

Попередні розділи захищали сам застосунок; цей — середовище, у якому він виконується: вебсервери, хмарні платформи, контейнери й оркестратори, безсерверні та периферійні обчислення. Хибна конфігурація інфраструктури — окрема категорія A02:2025 «Security Misconfiguration» OWASP Top 10 [231], і саме вона, а не вишукані вади коду, стоїть за багатьма великими інцидентами: відкриті сховища, надмірні права, забуті службові інтерфейси, витоки секретів. Розділ розглядає посилення конфігурації вебсерверів, модель спільної відповідальності в хмарах, керування доступом і секретами, безпеку контейнерів і оркестраторів, безсерверні й периферійні обчислення, інфраструктуру як код і виявлення помилок конфігурації, а також резервне копіювання й відновлення, із завершальним кейсом. Розділ застосовує принципи найменших привілеїв і заборони за замовчуванням (розділи 4, 9) до рівня інфраструктури.

12.1 Посилення конфігурації

12.1.1 Категорія хибної конфігурації

Категорія A02:2025 охоплює: типові (заводські) облікові дані й налаштування; надмірні дозволи й відкриті назовні службові інтерфейси (панелі керування, бази даних); увімкнені непотрібні функції й докладні повідомлення про помилки (зв'язок з A10, розділ 7); відсутні заголовки безпеки (розділ 6); неоновлене ПЗ. Спільна риса — ваду створено *налаштуванням*, а не кодом, тож вона не виявляється тестуванням логіки й часто лишається поза увагою розробників. Основою протидії є **еталонні конфігурації** (benchmarks) CIS [85] і автоматична перевірка відповідності їм, а також принцип *безпечних типових налаштувань*: система має бути захищеною «з коробки», а послаблення — свідомим винятком.

12.1.2 Вебсервери й базовий рівень

Посилення вебсервера й оточення включає: мінімізацію поверхні (вимкнення непотрібних модулів, портів, методів HTTP, лістингу каталогів); заборону службових інтерфейсів назовні; окремі облікові записи з найменшими привілеями для процесів; приховування версій і докладних помилок; заголовки безпеки (розділ 6) і TLS (розділ 5); своєчасне оновлення. Рівень захищеності зручно подавати як частку виконаних контролів еталона:

$$\text{Posture} = \frac{\sum_i w_i \, \mathbf{1}[\text{контроль } i \text{ виконано}]}{\sum_i w_i}, \tag{12.1}$$

де w_i — вага контролю за впливом. Показник (12.1) придатний для відстеження *дрейфу конфігурації* (поступового відходу від еталона) у часі й для порівняння середовищ; його автоматично обчислюють засоби перевірки відповідності. Важливо зважувати контролі за ризиком, а не рахувати рівнозначно, бо відкритий назовні інтерфейс бази даних важить незрівнянно більше за відсутність одного заголовка.

12.2 Хмара та модель спільної відповідальності

12.2.1 Розподіл відповідальності

У хмарі безпека *розподілена* між провайдером і клієнтом [59]: провайдер відповідає за безпеку *хмари* (фізична інфраструктура, гіпервізор, керовані сервіси), клієнт — за безпеку *в хмарі* (конфігурація, дані, доступ, код). Частка відповідальності клієнта залежить від моделі обслуговування (табл. 12.1): від інфраструктури як послуги (IaaS, клієнт відповідає за майже все, крім «заліза») до функцій як послуги (FaaS, провайдер бере на себе середовище виконання). Більшість хмарних інцидентів виникає саме в зоні клієнта — через хибну конфігурацію, а не компрометацію провайдера. Типова помилка — припущення, що «хмара безпечна сама по собі»; насправді провайдер забезпечує лише *інструменти* безпеки, а правильно їх налаштувати має клієнт.

Таблиця 12.1 — Розподіл відповідальності за моделями обслуговування

Рівень	Провайдер	Клієнт
IaaS	фізична інфраструктура, гіпервізор	ОС, середовище, застосунок, дані, доступ, мережа
PaaS	і платформа, середовище виконання	застосунок, дані, доступ, конфігурація сервісу
SaaS	і застосунок	дані, доступ, налаштування, керування користувачами
FaaS	і середовище виконання функцій	код функції, права, дані, залежності

12.2.2 Керування доступом у хмарі

Центральний механізм — **керування ідентичностями й доступом** (IAM): політики, що визначають, які суб'єкти (користувачі, ролі, сервіси) які дії над якими ресурсами можуть виконувати. Застосовують принципи розділу 9: найменші привілеї, заборона за замовчуванням, розділення обов'язків. Типові вади — надмірно широкі політики (дозвіл «усе на всьому»), довгочинні статичні ключі доступу замість тимчасових ролей,

відкриті публічно сховища об'єктів. Інцидент витоку даних Capital One 2019 р. [288, 181] поєднав SSRF (розділ 7) з надмірними правами ролі екземпляра: сервер зі зловжитим SSRF отримав облікові дані ролі й доступ до сховища. Урок — глибокий захист: навіть за вади застосунку мінімальні права ролі обмежують наслідки (7.2), а захищена служба метаданих (IMDSv2) ускладнює отримання облікових даних.

12.2.3 Керування секретами

Секрети — ключі API, паролі БД, токени, приватні ключі — потребують окремого керування. Поширена й небезпечна вада — секрети у вихідному коді й історії репозиторіїв; масштабне дослідження [203] виявило сотні тисяч секретів у публічних репозиторіях, причому витік у комерсійний репозиторій не менш небезпечний. Заходи: *сховища секретів* з контролем доступу й аудитом; *динамічні* короткочинні секрети замість статичних; автоматичне *сканування* коду й історії на секрети (розділ 15); *ротація* й негайне відкликання при підозрі; заборона секретів у змінних середовища там, де їх видно в журналах. Секрет, що потрапив у систему контролю версій, вважають скомпрометованим і ротують, а не просто видаляють, бо він лишається в історії.

12.3 Контейнери та оркестрація

12.3.1 Безпека образів і середовища виконання

Контейнеризація додає рівні, кожен з яких потребує захисту [274]. *Образи*: будувати з мінімальних базових образів, фіксувати версії, сканувати на відомі вразливості (розділ 13), не вбудовувати секрети, підписувати й перевіряти походження. *Середовище виконання*: запускати процеси не від суперкористувача, з мінімальними можливостями (capabilities) і файловою системою лише для читання, обмежувати ресурси (проти вичерпання, розділ 11), застосовувати профілі обмеження системних викликів. Контейнер — це ізоляція процесів, а не повноцінна межа безпеки рівня віртуальної машини, тож для різнодовірених навантажень застосовують сильнішу ізоляцію (окремі вузли, пісочниці середовища виконання).

12.3.2 Оркестратори

Оркестратор (Kubernetes та аналоги) має власну велику поверхню [213, 87]: сервер API, сховище стану кластера, облікові записи сервісів, мережеві політики. Посилення включає: автентифікацію й авторизацію доступу до API (RBAC, розділ 9); мережеві політики за принципом заборони за замовчуванням (мікросегментація — під з под не спілкується, доки не дозволено); обмеження привілеїв подів (заборона привілейованих контейнерів, політики безпеки); захист і шифрування сховища стану; ізоляцію через просто-

ри імен; керування секретами поза відкритими об'єктами конфігурації. Доступ до сервера API оркестратора рівнозначний контролю над кластером, тож його захищають як найчутливіший актив (нульова довіра, розділ 9).

12.3.3 Безсерверні та периферійні обчислення

Безсерверні функції (FaaS) знімають з клієнта керування середовищем, але додають специфіку [98]: велика кількість дрібних функцій ускладнює огляд прав (кожна має отримувати лише потрібні); події-тригери походять з різних джерел і потребують валідації як недовірене введення; залежності функцій — частина поверхні (розділ 13); ефемерність ускладнює моніторинг. Периферійні обчислення (edge) виконують код близько до користувача на вузлах CDN; їхні переваги для затримки й DDoS (розділ 11) супроводжуються обмеженим середовищем і потребою захищати код і дані на багатьох розподілених вузлах. Спільний принцип для обох — найменші привілеї на рівні *кожної* функції чи вузла й валідація всіх вхідних подій.

12.4 Інфраструктура як код і виявлення помилок конфігурації

12.4.1 Інфраструктура як код

Інфраструктура як код (Infrastructure as Code, IaC) описує конфігурацію декларативно у файлах, що зберігаються, рецензуються й версіонуються як код. Це головний важіль проти хибної конфігурації: замість ручних, невідтворюваних змін — декларований, перевірюваний стан, однаковий у всіх середовищах, що усуває дрейф (12.1). Проте сам код IaC може містити «запахи безпеки»: дослідження [247] систематизувало типові — вбудовані секрети, надто широкі права, вимкнене шифрування, відкриті мережеві правила. Тому IaC *сканують* статично до розгортання (розділ 15) тими самими засобами, що й застосунковий код, і застосовують політику як код (розділ 9) до самих описів інфраструктури.

12.4.2 Безперервне виявлення помилок конфігурації

Навіть правильно розгорнута інфраструктура дрейфує через ручні зміни й нові сервіси, тож потрібне *безперервне* оцінювання. Платформи захисту хмарних застосунків (CNAPP) і керування станом безпеки хмари (CSPM) [136] автоматично перевіряють конфігурацію проти еталонів і політик, виявляють відкриті сховища, надмірні права, невідповідність і подають показник (12.1) та його дрейф. Для пріоритезації знахідок їх зважують за досяжністю й впливом (відкрите назовні сховище з персональними даними — критично; те саме у внутрішній мережі — нижче). Безперервне виявлення змикається з моніторингом (розділ 16): зміна конфігурації, що відкриває ресурс, — подія безпеки.

12.5 Резервне копіювання й відновлення

Доступність і цілісність захищають не лише запобіжними заходами, а й здатністю *відновитися*. Вимоги до резервного копіювання: регулярність і відповідність цілям RPO (допустима втрата даних) і RTO (час відновлення); *незмінність* (immutable) і офлайн- чи ізольовані копії, недосяжні з робочого середовища, щоб їх не знищила компрометація чи шифрувальник; *шифрування* копій і контроль доступу до них; регулярна *перевірка відновлення* (копія, з якої не вдається відновитися, марна). Резервні копії — також ціль атак (знищення чи шифрування), тож їх захищають не слабше за робочі дані, а ключі шифрування копій зберігають окремо. Відновлення — частина плану реагування на інциденти (розділ 16) і неперервності діяльності (стандарт ISO 22301).

12.6 Кейс: посилення інфраструктури державного порталу

12.6.1 Вихідний стан

Портал державних послуг (розділ 4) розгорнуто в хмарі з частиною сервісів у контейнерах. Аудит за CIS Benchmarks [85], NIST SP 800-190 [274], Kubernetes Hardening Guide [213] і дослідженням безпеки хмарних сервісів в органах влади [44] виявив: показник (12.1) близько 0,6 з дрейфом; роль екземпляра з надмірними правами й стара служба метаданих (ризик, аналогічний Capital One); статичні ключі доступу; одне сховище об'єктів відкрите; секрети в змінних середовища й кілька — в історії репозиторію; контейнери запускаються від суперкористувача; мережеві політики кластера дозвільні за замовчуванням; IaC без сканування; резервні копії досяжні з робочого середовища й без перевірки відновлення.

12.6.2 План заходів

Таблиця 12.2 — План посилення інфраструктури порталу

#	Захід	Категорія
I1	Закрити відкрите сховище; ротувати секрети з історії; сховище секретів, динамічні короткочинні	секрети, доступ
I2	Найменші привілеї ролей; тимчасові ролі замість ключів; IMDSv2	IAM, Capital One
I3	Еталони CIS і безперервний CSPM; відстеження (12.1) і дрейфу	конфігурація
I4	Контейнери не від root, лише читання, обмеження можливостей і ресурсів; підпис образів	контейнери
I5	RBAC і мережеві політики кластера за забороною за замовчуванням; захист API та сховища стану	оркестратор
I6	Сканування IaC до розгортання; політика як код для інфраструктури	IaC
I7	Незмінні ізольовані резервні копії; шифрування; регулярна перевірка відновлення (RTO/RPO)	відновлення

Пріоритет — I1 і I2: відкрите сховище й надмірні права ролі — прямо експлуатовані вади з найтяжчими наслідками для персональних даних, а зв'язка SSRF+права (Capital One) показує, що мінімізація прав обмежує наслідки навіть за вади застосунку (7.2). I3 і I6 переводять конфігурацію з ручної в перевірювану й усувають дрейф. I4 і I5 посилюють контейнерний рівень і оркестратор за принципом заборони за замовчуванням. I7 забезпечує відновлюваність, зокрема проти шифрувальників, що є реальною загрозою державним ресурсам.

12.6.3 Результат

Після заходів публічно відкритих ресурсів немає; права мінімізовані, ключі тимчасові, служба метаданих захищена; показник (12.1) зростає й відстежується безперервно, дрейф виявляється; контейнери й кластер посилені; інфраструктура описана кодом і сканується; резервні копії ізольовані, зашифровані й перевірені на відновлення. Залишкові ризики: складність хмарних політик як джерело нових помилок (безперервний CSPM і рецензування IaC), компрометація провайдера чи керованого сервісу (поза зоною клієнта; договірні вимоги й шифрування на боці клієнта) і внутрішній доступ адміністраторів (розділення обов'язків, журналювання привілейованих дій, розділ 16).

Висновки до розділу

Безпека інфраструктури вебресурсу — окрема категорія ризику (A02:2025), бо ваду створює налаштування, а не код, і вона не виявляється тестуванням логіки. Основа захисту — еталонні конфігурації CIS, безпечні типові налаштування й мінімізація поверхні, а стан подають показником відповідності (12.1) з відстеженням дрейфу. У хмарі відповідальність розподілена, і

більшість інцидентів виникає в зоні клієнта через хибну конфігурацію, надмірні права й відкриті ресурси; інцидент Capital One показав, що мінімальні права ролі обмежують наслідки навіть за вади застосунку. Секрети керують через сховища з ротацією й динамічними короткочинними обліковими даними, а секрет у системі контролю версій вважають скомпрометованим. Контейнери потребують посилення образів і середовища виконання, а оркестратор — RBAC, мережевих політик за забороною за замовчуванням і захисту API та сховища стану; контейнер не є повноцінною межею безпеки. Безсерверні й периферійні обчислення вимагають найменших привілеїв на рівні кожної функції чи вузла й валідації подій. Інфраструктура як код робить конфігурацію перевірюваною й усуває дрейф, але сама потребує сканування, а безперервне виявлення помилок (CSPM/CNAPP) змикається з моніторингом. Відновлюваність забезпечують незмінні ізольовані зашифровані резервні копії з регулярною перевіркою відновлення.

Питання для самоперевірки

1. Чому хибна конфігурація (A02:2025) виділена в окрему категорію і чому її не виявляє тестування логіки? Наведіть п'ять типових проявів.
2. Поясніть показник стану захищеності (12.1). Навіщо зважувати контролі за ризиком і як показник допомагає виявляти дрейф конфігурації?
3. Опишіть модель спільної відповідальності (табл. 12.1). Чому припущення «хмара безпечна сама по собі» хибне?
4. Які типові вади IAM у хмарі? Як інцидент Capital One поєднав SSRF і надмірні права, і який висновок щодо глибокого захисту?
5. Чому секрет, що потрапив у систему контролю версій, вважають скомпрометованим? Перелічіть заходи керування секретами.
6. Які рівні додає контейнеризація і як посилюють образи й середовище виконання? Чому контейнер не є повноцінною межею безпеки?
7. Опишіть поверхню атаки оркестратора й заходи його посилення. Чому мережеві політики будують за забороною за замовчуванням?
8. Яку специфіку безпеки мають безсерверні й периферійні обчислення? Який спільний принцип застосовний до обох?
9. Чому інфраструктуру як код вважають головним важелем проти хибної конфігурації? Які «запахи безпеки» містить сам код IaC і як їх виявляють?
10. Що роблять платформи CSPM/CNAPP і чому потрібне безперервне, а не разове виявлення помилок конфігурації?

11. Сформулюйте вимоги до резервного копіювання. Чому копії мають бути незмінними та ізольованими і навіщо регулярно перевіряти відновлення?
12. Як пов'язані цілі RTO/RPO з резервним копіюванням і плануванням відновлення?
13. Поясніть на прикладі, чому відкрите назовні сховище з персональними даними має отримувати вищий пріоритет, ніж відсутність одного заголовка безпеки.
14. Як принципи найменших привілеїв і заборони за замовчуванням (розділи 4, 9) застосовуються на рівні інфраструктури?
15. Дослідницьке: розробіть зважену за ризиком метрику стану захищеності хмарної інфраструктури (12.1) і метод її безперервного вимірювання; оцініть кореляцію дрейфу конфігурації з інцидентами на наборі середовищ.
16. Дослідницьке: запропонуйте модель політики як код для хмарної інфраструктури державного порталу, що автоматично блокує розгортання з відкритими ресурсами й надмірними правами, і оцініть її вплив на кількість помилок конфігурації та швидкість розгортання.

РОЗДІЛ 13

Безпека ланцюга постачання програмного забезпечення та систем керування вмістом

Сучасний вебзастосунок лише на малу частину складається з власного коду; решта — сторонні бібліотеки, базові образи, інструменти збирання, системи керування вмістом і їхні розширення. Довіра до будь-якої ланки цього ланцюга переноситься на застосунок, тож компрометація постачальника вражає всіх його користувачів одразу. Масштаб і системність цієї загрози відображені новою категорією A03:2025 «Software Supply Chain Failures» OWASP Top 10 [231], яка замінила вужчу «вразливі й застарілі компоненти». Розділ розглядає природу транзитивного ризику залежностей, класи атак на ланцюг постачання (з реальними інцидентами xz, Polyfill, Shai-Hulud, SolarWinds як уроками захисту), специфікацію складу ПЗ (SBOM) і VEX, рамки цілісності збирання (SLSA, підписування Sigstore), безпеку конвеєра CI/CD, безпеку систем керування вмістом і сторонніх скриптів, а також вимоги Європейського акту про кіберстійкість (CRA), із завершальним кейсом. Розділ продовжує клієнтський вимір ризику (розділ 6) і безпеку інфраструктури (розділ 12).

13.1 Транзитивний ризик залежностей

13.1.1 Природа проблеми

Застосунок залежить не лише від бібліотек, які підключив явно (*прямі* залежності), а й від їхніх залежностей і так далі (*транзитивні*). Граф залежностей сучасного вебпроєкту налічує сотні й тисячі вузлів, більшість з яких розробник ніколи свідомо не обирав і не переглядав. Кожен вузол — це довіра: до його коду, до його супроводжувача й до його власного ланцюга постачання. Сукупний ризик зростає з розміром графа. Якщо граф містить n компонентів, кожен з яких за рік компрометується чи виявляє критичну ваду незалежно з імовірністю p, то ймовірність, що принаймні один вплине на застосунок,

$$P_{\text{ланцюг}} = 1 - \prod_{i=1}^{n} (1 - p_i) \approx 1 - e^{-\bar{p}\, n}, \tag{13.1}$$

де $\bar{p}$ — середня ймовірність. Для $n = 800$, $\bar{p} = 0{,}0008$ маємо $P_{\text{ланцюг}} \approx 1 - e^{-0{,}64} \approx 0{,}47$. Висновок кількісний і важливий: за великого графа навіть мала ймовірність на компонент дає значний сукупний ризик, тому ключовими стають *скорочення* й *контроль* графа, а не лише реагування на окремі

вади. Скорочення зайвих залежностей — це не лише гігієна, а прямий захід безпеки, що зменшує n.

13.1.2 Керування вразливостями компонентів

Базовий рівень — **аналіз складу ПЗ** (Software Composition Analysis, SCA): автоматичне зіставлення компонентів з базами відомих вразливостей (CVE, розділ 4) у конвеєрі. Він необхідний, але недостатній: по-перше, не всі вразливості відомі (нуль-день, навмисні бекдори); по-друге, наявність вразливого компонента ще не означає, що вразливість *досяжна* в застосунку, тож сирий перелік CVE потребує пріоритезації за досяжністю й впливом (EPSS, KEV, розділ 4), щоб не потонути в шумі. Пріоритет визначають поєднанням досяжності коду, наявності експлойта й критичності; саме цю оцінку досяжності формалізує механізм VEX (підрозділ 13.3).

13.2 Класи атак на ланцюг постачання

13.2.1 Таксономія та уроки інцидентів

Атаки на ланцюг постачання різняться тим, яку ланку вражено (табл. 13.1). Їх розглядаємо з погляду *захисту*: кожен інцидент указує, який контроль його б виявив або зупинив. *Плутанина залежностей* (dependency confusion) [77] використовує пріоритет публічного репозиторію над внутрішнім за збігу імені; захід — явне прив'язування внутрішніх пакетів до приватного джерела й зарезервовані простори імен. *Компрометація легітимного пакета* через захоплення облікового запису супроводжувача чи шкідливе оновлення (Polyfill, розділ 6; хробак Shai-Hulud у npm 2025 р. [279], що самопоширювався через викрадені токени публікації) — захід: фіксація версій і перевірка цілісності, фішингостійка MFA для супроводжувачів, виявлення аномальних оновлень. *Навмисний бекдор* у відкритому компоненті (xz/liblzma, CVE-2024-3094, 2024 р. [133], упроваджений після тривалого здобуття довіри до проєкту) — найважчий для виявлення; його стримують відтворюваним збиранням, рецензуванням і диверсифікацією. *Компрометація інструмента збирання чи оновлень* (SolarWinds, 2020 р. [93]) вражає всіх користувачів продукту; захід — цілісність і ізоляція конвеєра (підрозділ 13.4).

Таблиця 13.1 — Класи атак на ланцюг постачання і захисні контролі

Вражена ланка	Контроль, що виявляє/зупиняє
Ім'я пакета (плутанина)	прив'язка до приватного джерела, зарезервовані імена
Супроводжувач/оновлення	фіксація версій, перевірка цілісності, MFA, виявлення аномалій
Код компонента (бекдор)	відтворюване збирання, рецензування, диверсифікація
Інструмент збирання, оновлення	цілісність і ізоляція CI/CD, підписування артефактів
Публічний скрипт (CDN)	SRI, самостійне розміщення (розділ 6)

13.2.2 Чому ланцюг постачання особливий

Атаки на ланцюг постачання мають дві особливості, що роблять їх привабливими й небезпечними. По-перше, *масштаб*: одна компрометація вражає всіх користувачів компонента, тож зусилля нападника окупається багаторазово. По-друге, *довіра*: шкідливий код приходить легітимним, підписаним каналом оновлення, обходячи периметр, бо йому довіряють за визначенням. Тому захист будують не на недовірі до конкретного пакета, а на *системних* властивостях: прозорості складу (що саме використовується), цілісності походження (звідки це прийшло) і обмеженні наслідків (що скомпрометований компонент може зробити).

13.3 Прозорість складу: SBOM і VEX

Специфікація складу ПЗ (Software Bill of Materials, SBOM) [95] — машиночитний перелік усіх компонентів продукту з версіями й залежностями у стандартному форматі (SPDX [167] або CycloneDX [233]). SBOM відповідає на базове питання «з чого складається застосунок», без якого неможливі ні інвентаризація (пор. розділ 10), ні швидка оцінка впливу нової вразливості: коли оголошено критичну ваду в компоненті, організація з актуальними SBOM за хвилини визначає, які з її систем її містять. SBOM генерують автоматично під час збирання й зберігають як артефакт для кожної версії.

Сам перелік компонентів породжує багато потенційних, але недосяжних вразливостей, тому його доповнює **VEX** (Vulnerability Exploitability eXchange) — твердження постачальника про *придатність до експлуатації* конкретної вразливості в конкретному продукті (наприклад, «вразливість присутня, але код недосяжний, тож продукт не вразливий»). VEX перетворює сирий перелік на пріоритезований, зменшуючи шум і спрямовуючи зусилля на справді досяжні вади. Разом SBOM і VEX реалізують принцип прозорості: не можна захистити чи оцінити те, склад чого невідомий.

13.4 Цілісність збирання: SLSA, Sigstore, CI/CD

13.4.1 Рамка SLSA і походження

SLSA (Supply-chain Levels for Software Artifacts) [224] — рамка рівнів зрілості цілісності ланцюга збирання. Її ключове поняття — **походження** (provenance): підписане, перевірюване свідчення про те, *що, з чого і де* було зібрано, яке дає змогу переконатися, що артефакт справді походить з очікуваного джерела й процесу, а не підмінений. Вищі рівні SLSA вимагають ізольованого, параметризованого й відтворюваного збирання. **Відтворюване збирання** (reproducible builds) — властивість, за якої той самий вихідний код завжди дає побітово той самий артефакт, що дозволяє незалежно перевірити відсутність прихованих змін і є одним із небагатьох заходів проти навмисних бекдорів рівня xz.

13.4.2 Підписування артефактів

Підписування коду й артефактів дає змогу споживачеві перевірити їхнє походження й цілісність. Проєкт **Sigstore** [215] спростив підписування для відкритого ПЗ: воно спирається на короткочинні ключі, прив'язані до ідентичності через OpenID Connect (розділ 9), і на публічний журнал прозорості (за аналогією з прозорістю сертифікатів, розділ 5), що робить факти підписування аудитованими. Перевірка підпису перед використанням компонента чи розгортанням артефакта закриває клас підміни в дорозі й у сховищі.

13.4.3 Безпека конвеєра

Конвеєр CI/CD сам є привабливою ціллю: він має доступ до коду, секретів і середовищ розгортання, тож його компрометація (як SolarWinds) вражає продукт в обхід усіх застосункових заходів. Тому конвеєр захищають як виробниче середовище [275]: найменші привілеї й короткочинні облікові дані для кроків; ізоляція й ефемерність складальних агентів; захист і верифікація визначень конвеєра як коду (розділ 12); контроль цілісності залежностей (фіксація версій і перевірка хешів lock-файлів); обмеження й аудит доступу до сховищ артефактів і реєстрів. Ключова ідея — застосувати до процесу розроблення ті самі принципи (найменші привілеї, цілісність, нульова довіра), що й до застосунку.

13.5 Системи керування вмістом і сторонні скрипти

Значна частина вебресурсів працює на **системах керування вмістом** (CMS), і для них ланцюг постачання — це передусім *розширення* (плагіни, теми) сторонніх авторів. Галузеві звіти [282] стабільно показують, що переважна більшість зламів сайтів на поширених CMS спричинена не ядром, а вразливими чи занедбаними розширеннями. Заходи специфічні: вста-

новлювати лише потрібні розширення з надійних джерел; своєчасно оновлювати ядро й розширення (автооновлення для критичних); видаляти невикористовувані (кожне занедбане — відкрита поверхня, пор. (13.1)); обмежувати права адміністративного доступу й захищати його сильною автентифікацією (розділ 8); відокремлювати права на запис у файлову систему. Сторонні скрипти на сторінці (розділ 6) — клієнтський вимір того самого ризику; їх контролюють SRI, самостійним розміщенням і мінімізацією на чутливих сторінках. Спільний принцип для CMS і скриптів — мінімізувати кількість сторонніх компонентів і тримати решту актуальною.

13.6 Регуляторні вимоги: Акт про кіберстійкість

Європейський **Акт про кіберстійкість** (Cyber Resilience Act, CRA) [253] встановлює обов'язкові вимоги кібербезпеки до продуктів з цифровими елементами, зокрема до програмного забезпечення: безпека за задумом і за замовчуванням, відсутність відомих експлуатованих вразливостей на момент випуску, надання SBOM, процес оброблення вразливостей протягом життєвого циклу й *звітування* про активно експлуатовані вразливості й інциденти (відповідні зобов'язання зі звітування починають діяти з вересня 2026 р.). Для розробників вебпродуктів це перетворює практики цього розділу (SBOM, оброблення вразливостей, безпечні типові налаштування) з рекомендованих на обов'язкові для європейського ринку. Українські розробники, орієнтовані на ЄС, і державні системи в межах гармонізації (розділ 2) враховують CRA як вимогу, а не лише добру практику; дослідження ризиків ланцюга постачання для державних систем [48] наголошують на потребі національних підходів до SBOM і контролю походження.

13.7 Кейс: захист ланцюга постачання державного порталу

13.7.1 Вихідний стан

Портал державних послуг (розділ 4, сценарій R5) має граф залежностей близько 800 компонентів; окремий інформаційний підсайт працює на поширеній CMS із десятком плагінів. Аудит за SSDF [275], SLSA [224] і дослідженням ризиків ланцюга постачання [48] виявив: немає SBOM; SCA запущено разово, перелік вразливостей не пріоритезований; версії залежностей не зафіксовано, перевірки цілісності немає (ризик плутанини й шкідливих оновлень); внутрішні пакети під іменами, що збігаються з можливими публічними; артефакти не підписуються, походження не фіксується; конвеєр має широкі довгочинні облікові дані; на CMS два занедбані плагіни; сторонні скрипти без SRI (розділ 6). За (13.1) сукупний ризик ланцюга значний.

13.7.2 План заходів

Таблиця 13.2 — План захисту ланцюга постачання порталу

#	Захід	Клас (табл. 13.1)
S1	Генерація SBOM для кожної версії; SCA в CI з пріоритезацією (EPSS/KEV) і VEX	прозорість
S2	Фіксація версій і перевірка цілісності (lock-файли, хеші); прив'язка внутрішніх пакетів до приватного джерела	супроводжувач, плутанина
S3	Скорочення зайвих залежностей; видалення занедбаних плагінів CMS; автооновлення критичних	(13.1), CMS
S4	Підписування артефактів (Sigstore) і фіксація походження (SLSA); перевірка перед розгортанням	збирання
S5	Посилення CI/CD: найменші привілеї, короткочинні облікові дані, ізоляція агентів	конвеєр
S6	SRI і мінімізація сторонніх скриптів (розділ 6)	публічний скрипт
S7	Відповідність вимогам CRA: процес оброблення вразливостей і звітування	регуляторна

Пріоритет — S1 і S2: без SBOM неможливо оцінити вплив нової вразливості, а фіксація версій і перевірка цілісності закривають найпоширеніші й найдешевші для нападника вектори (плутанина, шкідливе оновлення). S3 прямо зменшує n у (13.1) і усуває занедбані плагіни — типову причину зламів CMS. S4 і S5 захищають від компрометації збирання (урок SolarWinds), застосовуючи до конвеєра принципи розділу 12. S7 робить практики обов'язковими з огляду на орієнтацію на ЄС.

13.7.3 Результат

Після заходів склад застосунку прозорий і відстежуваний, вплив нових вразливостей оцінюється швидко; версії зафіксовані й перевіряються на цілісність, внутрішні пакети захищені від плутанини; граф скорочено, занедбані компоненти видалено; артефакти підписані з перевірюваним походженням; конвеєр посилено; процес оброблення вразливостей відповідає CRA. Залишкові ризики: навмисний бекдор у довіреному компоненті рівня xz (частково стримується відтворюваним збиранням, рецензуванням і диверсифікацією, але повністю не усувається), компрометація провайдера підписування чи реєстру (журнали прозорості, моніторинг) і нуль-день у компоненті до його розкриття (швидке реагування за актуальним SBOM, розділ 16).

Висновки до розділу

Вебзастосунок здебільшого складається зі стороннього коду, тож довіра до кожної ланки ланцюга постачання переноситься на застосунок, а компрометація постачальника вражає всіх користувачів одразу — системність цієї загрози відображена новою категорією A03:2025. Транзитивний ризик зро-

стає з розміром графа залежностей (13.1), тому скорочення й контроль графа є прямими заходами безпеки, а аналіз складу ПЗ необхідний, але потребує пріоритезації за досяжністю. Класи атак — плутанина залежностей, компрометація супроводжувача чи оновлення, навмисний бекдор, компрометація інструментів збирання — мають спільні особливості (масштаб і зловживання довірою), тож захист будують на системних властивостях: прозорості складу (SBOM і VEX), цілісності походження (SLSA, підписування Sigstore, безпека CI/CD) і обмеженні наслідків. Системи керування вмістом вразливі передусім через занедбані розширення, а сторонні скрипти — клієнтський вимір того самого ризику; спільний принцип — мінімізувати кількість сторонніх компонентів і тримати решту актуальною. Акт про кіберстійкість ЄС перетворює ці практики на обов'язкові вимоги з наданням SBOM, обробленням вразливостей і звітуванням.

Питання для самоперевірки

1. Чому довіра до компонента ланцюга постачання переноситься на застосунок? Поясніть різницю між прямими й транзитивними залежностями.
2. За (13.1) обчисліть сукупний ризик для $n = 1200$, $\bar{p} = 0{,}0005$. Чому скорочення графа є заходом безпеки?
3. Чому аналіз складу ПЗ (SCA) необхідний, але недостатній? Як пріоритезують перелік вразливостей компонентів?
4. Опишіть класи атак на ланцюг постачання (табл. 13.1) і для кожного назвіть контроль, що його виявляє або зупиняє.
5. Які дві особливості роблять атаки на ланцюг постачання особливо небезпечними? Як вони визначають стратегію захисту?
6. Що таке SBOM і які задачі він розв'язує? Чому його доповнює VEX і що VEX усуває?
7. Поясніть поняття походження (provenance) у SLSA. Що таке відтворюване збирання і чому воно важливе проти навмисних бекдорів?
8. Як підписування Sigstore забезпечує цілісність артефактів? Яка аналогія з прозорістю сертифікатів (розділ 5)?
9. Чому конвеєр CI/CD є привабливою ціллю (урок SolarWinds) і як його захищають? Який спільний принцип із захистом застосунку?
10. Чому більшість зламів сайтів на CMS спричинена розширеннями, а не ядром? Перелічіть заходи захисту CMS.
11. Як пов'язані безпека сторонніх скриптів (розділ 6) і безпека ланцюга постачання? Який спільний принцип?

12. Які обов'язкові вимоги встановлює Акт про кіберстійкість ЄС і як вони пов'язані з практиками цього розділу?
13. Поясніть на прикладі плутанини залежностей, чому прив'язка внутрішніх пакетів до приватного джерела усуває цей клас.
14. Чому навмисний бекдор рівня xz найважчий для виявлення і які заходи його частково стримують?
15. Дослідницьке: розробіть метод пріоритезації вразливостей компонентів за досяжністю коду з автоматичною генерацією тверджень VEX і оцініть зменшення кількості хибно критичних знахідок на відкритому вебпроєкті.
16. Дослідницьке: запропонуйте національний підхід до застосування SBOM і контролю походження (SLSA) для державних інформаційних систем України з урахуванням вимог CRA і оцініть його застосовність та витрати на впровадження.

РОЗДІЛ 14

Безпека вебзастосунків зі штучним інтелектом і перспективні загрози

Вебзастосунки дедалі частіше вбудовують великі мовні моделі (LLM): чат-помічники, пошук із генеративними відповідями, автоматичне оброблення звернень, агенти, що виконують дії від імені користувача. Це додає новий клас ризиків, не зведених до попередніх розділів, бо модель обробляє *природну мову*, у якій дані й інструкції не розділені синтаксично — на відміну від структурного розділення, що рятувало від ін'єкцій (розділ 7). Специфіку відображають OWASP Top 10 для застосунків LLM [238] і для агентних застосунків [237], а також рамка керування ризиками ШІ NIST [283]. Розділ розглядає архітектуру застосунків LLM (RAG, агенти, протоколи інструментів), захист від ін'єкції підказок (прямої й непрямої), обмеження автономії й витоку даних, захисні патерни, роль ШІ в захисті вебресурсів, а також стисло перспективні технології (WebAssembly, Web3) із завершальним кейсом помічника державного порталу. Виклад — захисний: акцент на тому, як проєктувати безпечні застосунки ШІ, а не як ними зловживати.

14.1 Архітектура вебзастосунків з LLM

14.1.1 Типові компоненти

Застосунок LLM рідко є лише моделлю: зазвичай це система, де модель поєднана з даними й інструментами. **Генерація з доповненням пошуком** (Retrieval-Augmented Generation, RAG) [190] додає до запиту релевантні фрагменти з бази знань, щоб відповідь спиралася на актуальні дані організації, а не лише на навчання моделі. **Агенти** — застосунки, у яких модель не лише генерує текст, а й *викликає інструменти* (пошук, API, дії в системі) у циклі, плануючи кроки. Інструменти підключають через інтерфейси, зокрема протокол контексту моделі (MCP) [62], що стандартизує надання моделі доступу до даних і дій. Кожен компонент розширює можливості й водночас поверхню атаки: RAG вводить недовірені дані в контекст, агент надає моделі здатність *діяти*, а інструменти й протоколи — нові канали впливу.

14.1.2 Межі довіри в застосунку LLM

Ключ до безпеки — правильно провести *межі довіри* (розділ 4) у застосунку LLM. Головний принцип: *вивід моделі є недовіреними даними*, навіть якщо модель «своя», бо на нього впливає будь-який текст, що потрапив у кон-

текст (запит користувача, здобуті документи, відповіді інструментів). Так само недовіреним є будь-який текст, що входить у контекст із зовнішніх джерел. Звідси випливає архітектурна настанова: рішення про доступ і дії ухвалює не модель, а детермінований код застосунку з явними перевірками (розділ 9), а вивід моделі перед використанням обробляють як введення — валідують, екранують за контекстом (розділ 6), перевіряють на відповідність очікуваній схемі.

14.2 Ін'єкція підказок

14.2.1 Природа проблеми

Ін'єкція підказок (prompt injection) — найхарактерніший ризик застосунків LLM (LLM01 у переліку OWASP [238]). Її корінь той самий, що в усіх ін'єкцій (розділ 7): змішування каналів даних і керування. Але якщо для SQL чи оболонки канали можна *структурно розділити* (параметризація), то для моделі, що обробляє природну мову, інструкції та дані подано *тим самим* текстом, і надійного синтаксичного розділення наразі не існує. Тому ін'єкцію підказок не можна повністю усунути на вході фільтрацією — це фундаментальне обмеження, а не недопрацювання конкретної моделі [142]. Розрізняють *пряму* (недовірений текст у запиті користувача впливає на поведінку) і *непряму* (недовірений текст у здобутих документах, на вебсторінці, у відповіді інструмента впливає на модель, навіть без прямого введення зловмисником). Непряма особливо небезпечна для RAG і агентів, бо модель обробляє зовнішній вміст, який організація не контролює.

14.2.2 Захисні підходи

Оскільки ін'єкцію не можна усунути на вході, захист будують навколо *обмеження наслідків* за принципом, що модель не є межею безпеки (табл. 14.1). Ключові підходи: *найменші привілеї* інструментів і доступу моделі (модель отримує лише ті дії й дані, що потрібні для задачі); *детерміновані перевірки* до виконання будь-якої значущої дії, ухвалювані кодом, а не моделлю; *людина в контурі* (підтвердження користувачем) для незворотних чи чутливих дій; *розмежування* довіреного контексту (системні настанови) і недовіреного (дані) наскільки дозволяє інтерфейс моделі; *оброблення виводу* як недовіреного (екранування перед показом у браузері проти XSS, валідація перед передаванням в інструмент). Корисна метрика проєктної безпеки — *залишкова шкода* від успішної ін'єкції за (7.2): навіть якщо ін'єкція вплине на модель, мінімальні привілеї й обов'язкові перевірки обмежують, що саме станеться.

Таблиця 14.1 — Ризики застосунків LLM і захисні підходи

Ризик (OWASP LLM)	Захисний підхід
Ін'єкція підказок (пряма, непряма)	модель не межа безпеки; найменші привілеї; перевірки в коді; людина в контурі
Небезпечна обробка виводу	оброблення виводу як недовіреного; екранування, валідація
Надмірна автономія	обмеження дій і повноважень агента; підтвердження
Витік чутливих даних	мінімізація контексту; контроль доступу до джерел RAG
Отруєння даних/моделі	контроль джерел навчання й бази знань, походження
Відмова/вичерпання ресурсів	квоти, ліміти, бюджети запитів (розділ 11)

14.3 Автономія, інструменти й витік даних

14.3.1 Обмеження надмірної автономії

Чим більше дій довірено агентові, тим тяжчі наслідки ін'єкції чи помилки. **Надмірна автономія** (excessive agency) — ризик, коли агент має повноваження, непотрібні для задачі, або виконує незворотні дії без підтвердження. Захисні принципи повторюють розділ 9 у новому контексті: кожен інструмент агента отримує *мінімальні* права; дії класифікують за зворотністю й впливом, і чутливі потребують підтвердження людини; агент діє в межах явних *обмежень* (дозволені інструменти, ліміти на обсяг і вартість дій); усі дії *журналюють* для аудиту (розділ 16). Корисно оцінювати *потенціал шкоди* конфігурації агента

$$H = \sum_{t \in \mathcal{T}} a_t \cdot s_t \cdot (1 - h_t), \tag{14.1}$$

де $\mathcal{T}$ — множина доступних агентові інструментів; a_t — чи наданий інструмент; s_t — потенційна шкода від його зловживання; $h_t \in [0; 1]$ — частка, на яку підтвердження людиною й перевірки знижують цю шкоду. Мінімізація H — це вилучення непотрібних інструментів ($a_t = 0$) і введення підтверджень для найшкідливіших ($h_t \to 1$), що прямо відповідає принципу найменших привілеїв.

14.3.2 Витік чутливих даних

Застосунок LLM може розкрити чутливі дані кількома шляхами: включенням їх у контекст (системні настанови, здобуті документи), до яких отримує доступ неавторизований користувач; відтворенням навчальних даних; передаванням даних зовнішньому постачальнику моделі. Заходи: *мінімізація* даних у контексті (лише потрібне для задачі); *контроль доступу* до джерел RAG на рівні користувача (здобуваються лише документи, досту-

пні саме цьому користувачеві — інакше RAG обходить авторизацію розділу 9); *класифікація* даних і заборона передавати найчутливіші зовнішнім моделям; договірні й технічні гарантії щодо оброблення даних постачальником. Принцип: контекст моделі успадковує рівень доступу користувача, а не має власного привілейованого доступу до всіх даних.

14.4 Отруєння та цілісність джерел

Якість і безпечність виводу залежать від даних, якими живиться модель. **Отруєння** навчальних даних чи бази знань RAG (внесення шкідливого чи хибного вмісту) може спотворити поведінку або стати каналом непрямої ін'єкції. Таксономія атак на машинне навчання NIST [291] систематизує ці загрози. Захисні заходи стосуються *цілісності й походження* джерел (пор. розділ 13): контроль того, які документи потрапляють у базу знань, і довіра до їхніх джерел; відстеження походження вмісту; виявлення аномального чи підозрілого вмісту; обмеження того, хто може додавати дані до бази знань. Для застосунків, що здобувають вміст із зовнішнього вебу, увесь такий вміст вважають недовіреним (потенційний носій непрямої ін'єкції), і модель не повинна мати можливості на його підставі виконати привілейовану дію без перевірки в коді.

14.5 Штучний інтелект у захисті вебресурсів

ШІ корисний і в *захисті*: виявлення аномалій у трафіку й поведінці (розділи 11, 16), кореляція подій у SIEM, допомога в тріажі вразливостей і рецензуванні коду, генерація тестів. Проте до захисних застосувань ставлять ті самі вимоги: вивід моделі перевіряють, а не виконують автоматично для критичних рішень; модель не ухвалює остаточних блокувань без контрольованого балансу помилок (розділ 11); навчальні дані й сама модель є активами, що потребують захисту цілісності. Таким чином, ШІ в захисті — це *інструмент підтримки* рішень, а не автономний арбітр, і його впроваджують з тими самими застереженнями щодо надмірної автономії, що й будь-який агент.

14.6 Перспективні технології

14.6.1 WebAssembly

WebAssembly (Wasm) [144] — переносний бінарний формат для виконання коду в браузері й на сервері з майже нативною швидкістю. Його модель безпеки — *пісочниця* з явним наданням можливостей: модуль не має доступу до системи, доки його не надано хостом, що узгоджується з найменшими привілеями. Безпекові застереження: Wasm виконується в тому самому походженні, що й сторінка, тож скомпрометований модуль має її права (роз-

діл 6); вади в середовищах виконання Wasm поза браузером потребують уваги; Wasm не усуває логічних вад, а лише ізолює виконання. Для захисту Wasm застосовують ті самі принципи: мінімальні надані можливості, перевірка походження модулів (розділ 13), ізоляція.

14.6.2 Web3 і смарт-контракти

Децентралізовані застосунки й смарт-контракти мають власну специфіку ризиків [67]: незмінність розгорнутого коду робить вади невиправними без спеціальних механізмів, а публічність і безпосередній зв'язок з активами підвищують ціну помилки. Класичні класи вад (повторний вхід, арифметичні переповнення, помилки контролю доступу) добре задокументовані. З погляду цього посібника важливо, що вебінтерфейси до таких застосунків (гаманці, фронтенди) підлягають усім звичайним вимогам безпеки вебресурсів (розділи 5–9), а специфіка смарт-контрактів — предмет окремої дисципліни. Тому тут обмежимося тезою: інтеграція з Web3 не знімає, а додає вимоги, і фронтенд лишається типовою поверхнею атаки.

14.7 Регуляторний контекст

Застосування ШІ дедалі більше регулюється. Акт ЄС про штучний інтелект [252] запроваджує ризик-орієнтований підхід з вимогами до систем високого ризику (прозорість, нагляд людини, якість даних, стійкість), а рамка NIST AI RMF [283] надає добровільну методологію керування ризиками ШІ. Для державних вебзастосунків, що впроваджують ШІ (наприклад, автоматизоване оброблення звернень), це означає вимоги прозорості рішень, нагляду людини для значущих дій і захисту даних, що узгоджується із захисними принципами розділу. Дослідження загроз вебзастосункам на основі LLM [46] наголошують на потребі враховувати ці ризики ще на етапі проєктування, а не додавати захист згодом.

14.8 Кейс: безпечний помічник державного порталу

14.8.1 Постановка

Державний портал (розділ 4) упроваджує чат-помічника для громадян: відповідає на запитання з бази знань послуг (RAG) і здатний виконувати дії (перевірити статус заяви, записати на прийом) через інструменти. Аудит за OWASP Top 10 для LLM [238] і агентних застосунків [237] виявив ризики: RAG здобуває документи без урахування прав користувача (ризик витоку чужих даних і обходу авторизації); інструменти агента мають широкі права, зокрема незворотні дії без підтвердження; вивід моделі вставляється в сторінку без екранування (ризик XSS, розділ 6); частина бази знань поповнюється зі зовнішніх джерел без контролю (ризик отруєння й непрямої ін'єкції); немає журналювання дій агента й лімітів вартості.

14.8.2 Заходи

Таблиця 14.2 — Захист помічника державного порталу

#	Захід	Ризик
L1	Рішення про доступ і дії — у коді з перевірками; модель не ухвалює авторизації	ін'єкція підказок
L2	RAG здобуває лише документи, доступні цьому користувачеві (контроль доступу до джерел)	витік даних, обхід авторизації
L3	Найменші привілеї інструментів; підтвердження людиною незворотних дій; ліміти (14.1)	надмірна автономія
L4	Оброблення виводу моделі як недовіреного: екранування перед показом, валідація перед інструментом	небезпечна обробка виводу
L5	Контроль джерел і походження бази знань; зовнішній вміст — недовірений	отруєння, непряма ін'єкція
L6	Журналювання всіх дій агента; квоти й бюджети запитів	аудит, доступність
L7	Прозорість для користувача й нагляд людини для значущих дій (вимоги регуляторів)	регуляторна

Пріоритет — L1 і L2: оскільки ін'єкцію підказок не можна усунути на вході, безпека тримається на тому, що модель не є межею безпеки, а RAG не обходить авторизацію (інакше помічник стає каналом доступу до чужих персональних даних). L3 обмежує потенціал шкоди (14.1), вилучаючи зайві інструменти й вимагаючи підтвердження незворотних дій. L4 закриває перенесення ін'єкції у XSS. L5 стосується цілісності джерел і непрямої ін'єкції. L7 відповідає вимогам прозорості й нагляду людини.

14.8.3 Результат

Після заходів навіть успішна ін'єкція підказки не дає доступу до чужих даних чи незворотних дій без перевірки в коді й підтвердження; RAG успадковує права користувача; вивід моделі безпечно відображається; джерела бази знань контрольовані; дії агента журналюються й обмежені бюджетом; значущі рішення залишаються за людиною. Залишкові ризики: сама ін'єкція підказки лишається можливою (обмежено наслідки, а не усунено причину — це визнане фундаментальне обмеження), витонченіші атаки отруєння (контроль джерел і моніторинг), а також залежність від постачальника моделі (класифікація даних, договірні гарантії, розгляд у моделі ланцюга постачання, розділ 13).

Висновки до розділу

Вбудовування LLM у вебзастосунки додає клас ризиків, не зведених до попередніх, бо модель обробляє природну мову, де інструкції й дані не роз-

ділені синтаксично. Ключова архітектурна настанова — *вивід моделі є недовіреними даними*, а рішення про доступ і дії ухвалює детермінований код, а не модель. Ін'єкцію підказок (пряму й непряму) наразі не можна повністю усунути на вході — це фундаментальне обмеження, — тож захист будують навколо обмеження наслідків: найменших привілеїв інструментів і доступу, детермінованих перевірок, людини в контурі для чутливих дій і оброблення виводу як недовіреного. Надмірну автономію агента обмежують мінімізацією інструментів і підтвердженнями (14.1), а витік даних — мінімізацією контексту й успадкуванням прав користувача в RAG, щоб він не обходив авторизацію. Цілісність джерел захищає від отруєння й непрямої ін'єкції (зв'язок з ланцюгом постачання). ШІ корисний і в захисті, але як інструмент підтримки рішень, а не автономний арбітр. WebAssembly спирається на пісочницю з явними можливостями, а інтеграція з Web3 додає вимоги, не знімаючи звичайних. Регуляторні рамки (Акт ЄС про ШІ, NIST AI RMF) вимагають прозорості, нагляду людини й захисту даних, що узгоджується із захисними принципами розділу.

Питання для самоперевірки

1. Назвіть типові компоненти застосунку LLM (RAG, агенти, інструменти). Як кожен розширює поверхню атаки?
2. Чому вивід моделі слід вважати недовіреними даними? Яка головна архітектурна настанова випливає з цього?
3. У чому корінь ін'єкції підказок і чому її не можна усунути структурним розділенням, як для SQL (розділ 7)?
4. Порівняйте пряму й непряму ін'єкцію підказок. Чому непряма особливо небезпечна для RAG і агентів?
5. Перелічіть захисні підходи проти ін'єкції підказок (табл. 14.1). Чому захист будують навколо обмеження наслідків, а не усунення причини?
6. Що таке надмірна автономія агента? За (14.1) поясніть, як вилучення інструментів і підтвердження людиною знижують потенціал шкоди.
7. Якими шляхами застосунок LLM може розкрити чутливі дані? Чому RAG має успадковувати права користувача?
8. Що таке отруєння даних і бази знань? Як цілісність і походження джерел (зв'язок з розділом 13) захищають від нього й від непрямої ін'єкції?
9. Чому ШІ в захисті вебресурсів є інструментом підтримки, а не автономним арбітром? Які застереження до нього застосовні?

10. Опишіть модель безпеки WebAssembly. Чому скомпрометований Wasm-модуль небезпечний і які принципи його захисту?
11. Чому вебінтерфейси до застосунків Web3 підлягають усім звичайним вимогам безпеки вебресурсів? Що додає специфіка смарт-контрактів?
12. Які вимоги висувають Акт ЄС про ШІ та рамка NIST AI RMF до систем ШІ і як вони узгоджуються із захисними принципами розділу?
13. Поясніть, чому обробку виводу моделі перед показом у браузері треба виконувати як для недовіреного введення (зв'язок з розділом 6).
14. Чому для застосунку LLM рішення про доступ не можна делегувати самій моделі навіть за добрих системних настанов?
15. Дослідницьке: розробіть архітектурний патерн безпечного агента для державного сервісу, у якому модель не є межею безпеки, і оцініть залишкову шкоду (14.1) від успішної ін'єкції підказки для різних конфігурацій інструментів.
16. Дослідницьке: запропонуйте методику контролю доступу на рівні документів у RAG, що унеможливлює обхід авторизації (розділ 9) через здобуті фрагменти, і оцініть її вплив на якість відповідей та ризик витоку персональних даних.

РОЗДІЛ 15

Тестування захищеності вебресурсів

Попередні розділи описали, як *будувати* захист; цей — як *перевірити*, що він справді працює. Тестування захищеності виявляє вади до того, як їх знайде зловмисник, і дає об'єктивні свідчення рівня безпеки для ухвалення рішень (розділ 4). Воно охоплює методології (OWASP WSTG, PTES, NIST SP 800-115), автоматичні засоби (статичний, динамічний, інтерактивний аналіз, аналіз складу, фазинг), ручне тестування й оцінювання на проникнення в межах авторизації, програми винагород за вади й координоване розкриття, а також метрики покриття й ефективності. Виклад — з позиції власника ресурсу й інженера: як організувати перевірку, інтегрувати її в життєвий цикл і правильно інтерпретувати результати. Усе тестування виконують лише в межах письмового дозволу й узгодженого обсягу (розділ 2).

15.1 Методології та місце тестування в життєвому циклі

15.1.1 Стандарти

Систематичність відрізняє тестування від випадкового пошуку. **OWASP WSTG** [232] надає структурований перелік перевірок за категоріями (автентифікація, авторизація, сеанси, валідація введення тощо), зіставний з ASVS (розділ 2); він визначає, *що* перевіряти. **NIST SP 800-115** [264] описує технічний процес оцінювання. **PTES** [246] задає етапи робіт з оцінювання захищеності. Спільне — наголос на *повноті* (перевірено все істотне) і *повторюваності* (результат відтворюваний, а не залежить від інтуїції тестувальника). Для вебресурсу WSTG є природною основою плану тестування, а ASVS — переліком вимог, відповідність яким перевіряють.

15.1.2 Зсув уліво

Що пізніше виявлено ваду, то дорожче її виправлення. Принцип **зсуву вліво** (shift left) [205] переносить перевірки на ранні етапи — до проєктування (моделювання загроз, розділ 4), кодування (статичний аналіз, рецензування) і збирання (тести в конвеєрі). Залежність вартості виправлення від етапу апроксимують зростальною функцією:

$$C_{\text{випр}}(e) = C_0\, k^{\,e}, \qquad \text{ROI}_{\text{тест}} = \frac{\sum_e n_e \big(C_{\text{випр}}(E) - C_{\text{випр}}(e)\big)}{C_{\text{тест}}}, \tag{15.1}$$

де e — номер етапу виявлення (проєкт, код, тест, випуск, експлуатація); $k > 1$ — коефіцієнт подорожчання на етап; n_e — кількість вад, виявлених на етапі e замість останнього E; $C_{\text{тест}}$ — вартість тестування. Формула (15.1) кількісно обґрунтовує раннє тестування: перенесення виявлення з експлуатації на код економить k^{E-e}-кратну вартість. Проте зсув уліво доповнює, а не замінює пізні перевірки (динамічні, ручні), бо частина вад виявляється лише на працюючій системі.

15.2 Автоматичні засоби тестування

15.2.1 Класи засобів

Автоматичні засоби різняться тим, що й як аналізують (табл. 15.1). **SAST** (статичний аналіз) досліджує вихідний код без виконання, виявляючи небезпечні патерни й потоки даних від джерела до приймача [74]; бачить код повністю, але породжує хибні спрацювання й не бачить вад конфігурації й середовища. **DAST** (динамічний аналіз) тестує працюючий застосунок ззовні [115]; знаходить реальні, відтворювані проблеми, але обмежений покриттям (не досягає станів, яких не змусив пройти). **IAST** (інтерактивний аналіз) [241] поєднує обидва, спостерігаючи за застосунком зсередини під час тестів. **SCA** (аналіз складу) перевіряє залежності (розділ 13). Жоден клас не є повним; їх *поєднують*, бо вони виявляють різні перетинні множини вад.

Таблиця 15.1 — Класи засобів тестування захищеності

Клас	Що аналізує	Сильні/слабкі сторони
SAST	вихідний код без виконання	повне покриття коду; хибні спрацювання; не бачить середовища
DAST	працюючий застосунок ззовні	реальні вади; обмежене покриття станів
IAST	застосунок зсередини під час тестів	точність; потребує інструментування
SCA	залежності й компоненти	відомі CVE; потребує пріоритезації (розділ 13)
Фазинг	реакцію на згенеровані входи	глибокі й несподівані дефекти; ресурсомісткий

15.2.2 Фазинг

Фазинг (fuzzing) [195] автоматично генерує велику кількість різноманітних, зокрема некоректних, входів і спостерігає за аномаліями (збої, витоки, порушення інваріантів). Керований покриттям фазинг використовує зворотний зв'язок про досягнуті гілки коду, щоб спрямовувати генерацію в не-

пройдені ділянки, і ефективний для парсерів, обробників форматів і API. Для вебресурсів фазинг застосовують до розбору вхідних даних і до API за схемою (розділ 10). Ефективність зростає з покриттям; орієнтовну ймовірність виявити дефект, досяжний на частці θ входів, за N згенерованих входів оцінюють як

$$P_{\text{вияв}} = 1 - (1 - \theta)^N \approx 1 - e^{-\theta N}, \tag{15.2}$$

що показує спадну віддачу: подвоєння N не подвоює виявлення. Тому фазинг спрямовують (керування покриттям, словники, розумні мутації) замість простого нарощування обсягу.

15.2.3 Межі автоматизації

Автоматичні засоби ефективні для вад з *локальною синтаксичною ознакою*, але погано виявляють вади *логіки* й *авторизації* (BOLA, обхід кроків процесу, розділи 9, 10), бо ті потребують розуміння наміру застосунку. Дослідження сканерів [115] показали низьку повноту саме на логічних і авторизаційних вадах. Звідси висновок: автоматизація необхідна для масштабу й регулярності, але її доповнює ручне тестування (підрозділ 15.3), а результати потребують тріажу — відсіювання хибних спрацювань і пріоритезації за ризиком (розділ 4).

15.3 Ручне тестування й оцінювання на проникнення

Ручне тестування кваліфікованим фахівцем доповнює автоматизацію там, де потрібне розуміння контексту: логіка бізнес-процесів, ланцюги з кількох вад, авторизація на рівні об'єктів. **Оцінювання на проникнення** (penetration testing) — авторизована імітація дій порушника для перевірки стійкості в реальних умовах. Його правова й етична основа (розділ 2) безумовна: роботи виконують лише за *письмовим дозволом* власника, у *чітко визначеному обсязі* (scope) і з *правилами взаємодії* (дозволені цілі, час, методи, канали зв'язку, поводження зі знайденими даними). Результат — звіт з описом знахідок, їх впливу й рекомендацій (розділ 16). Для вебресурсу тестування структурують за WSTG, щоб забезпечити повноту, а не лише ефектні окремі знахідки. Важлива відмінність від реальної атаки — *мінімізація шкоди*: тестувальник підтверджує наявність вади, але не завдає збитків і не виходить за обсяг.

15.4 Програми винагород і координоване розкриття

15.4.1 Координоване розкриття вразливостей

Навіть ретельно протестований ресурс може містити невиявлені вади, які знайдуть сторонні дослідники. **Координоване розкриття** (coordinated

vulnerability disclosure, CVD) — процес, що дає дослідникові безпечний і легальний канал повідомити про ваду, а організації — час її виправити до оприлюднення. Стандарти ISO/IEC 29147 [165] (розкриття) і 30111 [166] (оброблення) описують, як організація *приймає й обробляє* повідомлення. Базовий елемент — *політика розкриття* (часто у файлі `security.txt`): контакт, обсяг, зобов'язання не переслідувати добросовісних дослідників, очікувані строки. Відсутність такого каналу шкідлива: дослідник або не повідомить, або оприлюднить ваду без попередження.

15.4.2 Програми винагород

Програми винагород за вади (bug bounty) [130] розширюють CVD матеріальним стимулом і залучають широке коло дослідників. Їхня цінність — у *різноманітті* поглядів і постійності, що доповнює періодичне оцінювання. Умови успіху: чіткий обсяг і правила; адекватні винагороди за рівнем впливу; спроможність організації швидко обробляти й виправляти (інакше потік звітів накопичується); захист добросовісних дослідників від переслідування. Для державних ресурсів програми й політики розкриття потребують узгодження з правовою базою (розділ 2), але самі є визнаною світовою практикою. Акт ЄС про кіберстійкість (розділ 13) робить процес оброблення вразливостей обов'язковим, тож CVD стає не лише доброю практикою, а вимогою.

15.5 Метрики покриття та ефективності

Тестування потребує об'єктивних метрик, інакше «протестовано» нічого не означає. Корисні показники: *покриття вимог* (частка перевірок WSTG/ASVS, фактично виконаних); *покриття коду й поверхні* тестами; *щільність знахідок* за критичністю; *час до виявлення й виправлення* (пов'язаний з моделлю (15.1)); *частка хибних спрацювань* (визначає корисність засобу). Ефективність набору засобів оцінюють на відомому наборі вад (еталонному чи штучно внесеному) як повноту й точність:

$$\text{Recall} = \frac{TP}{TP + FN}, \qquad \text{Precision} = \frac{TP}{TP + FP}, \qquad F_1 = \frac{2\,\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \tag{15.3}$$

де TP, FP, FN — істинно виявлені, хибно виявлені й пропущені вади. Повнота важлива для безпеки (пропуск — невиявлена вада), точність — для продуктивності (багато хибних спрацювань знецінюють засіб, бо їх перестають читати). Оскільки жоден засіб не має високими обома, набір будують так, щоб максимізувати сукупну повноту за прийнятної точності, а результати тріажують. Метрики інтегрують у конвеєр: збирання не прохо-

дить, якщо покриття критичних перевірок нижче порога чи з'явилися нові критичні знахідки.

15.6 Кейс: програма тестування державного порталу

15.6.1 Вихідний стан

Портал державних послуг (розділ 4) тестується нерегулярно: разовий зовнішній аудит раз на рік, автоматичний сканер без тріажу (багато хибних спрацювань, які ігнорують), немає статичного аналізу в конвеєрі, немає тестів авторизації (розділ 9), немає політики розкриття вразливостей і каналу для дослідників. Аудит зрілості за WSTG [232], NIST SP 800-115 [264] і дослідженням автоматизованого тестування [18] виявив: покриття вимог ASVS невідоме; логічні й авторизаційні вади не покриті (межі автоматизації); вартість виправлення висока через пізнє виявлення (15.1).

15.6.2 Програма заходів

Таблиця 15.2 — Програма тестування захищеності порталу

#	Захід	Мета
T1	SAST і SCA в конвеєрі з тріажем; поріг на критичні знахідки	зсув уліво, (15.1)
T2	Тести авторизації й бізнес-логіки (розділи 9, 10) в CI	межі автоматизації
T3	DAST/IAST на тестовому середовищі; фазинг парсерів і API за схемою	динамічні вади, (15.2)
T4	Періодичне ручне оцінювання на проникнення за WSTG у письмовому обсязі	логіка, ланцюги вад
T5	Політика розкриття (security.txt), канал для дослідників, процес за ISO 29147/30111	CVD
T6	Метрики покриття, повноти/точності (15.3), часу виправлення; інтеграція в конвеєр	вимірюваність

Пріоритет — T1 і T2: статичний аналіз і тести авторизації в конвеєрі дають найбільшу віддачу за (15.1) (раннє, регулярне, дешеве виявлення) і закривають саме ті класи (авторизація), які автоматичні сканери пропускають. T3 додає динамічну перевірку й фазинг. T4 покриває логічні вади й ланцюги, недосяжні автоматизації, суворо в межах дозволу й обсягу (розділ 2). T5 створює легальний канал для сторонніх знахідок, відсутність якого шкідлива. T6 робить тестування вимірюваним і керованим.

15.6.3 Результат

Після впровадження вади виявляються переважно на ранніх етапах і дешево; авторизаційні й логічні класи покриті тестами й періодичним ручним оцінюванням; динамічні дефекти й дефекти парсерів виявляє DAST і фазинг; сторонні дослідники мають легальний канал; рівень безпеки вимі-

рюється й відстежується. Залишкові ризики: вади, що їх не виявляє жоден метод до експлуатації (компенсуються моніторингом і готовністю до реагування, розділ 16); залежність якості ручного тестування від кваліфікації (вибір виконавців, повторюваність за WSTG); накопичення звітів CVD без спроможності виправляти (потрібен ресурс на оброблення).

Висновки до розділу

Тестування захищеності перевіряє, що побудований захист працює, і дає об'єктивні свідчення рівня безпеки для ухвалення рішень. Методології (WSTG, PTES, NIST SP 800-115) забезпечують повноту й повторюваність, а принцип зсуву вліво переносить перевірки на ранні, дешевші етапи (15.1), не замінюючи пізні. Автоматичні засоби — SAST, DAST, IAST, SCA, фазинг — виявляють різні перетинні множини вад і мають спадну віддачу за обсягом (15.2), але погано бачать вади логіки й авторизації, тож їх поєднують і доповнюють ручним тестуванням. Оцінювання на проникнення виконують лише за письмовим дозволом, у визначеному обсязі, з правилами взаємодії та мінімізацією шкоди. Координоване розкриття й програми винагород дають стороннім дослідникам легальний канал і доповнюють періодичне оцінювання, а Акт ЄС про кіберстійкість робить процес оброблення вразливостей обов'язковим. Об'єктивні метрики — покриття вимог, повнота й точність (15.3), час виявлення й виправлення — роблять тестування вимірюваним та інтегрованим у життєвий цикл.

Питання для самоперевірки

1. Яку роль відіграють методології WSTG, PTES і NIST SP 800-115? Чому повнота й повторюваність важливіші за окремі ефектні знахідки?
2. Поясніть принцип зсуву вліво. За (15.1) обчисліть економію від перенесення виявлення вади з етапу експлуатації ($e{=}5$) на код ($e{=}2$) за $k = 5$.
3. Порівняйте SAST, DAST, IAST і SCA (табл. 15.1) за тим, що аналізують, та їхніми сильними й слабкими сторонами. Чому їх поєднують?
4. Що таке фазинг і керування покриттям? За (15.2) поясніть спадну віддачу й оцініть $P_{\text{вияв}}$ для $\theta = 10^{-4}$, $N = 5 \cdot 10^4$.
5. Чому автоматичні засоби погано виявляють вади логіки й авторизації? Який висновок щодо ролі ручного тестування?
6. Які правові й етичні умови обов'язкові для оцінювання на проникнення (розділ 2)? Що означає мінімізація шкоди й чим тест відрізняється від реальної атаки?
7. Що таке координоване розкриття вразливостей і які стандарти його описують? Чому відсутність каналу для дослідників шкідлива?

8. У чому цінність програм винагород за вади і які умови їх успіху? Як із цим пов'язаний Акт ЄС про кіберстійкість?
9. Назвіть корисні метрики тестування. За (15.3) поясніть різницю між повнотою й точністю і чому жоден засіб не має високими обома.
10. Чому «протестовано» без метрик нічого не означає? Як інтегрувати метрики в конвеєр?
11. Чому зсув уліво доповнює, а не замінює пізні динамічні й ручні перевірки?
12. Як структурувати оцінювання на проникнення за WSTG, щоб забезпечити повноту? Що має містити звіт?
13. Поясніть, чому тести авторизації доцільно виконувати в конвеєрі автоматично (зв'язок з розділами 9, 10).
14. Як пріоритезувати знахідки автоматичних засобів і навіщо потрібен тріаж?
15. Дослідницьке: розробіть методику вимірювання покриття вимог ASVS автоматичними й ручними перевірками та дослідіть, яка комбінація засобів максимізує сукупну повноту (15.3) за прийнятної точності на відкритому вебзастосунку.
16. Дослідницьке: запропонуйте модель програми координованого розкриття вразливостей для державних вебресурсів України з урахуванням правової бази (розділ 2) і оцініть її вплив на час виявлення й виправлення вад.

РОЗДІЛ 16

Моніторинг, виявлення вторгнень і реагування на інциденти вебресурсів

Жоден набір запобіжних заходів не гарантує відсутності інцидентів, тож зрілий захист передбачає здатність *помітити* атаку, *відреагувати* на неї й *відновитися*. Нестача саме цієї здатності відображена категорією A09:2025 «Logging and Alerting Failures» OWASP Top 10 [231]: якщо подій не фіксують або на них не реагують, атака триває непоміченою місяцями. Розділ розглядає журналювання (що й як фіксувати), централізоване збирання й кореляцію подій (SIEM), телеметрію захисних засобів (WAF, RASP), виявлення аномалій і приманки, процес реагування на інциденти (NIST SP 800-61r3, ISO/IEC 27035, роль CERT-UA), цифрову криміналістику вебсерверів і стійкість та відновлення, із завершальним кейсом. Розділ замикає захисний цикл: моніторинг спирається на телеметрію всіх попередніх рівнів (CSP, WAF, авторизація, конвеєр) і живить оновлення оцінок ризику (розділ 4).

16.1 Журналювання

16.1.1 Що фіксувати

Основа виявлення й розслідування — якісні журнали. Фіксують події безпеки [234]: автентифікацію (успіхи й невдачі, зміни факторів), авторизаційні відмови, зміни привілеїв і конфігурації, адміністративні дії, значущі бізнес-операції, спрацювання захисних засобів, помилки й винятки (розділ 7). Кожен запис має містити достатній контекст: час (синхронізований, у єдиній зоні), суб'єкт, джерело, дію, об'єкт, результат і кореляційний ідентифікатор запиту. Водночас журнали *не повинні* містити зайвих чутливих даних — паролів, токенів, повних номерів карток, надмірних персональних даних, — бо журнал сам стає ціллю (вимоги захисту персональних даних, розділ 2). Журналювання має бути *структурованим* (поля, а не вільний рядок), що усуває ін'єкцію в журнали (розділ 7) і уможливлює автоматичний аналіз.

16.1.2 Цілісність і доступність журналів

Журнали — ціль атаки: порушник прагне приховати сліди, тому їх захищають від зміни й видалення. Заходи: передавання в *централізоване* сховище поза атакованою системою (щоб компрометація сервера не знищила журнали); контроль цілісності (підписування, незмінне сховище, пор. резервні копії розділу 12); обмеження доступу; достатній строк зберігання відповідно до вимог і потреб розслідування. Для юридично значущих дій застосо-

вують неспростовність (КЕП, розділ 8). Баланс: занадто мало журналів — не видно атаки (A09), занадто багато без структури й зберігання — шум і витрати; тому обсяг визначають за моделлю загроз (розділ 4) — фіксують те, що потрібне для виявлення пріоритетних сценаріїв і розслідування.

16.2 Централізоване збирання й кореляція

16.2.1 SIEM

Система керування подіями безпеки (SIEM) [75] збирає журнали з усіх джерел (вебсервери, застосунок, WAF, база даних, інфраструктура, CSP-звіти розділу 6), нормалізує їх і дає змогу *корелювати* події в різних джерелах, виявляючи ознаки, невидимі в окремому журналі. Приклади кореляції для вебресурсу: сплеск авторизаційних відмов від одного суб'єкта після успішного входу (можливе зловживання), незвичні адміністративні дії поза робочим часом, поява запитів до неінвентаризованих кінцевих точок (тіньові API, розділ 10), зміна конфігурації, що відкриває ресурс (розділ 12). SIEM — основа роботи центру безпеки (SOC), а правила кореляції відображають сценарії моделі загроз.

16.2.2 Баланс виявлення й хибних тривог

Виявлення підпорядковане теорії з розділу 3: за низької базової частоти атак навіть точні правила дають багато хибних тривог (парадокс базової частоти). Якщо правило має ймовірність виявлення p_d і хибної тривоги p_f, а частка справді зловмисних подій у потоці — π, то частка справжніх серед усіх тривог

$$\mathrm{PPV} = \frac{\pi\, p_d}{\pi\, p_d + (1-\pi)\, p_f}. \tag{16.1}$$

Для $\pi = 10^{-3}$, $p_d = 0{,}95$, $p_f = 0{,}02$ маємо $\mathrm{PPV} \approx 0{,}045$ — лише кожна двадцять друга тривога справжня. Тому мета — не максимальна чутливість, а *керований* потік тривог: пріоритезація, групування пов'язаних подій в інциденти, збагачення контекстом і автоматизація рутинного тріажу. Перевантажений хибними тривогами аналітик пропускає справжні — це операційний бік A09. Звідси практика: правила калібрують за (16.1), а найцінніші — ті, що корелюють кілька слабких сигналів у сильний.

16.3 Телеметрія захисних засобів і виявлення аномалій

16.3.1 WAF, RASP і звіти політик

Захисні засоби є й джерелами телеметрії. WAF/WAAP (розділ 11) постачає дані про заблоковані й підозрілі запити. **RASP** (runtime application self-protection) [97] діє *зсередини* застосунку, спостерігаючи за фактичним ви-

конанням (виклики до БД, файлової системи), і тому точніший за зовнішній WAF, хоча потребує інструментування й ресурсів. Звіти про порушення CSP (розділ 6) — цінне джерело: поява порушень на сторінці, де все переписано, сигналізує про можливий XSS чи сторонній скрипт. Ці джерела спрямовують у SIEM, замикаючи зв'язок клієнтських і серверних заходів із моніторингом.

16.3.2 Виявлення аномалій

Сигнатурне виявлення (відоме погане) доповнюють *виявленням аномалій* [86] — відхилень від базової лінії нормальної поведінки (обсяг і структура запитів, географія, патерни доступу до даних). Воно здатне помітити невідомі атаки, але схильне до хибних тривог за (16.1) і потребує якісної базової лінії. Моделі машинного навчання тут корисні як інструмент підтримки (розділ 14), але з тими самими застереженнями: вони не ухвалюють автоматичних блокувань критичних дій без контролю балансу помилок. Практично аномалії використовують для *пріоритезації* уваги, а не для автоматичних санкцій.

16.3.3 Приманки

Приманки (honeypots, honeytokens) [277] — навмисно привабливі фальшиві ресурси чи дані, будь-яке звернення до яких є сигналом, бо легітимних причин для нього немає. Приклади для вебресурсу: фальшиві адміністративні шляхи, підроблені записи в БД (honeytoken), чиє використання означає витік. Їхня цінність — висока точність (мало хибних тривог, бо нормальних звернень немає) і раннє виявлення. Приманки доповнюють, а не замінюють основне виявлення.

16.4 Реагування на інциденти

16.4.1 Процес

Реагування на інциденти (incident response) — організований процес, а не імпровізація. NIST SP 800-61r3 [214] у редакції, узгодженій з CSF 2.0, і ISO/IEC 27035 [164] описують цикл (рис. 16.1): *підготовка* (план, ролі, інструменти, навчання) → *виявлення й аналіз* (підтвердження, оцінка впливу й обсягу) → *стримування, викорінення й відновлення* → *діяльність після інциденту* (уроки, оновлення захисту й оцінок ризику). Підготовка — найважливіший етап: під час інциденту немає часу визначати ролі й шукати контакти. План містить класифікацію інцидентів, порядок ескалації, канали зв'язку (зокрема поза скомпрометованою системою), контакти регуляторів і CERT-UA, вимоги до збереження доказів.

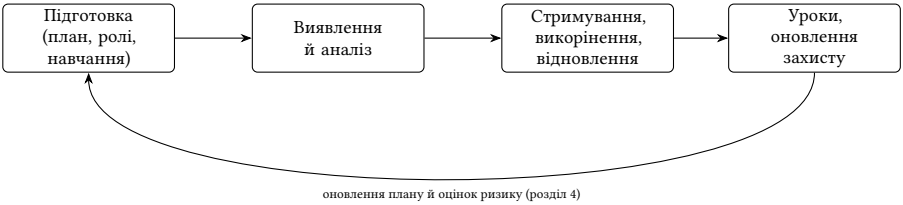


Рис. 16.1 — Цикл реагування на інциденти (NIST SP 800-61r3)

16.4.2 Швидкість виявлення й стримування

Шкода інциденту зростає з часом, поки порушник діє непоміченим. Галузеві звіти [194] вимірюють *час перебування* (dwell time) — від проникнення до виявлення, — який досі вимірюється днями й тижнями. Очікувану шкоду інциденту наближають як зростальну функцію часу до стримування:

$$L = L_0 + \int_0^{t_c} r(\tau)\, d\tau \;\approx\; L_0 + \bar{r}\,(t_d + t_r), \tag{16.2}$$

де L_0 — початкова шкода; $r(\tau)$ — швидкість накопичення шкоди (витік даних, поширення); $t_c = t_d + t_r$ — час до стримування як сума часу *виявлення* t_d і часу *реагування* t_r; $\bar{r}$ — середня швидкість. Формула (16.2) показує, що інвестиції у швидше виявлення (моніторинг, SIEM, приманки) і підготовлене реагування (план, автоматизація стримування) прямо зменшують шкоду, часто ефективніше за додаткові запобіжні заходи, коли ймовірність інциденту вже помірна. Це кількісний аргумент за баланс запобіжних і детективних заходів у моделі ризику (розділ 4).

16.4.3 Регуляторне звітування й CERT-UA

Реагування включає й *зобов'язання*: повідомлення регуляторів і суб'єктів даних у встановлені строки (GDPR, розділ 2; формула строків (2.3)), звітування про інциденти згідно з національними вимогами й взаємодію з CERT-UA [9], що координує реагування й надає індикатори компрометації. Акт ЄС про кіберстійкість (розділ 13) додає обов'язок звітувати про активно експлуатовані вразливості. Планування звітування заздалегідь критичне, бо строки короткі, а під час інциденту ресурс обмежений. Дослідження організації реагування в державних системах [3] наголошують на потребі заздалегідь визначених процедур і навчань.

16.5 Цифрова криміналістика та відновлення

16.5.1 Збереження доказів і розслідування

Після інциденту потрібно встановити, що сталося, яким був обсяг і як запобігти повторенню, — це вимагає *збереження доказів* [83]: фіксація стану систем, журналів, образів, з документуванням ланцюга збереження (chain of custody) для можливого юридичного використання. Для вебсерверів типовий артефакт — **вебшел** (web shell) [92]: завантажений порушником файл, що дає віддалене керування; його виявлення й аналіз (час появи, спосіб завантаження — зв'язок з розділом 7) допомагають відновити ланцюг атаки. Важливий баланс: поспішне «очищення» системи знищує докази, тому порядок дій визначає план реагування (стримувати, зберігши докази, а не хаотично перевстановлювати).

16.5.2 Стійкість і відновлення

Відновлення спирається на резервні копії (розділ 12) і цілі RTO/RPO. Критична вимога — переконатися, що відновлюють у *чистий* стан: відновлення з копії, що вже містить закладку порушника, повертає компрометацію. Тому перед відновленням викорінюють причину (закрита вада, ротовані облікові дані й секрети, видалені закладки) і перевіряють цілісність. Стійкість (resilience) означає здатність надавати критичні послуги навіть під час інциденту (керована деградація, розділ 11) і швидко відновлюватися; її планують у межах управління неперервністю діяльності (ISO 22301) і перевіряють навчаннями.

16.6 Кейс: моніторинг і реагування державного порталу

16.6.1 Вихідний стан

Портал державних послуг (розділ 4) веде журнали лише на рівні вебсервера, без подій застосунку й авторизаційних відмов; журнали зберігаються локально (знищуються при компрометації); SIEM немає, кореляції немає; CSP-звіти (розділ 6) і телеметрія WAF не збираються; немає плану реагування, ролей і каналу з CERT-UA; резервні копії не перевіряються на чистоту відновлення. Аудит за NIST SP 800-61r3 [214], ISO/IEC 27035 [164] і дослідженням реагування [3] оцінив час виявлення як потенційно тижні (A09), що за (16.2) означає велику очікувану шкоду.

16.6.2 План заходів

Таблиця 16.1 — Програма моніторингу й реагування порталу

#	Захід	Мета
M1	Структуроване журналювання подій безпеки застосунку без зайвих чутливих даних	A09, 16.1
M2	Централізоване незмінне сховище журналів поза робочим середовищем	цілісність доказів
M3	SIEM з правилами кореляції за сценаріями моделі загроз; калібрування за (16.1)	виявлення
M4	Збирання CSP-звітів, телеметрії WAF, подій конфігурації; приманки/honeytoken	телеметрія, рання тривога
M5	План реагування: ролі, ескалація, канали, зв'язок з CERT-UA; навчання	16.4, (16.2)
M6	Процедури збереження доказів і виявлення вебшелів	криміналістика
M7	Перевірка чистоти відновлення з копій; викорінення причини перед відновленням	стійкість

Пріоритет — M1–M3: без якісних журналів, їх збереження й кореляції атака лишається непоміченою (A09), а за (16.2) саме скорочення часу виявлення найсильніше зменшує шкоду. M4 додає цінні джерела (CSP-звіти виявляють XSS, приманки — високоточні сигнали). M5 забезпечує підготовленість — найважливіший етап реагування, зокрема заздалегідь налагоджений зв'язок з CERT-UA і план звітування у строки (2.3). M6 і M7 гарантують правильне розслідування й відновлення в чистий стан.

16.6.3 Результат

Після заходів події безпеки фіксуються й зберігаються незмінно поза робочим середовищем; SIEM корелює сигнали й дає керований потік пріоритезованих тривог; клієнтська й серверна телеметрія об'єднані; організація має план реагування, ролі й канал з CERT-UA; докази зберігаються, а відновлення відбувається в чистий стан. Очікувана шкода інциденту знижується за рахунок меншого часу виявлення й підготовленого реагування (16.2). Залишкові ризики: хибні тривоги за низької базової частоти (16.1) (пом'якшуються кореляцією й пріоритезацією), витончені атаки з довгим прихованим перебуванням (приманки, виявлення аномалій, полювання на загрози) і залежність якості реагування від навчань (регулярні тренування).

Висновки до розділу

Зрілий захист передбачає здатність помітити атаку, відреагувати й відновитися, а її нестача — це категорія A09:2025. Основа — якісне структуроване журналювання подій безпеки з достатнім контекстом, але без зайвих чутливих даних, із захистом цілісності й централізованим збереженням поза атакованою системою. SIEM корелює події різних джерел, виявляючи невидиме в окремому журналі, але за низької базової частоти атак навіть точні

правила дають багато хибних тривог (16.1), тож мета — керований, пріоритезований потік, а не максимальна чутливість. Телеметрія захисних засобів (WAF, RASP, CSP-звіти), виявлення аномалій і приманки доповнюють сигнатурне виявлення. Реагування на інциденти — організований цикл (NIST SP 800-61r3, ISO/IEC 27035), у якому підготовка є найважливішим етапом, а шкода зростає з часом до стримування (16.2), тож інвестиції у швидше виявлення й підготовлене реагування прямо її зменшують. Реагування включає звітування регуляторам у строки й взаємодію з CERT-UA. Криміналістика вимагає збереження доказів перед очищенням, а відновлення — викорінення причини й повернення в чистий стан. Моніторинг замикає захисний цикл, живлячи оновлення оцінок ризику.

Питання для самоперевірки

1. Чому нестача журналювання й реагування виділена в категорію A09:2025? Які події безпеки слід фіксувати і який контекст має містити запис?
2. Чому журнали не повинні містити зайвих чутливих даних і як їх захищають від зміни й знищення? Навіщо структуроване журналювання?
3. Що робить SIEM і які приклади кореляції подій корисні для вебресурсу? Як кореляція виявляє невидиме в окремому журналі?
4. За (16.1) обчисліть частку справжніх тривог для $\pi = 5 \cdot 10^{-4}$, $p_d = 0{,}9$, $p_f = 0{,}01$. Чому мета — керований потік, а не максимальна чутливість?
5. Порівняйте WAF і RASP як джерела телеметрії. Чому RASP точніший і яка його ціна?
6. Яку цінність мають звіти про порушення CSP (розділ 6) для моніторингу?
7. Що таке виявлення аномалій і приманки? Чому приманки мають високу точність, а виявлення аномалій схильне до хибних тривог?
8. Опишіть цикл реагування на інциденти (рис. 16.1). Чому підготовка — найважливіший етап?
9. За (16.2) поясніть, чому скорочення часу виявлення прямо зменшує шкоду. Коли інвестиції в детективні заходи ефективніші за додаткові запобіжні?
10. Які зобов'язання зі звітування включає реагування і чому їх планують заздалегідь? Яка роль CERT-UA?
11. Навіщо зберігати докази перед очищенням системи? Що таке вебшел і як його виявлення допомагає розслідуванню?

12. Чому відновлення має відбуватися в чистий стан і що для цього роблять перед ним? Як пов'язані стійкість і керована деградація (розділ 11)?
13. Як обсяг журналювання визначають за моделлю загроз (розділ 4)? Який баланс між надто малим і надто великим обсягом?
14. Чому моніторинг замикає захисний цикл і як він живить оновлення оцінок ризику?
15. Дослідницьке: розробіть набір правил кореляції SIEM для типових сценаріїв атак на державний вебпортал і оцініть їх за (16.1) на модельному потоці подій, мінімізуючи хибні тривоги за заданого рівня виявлення.
16. Дослідницьке: побудуйте модель залежності очікуваної шкоди інциденту від часу виявлення й реагування (16.2) для послуги електронного урядування та визначте оптимальний розподіл інвестицій між запобіжними й детективними заходами.

РОЗДІЛ 17

Безпечний життєвий цикл розроблення, управління безпекою та напрями наукових досліджень

Попередні розділи розглянули окремі механізми й етапи захисту вебресурсів. Цей, завершальний, об'єднує їх у *систему*: безпечний життєвий цикл розроблення, що вбудовує безпеку в кожен етап, моделі зрілості для оцінювання й розвитку програми, метрики управління, підготовку кадрів і напрями наукових досліджень. Головна теза посібника тут набуває остаточного вигляду: безпека вебресурсу — не набір окремих засобів, а властивість *процесу* й *платформи*, що робить безпечну поведінку типовою, а небезпечну — важкою. Розділ розглядає SSDLC і SSDF, моделі зрілості (SAMM, BSIMM), DevSecOps, безпечні за замовчуванням фреймворки й приватність за задумом, метрики програми безпеки, підготовку фахівців, напрями досліджень і методологію дисертаційного дослідження, із завершальним кейсом побудови програми безпеки.

17.1 Безпечний життєвий цикл розроблення

17.1.1 SSDLC і SSDF

Безпечний життєвий цикл розроблення (SSDLC) [201] вбудовує діяльність з безпеки в кожен етап — від вимог до виведення з експлуатації, — а не додає перевірку наприкінці. **Рамка безпечного розроблення** NIST (SSDF, SP 800-218) [275] систематизує практики в чотири групи: *підготувати організацію* (ролі, політики, інструменти), *захистити ПЗ* (цілісність, захист коду й артефактів — розділ 13), *виробляти захищене ПЗ* (моделювання загроз, безпечне проєктування й кодування, тестування) і *реагувати на вразливості* (оброблення й виправлення). SSDF є загальновизнаною основою й згадана у вимогах (зокрема CRA, розділ 13). Розділи цього посібника відображають практики SSDF: моделювання загроз (розділ 4), безпечне проєктування (розділи 6–10), тестування (розділ 15), реагування (розділ 16).

17.1.2 Безпека на кожному етапі

Таблиця 17.1 зіставляє етапи розроблення з діяльністю з безпеки й відповідними розділами посібника, показуючи його як єдину систему, а не перелік тем.

Таблиця 17.1 — Безпека на етапах життєвого циклу

Етап	Діяльність з безпеки	Розділи
Вимоги	вимоги безпеки (ASVS), приватність за задумом	2, 4
Проєктування	моделювання загроз, безпечні патерни	4, 6–10
Кодування	безпечні фреймворки, рецензування, SAST	6–12, 15
Збирання	цілісність ланцюга, SCA, SBOM	13
Тестування	DAST/IAST, фазинг, тести авторизації	15
Розгортання	посилення конфігурації, IaC	5, 12
Експлуатація	моніторинг, реагування, оновлення	11, 16

17.2 Моделі зрілості та DevSecOps

17.2.1 SAMM і BSIMM

Програму безпеки оцінюють і розвивають за *моделями зрілості*. **OWASP SAMM** [240] — приписова модель, що визначає практики за функціями (управління, проєктування, реалізація, верифікація, експлуатація) і рівні зрілості для кожної, даючи дорожню карту вдосконалення. **BSIMM** [206] — описова модель: результат спостереження за реальними програмами багатьох організацій, що дає змогу порівняти власну програму з галуззю. SAMM відповідає на питання «що робити далі», BSIMM — «як ми виглядаємо на тлі інших»; їх застосовують разом. Зрілість зручно подавати агрегованим показником за практиками:

$$\text{Maturity} = \frac{1}{|\mathcal{F}|} \sum_{f \in \mathcal{F}} \frac{\ell_f}{\ell_{\max}}, \tag{17.1}$$

де $\mathcal{F}$ — множина оцінюваних практик; ℓ_f — досягнутий рівень практики f; $\ell_{\max}$ — максимальний. Показник (17.1) корисний для відстеження прогресу в часі й виявлення найслабших практик, але його інтерпретують обережно: рівномірне підвищення всіх практик не завжди оптимальне — пріоритет визначають за ризиком (розділ 4), а не за прагненням вирівняти профіль.

17.2.2 DevSecOps

DevSecOps [212] інтегрує безпеку в конвеєр безперервної інтеграції й доставки, роблячи перевірки *автоматичними* й *безперервними* замість разових «воріт» наприкінці. Ключові практики: SAST, SCA, перевірка IaC і тести авторизації в конвеєрі (розділи 12, 15); генерація SBOM і підписування артефактів (розділ 13); політика як код (розділ 9); пороги, що зупиняють збирання за критичних знахідок. Важливий культурний складник — безпека як *спільна відповідальність* команд, а не функція окремого відділу-контролера; роль фахівців з безпеки зміщується від перевіряльників до тих,

хто надає інструменти й настанови, що роблять безпечну розробку шляхом найменшого опору.

17.3 Безпека як властивість платформи

17.3.1 Безпечні за замовчуванням фреймворки

Наскрізна теза посібника: надійніше *унеможливити* клас вад, ніж навчати кожного розробника його уникати. Безпечні за замовчуванням фреймворки й бібліотеки переносять безпеку з дисципліни на властивість платформи: шаблонізатори з автоматичним контекстним екрануванням усувають більшість XSS (розділ 6), ORM з параметризацією — SQL-ін'єкції (розділ 7), централізована авторизація із забороною за замовчуванням — порушення доступу (розділ 9). Міра успіху платформи — наскільки важко написати небезпечний код: ідеал, коли небезпечна дія потребує явного, помітного «аварійного виходу», який легко знайти й проконтролювати. Вибір і розвиток таких фреймворків — стратегічне рішення, що дає більший і дешевший ефект, ніж будь-який обсяг навчання чи тестування.

17.3.2 Безпека мов і приватність за задумом

Споріднений важіль — вибір *безпечних щодо пам'яті* мов і технологій, що усувають цілі класи вад на рівні мови [91]. Для нового коду, особливо низькорівневих компонентів, спільні настанови органів безпеки рекомендують переходити на такі мови. **Приватність за задумом** (privacy by design) [84] застосовує той самий принцип до персональних даних: мінімізація збору, обмеження цілей, захист за замовчуванням, прозорість — вбудовані в проєкт, а не додані згодом, що узгоджується з вимогами GDPR (розділ 2) і LINDDUN (розділ 4). І безпека за замовчуванням, і приватність за задумом утілюють спільний принцип: правильна поведінка має бути типовою, а не результатом зусилля.

17.4 Метрики програми безпеки

Управління потребує *вимірювання* [173]. Корисні метрики програми: покриття практик зрілості (17.1); частка застосунків з моделлю загроз і актуальним SBOM; щільність і час життя вразливостей за критичністю; час виявлення й реагування (розділ 16); покриття тестами безпеки (розділ 15); частка коду на безпечних фреймворках. Вимоги до метрик: вимірюваність (об'єктивність, автоматизоване збирання), дієвість (керують рішеннями), стійкість до маніпуляції (метрика, що стала ціллю, спотворюється — закон Гудгарта). Метрики подають керівництву разом з оцінками ризику (розділ 4), щоб обґрунтовувати інвестиції й пріоритети. Небезпека — вимірювати легковимірюване (кількість знайдених вад) замість важливого (залишко-

вий ризик); тому метрики обирають так, щоб вони відображали *результат* (зниження ризику), а не лише *активність*.

17.5 Підготовка кадрів

Найкращі процеси й платформи потребують кваліфікованих людей. Підготовка фахівців з безпеки вебтехнологій поєднує фундамент (мережі, криптографія, вебплатформа), практичні навички (безпечне кодування, тестування в межах дозволу, реагування) і *інженерне мислення* — розуміння причин вад і системних заходів, а не заучування окремих технік. Компетентнісний підхід [5] наголошує на формуванні здатності проєктувати захист, а не лише знаходити вади. Важлива складова — *етика й право* (розділ 2): фахівець діє лише в межах дозволу й мінімізує шкоду. Безперервне навчання обов'язкове, бо технології й загрози швидко змінюються; ефективні форми — практичні полігони (у контрольованому середовищі), участь у програмах розкриття, аналіз реальних інцидентів. Для організації навчання розробників безпечному кодуванню — практика SSDF і SAMM, а його ефективність вимірюють зниженням щільності вад, а не кількістю проведених годин.

17.6 Напрями наукових досліджень

Безпека вебресурсів лишається активною науковою галуззю. Перспективні напрями: *формальна верифікація* вебкомпонентів і протоколів [176] (успіх формального аналізу TLS 1.3, розділ 5, показує здійсненність); *автоматичний ремонт* вразливостей [187] і генерація безпечного коду з гарантіями; застосування й водночас *захист* ШІ (розділ 14) — зокрема теоретичні межі протидії ін'єкції підказок; *постквантовий перехід* вебплатформи (розділ 5), особливо постквантові підписи й керування сертифікатами; *безпека браузерних платформ* (ізоляція, побічні канали); *вимірювальні дослідження* стану безпеки реальних вебресурсів і економіки безпеки. Спільна риса сильних напрямів — прагнення *системних* рішень (усунути клас вад), а не латання окремих випадків, що відповідає головній тезі посібника.

17.7 Методологія дисертаційного дослідження

Для здобувачів наукового ступеня в галузі безпеки вебресурсів розділ окреслює методологічні вимоги. Дослідження має містити: чітко сформульовану *наукову проблему* й гіпотезу; *огляд* стану з критичним аналізом (а не переліком); обґрунтований *метод* (модель, алгоритм, методику) з доведенням коректності чи властивостей; *оцінювання* на відтворюваних даних з коректними метриками (розділ 15, (15.3)) і порівнянням з аналогами; аналіз *обмежень* і загроз validності. Специфіка галузі — вимога *етичності* й *відтворюваності*: експерименти з атакувальними техніками проводять лише

в контрольованому середовищі або з дозволу (розділ 2), а дані й код публікують для перевірки, де це не створює ризику. Практична цінність (застосовність до реальних систем, зокрема державних) посилює роботу, а зв'язок із національними пріоритетами (Стратегія кібербезпеки України [41]) обґрунтовує актуальність. Типова помилка — підміна наукового внеску інженерним описом системи без узагальнення, оцінювання й порівняння.

17.8 Кейс: побудова програми безпеки вебресурсів відомства

17.8.1 Постановка

Відомство супроводжує портфель вебресурсів (портал послуг, інформаційні сайти, внутрішні системи) з нерівномірною зрілістю: окремі практики (TLS, базовий WAF) є, але немає цілісної програми — моделювання загроз епізодичне, безпека в конвеєрі фрагментарна, метрик немає, навчання розробників не систематичне. Оцінювання за SAMM [240] дало показник зрілості (17.1) близько 0,35 з найслабшими практиками у верифікації й реагуванні. Ціль — побудувати керовану програму, що системно знижує ризик портфеля, а не окремі вади.

17.8.2 Програма

Таблиця 17.2 — Дорожня карта програми безпеки

#	Напрям	Розділи
E1	Вимоги безпеки (ASVS) і моделювання загроз для всіх нових систем	2, 4
E2	Перехід на безпечні за замовчуванням фреймворки; централізована авторизація як платформа	6–10
E3	DevSecOps: SAST, SCA, тести авторизації, SBOM, підписування в конвеєрі	13, 15
E4	Посилення й IaC для інфраструктури; безперервне виявлення помилок конфігурації	12
E5	Моніторинг, SIEM, план реагування, зв'язок з CERT-UA	16
E6	Метрики програми й зрілості (17.1); звітування керівництву з оцінками ризику	4
E7	Систематичне навчання розробників; політика координованого розкриття	15

Пріоритет визначено ризиком і важелем впливу. E2 (безпечні за замовчуванням платформи) дає найбільший системний ефект — усуває цілі класи вад дешевше за навчання й тестування. E1 забезпечує, що нові системи проєктуються безпечно від початку (зсув уліво, розділ 15). E3–E5 автоматизують перевірки, посилюють інфраструктуру й додають здатність виявляти й реагувати — найслабші практики за оцінкою. E6 робить програму керованою й обґрунтовує інвестиції, E7 розвиває людський складник. Про-

граму впроваджують ітеративно, починаючи з найризикованіших систем (портал послуг), а прогрес відстежують за (17.1) і метриками результату.

17.8.3 Результат

Після впровадження нові системи мають вимоги безпеки й модель загроз; цілі класи вад усунено на рівні платформи; конвеєр автоматично перевіряє код, залежності й авторизацію; інфраструктура посилена й контрольована; організація виявляє інциденти й реагує за планом; програма вимірюється й керується за ризиком; розробники навчені, є легальний канал для дослідників. Показник зрілості зростає, але головний результат — зниження *залишкового ризику* портфеля, а не самі цифри. Залишкові виклики: підтримання зрілості в часі (дрейф, плинність кадрів), баланс безпеки й швидкості постачання (DevSecOps пом'якшує, але не усуває) і залежність від зовнішніх сторін (постачальники, провайдери), що лишаються в моделі ризику.

Висновки до розділу

Безпека вебресурсу — властивість процесу й платформи, а не набір окремих засобів. Безпечний життєвий цикл (SSDLC, SSDF) вбудовує безпеку в кожен етап — від вимог і моделювання загроз до реагування, — а розділи посібника відображають його практики як єдину систему. Моделі зрілості SAMM (приписова) і BSIMM (описова) оцінюють і розвивають програму, а показник зрілості (17.1) відстежує прогрес, хоч пріоритети визначає ризик, а не вирівнювання профілю. DevSecOps робить перевірки автоматичними й безперервними, а безпеку — спільною відповідальністю. Головний важіль — безпека як властивість платформи: безпечні за замовчуванням фреймворки, безпечні щодо пам'яті мови й приватність за задумом усувають цілі класи вад дешевше за навчання й тестування, втілюючи принцип «правильна поведінка має бути типовою». Метрики програми обирають так, щоб вони відображали зниження ризику, а не активність, і подають керівництву з оцінками ризику. Підготовка кадрів формує інженерне мислення й етику, а не заучування технік. Перспективні наукові напрями — формальна верифікація, автоматичний ремонт, безпека й захист ШІ, постквантовий перехід — об'єднує прагнення системних рішень, що й є головним висновком усього посібника.

Питання для самоперевірки

1. Що таке безпечний життєвий цикл розроблення і які чотири групи практик визначає SSDF? Як розділи посібника відображають ці практики (табл. 17.1)?

2. Порівняйте приписову (SAMM) і описову (BSIMM) моделі зрілості. На які питання відповідає кожна і чому їх застосовують разом?
3. За (17.1) поясніть показник зрілості. Чому рівномірне підвищення всіх практик не завжди оптимальне?
4. Що таке DevSecOps і які практики безпеки інтегрують у конвеєр? У чому полягає культурний складник «спільної відповідальності»?
5. Поясніть тезу «безпека як властивість платформи». Чому унеможливлення класу вад надійніше за навчання уникати його? Наведіть три приклади.
6. Що таке безпечні щодо пам'яті мови і приватність за задумом? Який спільний принцип вони втілюють із безпекою за замовчуванням?
7. Які метрики програми безпеки корисні і яких вимог до них дотримуються? Поясніть небезпеку вимірювати активність замість результату (закон Гудгарта).
8. Що має поєднувати підготовка фахівця з безпеки вебтехнологій? Чому інженерне мислення й етика важливіші за заучування технік?
9. Назвіть перспективні напрями наукових досліджень і поясніть, що їх об'єднує. Як це пов'язано з головною тезою посібника?
10. Які методологічні вимоги до дисертаційного дослідження в галузі? У чому полягає типова помилка підміни наукового внеску інженерним описом?
11. Чому етичність і відтворюваність є специфічними вимогами до досліджень у цій галузі (зв'язок з розділом 2)?
12. Як пріоритезувати заходи програми безпеки за ризиком і важелем впливу (на прикладі кейсу)? Чому E2 дає найбільший системний ефект?
13. Чому показник зрілості не є самоціллю, а головним результатом є зниження залишкового ризику?
14. Як пов'язані між собою розділи посібника в єдину систему безпечного життєвого циклу (табл. 17.1)?
15. Дослідницьке: розробіть метрику результату програми безпеки, що відображає зниження залишкового ризику портфеля вебресурсів (а не активність), стійку до маніпуляції, і апробуйте її на модельному портфелі з оцінкою кореляції з фактичними інцидентами.
16. Дослідницьке: запропонуйте компетентнісну модель і методику підготовки фахівців з безпеки вебтехнологій для потреб електронного урядування України з урахуванням Стратегії кібербезпеки [41] і оцініть ефективність навчання за зниженням щільності вад у коді.

Список використаних джерел

[1] Байдур О. Кількісна методологія оцінки ризиків кібербезпеки при відсутності фінансових даних про втрати. *Кібербезпека: освіта, наука, техніка.* 2024. Т. 2, № 26. С. 95–114. DOI: 10.28925/2663-4023.2024.26.659.

[2] Балацька В., Ткачук Р., Маслова Н. Еволюція КСЗІ та інтеграція блокчейн-технологій у кіберзахисті державних інформаційних систем України. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 2, № 30. С. 316–332. DOI: 10.28925/2663-4023.2025.30.975.

[3] Бурячок В., Складанний П. Організація реагування на інциденти кібербезпеки в державних інформаційних системах. *Кібербезпека: освіта, наука, техніка.* 2024. Т. 3, № 27. С. 60–78.

[4] Бурячок В., Складанний П. Протидія розподіленим атакам на відмову в обслуговуванні державних інформаційних ресурсів. *Кібербезпека: освіта, наука, техніка.* 2023. Т. 2, № 22. С. 70–85.

[5] Бурячок В., Соколов В. Підготовка фахівців з кібербезпеки: компетентнісний підхід до безпеки вебтехнологій. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 1, № 29. С. 10–28.

[6] Горбенко І. Д., Горбенко Ю. І. Прикладна криптологія: теорія, практика, застосування: монографія. Харків: Форт, 2012. 878 с.

[7] Денисюк В. Прогнозування експлуатації CVE на основі відкритих даних NVD та KEV для ризико-орієнтованої пріоритизації. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 2, № 30. С. 428–444. DOI: 10.28925/2663-4023.2025.30.906.

[8] Державна служба спеціального зв'язку та захисту інформації України: офіційний вебсайт. URL: https://cip.gov.ua/.

[9] Державна служба спеціального зв'язку та захисту інформації України. Річний звіт CERT-UA про кіберінциденти. Київ, 2025. URL: https://cert.gov.ua/.

[10] Держспецзв'язку: Від 15 лютого Україна зазнала понад 3 000 DDoS-атак. *Урядовий портал.* 16.03.2022. URL: https://www.kmu.gov.ua/news/derzhspeczvyazku-vid-15-lyutogo-ukrayina-zaznala-ponad-3-000-ddos-atak.

[11] Деякі питання захисту інформаційних, електронних комунікаційних, інформаційно-комунікаційних, технологічних систем: постанова Кабінету Міністрів України від 18.06.2025 № 712. URL: https://www.kmu.gov.ua/npas/deiaki-pytannia-zakhystu-informatsiinykh-elektronnykh-komunikatsiinykh-informatsiino-s712180625.

[12] ДСТУ 4145-2002. Інформаційні технології. Криптографічний захист інформації. Цифровий підпис, що ґрунтується на еліптичних кривих. Формування та перевіряння. Київ: Держспоживстандарт України, 2003.

[13] ДСТУ 7624:2014. Інформаційні технології. Криптографічний захист інформації. Алгоритм симетричного блокового перетворення. Київ: Мінекономрозвитку України, 2015.

[14] ДСТУ 7564:2014. Інформаційні технології. Криптографічний захист інформації. Функція гешування. Київ: Мінекономрозвитку України, 2015.

[15] ДСТУ EN ISO/IEC 27001:2023. Інформаційна безпека, кібербезпека та захист приватності. Системи керування інформаційною безпекою. Вимоги (EN ISO/IEC 27001:2022, IDT). Київ: ДП «УкрНДНЦ», 2023.

[16] ДСТУ EN ISO/IEC 27002:2023. Інформаційна безпека, кібербезпека та захист приватності. Заходи інформаційної безпеки (EN ISO/IEC 27002:2022, IDT). Київ: ДП «УкрНДНЦ», 2023.

[17] Загальні вимоги до кіберзахисту об'єктів критичної інфраструктури: затв. постановою Кабінету Міністрів України від 19.06.2019 № 518. URL: https://zakon.rada.gov.ua/laws/show/518-2019-п.

[18] Корченко А., Складанний П. Автоматизоване тестування захищеності вебдодатків у процесі розробки. *Кібербезпека: освіта, наука, техніка.* 2024. Т. 2, № 26. С. 140–156.

[19] Корченко А., Складанний П. Моделі розмежування доступу в інформаційних системах критичної інфраструктури. *Кібербезпека: освіта, наука, техніка.* 2024. Т. 3, № 23. С. 120–135.

[20] Кримінальний кодекс України від 05.04.2001 № 2341-III. Розділ XVI. Кримінальні правопорушення у сфері використання електронно-обчислювальних машин (комп'ютерів), систем та комп'ютерних мереж і мереж електрозв'язку. URL: https://zakon.rada.gov.ua/laws/show/2341-14.

[21] Курій Є., Опірський І. ISO 27001: аналіз змін та особливості відповідності новій версії стандарту. *Кібербезпека: освіта, наука, техніка.* 2023. Т. 3, № 19. С. 46–55. DOI: 10.28925/2663-4023.2023.19.4655.

[22] Міністерство цифрової трансформації України. Дія.Підпис: технічні та правові засади використання. Київ, 2024. URL: https://diia.gov.ua/.

[23] Міністерство цифрової трансформації України. Інтегрована система електронної ідентифікації id.gov.ua: технічні засади. Київ, 2024. URL: https://id.gov.ua/.

[24] Національний банк України. Система BankID Національного банку України: правила та технічний опис. Київ, 2023. URL: https://bankid.bank.gov.ua/.

[25] НД ТЗІ 1.1-002-99. Загальні положення щодо захисту інформації в комп'ютерних системах від несанкціонованого доступу. Київ: ДСТСЗІ СБ України, 1999. URL: https://cip.gov.ua/ua/news/normativni-dokumenti-z-pitan-tzi.

[26] НД ТЗІ 2.5-004-99. Критерії оцінки захищеності інформації в комп'ютерних системах від несанкціонованого доступу. Київ: ДСТСЗІ СБ України, 1999.

[27] НД ТЗІ 3.7-003-05. Порядок проведення робіт із створення комплексної системи захисту інформації в інформаційно-телекомунікаційній системі. Київ: ДСТСЗІ СБ України, 2005.

[28] Новини 14 січня: російські хакери атакували 70 державних сайтів, українські активи падають. *Економічна правда*. 14.01.2022. URL: https://epravda.com.ua/news/2022/01/14/681463/.

[29] Огляд національної системи кібербезпеки України 2025: аналітичне дослідження. Київ: Національний координаційний центр кібербезпеки при РНБО України, 2026. URL: https://rnbo.gov.ua/files/2026/NATIONAL_CYBER_SCC/kiber_2025.pdf.

[30] Петрів П., Опірський І. Аналіз проблематики використання існуючих стандартів з веб-вразливостей. *Кібербезпека: освіта, наука, техніка*. 2023. Т. 2, № 22. С. 96–112. DOI: 10.28925/2663-4023.2023.22.96112. URL: https://csecurity.kubg.edu.ua/index.php/journal/article/view/512.

[31] Поночовний П. Модель упередження низькошвидкісних HTTP DDoS атак на кінцевого користувача. *Кібербезпека: освіта, наука, техніка*. 2024. Т. 2, № 26. С. 291–304. DOI: 10.28925/2663-4023.2024.26.695.

[32] Правила забезпечення захисту інформації в інформаційних, електронних комунікаційних та інформаційно-комунікаційних системах: затв. постановою Кабінету Міністрів України від 29.03.2006 № 373. URL: https://zakon.rada.gov.ua/laws/show/373-2006-п.

[33] Про внесення змін до деяких законів України щодо захисту інформації та кіберзахисту державних інформаційних ресурсів, об'єктів критичної інформаційної інфраструктури: Закон України від 27.03.2025 № 4336-IX. URL: https://zakon.rada.gov.ua/laws/show/4336-20.

[34] Про електронну ідентифікацію та електронні довірчі послуги: Закон України від 05.10.2017 № 2155-VIII (у редакції Закону № 3450-IX). URL: https://zakon.rada.gov.ua/laws/show/2155-19.

[35] Про електронну комерцію: Закон України від 03.09.2015 № 675-VIII. URL: https://zakon.rada.gov.ua/laws/show/675-19.

[36] Про затвердження Порядку інформаційного наповнення та технічного забезпечення Єдиного веб-порталу органів виконавчої влади та Порядку функціонування веб-сайтів органів виконавчої влади: наказ Держкомінформполітики, Держкомзв'язку від 25.11.2002 № 327/225. Зареєстровано в Мін'юсті 29.12.2002 за № 1021/7309. URL: https://zakon.rada.gov.ua/laws/show/z1021-02.

[37] Про захист інформації в інформаційно-комунікаційних системах: Закон України від 05.07.1994 № 80/94-ВР. URL: https://zakon.rada.gov.ua/laws/show/80/94-вр.

[38] Про захист персональних даних: Закон України від 01.06.2010 № 2297-VI. URL: https://zakon.rada.gov.ua/laws/show/2297-17.

[39] Про критичну інфраструктуру: Закон України від 16.11.2021 № 1882-IX. URL: https://zakon.rada.gov.ua/laws/show/1882-20.

[40] Про основні засади забезпечення кібербезпеки України: Закон України від 05.10.2017 № 2163-VIII (редакція від 19.10.2025). URL: https://zakon.rada.gov.ua/laws/show/2163-19.

[41] Про рішення Ради національної безпеки і оборони України від 14 травня 2021 року «Про Стратегію кібербезпеки України»: Указ Президента України від 26.08.2021 № 447/2021. URL: https://zakon.rada.gov.ua/laws/show/447/2021.

[42] Про хмарні послуги: Закон України від 17.02.2022 № 2075-IX. URL: https://zakon.rada.gov.ua/laws/show/2075-20.

[43] Складанний П., Соколов В. Багатофакторна автентифікація в інформаційних системах органів державної влади. *Кібербезпека: освіта, наука, техніка.* 2023. Т. 1, № 21. С. 46–58.

[44] Складанний П., Соколов В. Безпека хмарних сервісів в органах державної влади України. *Кібербезпека: освіта, наука, техніка.* 2024. Т. 1, № 25. С. 55–70.

[45] Соколов В., Складанний П. Безпека програмних інтерфейсів у системах електронного урядування. *Кібербезпека: освіта, наука, техніка.* 2024. Т. 4, № 24. С. 88–102.

[46] Соколов В., Складанний П. Загрози безпеці вебзастосунків на основі великих мовних моделей. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 2, № 28. С. 30–48.

[47] Соколов В., Складанний П. Методика оцінки комплексних збитків від інциденту інформаційної безпеки. *Кібербезпека: освіта, наука, техніка.* 2023. Т. 1, № 21. С. 99–120. DOI: 10.28925/2663-4023.2023.21.99120.

[48] Соколов В., Складанний П. Ризики ланцюга постачання програмного забезпечення для державних інформаційних систем. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 1, № 27. С. 40–58.

[49] У 2025 році кількість ворожих кібератак зросла на 37 %: кого атакують найчастіше (за даними CERT-UA). *dev.ua.* 08.01.2026. URL: https://dev.ua/news/u-2025-rotsi-kilkist-kiberatak-zrosla-na-37-1767858414.

[50] Чалий О., Толюпа С. Порівняння міжнародних стандартів аудиту інформаційної безпеки з перспективою автоматизації. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 3, № 31. С. 579–588. DOI: 10.28925/2663-4023.2025.31.1046.

[51] Шевченко С., Жданова Ю., Кія О. Напівавтоматизований інструмент багатостандартної оцінки кіберзрілості організації на основі NIST CSF 2.0, ISO/IEC 27001:2022, COBIT 2019 та CIS Controls v8. *Кібербезпека: освіта, наука, техніка.* 2025. Т. 3, № 31. С. 43–60. DOI: 10.28925/2663-4023.2025.31.1004.

[52] Штонда Р., Зозуля Р., Бокій О. Удосконалення парольної політики та використання сучасних менеджерів паролів для підвищення рівня кіберстійкості інформаційно-комунікаційних систем. *Кібербезпека: освіта, наука, техніка.* 2026. Т. 4, № 32. С. 431–447. DOI: 10.28925/2663-4023.2026.32.1114.

[53] 16 лютого — спільний брифінг Мінцифри, СБУ, Держспецзв'язку, Кіберполіції, РНБО та НБУ щодо хакерських атак 15 лютого. *Урядовий портал.* 16.02.2022. URL: https://www.kmu.gov.ua/events/16-lyutogo-spilnij-brifing-mincifri-sbu-derzhspeczvyazku-kiberpoliciyi-rnbo-ta-nbu-shchodo-hakerskih-atak-15-lyutogo.

[54] 2025 CWE Top 25 Most Dangerous Software Weaknesses. The MITRE Corporation, CISA, 2025. URL: https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html.

[55] 2025 Imperva Bad Bot Report: How AI is Supercharging the Bot Threat. *Imperva Blog* (Thales Group). 15.04.2025. URL: https://www.imperva.com/blog/2025-imperva-bad-bot-report-how-ai-is-supercharging-the-bot-threat/.

[56] 485 інформаційних систем в Україні авторизовані у першому півріччі 2026 року. *Урядовий портал*. 27.07.2026. URL: https://www.kmu.gov.ua/news/485-informatsiinykh-system-v-ukraini-avtoryzovani-u-pershomu-pivrichchi-2026-roku.

[57] Adrian D., Bhargavan K., Durumeric Z. et al. Imperfect Forward Secrecy: How Diffie-Hellman Fails in Practice. *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS'15)*. New York: ACM, 2015. P. 5–17. DOI: 10.1145/2810103.2813707.

[58] Akhawe D., Barth A., Lam P. E., Mitchell J., Song D. Towards a Formal Foundation of Web Security. *Proceedings of the 23rd IEEE Computer Security Foundations Symposium (CSF 2010)*. Edinburgh: IEEE, 2010. P. 290–304. DOI: 10.1109/CSF.2010.27.

[59] Amazon Web Services. Shared Responsibility Model. AWS Documentation. URL: https://aws.amazon.com/compliance/shared-responsibility-model/.

[60] Amazon Web Services. Use the Instance Metadata Service to Access Instance Metadata (IMDSv2). AWS Documentation. URL: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/configuring-instance-metadata-service.html.

[61] Anderson R. Security Engineering: A Guide to Building Dependable Distributed Systems. 3rd ed. Indianapolis: Wiley, 2020. 1232 p. URL: https://www.cl.cam.ac.uk/~rja14/book.html.

[62] Anthropic. Model Context Protocol (MCP) Specification. 2024–2025. URL: https://modelcontextprotocol.io/.

[63] Antonakakis M., April T., Bailey M. et al. Understanding the Mirai Botnet. *Proceedings of the 26th USENIX Security Symposium*. Vancouver: USENIX Association, 2017. P. 1093–1110.

[64] Apache Software Foundation. Apache Log4j Security Vulnerabilities (CVE-2021-44228). 2021. URL: https://logging.apache.org/log4j/2.x/security.html.

[65] Applebaum S., Gaber T., Ahmed A. Signature-based and Machine-Learning-based Web Application Firewalls: A Short Survey. *Procedia Computer Science*. 2021. Vol. 189. P. 359–367. DOI: 10.1016/j.procs.2021.05.105.

[66] Arends R., Austein R., Larson M., Massey D., Rose S. DNS Security Introduction and Requirements. RFC 4033. IETF, 2005. DOI: 10.17487/RFC4033.

[67] Atzei N., Bartoletti M., Cimoli T. A Survey of Attacks on Ethereum Smart Contracts (SoK). *Principles of Security and Trust (POST 2017)*. Berlin, Heidelberg: Springer, 2017. P. 164–186. DOI: 10.1007/978-3-662-54455-6_8.

[68] Axelsson S. The Base-Rate Fallacy and the Difficulty of Intrusion Detection. *ACM Transactions on Information and System Security*. 2000. Vol. 3, No. 3. P. 186–205. DOI: 10.1145/357830.357849.

[69] Barnes R., Hoffman-Andrews J., McCarney D., Kasten J. Automatic Certificate Management Environment (ACME). RFC 8555. IETF, 2019. DOI: 10.17487/RFC8555.

[70] Barth A. HTTP State Management Mechanism. RFC 6265. IETF, 2011. DOI: 10.17487/RFC6265. URL: https://www.rfc-editor.org/rfc/rfc6265.

[71] Barth A. The Web Origin Concept. RFC 6454. IETF, 2011. DOI: 10.17487/RFC6454. URL: https://www.rfc-editor.org/rfc/rfc6454.

[72] Barth A., Jackson C., Mitchell J. C. Robust Defenses for Cross-Site Request Forgery. *Proceedings of the 15th ACM Conference on Computer and Communications Security (CCS'08)*. New York: ACM, 2008. P. 75–88. DOI: 10.1145/1455770.1455782.

[73] Bernstein D. J., Lange T. Post-quantum Cryptography. *Nature*. 2017. Vol. 549. P. 188–194. DOI: 10.1038/nature23879.

[74] Bessey A., Block K., Chelf B. et al. A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World. *Communications of the ACM*. 2010. Vol. 53, No. 2. P. 66–75. DOI: 10.1145/1646353.1646374.

[75] Bhatt S., Manadhata P. K., Zomlot L. The Operational Role of Security Information and Event Management Systems. *IEEE Security & Privacy*. 2014. Vol. 12, No. 5. P. 35–41. DOI: 10.1109/MSP.2014.103.

[76] Bingler S., West M., Wilander J. (eds.) Cookies: HTTP State Management Mechanism (draft-ietf-httpbis-rfc6265bis). IETF HTTP Working Group. URL: https://datatracker.ietf.org/doc/draft-ietf-httpbis-rfc6265bis/.

[77] Birsan A. Dependency Confusion: How I Hacked Into Apple, Microsoft and Dozens of Other Companies. Medium, 2021. URL: https://medium.com/@alex.birsan/dependency-confusion-4a5d60fec610.

[78] Biryukov A., Dinu D., Khovratovich D. Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications. *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. Saarbrücken: IEEE, 2016. P. 292–302. DOI: 10.1109/EuroSP.2016.31.

[79] Biryukov A., Dinu D., Khovratovich D., Josefsson S. Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications. RFC 9106. IRTF, 2021. DOI: 10.17487/RFC9106.

[80] Bishop M. (Ed.). HTTP/3. RFC 9114. IETF, 2022. DOI: 10.17487/RFC9114. URL: https://www.rfc-editor.org/rfc/rfc9114.

[81] Bonneau J. The Science of Guessing: Analyzing an Anonymized Corpus of 70 Million Passwords. *2012 IEEE Symposium on Security and Privacy*. San Francisco: IEEE, 2012. P. 538–552. DOI: 10.1109/SP.2012.49.

[82] CA/Browser Forum. Baseline Requirements for the Issuance and Management of Publicly-Trusted TLS Server Certificates. Ballot SC-081: Introduce Schedule of Reducing Validity and Data Reuse Periods. 2025. URL: https://cabforum.org/working-groups/server/baseline-requirements/.

[83] Casey E. Digital Evidence and Computer Crime: Forensic Science, Computers, and the Internet. 3rd ed. Waltham: Academic Press, 2011. 840 p.

[84] Cavoukian A. Privacy by Design: The 7 Foundational Principles. Information and Privacy Commissioner of Ontario, 2011.

[85] Center for Internet Security. CIS Benchmarks for Web Servers, Linux, Kubernetes and Cloud Providers. URL: https://www.cisecurity.org/cis-benchmarks.

[86] Chandola V., Banerjee A., Kumar V. Anomaly Detection: A Survey. *ACM Computing Surveys*. 2009. Vol. 41, No. 3. Article 15. DOI: 10.1145/1541880.1541882.

[87] Chandramouli R. Security Strategies for Microservices-based Application Systems. NIST SP 800-204. Gaithersburg, MD: NIST, 2019. DOI: 10.6028/NIST.SP.800-204.

[88] Chen J., Jiang J., Duan H., Wan T., Chen S., Paxson V., Yang M. We Still Don't Have Secure Cross-Domain Requests: An Empirical Study of CORS. *Proceedings of the 27th USENIX Security Symposium*. Baltimore: USENIX Association, 2018. P. 1079–1093.

[89] Chiara P. G. The Cyber Resilience Act: the EU Commission's proposal for a horizontal regulation on cybersecurity for products with digital elements. *International Cybersecurity Law Review*. 2022. Vol. 3, no. 2. P. 255–272. DOI: 10.1365/s43439-022-00067-6.

[90] CISA, FBI. Update: Destructive Malware Targeting Organizations in Ukraine. Cybersecurity Advisory AA22-057A. 26.02.2022 (rev. 28.04.2022). URL: https://www.cisa.gov/news-events/cybersecurity-advisories/aa22-057a.

[91] CISA, NSA, FBI et al. The Case for Memory Safe Roadmaps. Joint Guidance, 2023. URL: https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps.

[92] CISA, NSA. Detect and Prevent Web Shell Malware. Joint Cybersecurity Advisory, 2020. URL: https://www.cisa.gov/.

[93] CISA. Emergency Directive 21-01: Mitigate SolarWinds Orion Code Compromise. 2020. URL: https://www.cisa.gov/news-events/directives/ed-21-01.

[94] CISA. Known Exploited Vulnerabilities Catalog. Cybersecurity and Infrastructure Security Agency, 2021–2026. URL: https://www.cisa.gov/known-exploited-vulnerabilities-catalog.

[95] CISA. Software Bill of Materials (SBOM). URL: https://www.cisa.gov/sbom.

[96] CISA. Stakeholder-Specific Vulnerability Categorization Guide. Cybersecurity and Infrastructure Security Agency, 2022. URL: https://www.cisa.gov/stakeholder-specific-vulnerability-categorization-ssvc.

[97] Cisco / Signal Sciences. Runtime Application Self-Protection (RASP): Concepts and Limitations. (технічний огляд).

[98] Cloud Security Alliance. The 12 Most Critical Risks for Serverless Applications. CSA, 2019. URL: https://cloudsecurityalliance.org/.

[99] Cloudflare. Blocking a Record 31.4 Tbps DDoS Attack from the Aisuru Botnet. Cloudflare Blog, 2026. URL: https://blog.cloudflare.com/.

[100] Cloudflare. DDoS Threat Report for 2025 Q3. Cloudflare Blog, 2025. URL: https://blog.cloudflare.com/ddos-threat-report-2025-q3/.

[101] Commission Implementing Regulation (EU) 2024/2690 of 17 October 2024 laying down rules for the application of Directive (EU) 2022/2555 as regards technical

and methodological requirements of cybersecurity risk-management measures and further specification of the cases in which an incident is considered to be significant. *Official Journal of the EU*. L series, 18.10.2024. URL: https://eur-lex.europa.eu/eli/reg_impl/2024/2690/oj.

[102] Commission Implementing Regulation (EU) 2024/482 of 31 January 2024 laying down rules for the application of Regulation (EU) 2019/881 as regards the adoption of the European Common Criteria-based cybersecurity certification scheme (EUCC). *Official Journal of the EU*. L series, 07.02.2024. URL: https://eur-lex.europa.eu/eli/reg_impl/2024/482/oj.

[103] Commission Implementing Regulation (EU) 2025/2392 on the technical description of the categories of important and critical products with digital elements pursuant to Regulation (EU) 2024/2847. *Official Journal of the EU*. L series, 01.12.2025. URL: https://eur-lex.europa.eu/eli/reg_impl/2025/2392/oj/eng.

[104] Common Attack Pattern Enumeration and Classification (CAPEC). The MITRE Corporation. URL: https://capec.mitre.org/.

[105] Common Weakness Enumeration (CWE). The MITRE Corporation. URL: https://cwe.mitre.org/.

[106] Content Security Policy Level 3. World Wide Web Consortium (W3C). URL: https://www.w3.org/TR/CSP3/.

[107] Cooper D. et al. Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile. RFC 5280. IETF, 2008. DOI: 10.17487/RFC5280. URL: https://www.rfc-editor.org/rfc/rfc5280.

[108] Croley S. (Chick3nman). Hashcat v6.2.6 Benchmark on the Nvidia RTX 4090. GitHub Gist, 2022. URL: https://gist.github.com/Chick3nman/32e662a5bb63bc4f51b847bb422222fd.

[109] Crosby S. A., Wallach D. S. Denial of Service via Algorithmic Complexity Attacks. *Proceedings of the 12th USENIX Security Symposium*. Washington, D.C.: USENIX, 2003. P. 29–44. URL: https://www.usenix.org/conference/12th-usenix-security-symposium/denial-service-algorithmic-complexity-attacks.

[110] Das A., Bonneau J., Caesar M., Borisov N., Wang X. The Tangled Web of Password Reuse. *Network and Distributed System Security Symposium (NDSS 2014)*. San Diego: Internet Society, 2014. DOI: 10.14722/ndss.2014.23357.

[111] DDoS-атака на мільйони доларів. Банки та урядові сайти зазнали наймасштабнішої атаки в історії України. *Forbes Україна*. 16.02.2022. URL: https://forbes.ua/news/ddos-ataka-na-milyoni-dolariv-banki-ta-uryadovi-sayti-zaznali-naymasshtabnishoi-ataki-v-istorii-ukraini-16022022-3706.

[112] Deng M., Wuyts K., Scandariato R., Preneel B., Joosen W. A Privacy Threat Analysis Framework: Supporting the Elicitation and Fulfillment of Privacy Requirements. *Requirements Engineering*. 2011. Vol. 16, No. 1. P. 3–32. DOI: 10.1007/s00766-010-0115-7.

[113] Directive (EU) 2019/882 of the European Parliament and of the Council of 17 April 2019 on the accessibility requirements for products and services (European

Accessibility Act). *Official Journal of the EU*. L 151. P. 70–115. URL: https://eur-lex.europa.eu/eli/dir/2019/882/oj.

[114] Directive (EU) 2022/2555 of the European Parliament and of the Council of 14 December 2022 on measures for a high common level of cybersecurity across the Union (NIS 2 Directive). *Official Journal of the EU*. L 333. P. 80–152. URL: https://eur-lex.europa.eu/eli/dir/2022/2555/oj.

[115] Doupé A., Cova M., Vigna G. Why Johnny Can't Pentest: An Analysis of Black-box Web Vulnerability Scanners. *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2010)*. Berlin, Heidelberg: Springer, 2010. P. 111–131. DOI: 10.1007/978-3-642-14215-4_7.

[116] Durumeric Z., Bailey M., Halderman J. A. An Internet-Wide View of Internet-Wide Scanning. *Proceedings of the 23rd USENIX Security Symposium*. San Diego: USENIX Association, 2014. P. 65–78. URL: https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/durumeric.

[117] Durumeric Z., Li F., Kasten J., Amann J., Beekman J., Payer M., Weaver N., Adrian D., Paxson V., Bailey M., Halderman J. A. The Matter of Heartbleed. *Proceedings of the 2014 Internet Measurement Conference (IMC'14)*. New York: ACM, 2014. P. 475–488. DOI: 10.1145/2663716.2663755.

[118] Durumeric Z., Wustrow E., Halderman J. A. ZMap: Fast Internet-wide Scanning and Its Security Applications. *Proceedings of the 22nd USENIX Security Symposium*. Washington, D.C.: USENIX Association, 2013. P. 605–620. URL: https://www.usenix.org/conference/usenixsecurity13/technical-sessions/paper/durumeric.

[119] ENISA Threat Landscape 2025. Athens: European Union Agency for Cybersecurity, 2025. URL: https://www.enisa.europa.eu/publications/enisa-threat-landscape-2025.

[120] Fahl S. Web & Mobile Security Knowledge Area. Version 1.0.1. *The Cyber Security Body of Knowledge (CyBOK)*. © Crown Copyright, The National Cyber Security Centre, 2021. URL: https://www.cybok.org/.

[121] Fetch Living Standard. WHATWG. URL: https://fetch.spec.whatwg.org/.

[122] Fett D., Campbell B., Bradley J., Lodderstedt T., Jones M., Waite D. OAuth 2.0 Demonstrating Proof of Possession (DPoP). RFC 9449. IETF, 2023. DOI: 10.17487/RFC9449.

[123] Fett D., Küsters R., Schmitz G. A Comprehensive Formal Security Analysis of OAuth 2.0. *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS'16)*. New York: ACM, 2016. P. 1204–1215. DOI: 10.1145/2976749.2978385.

[124] Fette I., Melnikov A. The WebSocket Protocol. RFC 6455. IETF, 2011. DOI: 10.17487/RFC6455.

[125] FIDO Alliance. FIDO2: WebAuthn & CTAP — Specifications Overview. URL: https://fidoalliance.org/fido2/.

[126] FIDO Alliance. Passkeys: Multi-Device FIDO Credentials. URL: https://fidoalliance.org/passkeys/.

[127] Fielding R. T. Architectural Styles and the Design of Network-based Software Architectures: doctoral dissertation. Irvine: University of California, 2000. 162 p. URL: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm.

[128] Fielding R. T. Architectural Styles and the Design of Network-based Software Architectures: PhD dissertation. University of California, Irvine, 2000.

[129] Fielding R., Nottingham M., Reschke J. (Ed.). HTTP Semantics. RFC 9110. IETF, 2022. DOI: 10.17487/RFC9110. URL: https://www.rfc-editor.org/rfc/rfc9110.

[130] Finifter M., Akhawe D., Wagner D. An Empirical Study of Vulnerability Rewards Programs. *Proceedings of the 22nd USENIX Security Symposium*. Washington: USENIX Association, 2013. P. 273–288.

[131] FIRST. Common Vulnerability Scoring System Version 4.0: Specification Document. Forum of Incident Response and Security Teams, 2023. URL: https://www.first.org/cvss/v4-0/specification-document.

[132] Florêncio D., Herley C., van Oorschot P. C. An Administrator's Guide to Internet Password Research. *Proceedings of the 28th USENIX Conference on Large Installation System Administration (LISA'14)*. Seattle: USENIX Association, 2014. P. 44–61.

[133] Freund A. Backdoor in Upstream xz/liblzma Leading to SSH Server Compromise (CVE-2024-3094). oss-security Mailing List, 2024. URL: https://www.openwall.com/lists/oss-security/2024/03/29/4.

[134] Freund J., Jones J. Measuring and Managing Information Risk: A FAIR Approach. Oxford: Butterworth-Heinemann, 2014. 408 p.

[135] Frohoff C., Lawrence G. Marshalling Pickles: How Deserializing Objects Can Ruin Your Day. *AppSecCali*. Santa Monica: OWASP, 2015.

[136] Gartner. Market Guide for Cloud-Native Application Protection Platforms (CNAPP). 2023.

[137] Gil O. Web Cache Deception Attack. *Black Hat USA*. Las Vegas, 2017.

[138] Google Cloud. How It Works: The Novel HTTP/2 Rapid Reset DDoS Attack (CVE-2023-44487). Google Cloud Blog, 2023. URL: https://cloud.google.com/blog/products/identity-security/how-it-works-the-novel-http2-rapid-reset-ddos-attack.

[139] Gordon L. A., Loeb M. P. The Economics of Information Security Investment. *ACM Transactions on Information and System Security*. 2002. Vol. 5, No. 4. P. 438–457. DOI: 10.1145/581271.581274.

[140] GraphQL Foundation. GraphQL Specification. Linux Foundation, 2021. URL: https://spec.graphql.org/.

[141] Grassi P. A., Garcia M. E., Fenton J. L. et al. Digital Identity Guidelines: Authentication and Authenticator Management. NIST SP 800-63B-4. Gaithersburg, MD: NIST, 2025. DOI: 10.6028/NIST.SP.800-63b-4.

[142] Greshake K., Abdelnabi S., Mishra S., Endres C., Holz T., Fritz M. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. *Proceedings of the 16th ACM Workshop on*

Artificial Intelligence and Security (AISec'23). New York: ACM, 2023. P. 79–90. DOI: 10.1145/3605764.3623985.

[143] gRPC Authors. gRPC: A High Performance, Open Source Universal RPC Framework. Cloud Native Computing Foundation. URL: https://grpc.io/.

[144] Haas A., Rossberg A., Schuff D. L. et al. Bringing the Web up to Speed with WebAssembly. *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI'17)*. New York: ACM, 2017. P. 185–200. DOI: 10.1145/3062341.3062363.

[145] Hallam-Baker P., Stradling R., Hoffman-Andrews J. DNS Certification Authority Authorization (CAA) Resource Record. RFC 8659. IETF, 2019. DOI: 10.17487/RFC8659.

[146] Harbison M., Falcone R., Grunzweig J. Threat Brief: Ongoing Russia and Ukraine Cyber Activity. *Palo Alto Networks Unit 42*. 20.01.2022. URL: https://unit42.paloaltonetworks.com/ukraine-cyber-conflict-cve-2021-32648-whispergate/.

[147] Hardt D. (Ed.). The OAuth 2.0 Authorization Framework. RFC 6749. IETF, 2012. DOI: 10.17487/RFC6749. URL: https://www.rfc-editor.org/rfc/rfc6749.

[148] Hardt D., Parecki A., Lodderstedt T. The OAuth 2.1 Authorization Framework (draft-ietf-oauth-v2-1). IETF OAuth Working Group. URL: https://datatracker.ietf.org/doc/draft-ietf-oauth-v2-1/.

[149] Hartig O., Pérez J. Semantics and Complexity of GraphQL. *Proceedings of the 2018 World Wide Web Conference (WWW'18)*. Lyon: International World Wide Web Conferences Steering Committee, 2018. P. 1155–1164. DOI: 10.1145/3178876.3186014.

[150] Hodges J., Jackson C., Barth A. HTTP Strict Transport Security (HSTS). RFC 6797. IETF, 2012. DOI: 10.17487/RFC6797.

[151] Hoffman A. Web Application Security: Exploitation and Countermeasures for Modern Web Applications. 2nd ed. Sebastopol, CA: O'Reilly Media, 2024. 440 p.

[152] HTML Living Standard. WHATWG. URL: https://html.spec.whatwg.org/.

[153] Hu V. C., Ferraiolo D., Kuhn R. et al. Guide to Attribute Based Access Control (ABAC) Definition and Considerations. NIST SP 800-162. Gaithersburg, MD: NIST, 2014. DOI: 10.6028/NIST.SP.800-162.

[154] Huang L.-S., Moshchuk A., Wang H. J., Schechter S., Jackson C. Clickjacking: Attacks and Defenses. *Proceedings of the 21st USENIX Security Symposium*. Bellevue: USENIX Association, 2012. P. 413–428.

[155] Hubbard D. W., Seiersen R. How to Measure Anything in Cybersecurity Risk. 2nd ed. Hoboken: Wiley, 2023. 368 p. ISBN 978-1-119-89230-4.

[156] Hunt T. Have I Been Pwned: Pwned Passwords. URL: https://haveibeenpwned.com/Passwords.

[157] IBM Security, Ponemon Institute. Cost of a Data Breach Report 2025. Armonk: IBM, 2025. URL: https://www.ibm.com/reports/data-breach.

[158] Important Updates Announced for Merchants Validating to Self-Assessment Questionnaire A. *PCI Perspectives Blog*. PCI Security Standards Council, 2025. URL: https://blog.pcisecuritystandards.org/important-updates-announced-for-merchants-validating-to-self-assessment-questionnaire-a.

[159] Information Commissioner's Office. Penalty Notice: British Airways plc. Case ref. COM0783542. Wilmslow: ICO, 2020.

[160] ISO 31000:2018. Risk management — Guidelines. Geneva: ISO, 2018. URL: https://www.iso.org/standard/65694.html.

[161] ISO/IEC 15408-1:2022. Information security, cybersecurity and privacy protection — Evaluation criteria for IT security — Part 1: Introduction and general model. Geneva: ISO, 2022. URL: https://www.iso.org/standard/72891.html.

[162] ISO/IEC 27005:2022. Information security, cybersecurity and privacy protection — Guidance on managing information security risks. Geneva: ISO, 2022. URL: https://www.iso.org/standard/80585.html.

[163] ISO/IEC 27034-1:2011. Information technology — Security techniques — Application security — Part 1: Overview and concepts. Geneva: ISO, 2011. URL: https://www.iso.org/standard/44378.html.

[164] ISO/IEC 27035-1:2023. Information technology — Information security incident management — Part 1: Principles and process. Geneva: ISO, 2023.

[165] ISO/IEC 29147:2018. Information technology — Security techniques — Vulnerability disclosure. Geneva: ISO, 2018. URL: https://www.iso.org/standard/72311.html.

[166] ISO/IEC 30111:2019. Information technology — Security techniques — Vulnerability handling processes. Geneva: ISO, 2019. URL: https://www.iso.org/standard/69725.html.

[167] ISO/IEC 5962:2021. Information technology — SPDX Specification V2.2.1. Geneva: ISO, 2021. URL: https://www.iso.org/standard/81870.html.

[168] Istio Authors. Istio Security: Mutual TLS, Authorization Policies. URL: https://istio.io/latest/docs/concepts/security/.

[169] Iyengar J., Thomson M. (eds.) QUIC: A UDP-Based Multiplexed and Secure Transport. RFC 9000. IETF, 2021. DOI: 10.17487/RFC9000.

[170] Jacobs J., Romanosky S., Adjerid I., Baker W. Improving Vulnerability Remediation through Better Exploit Prediction. *Journal of Cybersecurity*. 2020. Vol. 6, No. 1. Article tyaa015. DOI: 10.1093/cybsec/tyaa015.

[171] Jacobs J., Romanosky S., Edwards B., Adjerid I., Roytman M. Exploit Prediction Scoring System (EPSS). *Digital Threats: Research and Practice*. 2021. Vol. 2, No. 3. Article 20. DOI: 10.1145/3436242.

[172] Jacobs J., Romanosky S., Suciu O., Edwards B., Sarabi A. Enhancing Vulnerability Prioritization: Data-Driven Exploit Predictions with Community-Driven Insights. *2023 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. Delft: IEEE, 2023. P. 194–206. DOI: 10.1109/EuroSPW59978.2023.00027.

[173] Jaquith A. Security Metrics: Replacing Fear, Uncertainty, and Doubt. Boston: Addison-Wesley, 2007. 336 p.

[174] Jones M., Bradley J., Sakimura N. JSON Web Token (JWT). RFC 7519. IETF, 2015. DOI: 10.17487/RFC7519. URL: https://www.rfc-editor.org/rfc/rfc7519.

[175] Kettle J. Server-Side Template Injection: RCE for the Modern Web App. *Black Hat USA*. Las Vegas, 2015.

[176] Klein G., Elphinstone K., Heiser G. et al. seL4: Formal Verification of an OS Kernel. *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP'09)*. New York: ACM, 2009. P. 207–220. DOI: 10.1145/1629575.1629596.

[177] Kleinrock L. Queueing Systems. Vol. 1: Theory. New York: Wiley, 1975. 417 p.

[178] Knittel L., Mainka C., Niemietz M., Noß D. T., Schwenk J. XSinator.com: From a Formal Model to the Automatic Evaluation of Cross-Site Leaks in Web Browsers. *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS'21)*. New York: ACM, 2021. P. 1771–1788. DOI: 10.1145/3460120.3484739.

[179] Kocher P., Horn J., Fogh A., Genkin D., Gruss D., Haas W., Hamburg M., Lipp M., Mangard S., Prescher T., Schwarz M., Yarom Y. Spectre Attacks: Exploiting Speculative Execution. *2019 IEEE Symposium on Security and Privacy (SP)*. San Francisco: IEEE, 2019. P. 1–19. DOI: 10.1109/SP.2019.00002.

[180] Kordy B., Piètre-Cambacédès L., Schweitzer P. DAG-based Attack and Defense Modeling: Don't Miss the Forest for the Attack Trees. *Computer Science Review*. 2014. Vol. 13–14. P. 1–38. DOI: 10.1016/j.cosrev.2014.07.001.

[181] Krebs B. What We Can Learn from the Capital One Hack. *KrebsOnSecurity*. 2019. URL: https://krebsonsecurity.com/2019/08/what-we-can-learn-from-the-capital-one-hack/.

[182] Kwiatkowski K., Kampanakis P., Westerbaan B., Stebila D. Post-quantum Hybrid ECDHE-MLKEM Key Agreement for TLS 1.3. RFC 10024. IETF, 2026. URL: https://www.rfc-editor.org/rfc/rfc10024.

[183] Kührer M., Hupperich T., Rossow C., Holz T. Exit from Hell? Reducing the Impact of Amplification DDoS Attacks. *Proceedings of the 23rd USENIX Security Symposium*. San Diego: USENIX Association, 2014. P. 111–125.

[184] Lassak L., Pan E., Ur B., Golla M. Why Aren't We Using Passkeys? Obstacles Companies Face Deploying FIDO2 Passwordless Authentication. *Proceedings of the 33rd USENIX Security Symposium*. Philadelphia: USENIX Association, 2024.

[185] Lauinger T., Chaabane A., Arshad S., Robertson W., Wilson C., Kirda E. Thou Shalt Not Depend on Me: Analysing the Use of Outdated JavaScript Libraries on the Web. *Proceedings of the Network and Distributed System Security Symposium (NDSS 2017)*. San Diego: Internet Society, 2017. DOI: 10.14722/ndss.2017.23414.

[186] Laurie B., Langley A., Kasper E. Certificate Transparency. RFC 6962. IETF, 2013. DOI: 10.17487/RFC6962.

[187] Le Goues C., Pradel M., Roychoudhury A. Automated Program Repair. *Communications of the ACM*. 2019. Vol. 62, No. 12. P. 56–65. DOI: 10.1145/3318162.

[188] Lekies S., Stock B., Johns M. 25 Million Flows Later: Large-scale Detection of DOM-based XSS. *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security (CCS'13)*. New York: ACM, 2013. P. 1193–1204. DOI: 10.1145/2508859.2516703.

[189] Let's Encrypt. Ending OCSP Support in 2025. Internet Security Research Group, 2024. URL: https://letsencrypt.org/2024/12/05/ending-ocsp/.

[190] Lewis P., Perez E., Piktus A. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. 2020. P. 9459–9474.

[191] Liu D., Hao S., Wang H. All Your DNS Records Point to Us: Understanding the Security Threats of Dangling DNS Records. *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS'16)*. New York: ACM, 2016. P. 1414–1425. DOI: 10.1145/2976749.2978387.

[192] Lodderstedt T., Bradley J., Labunets A., Fett D. Best Current Practice for OAuth 2.0 Security. RFC 9700. IETF, 2025. DOI: 10.17487/RFC9700. URL: https://www.rfc-editor.org/rfc/rfc9700.

[193] Manadhata P. K., Wing J. M. An Attack Surface Metric. *IEEE Transactions on Software Engineering*. 2011. Vol. 37, No. 3. P. 371–386. DOI: 10.1109/TSE.2010.60.

[194] Mandiant. M-Trends 2024: Special Report. Google Cloud / Mandiant, 2024. URL: https://www.mandiant.com/m-trends.

[195] Manès V. J. M., Han H., Han C. et al. The Art, Science, and Engineering of Fuzzing: A Survey. *IEEE Transactions on Software Engineering*. 2021. Vol. 47, No. 11. P. 2312–2331. DOI: 10.1109/TSE.2019.2946563.

[196] Marlinspike M. New Tricks for Defeating SSL in Practice. *Black Hat DC*. Arlington, 2009.

[197] Massey J. L. Guessing and Entropy. *Proceedings of the 1994 IEEE International Symposium on Information Theory*. Trondheim: IEEE, 1994. P. 204. DOI: 10.1109/ISIT.1994.394764.

[198] Matte C., Bielova N., Santos C. Do Cookie Banners Respect my Choice? Measuring Legal Compliance of Banners from IAB Europe's Transparency and Consent Framework. *2020 IEEE Symposium on Security and Privacy (SP)*. San Francisco, CA: IEEE, 2020. P. 791–809. DOI: 10.1109/SP40000.2020.00076.

[199] Mauw S., Oostdijk M. Foundations of Attack Trees. *Information Security and Cryptology — ICISC 2005*. Lecture Notes in Computer Science, vol. 3935. Berlin, Heidelberg: Springer, 2006. P. 186–198. DOI: 10.1007/11734727_17.

[200] McDermott J., Fox C. Using Abuse Case Models for Security Requirements Analysis. *Proceedings 15th Annual Computer Security Applications Conference (ACSAC'99)*. Phoenix: IEEE, 1999. P. 55–64. DOI: 10.1109/CSAC.1999.816013.

[201] McGraw G. Software Security: Building Security In. Boston: Addison-Wesley, 2006. 448 p.

[202] McKay K. A., Cooper D. A. Guidelines for the Selection, Configuration, and Use of Transport Layer Security (TLS) Implementations. NIST SP 800-52, Rev. 2. Gaithersburg, MD, 2019. DOI: 10.6028/NIST.SP.800-52r2. URL: https://csrc.nist.gov/pubs/sp/800/52/r2/final.

[203] Meli M., McNiece M. R., Reaves B. How Bad Can It Git? Characterizing Secret Leakage in Public GitHub Repositories. *Proceedings of the 2019 Network and Distributed System Security Symposium (NDSS)*. San Diego: Internet Society, 2019. DOI: 10.14722/ndss.2019.23418.

[204] Microsoft Threat Intelligence. From Cookie Theft to BEC: Attackers Use AiTM Phishing Sites as Entry Point to Further Financial Fraud. Microsoft Security Blog, 2022. URL: https://www.microsoft.com/en-us/security/blog/2022/07/12/.

[205] Microsoft. Security Development Lifecycle (SDL) Practices. Microsoft, 2023. URL: https://www.microsoft.com/en-us/securityengineering/sdl/practices.

[206] Migues S., Steven J., Ware M. Building Security In Maturity Model (BSIMM15). Black Duck / Synopsys, 2024.

[207] MITRE ATT&CK®: Adversarial Tactics, Techniques, and Common Knowledge (Enterprise matrix). The MITRE Corporation. URL: https://attack.mitre.org/.

[208] Moriarty K., Farrell S. Deprecating TLS 1.0 and TLS 1.1. RFC 8996. IETF, 2021. DOI: 10.17487/RFC8996.

[209] Moriarty K., Kaliski B., Rusch A. PKCS #5: Password-Based Cryptography Specification Version 2.1. RFC 8018. IETF, 2017. DOI: 10.17487/RFC8018.

[210] Moura G. C. M., Schmidt R. de O., Heidemann J. et al. Anycast vs. DDoS: Evaluating the November 2015 Root DNS Event. *Proceedings of the 2016 Internet Measurement Conference (IMC'16)*. New York: ACM, 2016. P. 255–270. DOI: 10.1145/2987443.2987446.

[211] Mozilla Foundation. Public Suffix List. URL: https://publicsuffix.org/.

[212] Myrbakken H., Colomo-Palacios R. DevSecOps: A Multivocal Literature Review. *Software Process Improvement and Capability Determination (SPICE 2017)*. Cham: Springer, 2017. P. 17–29. DOI: 10.1007/978-3-319-67383-7_2.

[213] National Security Agency, CISA. Kubernetes Hardening Guide. Version 1.2. 2022. URL: https://media.defense.gov/2022/Aug/29/2003066362/-1/-1/0/CTR_KUBERNETES_HARDENING_GUIDANCE_1.2_20220829.PDF.

[214] Nelson A., Rekhi S., Souppaya M., Scarfone K. Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile. NIST SP 800-61, Rev. 3. Gaithersburg, MD, 2025. DOI: 10.6028/NIST.SP.800-61r3. URL: https://csrc.nist.gov/pubs/sp/800/61/r3/final.

[215] Newman Z., Meyers J. S., Torres-Arias S. Sigstore: Software Signing for Everybody. *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS'22)*. New York: ACM, 2022. P. 2353–2367. DOI: 10.1145/3548606.3560596.

[216] Nikiforakis N., Invernizzi L., Kapravelos A., Van Acker S., Joosen W., Kruegel C., Piessens F., Vigna G. You Are What You Include: Large-scale Evaluation of Remote JavaScript Inclusions. *Proceedings of the 2012 ACM Conference on Computer and Communications Security (CCS '12)*. New York: ACM, 2012. P. 736–747. DOI: 10.1145/2382196.2382274.

[217] NIST. Guide for Conducting Risk Assessments. NIST SP 800-30, Rev. 1. Gaithersburg, MD, 2012. DOI: 10.6028/NIST.SP.800-30r1. URL: https://csrc.nist.gov/pubs/sp/800/30/r1/final.

[218] NIST. Module-Lattice-Based Digital Signature Standard. FIPS 204. Gaithersburg, MD, 2024. DOI: 10.6028/NIST.FIPS.204. URL: https://csrc.nist.gov/pubs/fips/204/final.

[219] NIST. Module-Lattice-Based Key-Encapsulation Mechanism Standard. FIPS 203. Gaithersburg, MD, 2024. DOI: 10.6028/NIST.FIPS.203. URL: https://csrc.nist.gov/pubs/fips/203/final.

[220] NIST. Security and Privacy Controls for Information Systems and Organizations. NIST SP 800-53, Rev. 5. Gaithersburg, MD, 2020. DOI: 10.6028/NIST.SP.800-53r5. URL: https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final.

[221] NIST. The NIST Cybersecurity Framework (CSF) 2.0. NIST CSWP 29. Gaithersburg, MD, 2024. DOI: 10.6028/NIST.CSWP.29. URL: https://csrc.nist.gov/pubs/cswp/29/the-nist-cybersecurity-framework-20/final.

[222] Nygard M. T. Release It! Design and Deploy Production-Ready Software. 2nd ed. Raleigh: Pragmatic Bookshelf, 2018. 376 p.

[223] Open Policy Agent. Policy-based Control for Cloud Native Environments. Cloud Native Computing Foundation. URL: https://www.openpolicyagent.org/.

[224] Open Source Security Foundation. Supply-chain Levels for Software Artifacts (SLSA) v1.0. Linux Foundation, 2023. URL: https://slsa.dev/.

[225] OpenAPI Initiative. OpenAPI Specification v3.1.0. Linux Foundation, 2021. URL: https://spec.openapis.org/oas/v3.1.0.

[226] OWASP Application Security Verification Standard (ASVS). Version 5.0.0. OWASP Foundation, 2025. URL: https://owasp.org/www-project-application-security-verification-standard/.

[227] OWASP Cheat Sheet Series. OWASP Foundation. URL: https://cheatsheetseries.owasp.org/.

[228] OWASP pytm: A Pythonic Framework for Threat Modeling. OWASP Foundation. URL: https://github.com/OWASP/pytm.

[229] OWASP Software Assurance Maturity Model (SAMM). Version 2.0. OWASP Foundation, 2020. URL: https://owaspsamm.org/model/.

[230] OWASP Threat Dragon. OWASP Foundation. URL: https://owasp.org/www-project-threat-dragon/.

[231] OWASP Top 10:2025. The Ten Most Critical Web Application Security Risks. OWASP Foundation, 2025. URL: https://owasp.org/Top10/2025/.

[232] OWASP Web Security Testing Guide (WSTG). Version 4.2. OWASP Foundation, 2020. URL: https://owasp.org/www-project-web-security-testing-guide/.

[233] OWASP. CycloneDX Bill of Materials Specification. OWASP Foundation. URL: https://cyclonedx.org/.

[234] OWASP. Logging Cheat Sheet. OWASP Cheat Sheet Series. URL: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html.

[235] OWASP. OWASP API Security Top 10 — 2023. OWASP Foundation, 2023. URL: https://owasp.org/API-Security/editions/2023/en/0x00-header/.

[236] OWASP. OWASP Automated Threats to Web Applications (OAT). OWASP Foundation, 2018. URL: https://owasp.org/www-project-automated-threats-to-web-applications/.

[237] OWASP. OWASP Top 10 for Agentic Applications. OWASP GenAI Security Project, 2025. URL: https://genai.owasp.org/.

[238] OWASP. OWASP Top 10 for Large Language Model Applications 2025. OWASP GenAI Security Project, 2025. URL: https://genai.owasp.org/llm-top-10/.

[239] OWASP. Session Management Cheat Sheet. OWASP Cheat Sheet Series. URL: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html.

[240] OWASP. Software Assurance Maturity Model (SAMM) v2. OWASP Foundation, 2020. URL: https://owaspsamm.org/.

[241] Pan J., Mao X. Interactive Application Security Testing: A Systematic Comparison. *IEEE Access*. 2021. (оглядова стаття).

[242] Pang R., Caceres R., Burrows M. et al. Zanzibar: Google's Consistent, Global Authorization System. *Proceedings of the 2019 USENIX Annual Technical Conference*. Renton: USENIX Association, 2019. P. 33–46.

[243] PCI Security Standards Council. Payment Card Industry Data Security Standard: Requirements and Testing Procedures. Version 4.0.1. Wakefield, MA: PCI SSC, 2024. URL: https://www.pcisecuritystandards.org/document_library/.

[244] Poolsappasit N., Dewri R., Ray I. Dynamic Security Risk Management Using Bayesian Attack Graphs. *IEEE Transactions on Dependable and Secure Computing*. 2012. Vol. 9, No. 1. P. 61–74. DOI: 10.1109/TDSC.2011.34.

[245] Private Access Tokens / Privacy Pass. IETF Privacy Pass Working Group. RFC 9576 (Architecture). IETF, 2024. URL: https://datatracker.ietf.org/wg/privacypass/documents/.

[246] PTES. The Penetration Testing Execution Standard. 2014. URL: http://www.pentest-standard.org/.

[247] Rahman A., Parnin C., Williams L. The Seven Sins: Security Smells in Infrastructure as Code Scripts. *Proceedings of the 41st International Conference on Software Engineering (ICSE'19)*. Montreal: IEEE, 2019. P. 164–175. DOI: 10.1109/ICSE.2019.00033.

[248] Ray D., Ligatti J. Defining Code-injection Attacks. *Proceedings of the 39th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL'12)*. New York: ACM, 2012. P. 179–190. DOI: 10.1145/2103656.2103678.

[249] Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation). *Official Journal of the EU*. L 119. P. 1–88. URL: https://eur-lex.europa.eu/eli/reg/2016/679/oj.

[250] Regulation (EU) 2022/2065 of the European Parliament and of the Council of 19 October 2022 on a Single Market For Digital Services (Digital Services Act). *Official Journal of the EU*. L 277. P. 1–102. URL: https://eur-lex.europa.eu/eli/reg/2022/2065/oj.

[251] Regulation (EU) 2024/1183 of the European Parliament and of the Council of 11 April 2024 amending Regulation (EU) No 910/2014 as regards establishing the European Digital Identity Framework. *Official Journal of the EU*. L series, 30.04.2024. URL: https://eur-lex.europa.eu/eli/reg/2024/1183/oj.

[252] Regulation (EU) 2024/1689 of the European Parliament and of the Council laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). *Official Journal of the European Union*. 2024.

[253] Regulation (EU) 2024/2847 of the European Parliament and of the Council of 23 October 2024 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act). *Official Journal of the EU*. L series, 20.11.2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj.

[254] Reis C., Moshchuk A., Oskov N. Site Isolation: Process Separation for Web Sites within the Browser. *Proceedings of the 28th USENIX Security Symposium*. Santa Clara: USENIX Association, 2019. P. 1661–1678.

[255] Rescorla E. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446. IETF, 2018. DOI: 10.17487/RFC8446. URL: https://www.rfc-editor.org/rfc/rfc8446.

[256] Rescorla E., Barnes R., Tschofenig H. Mutual TLS. In: Transport Layer Security (TLS) — RFC 8446. IETF, 2018.

[257] Rescorla E., Oku K., Sullivan N., Wood C. A. TLS Encrypted Client Hello. RFC 9849. IETF. URL: https://www.rfc-editor.org/rfc/rfc9849.

[258] Rose S., Borchert O., Mitchell S., Connelly S. Zero Trust Architecture. NIST Special Publication 800-207. Gaithersburg, MD, 2020. DOI: 10.6028/NIST.SP.800-207. URL: https://csrc.nist.gov/pubs/sp/800/207/final.

[259] Sakimura N., Bradley J., Agarwal N. Proof Key for Code Exchange by OAuth Public Clients. RFC 7636. IETF, 2015. DOI: 10.17487/RFC7636.

[260] Sakimura N., Bradley J., Jones M., de Medeiros B., Mortimore C. OpenID Connect Core 1.0 incorporating errata set 2. OpenID Foundation, 2023. URL: https://openid.net/specs/openid-connect-core-1_0.html.

[261] Salt Security. State of API Security Report. 2024. URL: https://salt.security/api-security-trends.

[262] Sandhu R. S., Coyne E. J., Feinstein H. L., Youman C. E. Role-Based Access Control Models. *IEEE Computer*. 1996. Vol. 29, No. 2. P. 38–47. DOI: 10.1109/2.485845.

[263] Sansec. Polyfill Supply Chain Attack Hits 100K+ Sites. 2024. URL: https://sansec.io/research/polyfill-supply-chain-attack.

[264] Scarfone K., Souppaya M., Cody A., Orebaugh A. Technical Guide to Information Security Testing and Assessment. NIST SP 800-115. Gaithersburg, MD: NIST, 2008. DOI: 10.6028/NIST.SP.800-115.

[265] Schneier B. Attack Trees. *Dr. Dobb's Journal*. 1999. Vol. 24, No. 12. P. 21–29.

[266] Shcherbakov M., Balliu M., Staicu C.-A. Silent Spring: Prototype Pollution Leads to Remote Code Execution in Node.js. *Proceedings of the 32nd USENIX Security Symposium*. Anaheim: USENIX Association, 2023. P. 5521–5538.

[267] Sheffer Y., Hardt D., Jones M. JSON Web Token Best Current Practices. RFC 8725. IETF, 2020. DOI: 10.17487/RFC8725.

[268] Sheyner O., Haines J., Jha S., Lippmann R., Wing J. M. Automated Generation and Analysis of Attack Graphs. *Proceedings 2002 IEEE Symposium on Security and Privacy*. Berkeley: IEEE, 2002. DOI: 10.1109/SECPRI.2002.1004377.

[269] Shirey R. Internet Security Glossary, Version 2. RFC 4949. IETF, 2007. DOI: 10.17487/RFC4949. URL: https://www.rfc-editor.org/rfc/rfc4949.

[270] Shostack A. Threat Modeling: Designing for Security. Indianapolis: Wiley, 2014. 624 p.

[271] Sindre G., Opdahl A. L. Eliciting Security Requirements with Misuse Cases. *Requirements Engineering*. 2005. Vol. 10, No. 1. P. 34–44. DOI: 10.1007/s00766-004-0194-4.

[272] Soldatov S., Zhora V. The IT Army of Ukraine: Crowdsourced Cyber Operations. *Journal of Cyber Policy*. 2023. (аналітичний огляд).

[273] Sonnenreich W., Albanese J., Stout B. Return On Security Investment (ROSI) — A Practical Quantitative Model. *Journal of Research and Practice in Information Technology*. 2006. Vol. 38, No. 1. P. 45–56.

[274] Souppaya M., Morello J., Scarfone K. Application Container Security Guide. NIST SP 800-190. Gaithersburg, MD: NIST, 2017. DOI: 10.6028/NIST.SP.800-190.

[275] Souppaya M., Scarfone K., Dodson D. Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities. NIST SP 800-218. Gaithersburg, MD, 2022. DOI: 10.6028/NIST.SP.800-218. URL: https://csrc.nist.gov/pubs/sp/800/218/final.

[276] SPIFFE Authors. SPIFFE: Secure Production Identity Framework for Everyone. Cloud Native Computing Foundation. URL: https://spiffe.io/.

[277] Spitzner L. Honeypots: Tracking Hackers. Boston: Addison-Wesley, 2002. 480 p.

[278] Späth C., Mainka C., Mladenov V., Schwenk J. SoK: XML Parser Vulnerabilities. *10th USENIX Workshop on Offensive Technologies (WOOT'16)*. Austin: USENIX Association, 2016.

[279] StepSecurity. Shai-Hulud: The Self-Propagating npm Supply Chain Worm. 2025. URL: https://www.stepsecurity.io/blog/.

[280] Stuttard D., Pinto M. The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws. 2nd ed. Indianapolis: Wiley, 2011. 912 p.

[281] Su Z., Wassermann G. The Essence of Command Injection Attacks in Web Applications. *Proceedings of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL'06)*. New York: ACM, 2006. P. 372–382. DOI: 10.1145/1111037.1111070.

[282] Sucuri. Website Threat Research Report: CMS and Plugin Vulnerabilities. 2024. URL: https://sucuri.net/reports/.

[283] Tabassi E. Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1. Gaithersburg, MD: NIST, 2023. DOI: 10.6028/NIST.AI.100-1.

[284] Temoshok D., Fenton J. L., Choong Y.-Y., Lefkovitz N., Regenscheid A., Galluzzo R., Richer J. P. Digital Identity Guidelines. NIST SP 800-63-4. Gaithersburg, MD, 2025. DOI: 10.6028/NIST.SP.800-63-4. URL: https://csrc.nist.gov/pubs/sp/800/63/4/final.

[285] Thomson M., Turner S. (eds.) Using TLS to Secure QUIC. RFC 9001. IETF, 2021. DOI: 10.17487/RFC9001.

[286] Threat Modeling Manifesto Working Group. Threat Modeling Manifesto. 2020. URL: https://www.threatmodelingmanifesto.org/.

[287] Tsai O. A New Era of SSRF — Exploiting URL Parser in Trending Programming Languages! *Black Hat USA*. Las Vegas, 2017.

[288] U.S. Senate. Capital One Data Breach: Committee Staff Report. 2019 (аналіз помилок конфігурації хмари та SSRF).

[289] UcedaVélez T., Morana M. M. Risk Centric Threat Modeling: Process for Attack Simulation and Threat Analysis. Hoboken: Wiley, 2015. 696 p. ISBN 978-0-470-50096-5.

[290] Urban T., Vekaria Y., Shafiq Z., Böttger C. Third Parties. *The 2024 Web Almanac*. HTTP Archive, 2024. URL: https://almanac.httparchive.org/en/2024/third-parties.

[291] Vassilev A., Oprea A., Fordyce A., Anderson H. Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations. NIST AI 100-2e2025. Gaithersburg, MD: NIST, 2025. DOI: 10.6028/NIST.AI.100-2e2025.

[292] Vissers T., Van Goethem T., Joosen W., Nikiforakis N. Maneuvering Around Clouds: Bypassing Cloud-based Security Providers. *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS'15)*. New York: ACM, 2015. P. 1530–1541. DOI: 10.1145/2810103.2813633.

[293] von Ahn L., Blum M., Hopper N. J., Langford J. CAPTCHA: Using Hard AI Problems for Security. *Advances in Cryptology — EUROCRYPT 2003*. Berlin, Heidelberg: Springer, 2003. P. 294–311. DOI: 10.1007/3-540-39200-9_18.

[294] W3C / Google. Device Bound Session Credentials (DBSC). Web Incubator Community Group. URL: https://github.com/WICG/dbsc.

[295] W3C. Fetch Metadata Request Headers. W3C Working Draft. URL: https://www.w3.org/TR/fetch-metadata/.

[296] W3C. Subresource Integrity. W3C Recommendation, 23 June 2016. URL: https://www.w3.org/TR/SRI/.

[297] W3C. Trusted Types. W3C Working Draft. Web Application Security Working Group. URL: https://www.w3.org/TR/trusted-types/.

[298] Wang D., Cheng H., Wang P., Huang X., Jian G. Zipf's Law in Passwords. *IEEE Transactions on Information Forensics and Security*. 2017. Vol. 12, No. 11. P. 2776–2791. DOI: 10.1109/TIFS.2017.2721359.

[299] Ward R., Beyer B. BeyondCorp: A New Approach to Enterprise Security. *;login:*. 2014. Vol. 39, No. 6. P. 6–11.

[300] Web Authentication: An API for accessing Public Key Credentials. Level 3. World Wide Web Consortium (W3C). URL: https://www.w3.org/TR/webauthn-3/.

[301] Weichselbaum L., Spagnuolo M., Lekies S., Janc A. CSP Is Dead, Long Live CSP! On the Insecurity of Whitelists and the Future of Content Security Policy. *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS'16)*. New York: ACM, 2016. P. 1376–1387. DOI: 10.1145/2976749.2978363.

[302] Xiong W., Lagerström R. Threat Modeling — A Systematic Literature Review. *Computers & Security*. 2019. Vol. 84. P. 53–69. DOI: 10.1016/j.cose.2019.03.010.

[303] XS-Leaks Wiki: Cross-Site Leaks. URL: https://xsleaks.dev/.

[304] Yuan D., Luo Y., Zhuang X., Rodrigues G. R., Zhao X., Zhang Y., Jain P. U., Stumm M. Simple Testing Can Prevent Most Critical Failures: An Analysis of Production Failures in Distributed Data-Intensive Systems. *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI'14)*. Broomfield: USENIX Association, 2014. P. 249–265.

[305] Zalewski M. The Tangled Web: A Guide to Securing Modern Web Applications. San Francisco: No Starch Press, 2011. 320 p.

Додаток. Завдання для самостійної роботи

Завдання згруповано за тематичними блоками посібника. Для кожної теми зазначено зміст роботи та форму подання результату. Результати доцільно оформлювати як фрагменти дослідницької роботи магістранта: з постановкою задачі, обґрунтуванням методу, числовими результатами та висновками. Багато завдань спираються на дослідницькі питання самоперевірки відповідних розділів. Завдання, що передбачають практичне виконання атак, виконуються лише на власних навчальних стендах (наприклад, навмисно вразливих застосунках OWASP) або в межах письмово погодженого тестування.

Блок I. Засади

1. **Аналіз вебресурсу як об'єкта захисту** (розділ 1). Для обраного реального вебресурсу визначити архітектуру, домени довіри, сторонні компоненти, побудувати багаторівневу поверхню атаки та модель порушника. *Форма:* аналітична записка зі схемами.
2. **Визначення застосовних нормативних вимог** (розділ 2). Для державного вебпорталу або інтернет-магазину скласти реєстр обов'язкових і добровільних вимог (законодавство України, ЄС, PCI DSS, ASVS), обчислити індекс розриву відповідності. *Форма:* реєстр вимог і розрахунок.
3. **Кількісне оцінювання захищеності** (розділ 3). Для заданого вебзастосунку розрахувати стійкість секретів і токенів, ефективність обмеження частоти спроб, частку істинних тривог системи виявлення та запас стійкості до перевантаження. *Форма:* розрахунково-аналітична записка.
4. **Моделювання загроз і оцінювання ризиків** (розділ 4). Побудувати діаграму потоків даних, модель загроз (STRIDE, LINDDUN), дерево атак і кількісну оцінку ризику з обґрунтуванням заходів. *Форма:* модель загроз і реєстр ризиків.

Блок II. Механізми та технології захисту вебзастосунків

5. **Профіль захисту транспортного рівня** (розділ 5). Проаналізувати конфігурацію TLS, сертифікатів, HSTS і DNS для групи вебресурсів (наприклад, державних доменів), сформулювати профіль конфігурації з урахуванням постквантового переходу. *Форма:* звіт з результатами вимірювань.

6. **Політика безпеки клієнтської частини** (розділ 6). Розробити й обґрунтувати політику CSP, заголовки ізоляції та атрибути cookie для вебзастосунку зі сторонніми скриптами; оцінити залишкові ризики XSS і CSRF. *Форма:* проєктне рішення.
7. **Захист від ін'єкцій** (розділ 7). На навчальному стенді дослідити класи ін'єкцій і SSRF, розробити заходи на рівні архітектури та коду, перевірити їхню ефективність. *Форма:* звіт з результатами експериментів.
8. **Схема автентифікації** (розділ 8). Спроєктувати схему автентифікації й керування сеансами з ключами доступу (passkeys) і процедурою відновлення облікового запису; визначити рівень AAL. *Форма:* проєктне рішення й аналіз загроз.
9. **Модель авторизації та федеративного входу** (розділ 9). Розробити модель контролю доступу (RBAC/ABAC/ReBAC) та профіль OAuth 2.0/OpenID Connect для багатокористувацького застосунку. *Форма:* модель і профіль конфігурації.
10. **Аудит безпеки API** (розділ 10). Для заданої специфікації OpenAPI або GraphQL-схеми виявити ризики за OWASP API Security Top 10, запропонувати політики шлюзу API. *Форма:* звіт аудиту.
11. **Стійкість до відмови в обслуговуванні** (розділ 11). Розробити багаторівневу схему захисту вебресурсу від DDoS і ботів, оцінити необхідну ємність і вартість. *Форма:* архітектура й розрахунок.

Блок III. Платформи, інфраструктура та нові класи загроз

12. **Захищене розгортання у хмарі** (розділ 12). Спроєктувати розгортання вебзастосунку в контейнерному середовищі з розмежуванням відповідальності, керуванням секретами й контролем конфігурацій. *Форма:* проєкт і перелік контрольних перевірок.
13. **Безпека ланцюга постачання** (розділ 13). Сформувати SBOM для реального проєкту, проаналізувати транзитивні залежності й вразливості, запропонувати рівень SLSA і заходи для конвеєра CI/CD. *Форма:* SBOM, аналіз і план заходів.
14. **Захист застосунку зі штучним інтелектом** (розділ 14). Побудувати модель загроз для вебзастосунку з великою мовною моделлю та агентами, запропонувати й експериментально перевірити захисні патерни проти ін'єкції підказок. *Форма:* модель загроз і звіт.

Блок IV. Операційна безпека та управління

15. **Програма тестування захищеності** (розділ 15). Розробити програму тестування вебзастосунку з поєднанням SAST, DAST, SCA і ручного

тестування; визначити метрики покриття й ефективності. *Форма:* програма й методика.

16. **Моніторинг і реагування** (розділ 16). Спроєктувати схему журналювання, правила кореляції та сценарії реагування на інциденти вебресурсу; оцінити показники MTTD і MTTR. *Форма:* проєкт системи моніторингу і сценарії.
17. **Програма безпеки та наукове дослідження** (розділ 17). Оцінити зрілість процесів безпечного розроблення організації за OWASP SAMM або виконати систематичний огляд за обраним перспективним напрямом як основу дисертаційного дослідження. *Форма:* оцінка зрілості з дорожньою картою або огляд із постановкою наукової задачі.